

[강의] 언리얼 2D Game 기초

작성자: [오희성\[Heesung Oh\]](#)

00. 문서	5
01. 언리얼 설치와 프로젝트 생성	5
02. 언리얼 문법	5
03. 2D Tile Map	5
3.1 Tile Map 생성	5
3.1.1 리소스 다운로드	5
3.1.2 Tile Map Texture 임포트	5
3.1.3 페이퍼2D 텍스처 적용	5
3.2 2D Tile Map 구성	6
3.2.1 타일 맵 구성	6
3.2.2 타일 맵 작성	8
3.2.3 월드 구성	8
3.2.4 타일 맵 Layer	9
04. 2D Game Project 설정	9
4.1 Project 설정	10
4.2 PostProcessVolume 설정	11
4.3 2D Tile 과제	12
05. Paper2D Character	12
5.1 Paper 2D Sprite 생성	12
5.1.1 Paper2D 생성 방법	12
5.1.2 Hero2D 폴더 생성 / 텍스처 이동 / Paper2D Texture setting 적용	13
5.1.3 Sprite 생성	13
5.1.4 스프라이트 뷰	15
5.1.5 소스 리전 편집	15
5.1.6 콜리전 영역 편집	16
5.1.7 렌더지오 편집	16
5.1.8 콜리전 영역 편집	16
참고	17
과제	17
5.2 Paper 2D Flipbook	17
5.2.1 Flipbook 생성	17
5.2.2 Hero2D 플립북 Idle	17
5.2.3 Hero2D 플립북 Run	17
5.2.4 Hero2D 플립북 JumpRise	18
5.2.5 Hero2D 플립북 JumpLand	18
5.2.6 Hero2D FB_Hero* 파일 이동	18
5.2.7 Flipbook 애니메이션 속도	18
5.2.8 Flipbook Attack Dead 추가	19
5.2.8.1 Attack, Dead 텍스처 임포트	19
5.2.8.2 Hero2D 플립북 Attack /Dead 스프라이트 생성	19
5.2.8.3 Hero2D 플립북 Attack 생성	20
5.2.8.4 Hero2D 플립북 Dead 생성	20
5.3 PaperFlipbookActor	21
5.3.1 Paper2D 관련 Actor	21
5.3.2 PaperFlipbookActor	21
5.3.3 PaperFlipbookActor 생성	22
5.3.4 PaperFlipbookActor 편집 창	23
5.3.4.1 FlipbookActor 편집	23
5.3.4.2 Flipbook 추가하기	24
5.3.4.3 Flipbook 편집 완료	25
5.3.4.4 블루프린트 Flipbook 월드 배치	25
5.4 PaperSpriteActor	25
5.4.1 PaperSpriteActor 설정	26
5.4.1.1 모빌리티 무버블 변경	26
5.4.1.2 스프라이트 변경	26
5.4.2 PaperSpriteActor 배경 생성	26
5.4.2.1 배경 스프라이트 생성	26
5.4.2.2 배경 PaperSpriteActor 생성	27
5.4.2.3 배경 PaperSpriteActor 편집	27
5.4.2.4 배경 PaperSpriteActor 편집	28
5.5 PaperCharacter 구성	28

5.5.1 PaperCharacter 컴포넌트 구성	28
5.5.1.1 스프라이트(PaperFlipbookComponent) 구성	29
5.5.1.2 캡슐 컴포넌트 (CollisionCylinder) 구성	29
5.5.1.3 카메라 구성	29
1) Sprint Arm Component 추가	30
2) 카메라 추가	30
5.5.2 PaperCharacter 월드 적용	31
06. Paper2DCharacter 애니메이션	32
6.1 애니메이션	32
6.1.1 Idle 애니메이션	32
6.1.2 Run 애니메이션	34
6.1.3 Jump, Attack 애니메이션	35
6.1.4 카테고리 설정	35
6.2 PaperCharacter 스크립트	37
6.2.1 애니메이션 변경 Run	37
6.2.2 애니메이션 변경 Idle	39
6.2.3 애니메이션 변경 JumpRise	40
6.2.4 애니메이션 변경 Attack	41
6.2.5 Mob Character	41
07. 인풋 시스템	43
7.1 입력 액션(Input Action) 생성	44
7.2 Input Action Value Type 과 IA_2DMove	44
7.2.1 Value Type	44
7.2.2 IA_2DMove	45
7.3 Input Mapping Context	45
7.3.1 Input Mapping 생성	45
7.3.2 Input Mapping에 Action Control 설정	45
7.3.3 입력 매핑 추가	46
7.3.4 Input Mapping Attack1 설정	46
7.4 Input Mapping Context Modifiers 설정	46
7.4.1 Input Mapping Move 설정	47
7.4.2 Negate 모디파이어 설정	47
7.5 Enhanced Input 사용하기	47
순서 개요	47
7.5.1 블루프린트 구성 단계	47
7.5.2 중단점으로 실행 확인	48
7.5.3 InitInput 노드 생성	49
7.6 Input 에 애니메이션 연결	49
08. 2D Move	50
8.1 Add Movement Input	50
8.2 캐릭터 무브먼트 컴포넌트	51
8.2.1 IsMovingOn Ground	51
8.2.2 Vector Length Squared	52
8.3 좌/우 반전	52
8.3.1 Compare Float Node	52
8.3.2 Set Control Rotation	53
8.3.3 Make Rotator	53
8.4 직교(Orthographic) 카메라	54
8.5 Jump	54
8.5.1 Jump / Stop Jump	54
8.5.2 Jump Animation	55
8.6 캐릭터 무브먼트	56
09. Collision / Attack	56
9.1 Collision	56
9.1.1 충돌 판정을 위한 컴포넌트	57
9.1.2 PaperCharacter와 콜리전	57
9.1.3 콜리전 설정	57
9.1.4 충돌 이벤트	58
9.1.5 Hit Event	58
9.2 Mob Collision	58
9.2.1 Box Collision	58
9.2.2 캡슐 콜리전 설정	60
9.2.3 Sprite 컴포넌트 충돌 해제	61
9.2.4 최종 Collision 구조 요약	61
9.3 Character Collision	61
9.3.1 Box Collision	61

9.3.2 캡슐 콜리전 설정	62
9.3.3 Sprite 컴포넌트 충돌 해제	62
9.4 ActorBeginOverlap / ActorEndOverlap	62
9.4.1 Attack Target List	63
9.4.1.1 Target List 추가	64
9.4.1.2 Target List 제거	65
9.5 Attack	65
9.5.1 재생 종료	66
9.5.2 Looping	66
9.5.3 SetupFlipbook 함수 생성	66
9.5.4 Apply Damage	67
9.5.5 AnyDamage 이벤트	68
10. Effect	69
10.1 Spawn Actor	69
10.2 Explosion 분석	70
10.3 2D Explosion 액터	71
10.3.1 2D Explosion 적용	73
10.3.2 Effect 동기화	73
10.3.3 Mob Damage Animation 동기화	74
11. Game Instance	76
11.1 Game Instance 프로젝트 설정	76
11.2 Game Project 변수	77
11.3 Game Project 함수	78
11.4 IsPlayerAlive	78
11.5 AddScore	78
11.6 AddHp	79
11.7 GetHeroHpPercent	79
12. Game Ui	80
12.1 UMG	80
12.2 UMG 구성 요소	80
12.3 HUD 화면 디자인	81
12.3.1 캔버스 패널	81
12.3.2 텍스트	82
12.3.3 프로그레스 바	83
12.3.4 앵커 및 위치	84
12.5 HUD 화면 그래프	85
12.5.1 Score	85
12.5.2 체력 바	86
12.6 위젯 블루프린트 게임 연결	87
12.6.1 위젯 노드 생성	87
12.7 Mob의 AnyDamage에 점수 연결	88
13. 기본 AI 행동(Basic AI Behaviors)	89
13.1 Transform	89
13.2 Ai 이동 / 공격 관련 변수	89
13.3Begin 이벤트	90
13.4 Tick 이벤트	91
13.5 BeginOverlap 이벤트	91
13.6 EndOverlap 이벤트	91
13.7 재생 종료 이벤트	92
13.8 Pure 함수	92
13.9 IsAlive 함수	92
13.10 HasTargetPostion 함수	93
13.11 HasTargetActor 함수	93
13.12 CanAttack 함수	93
13.13 DirectionXToTarget 함수	94
13.14 IsInAttackRange 함수	94
13.15 IsInTargetPosition	95
13.16 SetupTargetPosition	95
13.17 SetupTargetPlayer	96
13.18 ClearTargetPosition	96
13.19 ClearTargetActor	96
13.20 MovePositionX	96
13.21 MoveToTargetPosition	97
13.22 MoveToPlayer	97
13.23 AttackPlayer	98
13.24 CheckBehavior	98

13.25 UpdateBehavior	99
13.26 플레이어 설정	99
14. Ai Controller	101
14.1 AIController 생성 및 Character에 연결	101
14.2 OnPossess	101
14.3 AIController 변수/함수 설정	102
14.3.1 함수 복사	102
13.3.2 에러 수정	103
13.3.3 Add Movement Input 타깃 수정	104
13.3.4 SetActorRotation 타깃 수정	105
13.3.5 Get Actor Location 수정	105
13.3.6 BeginPlay 이벤트	106
13.3.7 Tick 이벤트	106
14.4 몹 캐릭터 클래스	106
14.4.1 controllerAi 변수 추가	106
14.4.2 Tick 이벤트 정리	106
14.4.3 Hit 이벤트 정리	107
14.4.4 ActorBeginOverlap 이벤트	107
14.4.5 ActorEndOverlap 이벤트	107
14.4.6 AttackPlayer 함수	107
14.4.7 AI_EnemyMob 으로 복사된 변수 및 함수 삭제	108
15. 게임 완성	109
15.1 Game Data 초기화	109
15.2 게임 흐름 구조	109
15.3 UiLobby	110
15.3.1 위젯 생성	110
15.3.2 화면 구성: 캔버스 / 이미지 마스크 / 게임 타이틀	111
15.3.3 화면 구성: 게임 시작 버튼	113
15.3.4 게임 시작 버튼(ButtonGameStart) 이벤트	115
15.3.5 게임 종료 버튼(ButtonGameExit) 이벤트	115
15.4 UiPlayResult	115
15.4.1 화면 디자인	116
15.4.1.1 TextScoreTitle	116
15.4.1.2 TXT_GAME_SCORE	117
15.4.1.3 완성된 디자인	117
15.4.2 ButtonGameLobby 이벤트	118
15.4.3 ButtonGameExit	118
15.5 UiPopGameOver	119
15.5.1 화면 디자인	119
15.5.1.1 TextScoreTitle	119
15.5.1.2 완성된 디자인	120
15.5.2 ButtonOk 이벤트	120
15.6 게임 시작 LEVEL(LEVEL_LOBBY)	121
15.6.1 LEVEL_LOBBY 생성	121
15.6.2 레벨 초기 구성	121
15.6.3 실행 시 나타나는 문제	122
15.6.4 Dummy Character	122
15.6.5 Dummy GameMode	122
15.6.6 Dummy GameMode 적용	123
15.6.7 마우스 포인터 보이기	123
15.6.8 LEVEL2D_LOBBY에 UiLobby 연결하기	124
15.7 게임 플레이 결과 LEVEL(LEVEL2D_PLAY_RESULT)	125
15.7.1 LEVEL_LOBBY 복제하기	125
15.7.2 레벨 블루프린트 수정	125
15.8 게임 플레이 LEVEL(LEVEL2D_01)	125
15.8.1 IsGameOver 함수 생성	126
16. 패키징	126
16.1 기본 맵 설정(Default Maps)	126
16.2 빌드에 포함될 레벨 지정	127
16.3 Windows 실행 파일 패키징	127
맺음말	128

00. 문서

- 목표:
 - Paper2D, PaperZD 를 사용하여 언리얼 2D게임의 제작 기술을 습득한다.
- 프로젝트 전체 리소스
 - <https://github.com/3dapi/Ue2DProj>
- 관련 자료
 - Unreal Tile Map: [언리얼 엔진의 페이퍼 2D 타일 세트 및 타일 맵](#)
 - 언리얼 페이퍼2D: [언리얼 엔진의 페이퍼 2D](#)
 - Resource: <https://raw.githubusercontent.com/3dapi/GameResource/main/2DGameTutorial/LegacyFantasyForest.zip>

01. 언리얼 설치와 프로젝트 생성

[강의] 언리얼01 설치 / 프로젝트 생성

02. 언리얼 문법

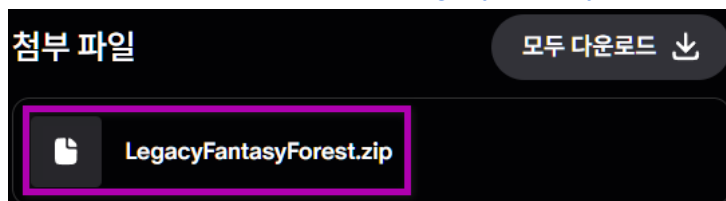
[강의] 언리얼02 문법

03. 2D Tile Map

3.1 Tile Map 생성

3.1.1 리소스 다운로드

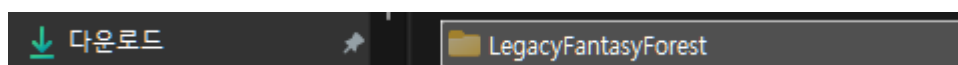
<https://www.patreon.com/posts/legacy-fantasy-95422822>



3.1.2 Tile Map Texture 임포트

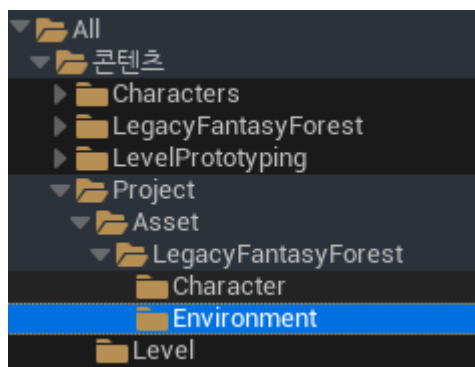
콘텐츠/Project/Asset 폴더 생성

리소스 zip 파일 압축 풀기



폴더를 콘텐츠/Project/Asset에 끌어서 놓기

콘텐츠 폴더에서 리소스 임포트 확인



3.1.3 페이퍼2D 텍스처 적용

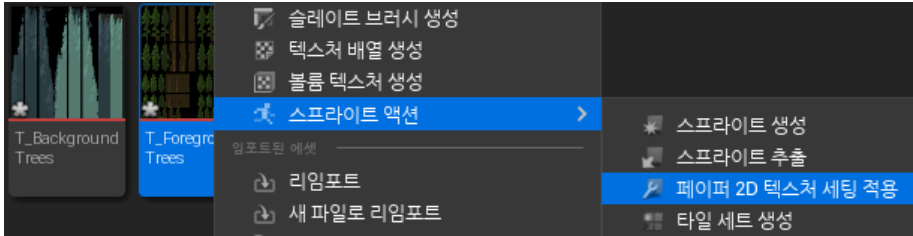
텍스처를 확대하면 3D 필터링이 적용되어 2D 분위기가 안남



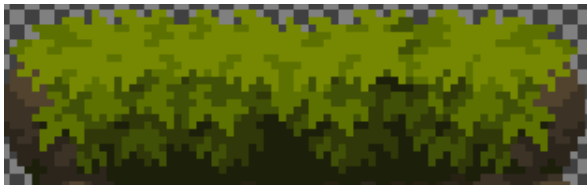
임포트한 모든 텍스처를 전부 선택



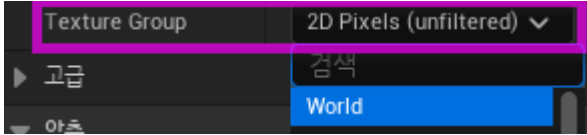
페이퍼 2D 텍스처 세팅 적용



페이퍼 2D 텍스처 세팅 적용



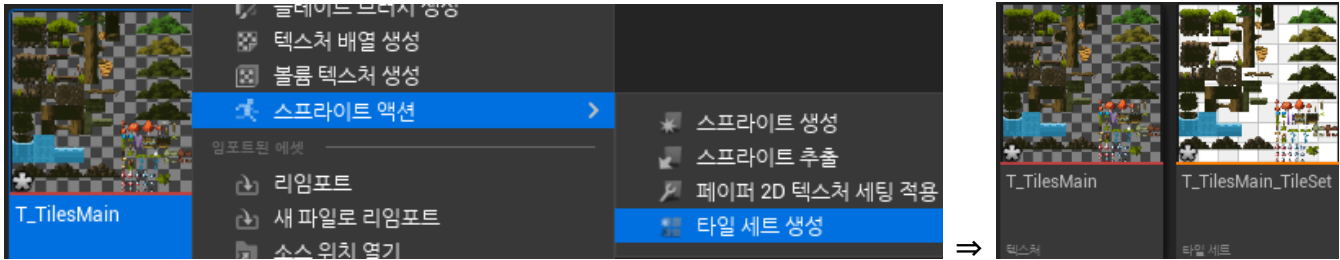
3D 텍스처로 환원하려면 텍스처 ⇒ 디테일 패널 ⇒ Texture Group ⇒ World 적용



3.1.4 Tile Map 생성

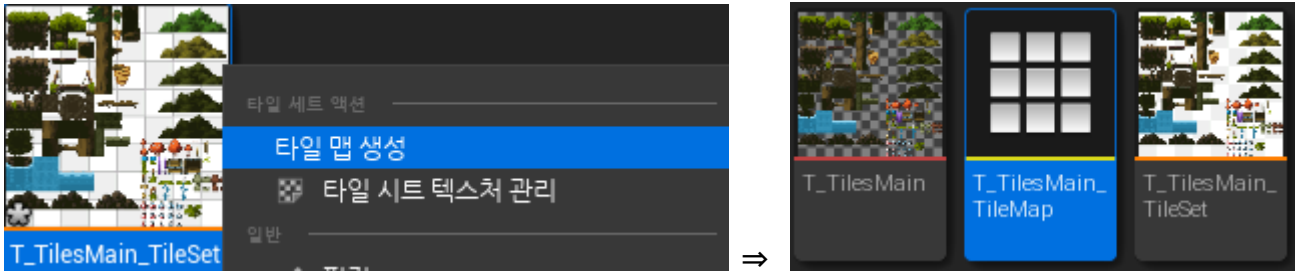
타일 맵 생성 과정은 타일 텍스처 ⇒ 타일 세트 생성 ⇒ 타일 맵 생성 순으로 진행 한다.
먼저 타일맵에서 타일 세트를 생성한다

타일 맵 텍스처 선택 ⇒ 우버튼 ⇒ 스프라이트 액션 ⇒ 타일 세트 생성



타일 세트에서 타일 맵을 생성

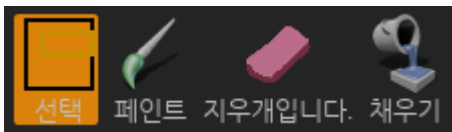
타일 세트 선택 ⇒ 우버튼 ⇒ 타일 맵 생성



3.2 2D Tile Map 구성

3.2.1 타일 맵 구성

주의 메뉴 ⇒ 선택 버튼: shift 키를 눌러야 보임.



타일 선택

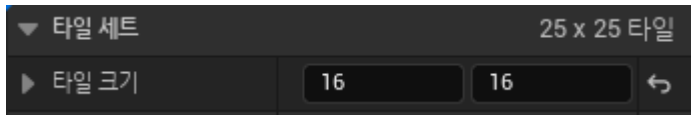
디폴트 상태에서 타일을 선택하면 의도한 영역을 벗어나 있음.



타일 맵을 닫음.

타일 세트 변경

싱글타일 에디터 ⇒ 타일 세트 오픈

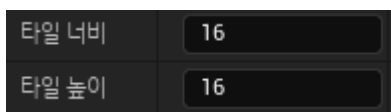


타일 세트 뷰에서 영역 확인



타일 너비, 높이 확인

타일맵 오픈 ⇒ 디테일 ⇒ 셋업 ⇒ 타일 너비, 높이 확인

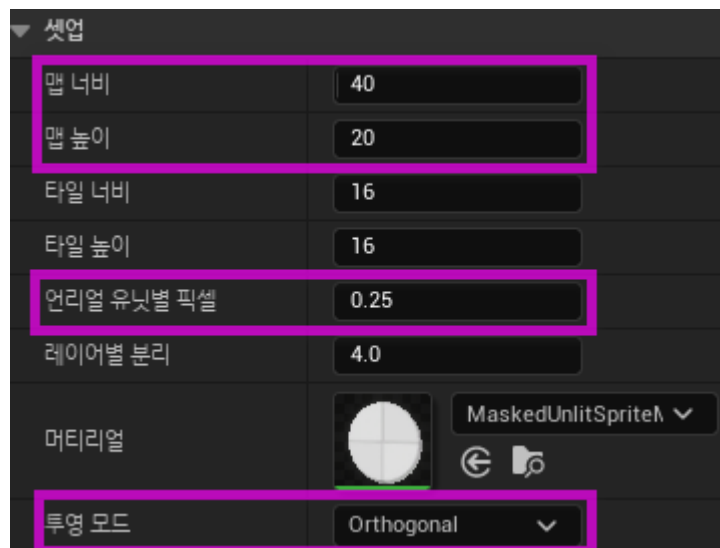


맵 셋업 너비/높이/픽셀

맵의 너비, 높이 ⇒ 타일의 숫자

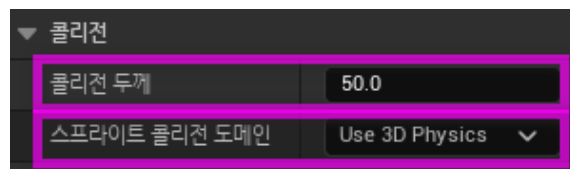
언리얼 유닛별 픽셀 ⇒ 100cm 당 차지하는 픽셀을 수. 작을 수록 화면에 확대

ex) 0.25 ⇒ 1픽셀이 400cm. 0.1 ⇒ 1픽셀이 1000cm



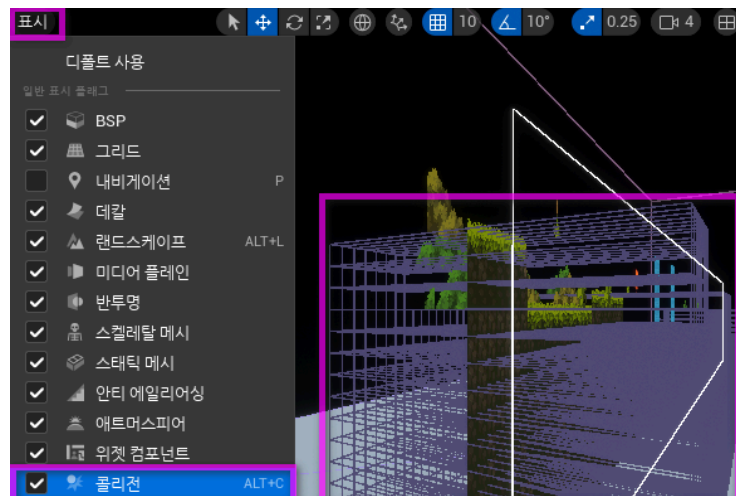
맵 셋업 너비/높이/픽셀

콜리전 두께: 3D 공간에서 평면과 수직인 콜리전 돌출 두께



다음은 콜리전 두께를 800.0 정하고 에디터에서 확인한 화면

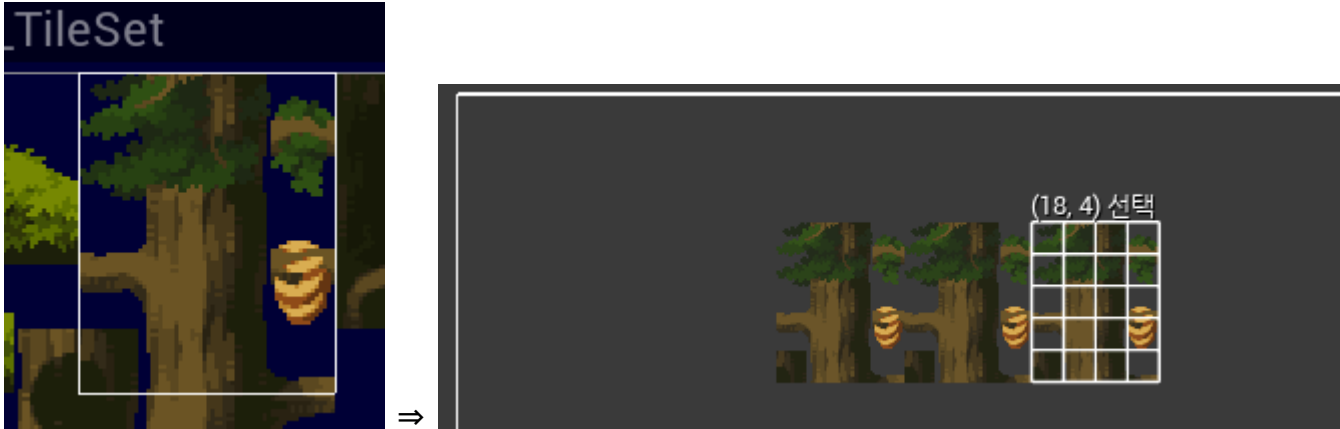
콜리전 보이기: 뷰포트 ⇒ 콜리전 체크 또는 ALT+C



실행 해서 타일 맵으로 3인칭 캐릭터를 이동하면 충돌하는 것을 볼 수 있음.

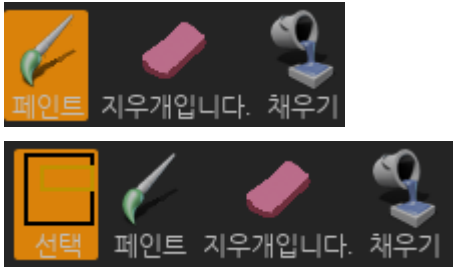
3.2.2 타일 맵 작성

타일 셋에서 영역으로 선택 ⇒ 타일 맵 뷰포트의 맵 영역 안 클릭으로 타일 맵을 구성

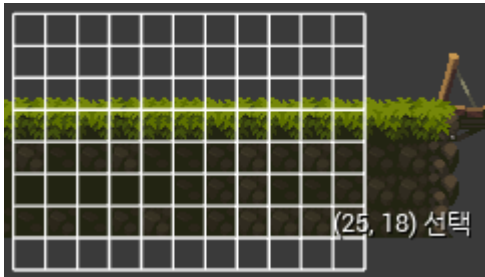


타일 맵 구성 복사

페인트 상태로 설정하고, Shift를 눌러 선택이 메뉴에 보이도록 한다.

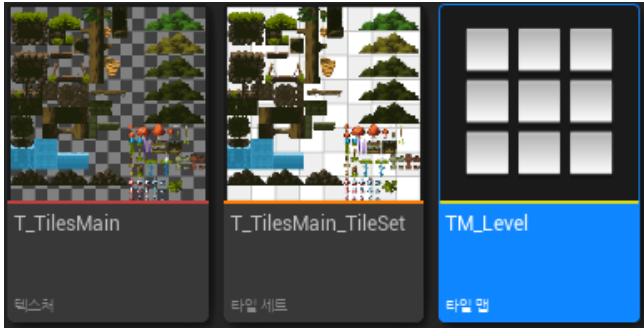


shift 누른 상태에서 맵 구성 뷰포트 창에서 마우스 좌버튼 누르고 드래그 하면 영역에 복사



타일 맵 이름 변경

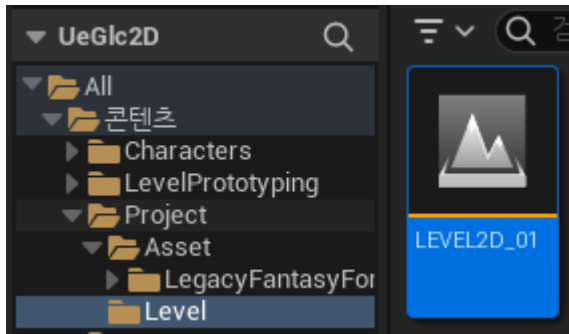
T_TilesMain_TileMap ⇒ TM_Level



3.2.3 월드 구성

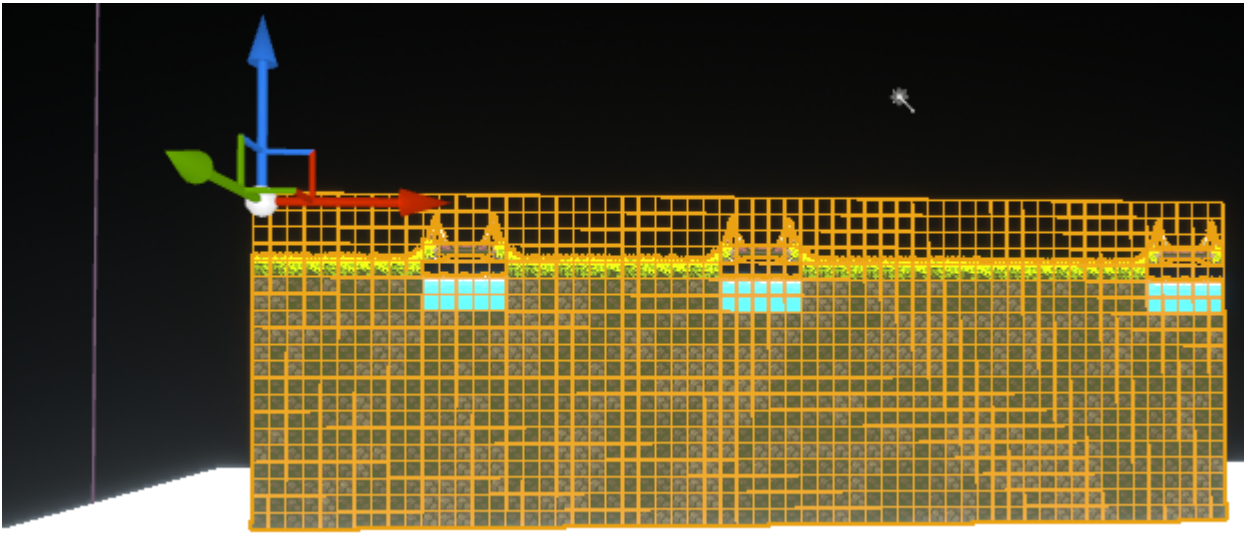
신규 레벨 생성

[강의] 언리얼01 문법 에서 배웠던 "신규 레벨 생성" 과정을 따라 신규 레벨에 "콘텐츠/Project/Level/LEVEL2D_01" 이름으로 저장



타일 맵 TM_Level을 월드에 배치

TM_Level를 선택하고 마우스로 끌어다 월드에 배치



Transform 위치 $x = 0, y = 0$ 으로 설정하고 기즈모를 이용해서 z값을 적당히 조정



색상 조정

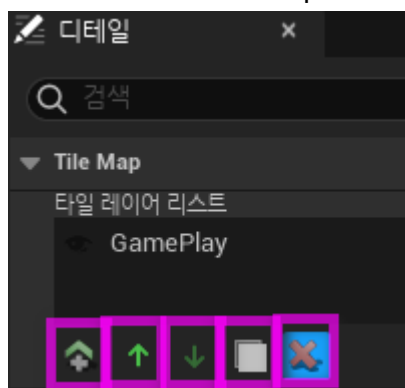
실행 하면 원하는 색상보다 밝은 색상이 나옴
2D Game Project 설정에서 이를 보완 하자.

3.2.4 타일 맵 Layer

레이어로 입체감을 표현

레이어 관리

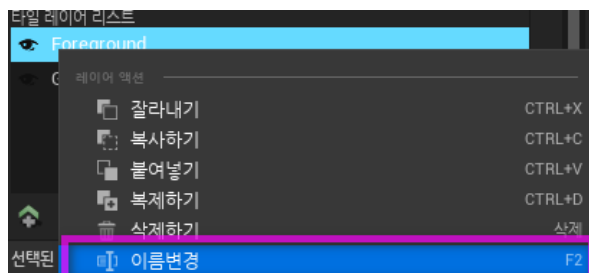
디테일 패널 ⇒ Tile Map 하단 5개 버튼



각각 레이어 추가, 레이어 앞으로 이동, 레이어 뒤로 이동, 레이어 복제, 레이어 삭제

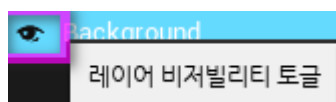
레이어 추가 버튼을 눌러서 2개의 레이어를 추가하고 각각의 이름을 **Foreground**, **Background**로 정한다.

이름 수정은 해당 레이어를 선택하고 마우스로 천천히 두 번 클릭하거나 마우스 우측 버튼 누르면 수정 메뉴를 클릭해서 수정할 수 있다



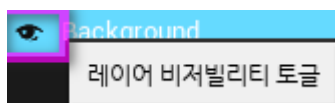
레이어 비저빌리티

레이어 비저 빌러티를 누를 때마다 레이어가 화면에 보이거나 숨겨진다



게임 적용레이어 비저빌리티

레이어 비저 빌러티를 누를 때마다 레이어가 화면에 보이거나 숨겨진다

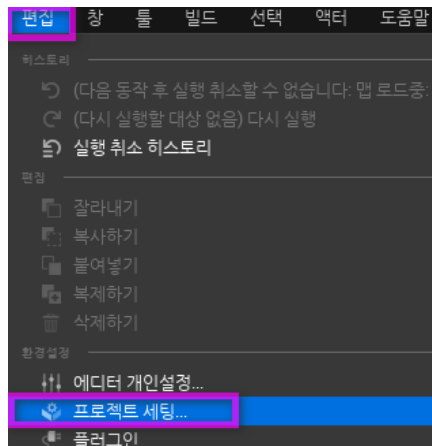


04. 2D Game Project 설정

언리얼은 3D 렌더링이 기본. 2D 게임에 맞게 프로젝트를 설정.

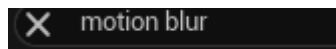
4.1 Project 설정

메뉴 ⇒ 편집 ⇒ 프로젝트 설정 오픈



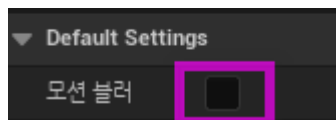
모션 블러 끄기

검색 필드에 motion blur 입력



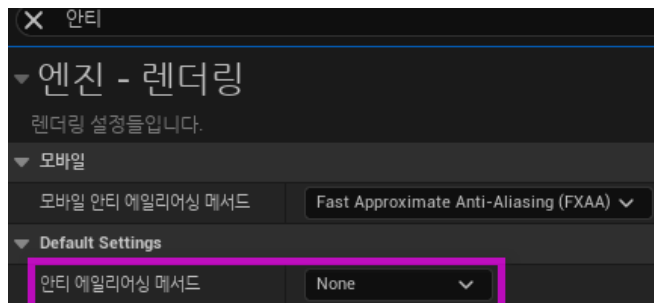
모션 블러 체크 해제

Default Settings ⇒ 모션 블러



안티 에일리어싱 끄기

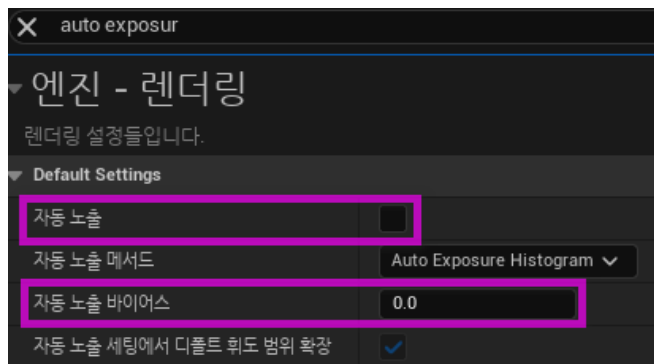
검색 필드에 안티 입력



자동 노출 끄기

검색 필드에 auto exposur 입력

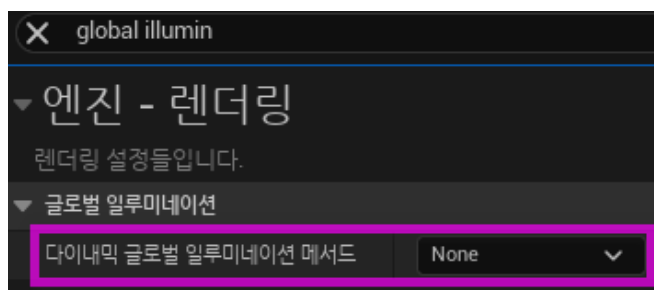
자동 노출 끄기, 자동 노출 바이어스 0으로 설정



글로벌 일루미네이션 끄기

검색 필드에 global illumin 입력

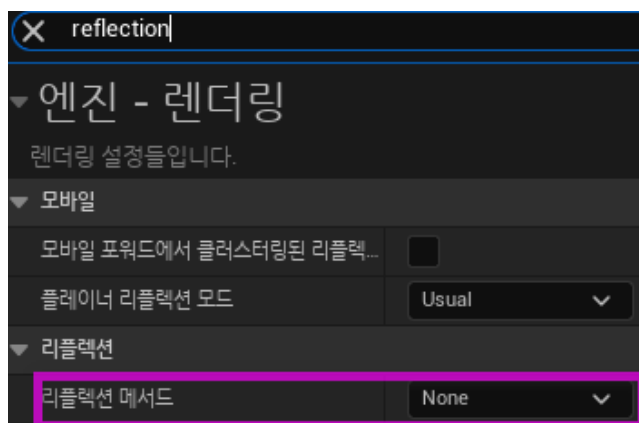
다이내믹 글로벌 일루미네이션 메서드 ⇒ None으로 설정



리플렉션 끄기

검색 필드에 reflection 입력

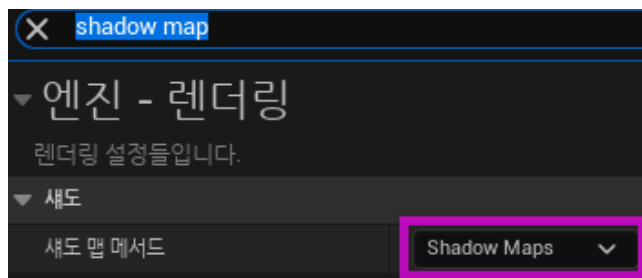
리플렉션 메서드 ⇒ None으로 설정



새도 맵 설정

검색 필드에 shadow map 입력

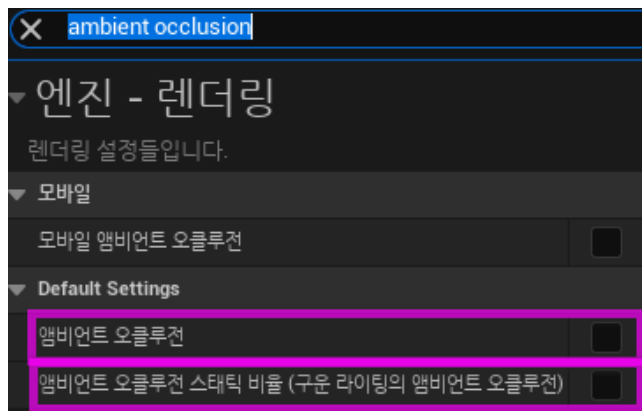
새도 맵 메서드 Shadow Maps 로 변경



앰비언트 오클루전

검색 필드에 ambient occlusion 입력

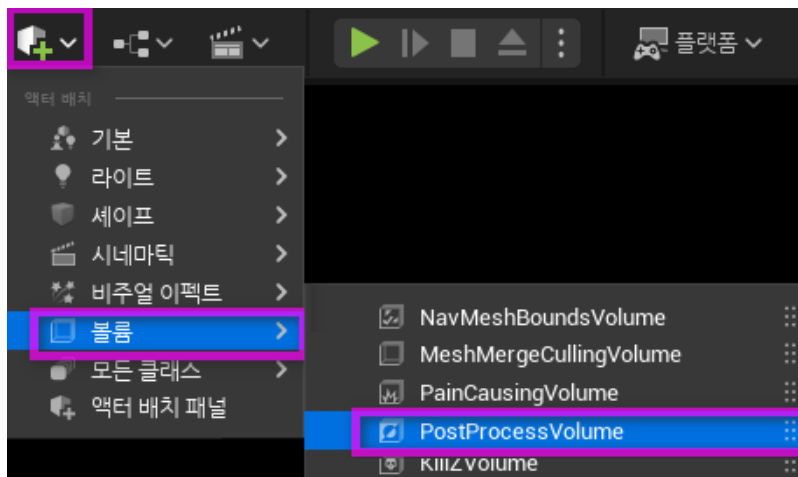
앰비언트 오클루전 체크 해제, 앰비언트 오클루전 스테틱 비율 체크 해제



4.2 PostProcessVolume 설정

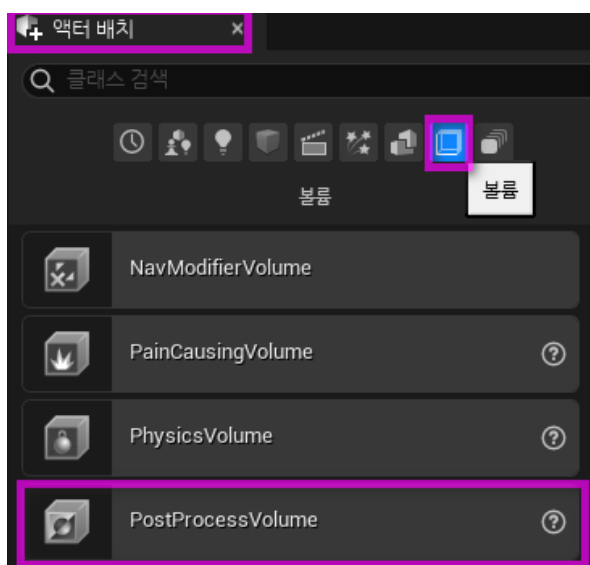
PostProcessVolume 추가 방법 2가지

툴바 ⇒ 추가 버튼("+") ⇒ 볼륨 ⇒ PostProcessVolume 선택



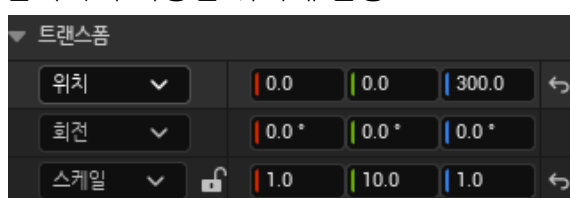
액터 배치 패널에서 추가

볼륨 선택 ⇒ PostProcessVolume 선택



PostProcessVolume Transform 설정

관리하기 적당한 위치에 선정

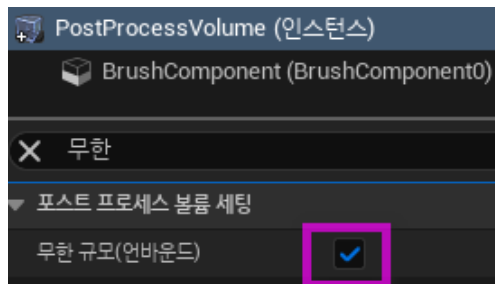


무한 규모(언바운드)

카메라 위치에 상관 없이 PostProcessVolume 이 적용 되도록 설정

PostProcessVolume 인스턴스 선택 ⇒ 디테일 ⇒ 검색 ⇒ "무한" 입력

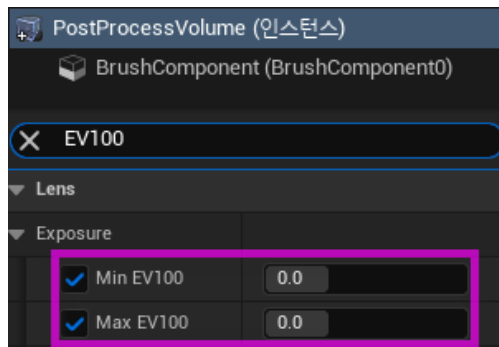
무한 규모(언바운드) 체크



Exposure 설정

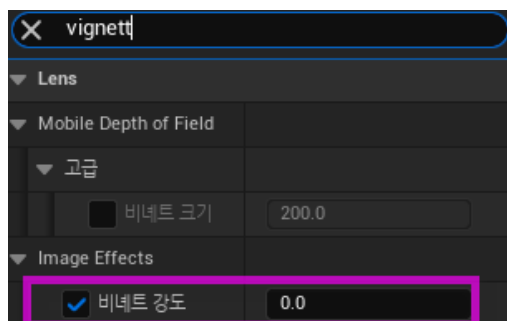
EV100 검색

Min EV100, MaxEV100 ⇒ 0으로 설정



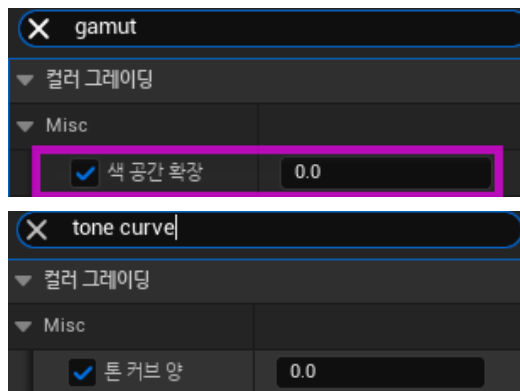
비네프트 효과 끄기

피사체 주먹 효과를 주기 위해 화면 가장자리 약간 어둡게 만드는 요소 제거



색 공간 확장, 톤 커브 양 설정

Misc 로 검색하거나 gamut, tone curve를 입력해서 색공간 확장, 톤커브 양을 0으로 설정



4.3 2D Tile 과제

1. 신규 타일 맵을 생성해본다.

2. 다음 질문을 답할 수 있어야 한다.

Texture Atlas란 무엇인가?

Texture Atlas를 사용하는 이유?

게임에서 스프라이트란 무엇인가?

3. 실습에 사용할 애니메이션 이미지 준비한다.

character 2종 구하기 필수

idle / run or walk / attack / jump

★ 반드시 출처와 라이선스를 표기할 것

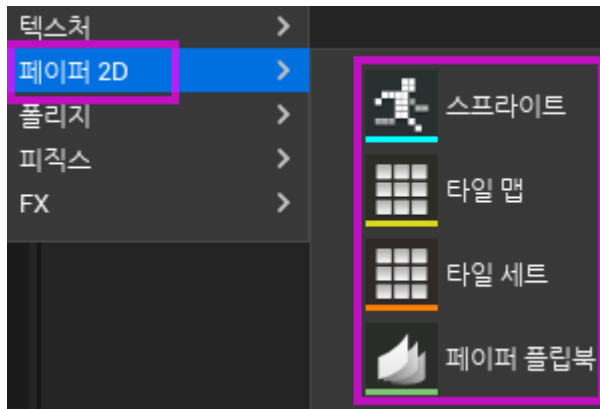
05. Paper2D Character

5.1 Paper 2D Sprite 생성

스프라이트(Sprite)는 2D 게임에서 화면에 렌더링되는 개별 이미지 단위를 의미한다. C언어에서 여러 데이터를 하나로 묶어 관리하기 위해 구조체(Struct)를 사용하듯, 언리얼 엔진의 스프라이트는 원본 텍스처에서 자신이 렌더링될 영역(UV 좌표)과 충돌(Collision) 영역, 렌더링 최적화 정보 등을 하나의 데이터 객체로 묶어 관리하는 예셋이다.

5.1.1 Paper2D 생성 방법

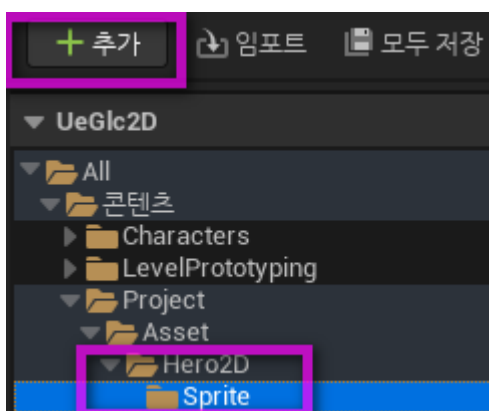
방법1: 콘텐츠 브라우저 ⇒ 추가("+") 버튼 또는 폴더의 빈 영역 마우스 우 클릭



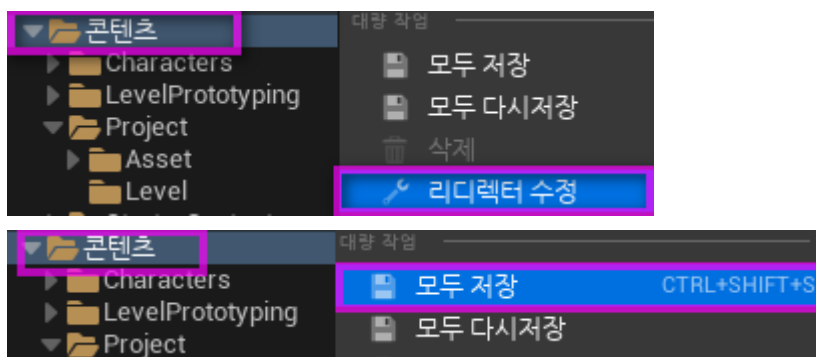
방법2: 텍스처 ⇒ 페이퍼2D 텍스처 세팅을 완료하고 해당 텍스처로 타일맵을 생성하거나 스프라이트 추출 ⇒ 플립북 생성 으로 진행
작업 속도가 빠르고 절차의 일관성이 유지 됨

5.1.2 Hero2D 폴더 생성 / 텍스처 이동 / Paper2D Texture setting 적용

스프라이트를 관리할 **콘텐츠/Project/Asset/Hero2D/Sprite** 폴더를 생성한다. 파일의 이동이 발생하면 파일의 참조(Reference) 주소가 변경되므로, C언어에서 댕글링 포인터(Dangling Pointer) 문제가 발생하지 않도록 리더렉터 수정 및 모두 저장(Save All)을 실행하여 참조 무결성을 유지해야 한다. 캐릭터 텍스처를 이동한 후, 3D 필터링으로 인해 픽셀이 뭉개지는 것을 막기 위해 반드시 [스프라이트 액션] -> [페이퍼 2D 텍스처 세팅 적용]을 실행한다.

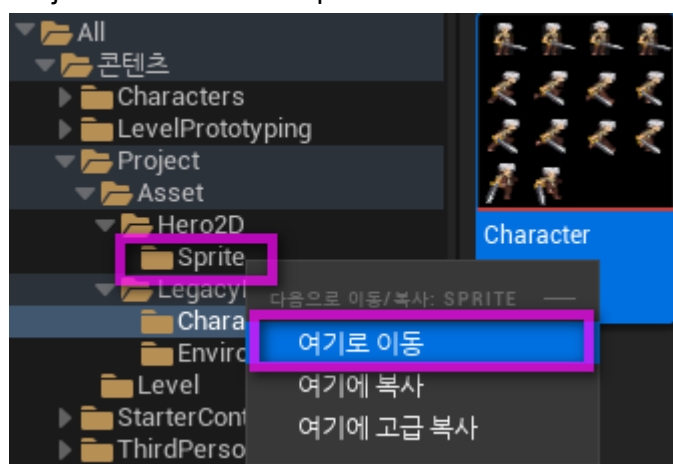


리소스의 이동이 있으면 레퍼런스가 올바르게 설정되도록 리더렉터 수정 / 모두 저장 실행



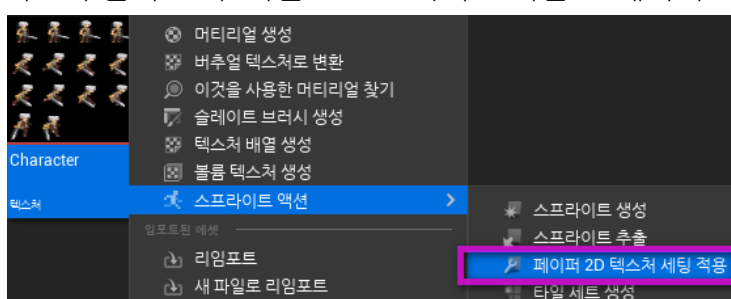
Character Texture 이동

콘텐츠/Project/Asset/Hero2D/Sprite 폴더로 이동



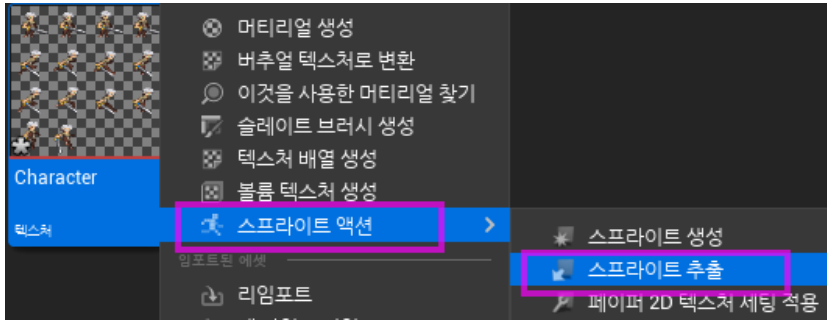
Paper2D 텍스처 세팅 적용

텍스처 선택 ⇒ 우 버튼 ⇒ 스프라이트 액션 ⇒ 페이퍼 2D 텍스처 세팅 적용



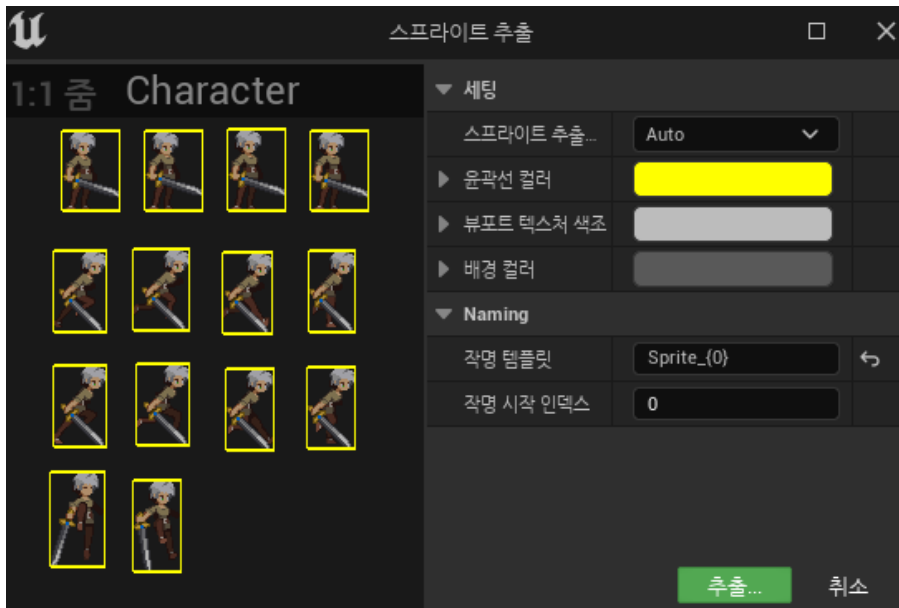
5.1.3 Sprite 생성

텍스처 선택 ⇒ 우 버튼 ⇒ 스프라이트 액션 ⇒

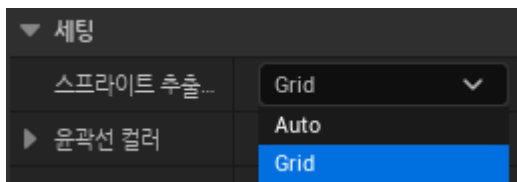


디폴트 선택

스프라이트의 크기가 다르면 센터가 안 맞아 애니메이션이 틀어짐



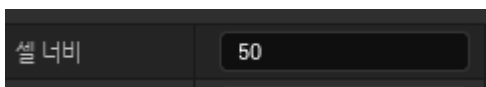
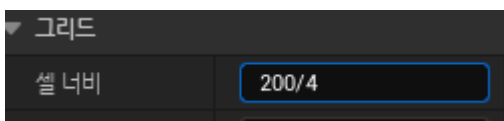
세팅 ⇒ 스프라이트 추출 ⇒ Grid로 선택



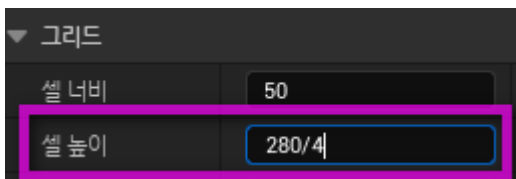
전체가 선택됨



셀 너비 값을 가로 스프라이트 숫자로 나눔



셀 높이 값을 세로 스프라이트 숫자로 나눔



스프라이트 추출

스프라이트 분할 미리보기 확인

추출 버튼을 실행

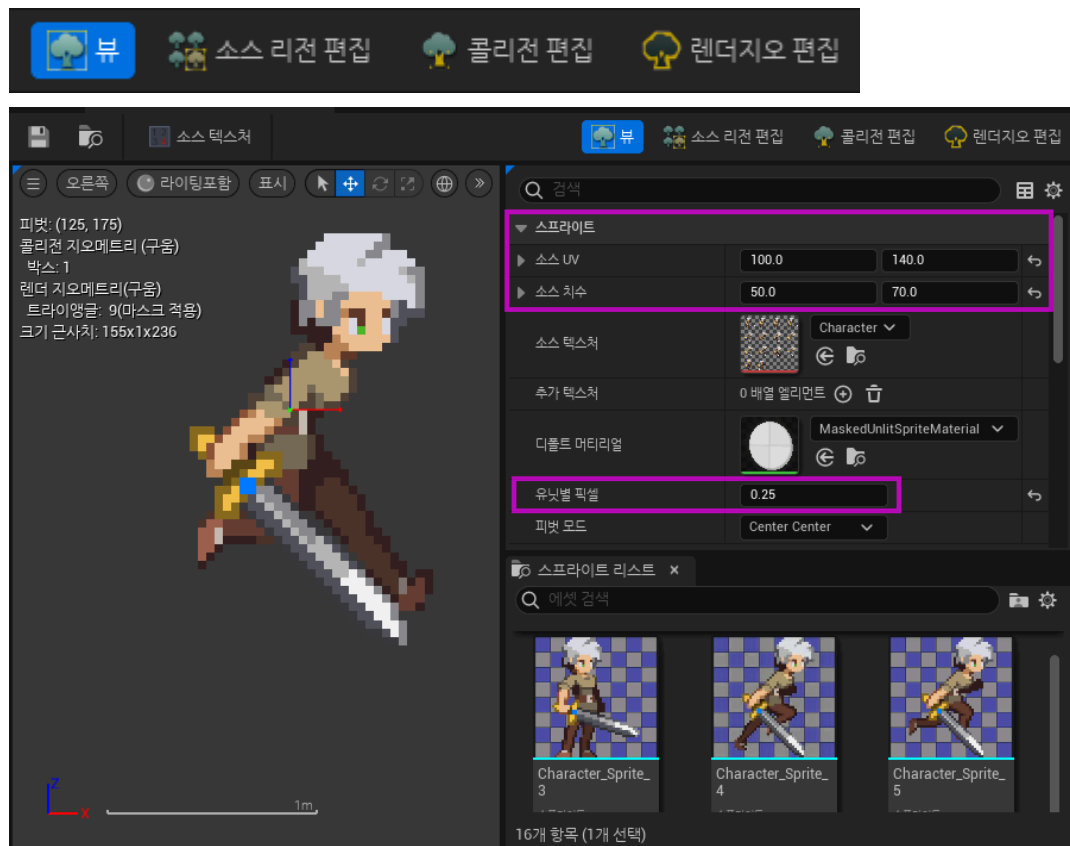


추출된 스프라이트



5.1.4 스프라이트 뷰

스프라이트 리소스를 클릭하면 스프라이트 에디터 실행



소스 UV

좌상단 기준 픽셀 시작

소스 치수

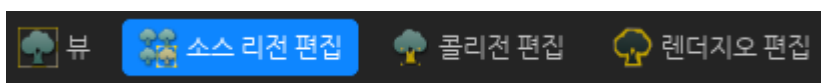
스프라이트 픽셀 크기

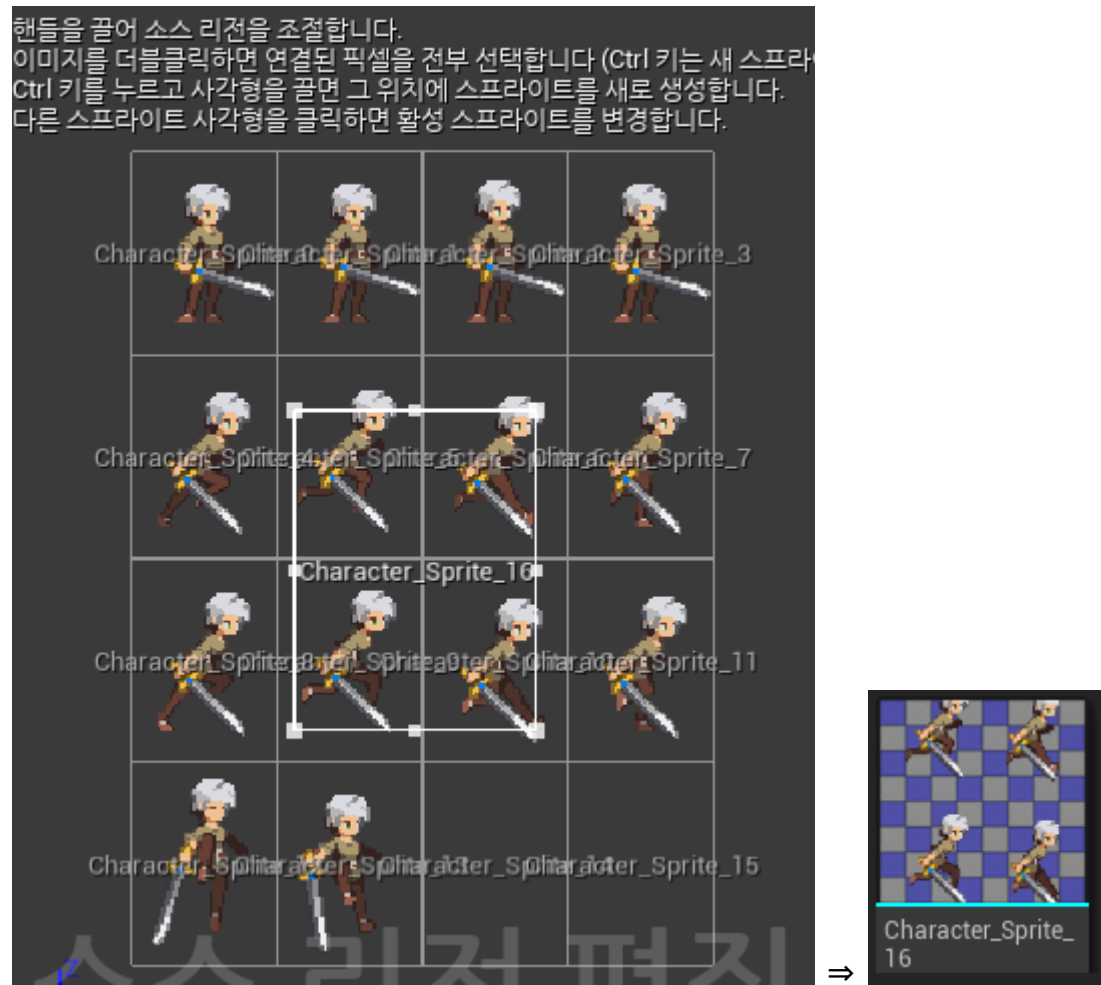
유닛별 픽셀

100cm 당 차지하는 픽셀을 수

5.1.5 소스 리전 편집

스프라이트 추가





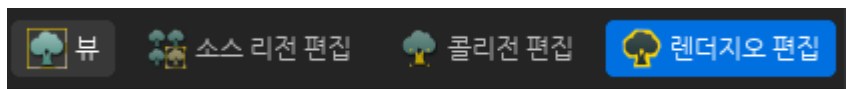
5.1.6 콜리전 영역 편집

충돌영역을 디테일하게 설정



5.1.7 렌더지오 편집

폴리곤을 추가해서 불필요한 렌더링을 제거 < 렌더링 최적화 참조



5.1.8 콜리전 영역 편집



참고

콜리전 편집, 렌더지오 편집은 거의 안함

특히 콜리전(충돌)의 경우 블루프린트에서 처리하기에 액터의 충돌박스를 사용하는 것이 유지보수에 유리

과제

화면에 배치할 **Atlas** 이미지 준비

문열기/ 닫기

스프링

이펙트 폭발, 불꽃 등 2종 이상

★ 반드시 출처와 라이선스를 표기할 것

5.2 Paper 2D Flipbook

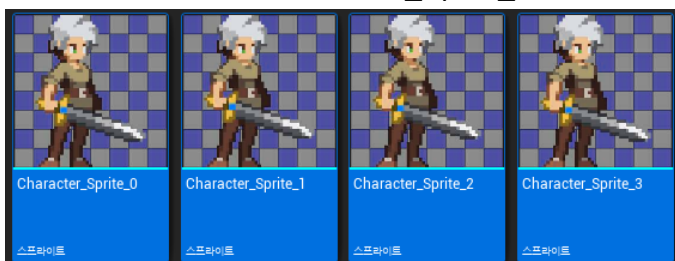
5.2.1 Flipbook 생성

언리얼 문서: [언리얼 엔진의 페이퍼 2D 플립북](#)

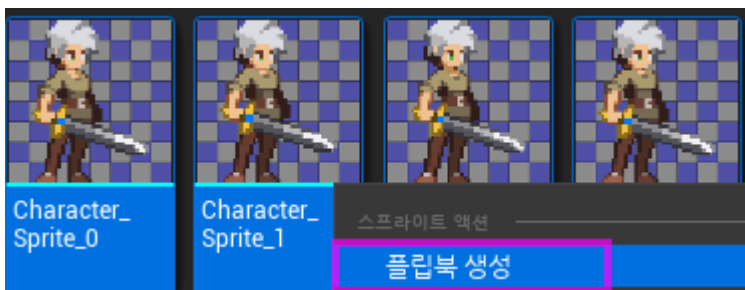
5.2.2 Hero2D 플립북 Idle

컨텐츠 브라우저의 추가 버튼 ⇒ Paper2D ⇒ 플립북을 선택 하는 방식으로 생성해도 되지만 만들어진 스프라이트 리스트를 선택해서도 생성이 가능

Idle 스프라이트에 해당하는 Character_Sprite_0~3 을 전부 선택



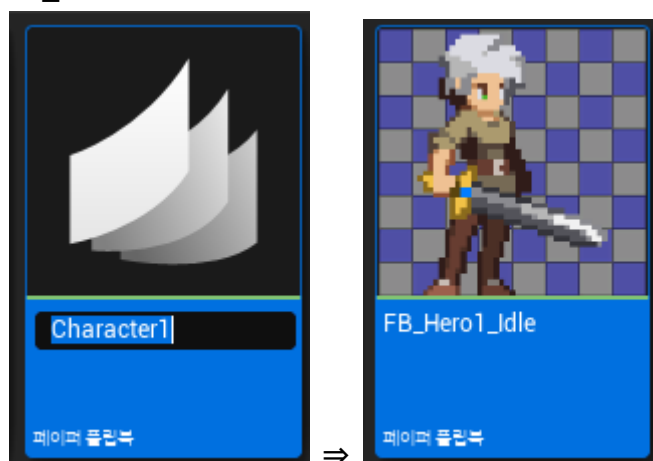
선택한 상태에서 마우스 우버튼 클릭



플립북 이름 설정

FB 접두어를 사용해서

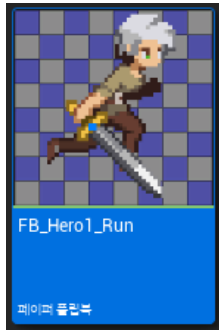
FB_Hero1_Idle을 입력



5.2.3 Hero2D 플립북 Run

Idle 과 같은 방법으로 Character_Sprite_4~Character_Sprite_11 까지 선택 후에 플립북 생성, 이름을 FB_Hero01_Run으로 변경





5.2.4 Hero2D 플립북 JumpRise

Character_Sprite_12 선택 ⇒ 플립북 생성 ⇒ FB_Hero1_JumpRise으로 변경



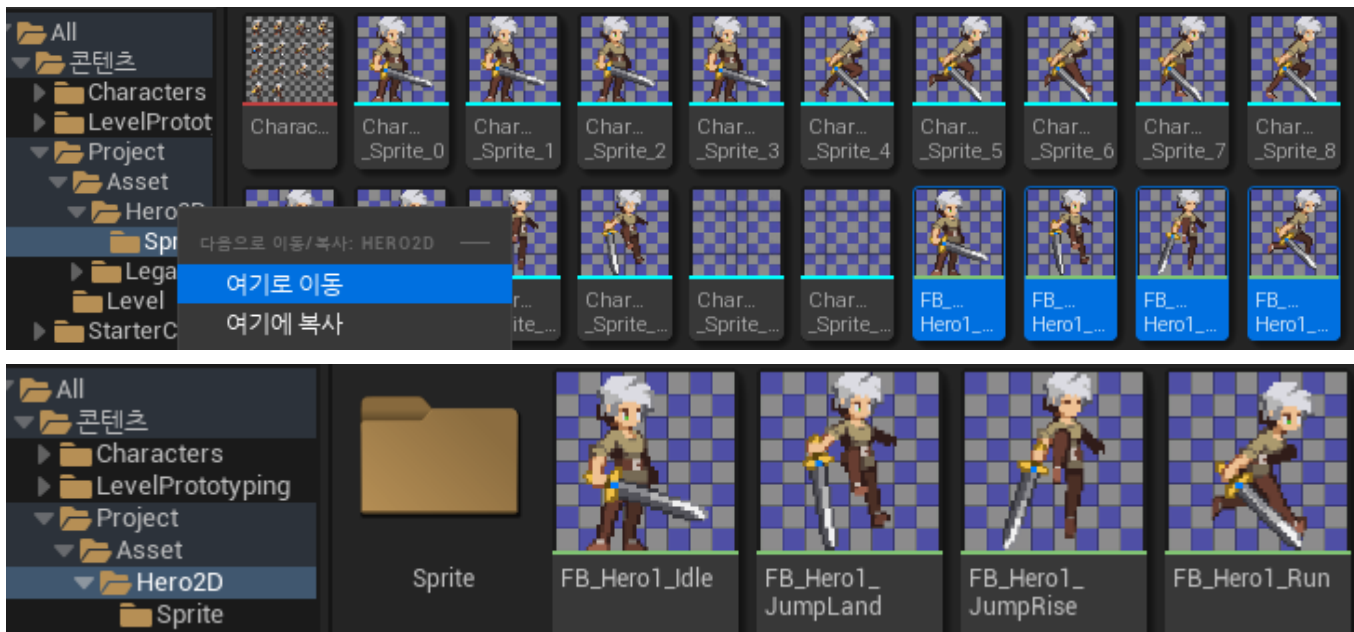
5.2.5 Hero2D 플립북 JumpLand

Character_Sprite_13 선택 ⇒ 플립북 생성 ⇒ FB_Hero1_JumpLand으로 변경



5.2.6 Hero2D FB_Hero* 파일 이동

FB_Hero1* 에 해당하는 페이퍼북 파일을 선택해서 /콘텐츠/Project/Asset/Hero2D 폴더로 이동
이동 후 콘텐츠 폴더 선택 후 리디렉터 수정, 모두 저장 실행



5.2.7 Flipbook 애니메이션 속도

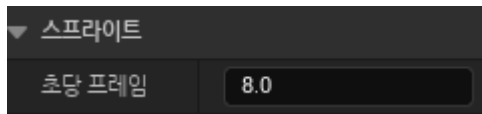
FB_Hero1_Idle 을 클릭해서 열면 애니메이션이 아주 빠르게진행되고 있음을 볼 수 있다.

디테일 패널 ⇒ 스프라이트 ⇒ 초당 프레임을 적당히 조정해서 자연스럽게 맞춘다

15.0 ⇒ 3.0 ~ 8.0



FB_Hero1_Run의 초당 프레임

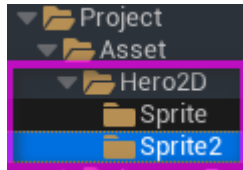


5.2.8 Flipbook Attack Dead 추가

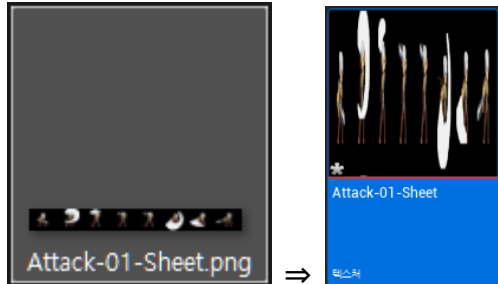
5.2.8.1 Attack, Dead 텍스처 импорт

<https://raw.githubusercontent.com/3dapi/GameResource/main/2DGameTutorial/LegacyFantasyHighForest2.3.zip>

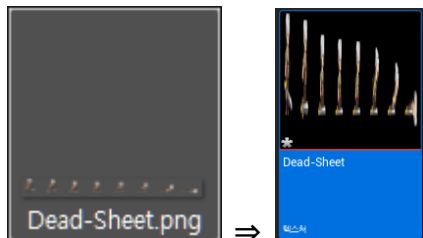
/콘텐츠/Project/Asset/Hero2D/Sprite2 폴더 생성



Sprite2 폴더에 다운로드 한 Character/Attack-01/Attack-01-Sheet.png импорт

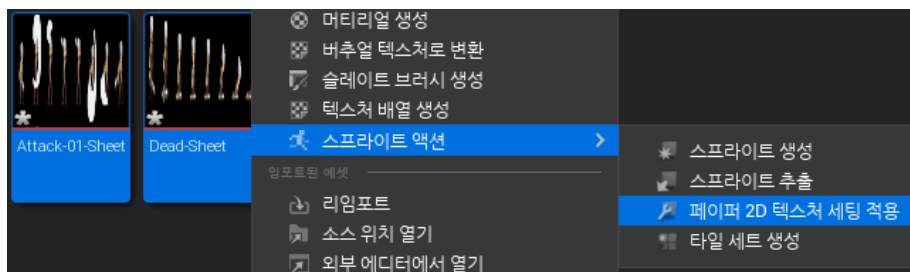


Sprite2 폴더에 다운로드 한 Character/Dead/Dead-Sheet.png импорт



페이퍼2D 텍스처 세팅 적용

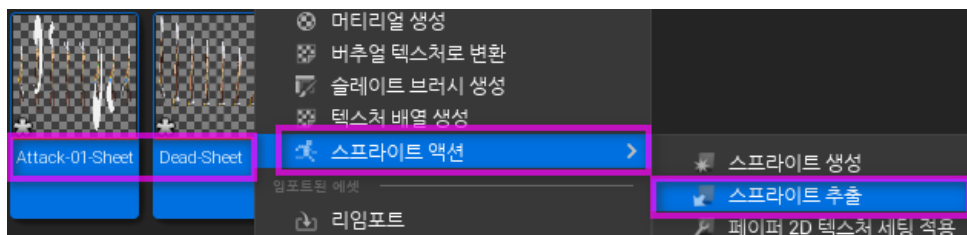
임포트한 Attack, Dead 텍스처를 선택 ⇒ 페이퍼2D 텍스처 세팅 적용



5.2.8.2 Hero2D 플립북 Attack /Dead 스프라이트 생성

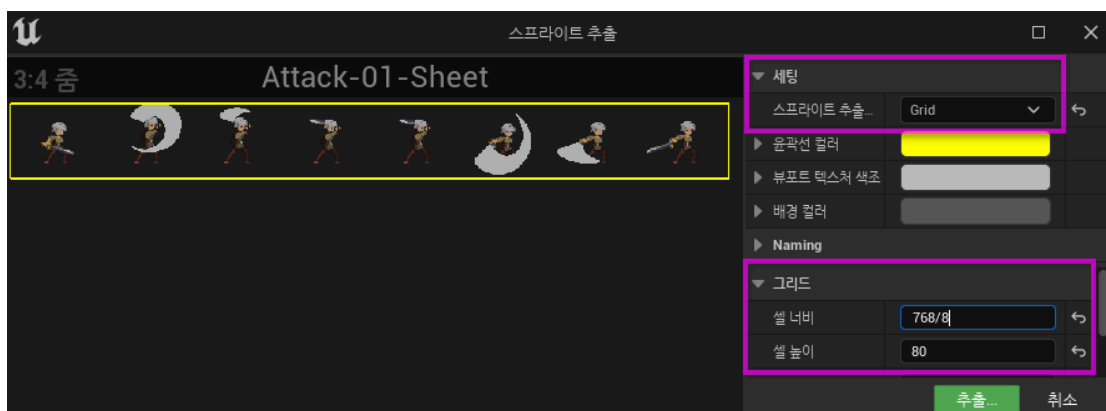
앞서 진행한 플립북 생성에 따라 텍스처를 선택하고 Flipbook을 생성

Attack, Dead 텍스처를 동시에 선택 ⇒ 우 클릭 ⇒ 스프라이트 액션 ⇒ 스프라이트 추출



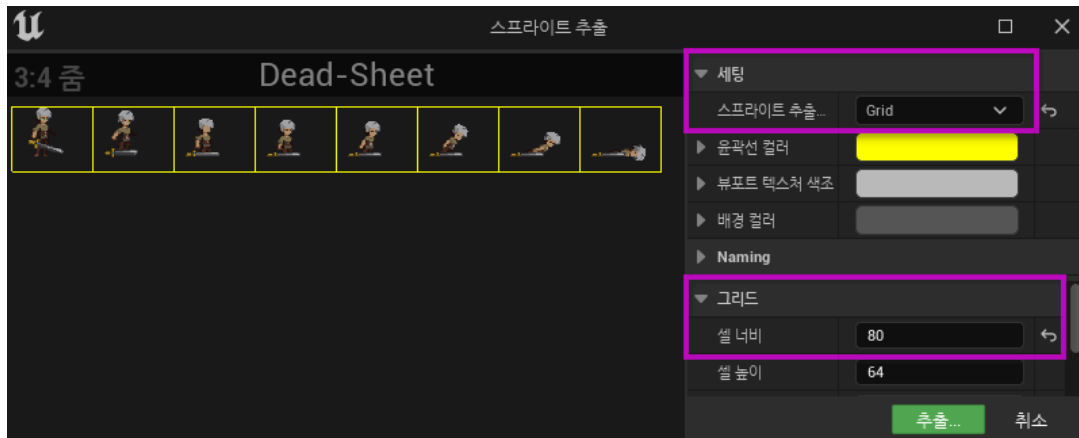
스프라이트 추출 화면에서 세팅 ⇒ 스프라이트 추출 Grid 선택

셀 너비를 attack 숫자로 나눔. 추출 진행

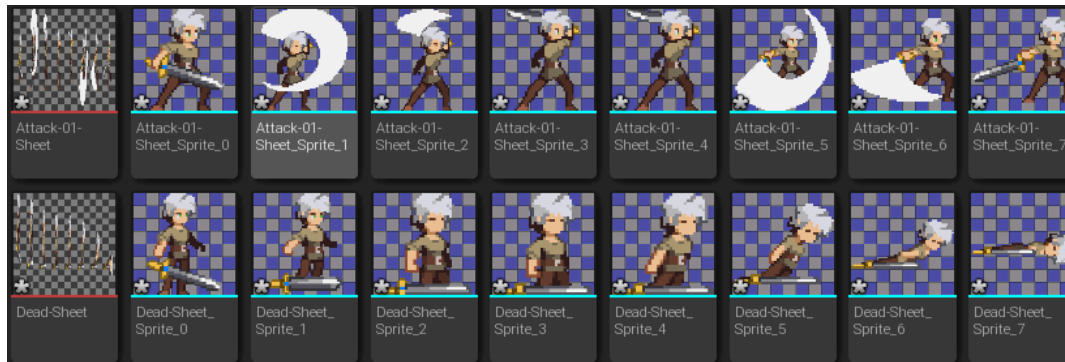


스프라이트 추출 화면에서 세팅 ⇒ 스프라이트 추출 Grid 선택

셀 너비를 dead 숫자로 나눔. 추출 진행



스프라이트 추출 결과를 확인



5.2.8.3 Hero2D 플립북 Attack 생성

attack에 해당하는 스프라이트를 선택 ⇒ 우 버튼 ⇒ 플립북 생성

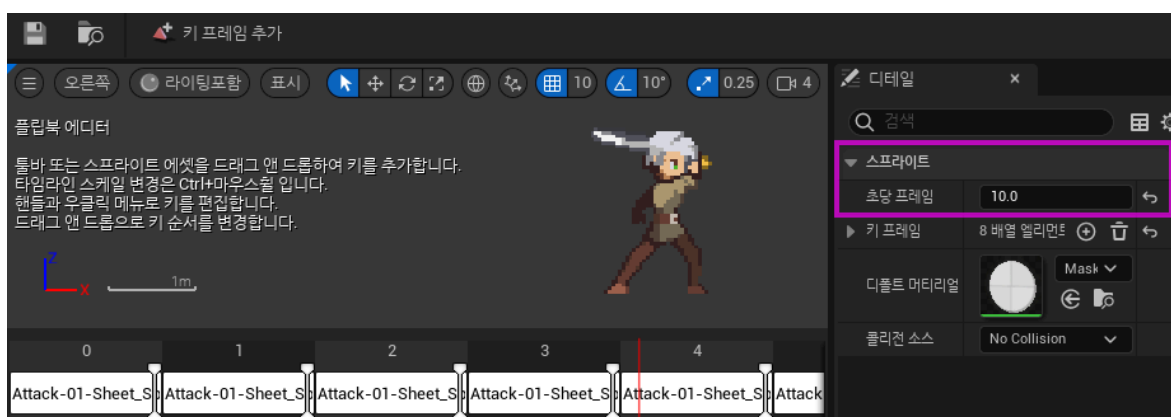


이름을 FB_Hero1_Attack으로 변경



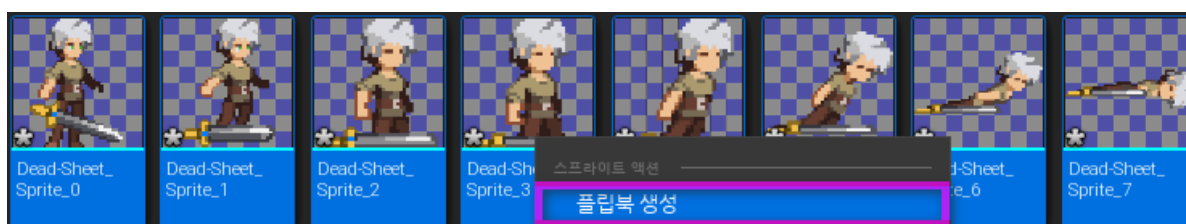
애니메이션 속도 조정

디테일 패널 ⇒ 스프라이트 ⇒ 초당 프레임
15⇒ 7~10

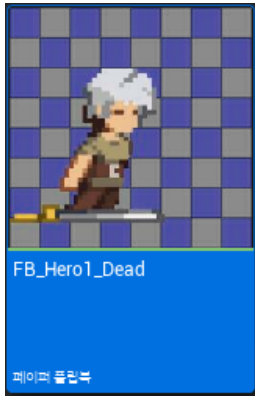


5.2.8.4 Hero2D 플립북 Dead 생성

Dead에 해당하는 스프라이트를 선택 ⇒ 우 버튼 ⇒ 플립북 생성



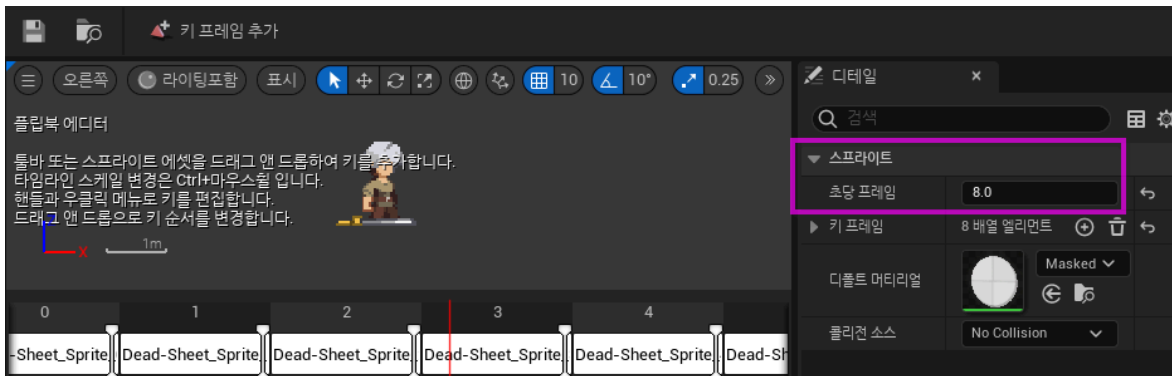
이름을 FB_Hero1_Dead으로 변경



애니메이션 속도 조정

디테일 패널 ⇒ 스프라이트 ⇒ 초당 프레임

15⇒ 7~10



5.3 PaperFlipbookActor

5.3.1 Paper2D 관련 Actor

엔리얼 액터: 레벨에 배치할 수 있는 오브젝트

액터를 만들어야 월드를 구성할 수 있음

Paper2D와 엔리얼 클래스 계층 구조

상속(Inheritance): 하위 클래스(자식 클래스)가 상위 클래스(부모 클래스)의 권한, 속성, 행위를 이어받아 사용할 수 있으며, 필요에 따라 새로운 기능을 추가하거나 기존 기능을 재정의(override) 할 수 있는 객체지향의 핵심 개념

상속 관계는 Is-A 로 표현 가능

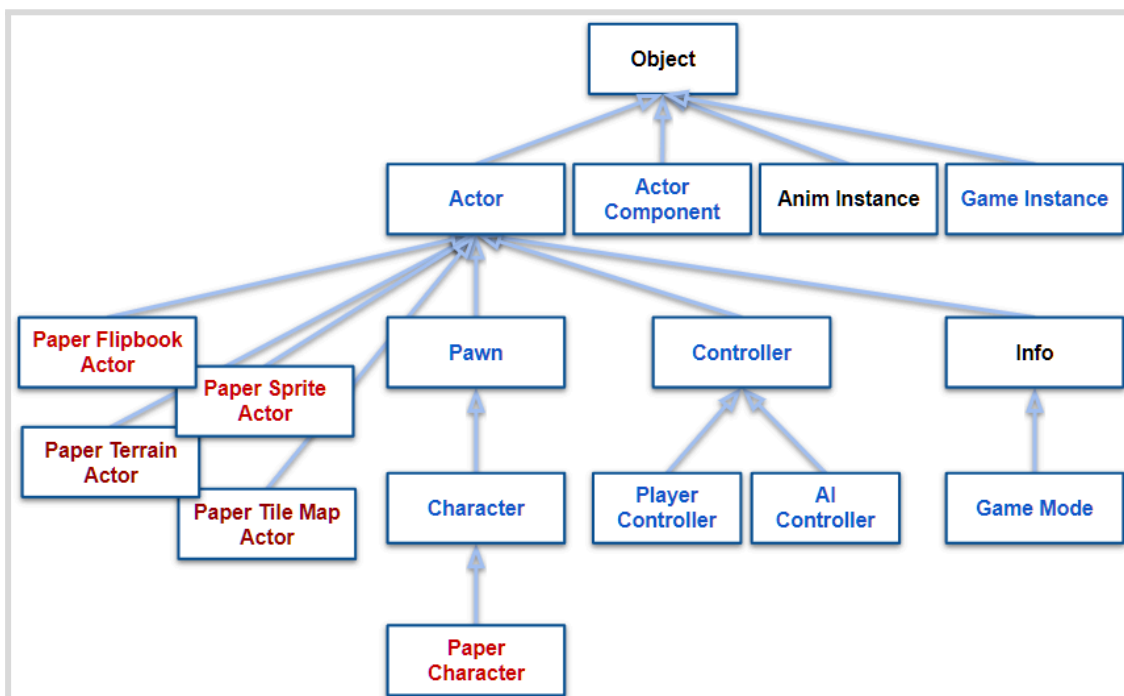
Paper Character is Actor.

PapperFlipbookActor is Actor.

Character is Actor.

엔리얼 클래스 계층 구조도

화살표 의존성



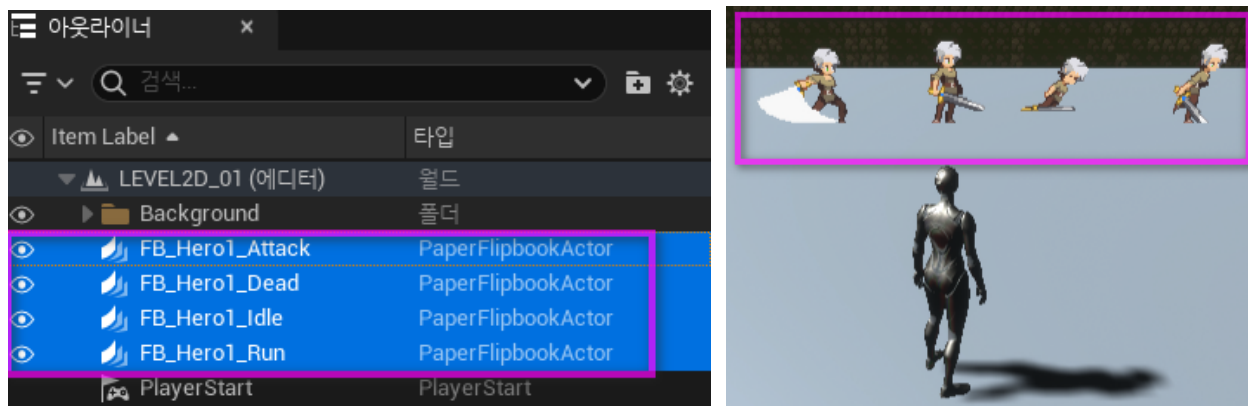
예를 들어, PaperCharacter 클래스는 Character 클래스의 모든 것을 사용 할 수 있으며, 속성과 행위를 추가해서 사용

2D게임에서 사용도가 높은 클래스

PaperCharacter, Actor, PaperFlipbookActor, PaperSpriteActor, Controller

5.3.2 PaperFlipbookActor

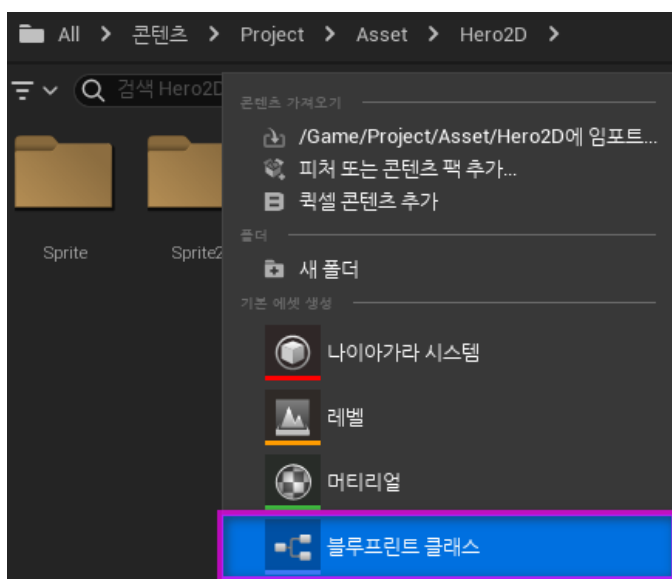
FB_Hero1_Attack, FB_Hero1_Idle, FB_Hero1_Dead, FB_Hero1_Run 을 선택해서 월드에 배치하면 자동으로 PaperFlipbookActor가 만들어진다.



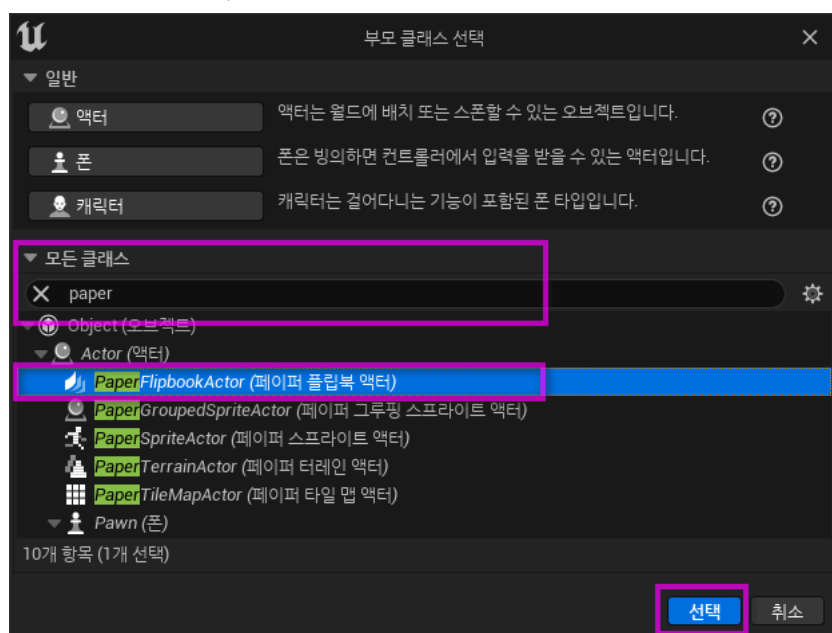
단순 연출이라면 블립북을 이렇게 월드에 배치해도 좋다.
 게임 시스템과 상호 작용하는 액터라면 프로그램이 필요하다.
 언리얼 엔진은 스크립트 기반의 블루프린트(Blueprint)와 컴파일 기반의 C++ 두 가지 프로그래밍 방식을 제공한다.
 블루프린트는 간단히 정의하면 언리얼에서 제공하는 노드 기반 비주얼 스크립팅이다.
 노드(Node)는 일종의 코드 덩치로, 블루프린트에서 특정 기능이나 연산을 수행하는 코드 단위(기능 블록)이다.
 C++로 작성하는 코드와 블루프린트로 작성하는 스크립트는 대부분 1:1로 대응되며,
 블루프린트는 내부적으로 C++ 클래스를 기반으로 동작한다.
 따라서 C++ 없이도 블루프린트만으로 완전한 게임을 제작하는 것이 가능하다.
 다만, 모든 C++ 기능이 블루프린트에서 노출되는 것은 아니므로,
 세밀한 성능 최적화나 저수준 시스템 제어는 C++로 작성해야 한다.
 일반적인 스크립팅 언어와 마찬가지로, 엔진 내 객체 지향형(OO) 클래스 또는 오브젝트를 정의하는 데 사용한다.

5.3.3 PaperFlipbookActor 생성

콘텐츠 브라우저에서 "/콘텐츠/Project/Asset/Hero2D"를 선택한다
 폴더의 빈 곳에 우 클릭 하거나 추가 버튼("+")를 누른다
 블루프린트 클래스 선택



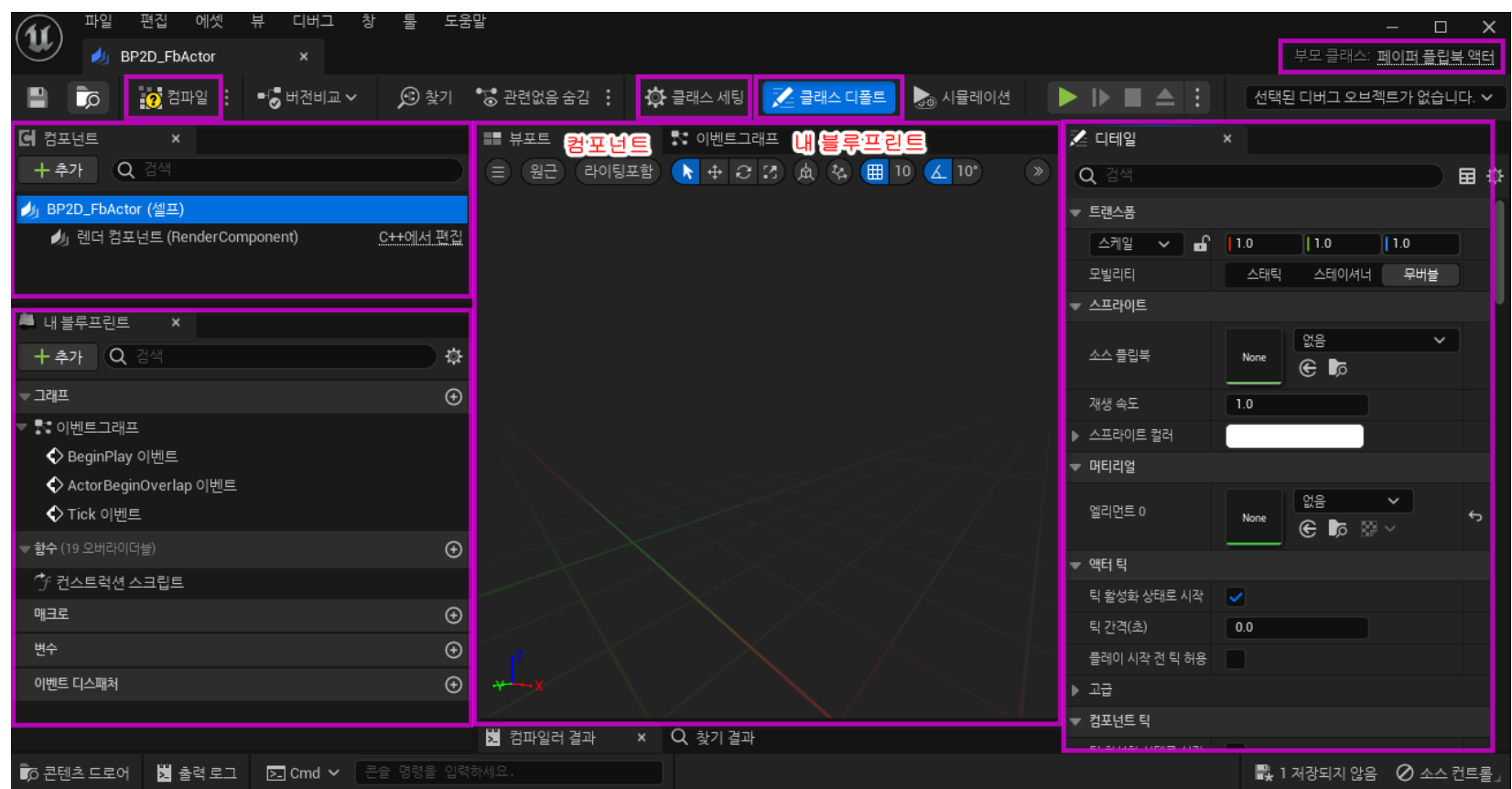
PaperFlipbookActor 선택
 모든 클래스 밑에 있는 검색에 paper 입력
 PaperFlipbookActor 선택



이름을 "BP2D_FbActor"로 설정

5.3.4 PaperFlipbookActor 편집 창

5.3.4.1 FlipbookActor 편집



부모 클래스

먼저 블루프린트 에디터 우상단의 ‘Parent Class(부모 클래스)’를 확인하여, 현재 블루프린트가 어떤 클래스의 자식인지를 파악한다.

툴바: 컴파일 (단축키 F7)

블루프린트나 컴포넌트의 값을 변경한 뒤에는 컴파일(Compile) 해야, 그 변경 사항이 월드(에디터 뷰포트 및 실행 환경)에 적용된다.

툴바: 클래스 디폴트

블루프린트 클래스의 기본 속성을 설정하는 메뉴로, 인스턴스를 만들지 않고도 클래스 전체의 초기값을 변경할 수 있다.

컴포넌트 패널(Components Panel)

블루프린트의 구성 요소를 관리하는 영역으로, 루트 컴포넌트를 중심으로 다양한 컴포넌트를 추가하여 액터의 구조를 정의한다.

충돌(콜리전), 스프라이트, 카메라, 라이트 등의 요소를 추가·삭제하거나 속성을 변경하여 액터의 형태와 동작을 구성할 수 있다.

컴포넌트 계층 구조에서 구성 요소를 클릭하면 뷰포트(Viewport)가 활성화되고, 디테일 패널(Details Panel)에서 해당 컴포넌트의 세부 속성 값을 확인하거나 수정할 수 있다.

내 블루프린트(My Blueprint)

컴포넌트 패널이 정적인 구성 요소를 다룬다면, 내 블루프린트(My Blueprint)는 블루프린트의 동적 동작을 제어하는 프로그래밍 영역이다.

이 패널에서는 변수(Variables), 함수(Functions), 이벤트(Event Graph), 매크로(Macros) 등의 요소를 한눈에 관리할 수 있다.

새로운 변수, 함수, 이벤트·매크로를 추가하거나 삭제할 수 있으며, 이미 생성된 요소를 더블 클릭하여 그래프 편집 창으로 바로 이동할 수도 있다.

디테일 패널

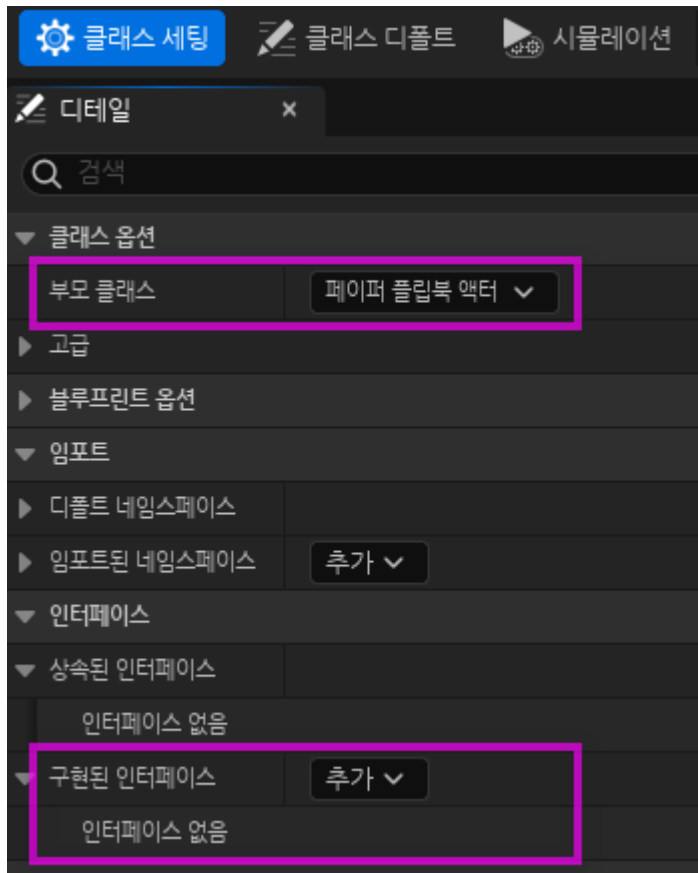
디테일 패널은 선택한 객체(액터, 컴포넌트, 블루프린트 등)의 세부 속성(Property)을 확인하고 수정할 수 있는 영역이다.

컴포넌트 패널이나 뷰포트에서 특정 요소를 클릭하면, 그 객체의 위치·회전·크기 같은 트랜스폼(Transform) 값과 머티리얼, 콜리전, 렌더링, 물리, 입력(Input) 등과 같은 세부 설정 항목이 이 패널에 표시된다.

디테일 패널은 객체의 속성을 직접 편집하는 창으로, 에디터 내에서 대부분의 설정 변경이 이곳에서 이루어진다.

툴바: 클래스 세팅

클래스 세팅 패널



부모 클래스

현재 블루프린트가 어떤 클래스를 상속받는지 결정하며 부모 클래스 변경가능

ex) Actor → PaperFlipbookActor

인터페이스 추가

Interface는 여러 블루프린트 간에 공통 함수 호출이나 메시지 전달이 가능

(예: BPI_DamageReceiver 인터페이스를 추가하면, 다른 액터가 "피해를 받는 기능"을 메시지 형태로 호출 가능)

5.3.4.2 Flipbook 추가하기

무버블(Movable) 설정 확인

컴포넌트 패널(Components Panel) 에서 최상단 루트 컴포넌트(Self) 더블 클릭

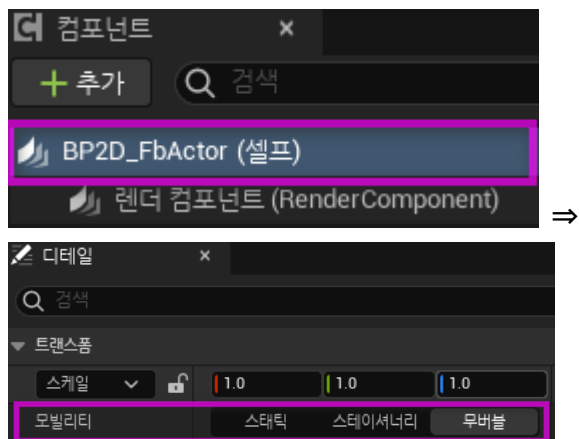
디테일 패널(Details Panel) 의 Transform 또는 Mobility(모빌리티) 항목을 무버블(Movable)로 변경

참고: Mobility 옵션

Static : 완전히 고정 (이동 불가)

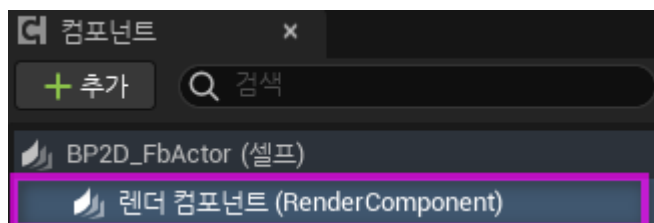
Stationary : 일부 이동 가능 (라이트 등)

Movable : 완전 이동 가능

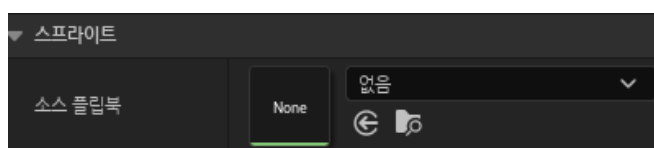


Flipbook 추가

렌더 컴포넌트 더블 클릭

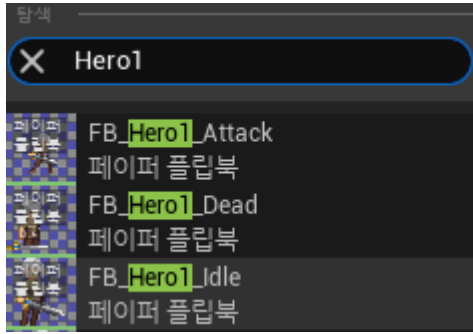


스프라이트 ⇒ 소스 플립북 ⇒ 없음 영역 클릭

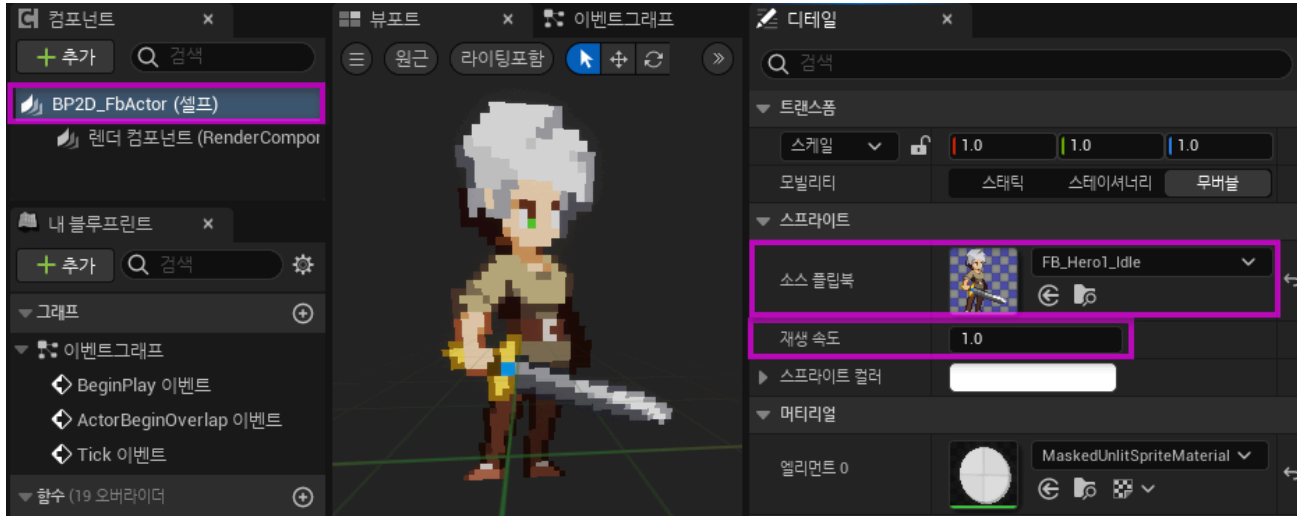


에셋 검색창을 클릭하거나 필요한 에셋의 이름으로 검색

FB_Hero1_Idle을 선택해보자

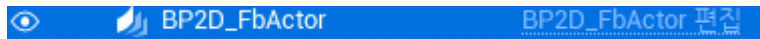


5.3.4.3 Flipbook 편집 완료



컴포넌트 패널에서 BP2D_FbActor (Self)를 선택하면, 디테일 패널에서 해당 플립북(FB_Hero1_Idle)의 속성을 확인할 수 있다. Mobility(무버블) 속성을 통해 객체의 이동 가능 여부를 설정할 수 있다. 소스 플립북(Source Flipbook) 과 재생 속도(Play Rate) 는 블루프린트에서 애니메이션 제어나 상태 전환 시 가장 자주 사용되는 요소이다. 디테일 패널의 대부분의 요소는 블루프린트 스크립팅을 통해 제어 가능하다. 툴바의 컴파일 버튼 또는 F7 컴파일 단축키로 컴파일하고 저장

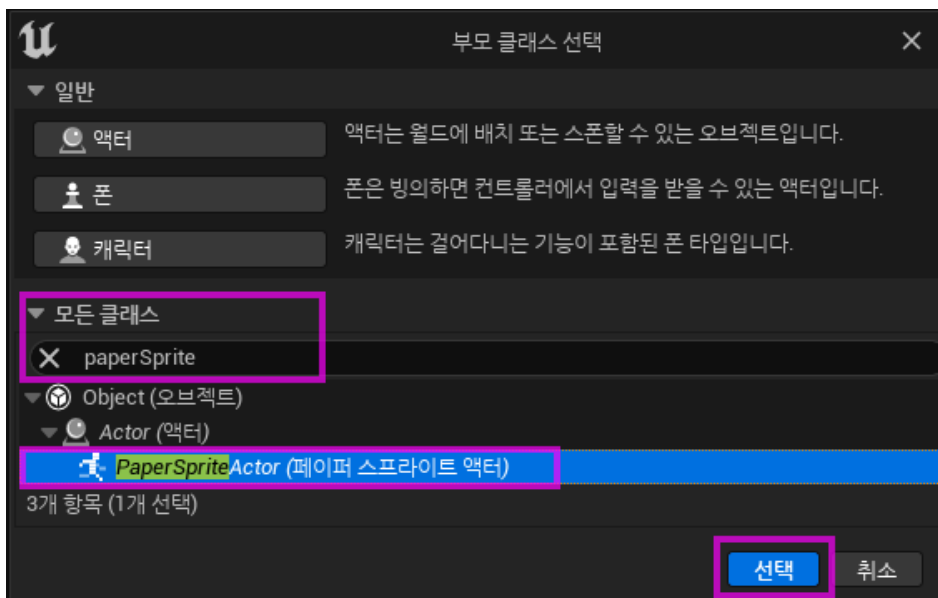
5.3.4.4 블루프린트 Flipbook 월드 배치

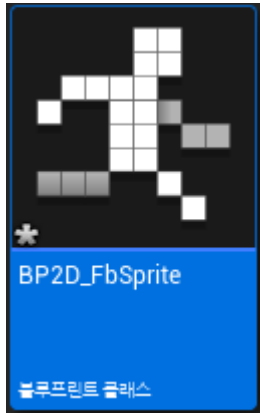


레벨 에디터(Level Editor) 로 돌아와서, 블루프린트로 작성한 플립북 액터(Blueprint Flipbook Actor) 를 월드에 배치한다. 아웃라이너(Outliner) 패널에서 새로 배치된 액터를 확인하면, 이름이 "블루프린트이름 (편집)" 형태로 표시되어 있는 것을 볼 수 있다. "블루프린트이름 (편집)"을 클릭하면 해당 블루프린트 편집창(Blueprint Editor) 이 열린다.

5.4 PaperSpriteActor

PaperFlipbookActor 보다 사용 빈도는 낮지만 간단하게 만들 수 있어서 정적인 2D를 표현하는데 간혹 쓰인다. 콘텐츠 브라우저(Content Browser) 에서 "/콘텐츠/Project/Asset/Hero2D" 폴더를 선택한다. 폴더의 빈 공간을 우클릭 → 블루프린트 클래스(Blueprint Class) 를 선택한다. 부모 클래스 선택 창에서 상단의 '모든 클래스(All Classes)' 를 클릭한 후 검색창에 paperSprite를 입력하고, PaperSpriteActor 클래스를 선택한다. 새 블루프린트 이름을 BP2D_FbSprite 로 지정하고 저장한다.

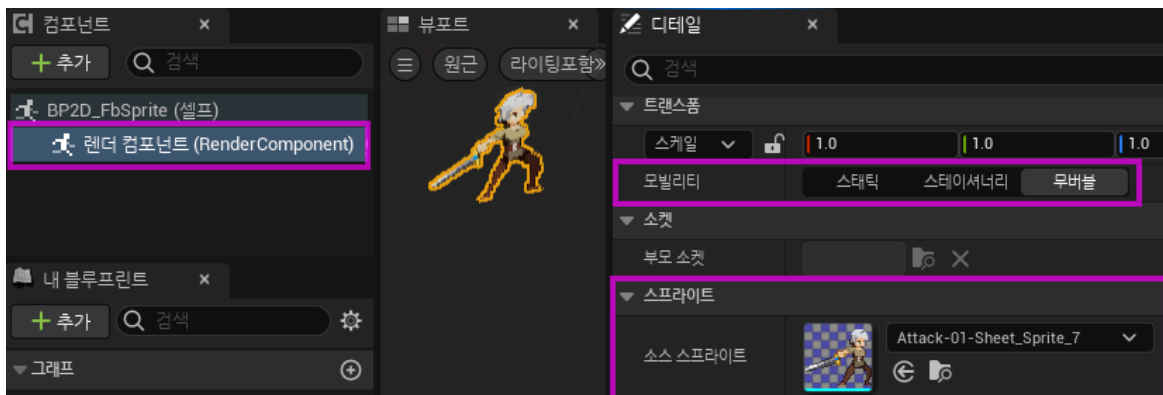




5.4.1 PaperSpriteActor 설정

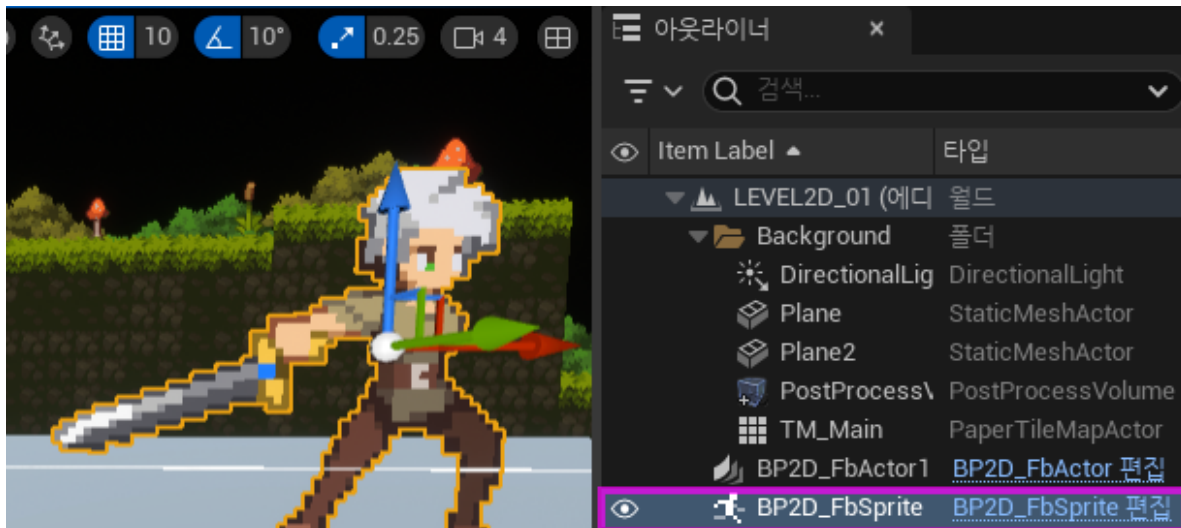
5.4.1.1 모빌리티 무버블 변경

PaperSpriteActor 모빌리티 디폴트는 스테틱이다. 앞서 언급했듯이 스테틱이면 블루프린트 스크립트가 적용이 안된다. 무버블로 변경



5.4.1.2 스프라이트 변경

디테일 패널 ⇒ 스프라이트 ⇒ 소스 스프라이트 콤보 박스 클릭 원하는 스프라이트 선택
컴파일 ⇒ 저장
다음은 테스트로 월드에 배치한 화면이다

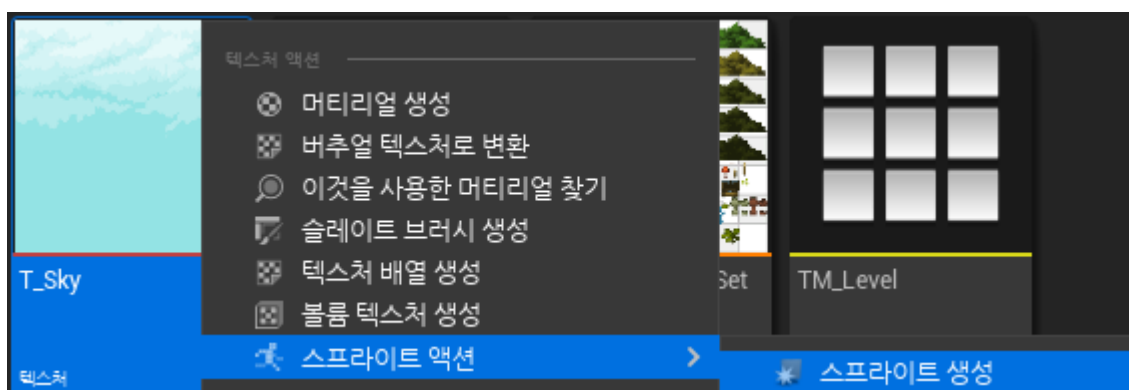


5.4.2 PaperSpriteActor 배경 생성

복습하는 의미로 배경 스프라이트를 생성해 본다

5.4.2.1 배경 스프라이트 생성

콘텐츠/Project/Asset/LegacyFantasyForest/Environment 이동
T_Sky 텍스처 선택 ⇒ 우 클릭 ⇒ 스프라이트 액션 ⇒ 스프라이트 생성



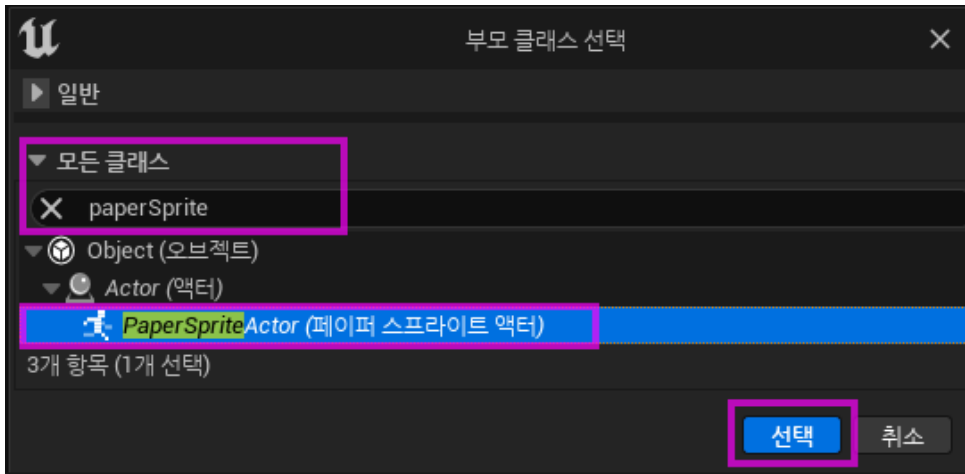
T_Sky_Sprite로 이름을 변경



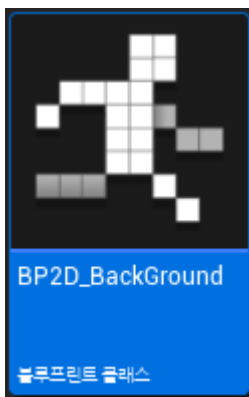
5.4.2.2 배경 PaperSpriteActor 생성

콘텐츠/Project/Asset/LegacyFantasyForest 로 이동

폴더 빈 곳에 우 클릭 ⇒ 블루프린트 클래스 ⇒ 부모 클래스 선택 ⇒ 모든 클래스 ⇒ paperSprite 입력 ⇒ PaperSpriteActor 선택



생성된 블루프린트 클래스 이름을 BP2D_BackGround 로 변경



5.4.2.3 배경 PaperSpriteActor 편집

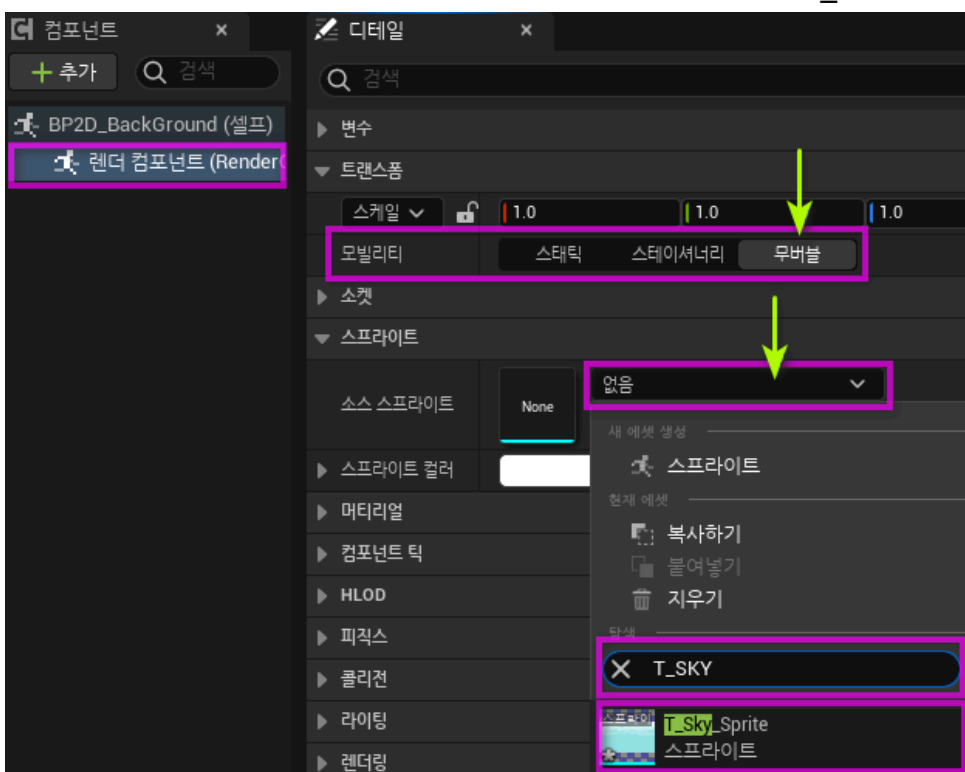
BP2D_BackGround 더블 클릭 ⇒ 에디터 오픈

컴포넌트 패널 ⇒ 렌더 컴포넌트 선택

디테일 패널

모빌리티 무버블 변경

스프라이트 ⇒ 소스 스프라이트 ⇒ 콤보 박스 클릭 ⇒ 검색창 T_SKY 입력 ⇒ T_Sky_Sprite 선택



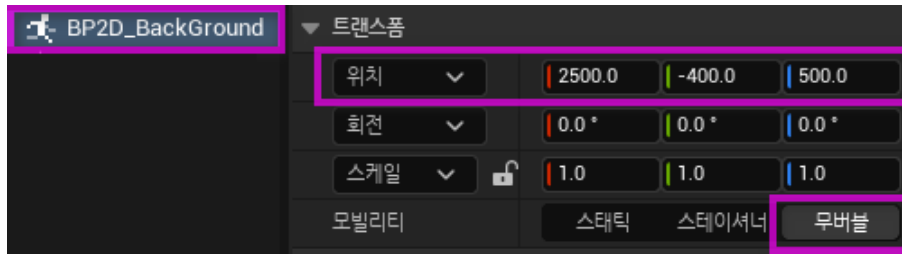
뷰포트에서 스프라이트가 보이는지 확인.

툴바 ⇒ 컴파일 또는 F7 키로 컴파일 실행



5.4.2.4 배경 PaperSpriteActor 편집

월드 에 배치
아웃라이너에서 BP2D_BackGround 선택
무버블 확인
트랜스폼 위치 수정
적당한 값을 입력

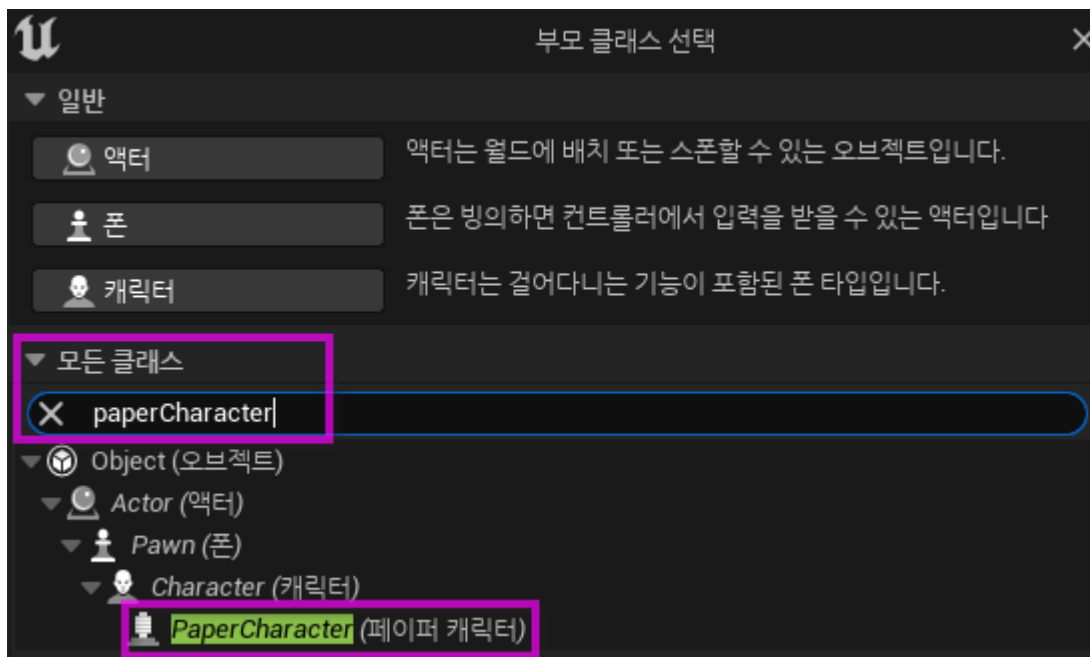


무버블인지 확인
실행 결과



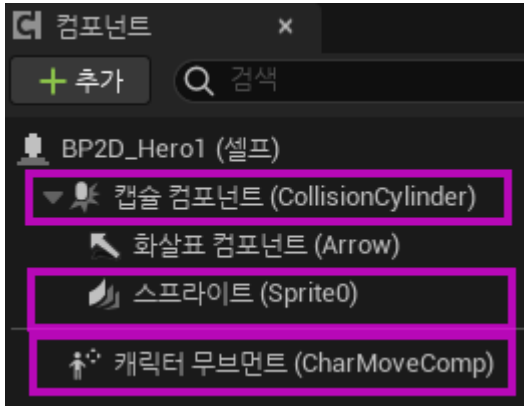
5.5 PaperCharacter 구성

콘텐츠 브라우저(Content Browser) 에서 "/콘텐츠/Project/Asset/Hero2D" 폴더를 선택한다.
폴더의 빈 공간을 우클릭 → 블루프린트 클래스(Blueprint Class) 를 선택한다.
부모 클래스 선택 창에서 상단의 '모든 클래스(All Classes)' 를 클릭한 후 검색창에 paperCharacter를 입력하고, PaperCharacter 클래스를 선택한다.
새 블루프린트 이름을 BP2D_Hero1 로 지정하고 저장한다.



5.5.1 PaperCharacter 컴포넌트 구성

기본 PaperCharacter 컴포넌트



캡슐 컴포넌트 (Capsule Component)

캡슐 형태의 충돌(콜리전) 및 물리적 영역 정의

스프라이트 컴포넌트에서도 충돌을 처리할 수 있지만, 대부분의 프로젝트에서는 스프라이트 충돌을 비활성화하고 캡슐 컴포넌트만 충돌 처리용으로 사용함

기능: 벽 충돌, 지면 감지, 점프 착지 등 캐릭터의 물리적 중심

화살표 컴포넌트 (Arrow Component)

액터의 전방(Forward Direction)을 시각적으로 표시하는 에디터 전용 시각 가이드(Visual Gizmo)

게임 실행 중에는 숨김(bHiddenInGame = true)

기본적으로 +X 축을 전방으로 가리킴

캐릭터 방향, 투사체 발사 방향, 시선 방향 확인 등에 활용

스프라이트(Sprite0) (Paper Flipbook Component)

캐릭터의 시각적 표현 (애니메이션, 스프라이트)

주요 함수: SetRelativeRotation(), SetFlipbook(), Play(), PlayFromStart(), Stop()

좌표 기준: X축: 전방, Y축: 깊이 방향, Z축: 회전 기준 축 (2D 회전 시 주로 사용)

충돌보다는 시각적 표현에 초점, 충돌은 Capsule이 담당

캐릭터 무브먼트 컴포넌트 (Character Movement Component)

캐릭터의 실제 물리적 동작 처리(중력, 마찰, 점프, 낙하, 걷기, 이동 속도 등)

PaperCharacter는 3D 캐릭터 무브먼트를 그대로 사용하지만 Y축을 고정하여 2D 이동 전용으로 사용함

내부 동작: PhysWalking, PhysFalling 등 상태 기반 물리 처리

주요 프로퍼티: MaxWalkSpeed, JumpZVelocity, AirControl, GravityScale

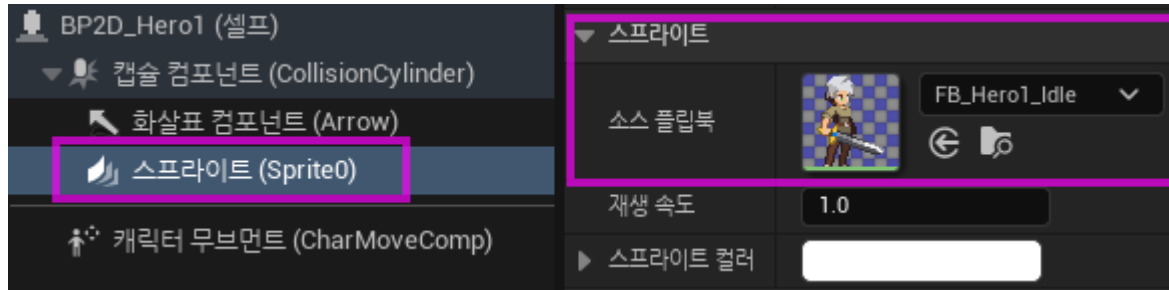
주요 함수: AddMovementInput(), LaunchCharacter(), Jump(), StopJumping()

캡슐 컴포넌트와 캐릭터 무브먼트 구성은 이후 이동과 충돌에서 구현하기로 하고, 지금은 스프라이트를 구성해본다
스프라이트(Sprite0)를 더블 클릭하면

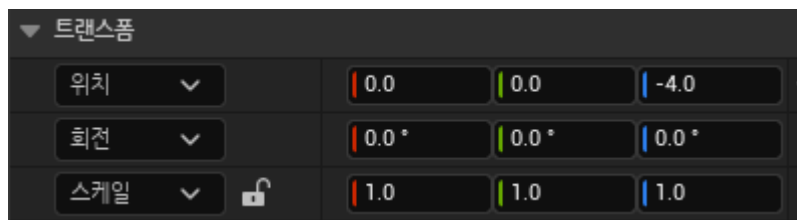
5.5.1.1 스프라이트(PaperFlipbookComponent) 구성

PaperFlipbookActor와 유사

컴포넌트 패널의 스프라이트 더블 클릭 ⇒ 디테일 패널의 스프라이트 ⇒ 소스 플립북 ⇒ Fb_Hero1_Idle 설정

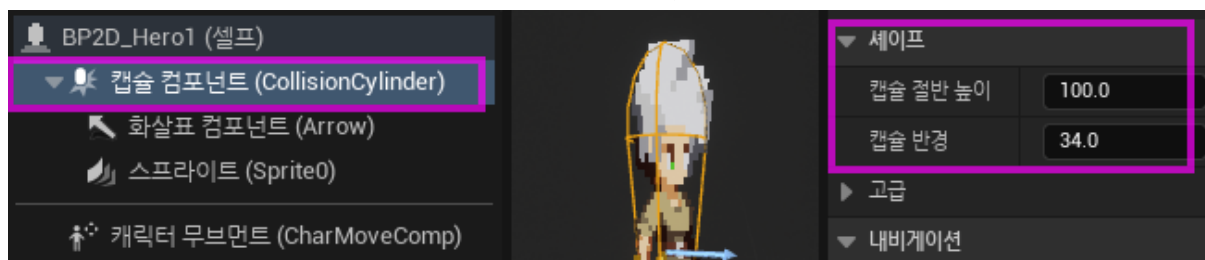


캐릭터의 발이 살짝 물히도록 위치 z 값을 -4.0으로 조정



5.5.1.2 캡슐 컴포넌트 (CollisionCylinder) 구성

뷰포트를 보면서 디테일 패널 ⇒ 세이프 ⇒ 캡슐 절반 높이, 캡슐 반경 조정



5.5.1.3 카메라 구성

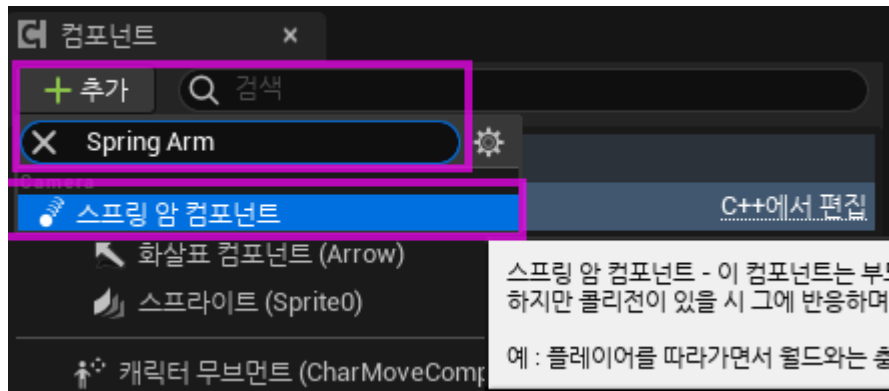
ThirdPerson 캐릭터처럼 카메라가 Main Player를 쫓아다니는 카메라를 구성하자.

이를 구현하기 위해서 Spring Arm 과 카메라가 필요하다.

먼저 Spring Arm을 추가하자

1) Sprint Arm Component 추가

컴포넌트 패널 ⇒ 추가("+") 버튼 ⇒ Spring Arm 입력



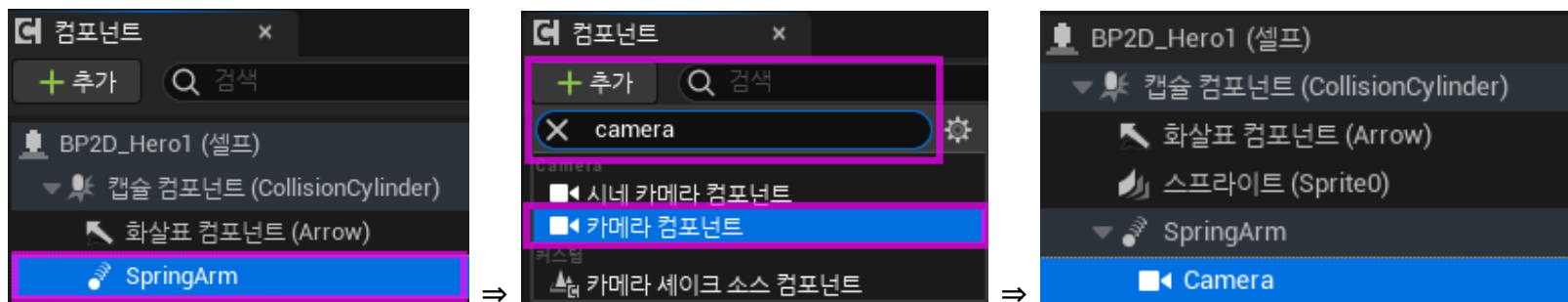
2) 카메라 추가

SpringArm 하위에 카메라를 부착할 것이다.

반드시 SpringArm 선택한 후에 진행해야 카메라가 SpringArm 하위에 붙는다

SpringArm 선택한 후 ⇒ 추가("+") 버튼 ⇒ camera

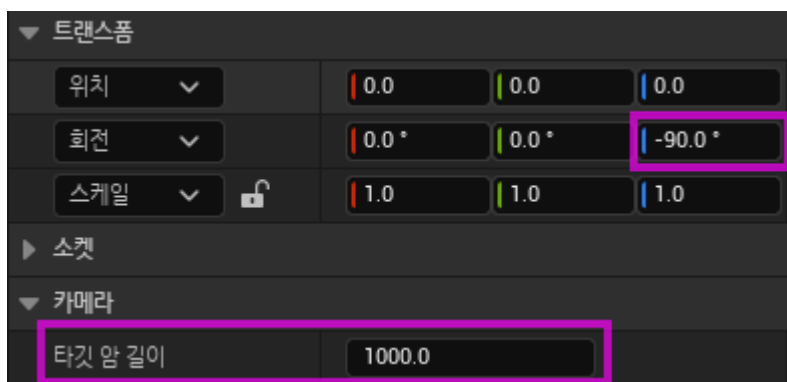
주의! 한글 카메라로 입력하면 CineCamera가 붙는다. 반드시 영어로



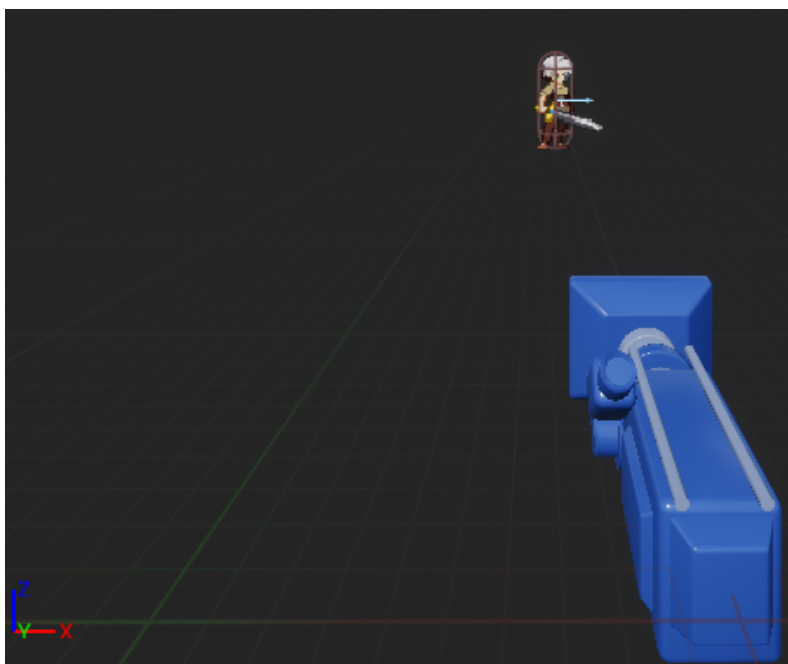
SpringArm 수정

SpringArm을 클릭하고 트랜스폼 회전의 Z 에 -90도 입력

카메라 타겟 암길이 1000 으로 설정



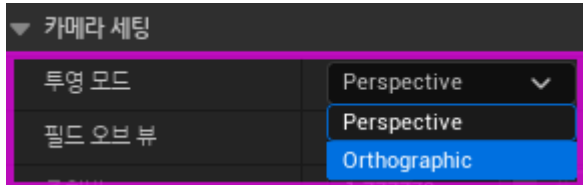
루크 컴포넌트를 클릭하고 뷰포트를 보면 카메라가 Y으로 캐릭터를 바라보고 있음을 알 수 있다.



Camera 투영 모드 변경

SpringArm 하위에 있는 카메 선택

투영 모드 를 Perspective ⇒ Orthographic 으로 변경

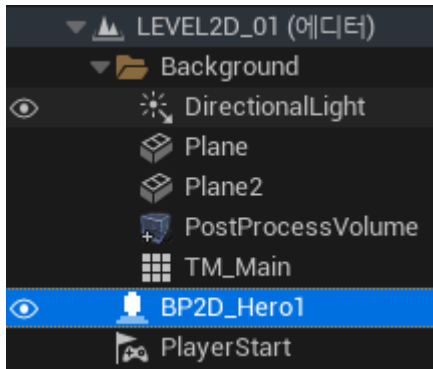


Perspective 소실점이 있어 원근감을 표시

Orthographic(직교 투영): 소실점이 없어서, 카메라와의 거리(z)에 무관하게 물체 크기가 변하지 않고 평면적으로 표현

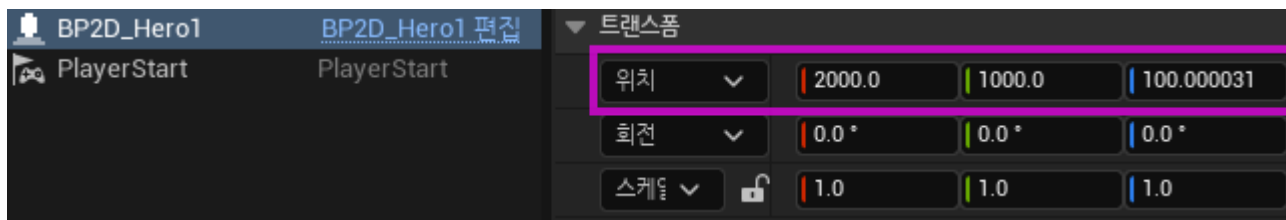
5.5.2 PaperCharacter 월드 적용

BP2D_Hero1을 월드에 배치하고 테스트로 진행했던 PaperFlipbookActor PaperSpriteActor 는 전부 삭제



BP2D_Hero1

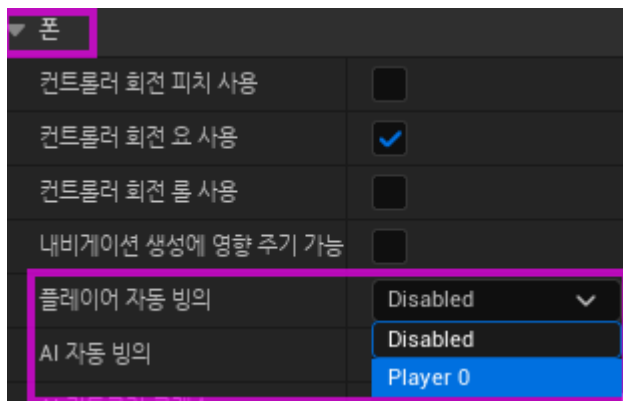
위치: 2000, 1000, 100 으로 조정



폰 (Pawn)

트랜스폼을 설정하고 스크롤 하다보면 거의 끝 부분에 폰 (Pawn) 있음

플레이어 자동 빙의 Disabled ⇒ Player 0으로 변경. 이렇게 하면 GameMode의 디폴트 폰 클래스를 변경 안하고 테스트 할 수 있음. 테스트가 완료되면 다시 Disabled로 변경



실행 결과

Play 하면 BP_ThirdPerson 대신 카메라가 BP2D_Hero1를 보고 있음

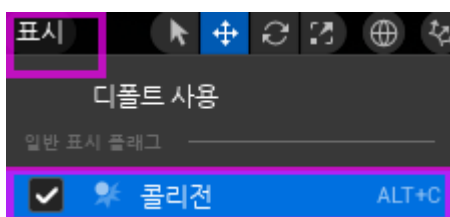


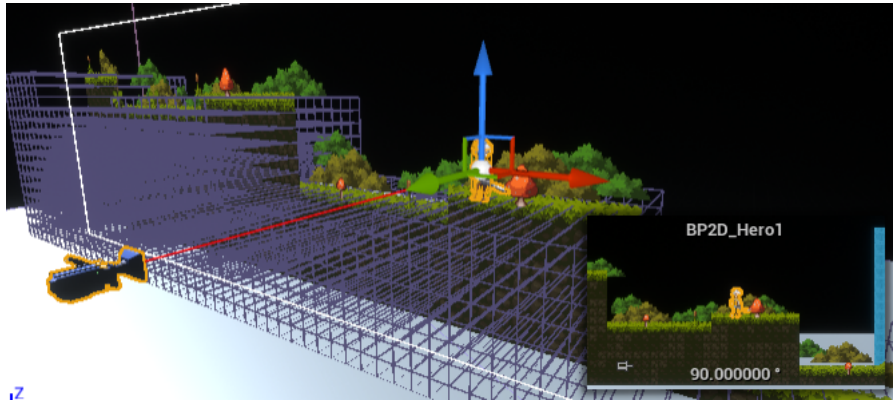
타일맵에 적용

타일 맵의 적당한 위치에 올려 놓는다

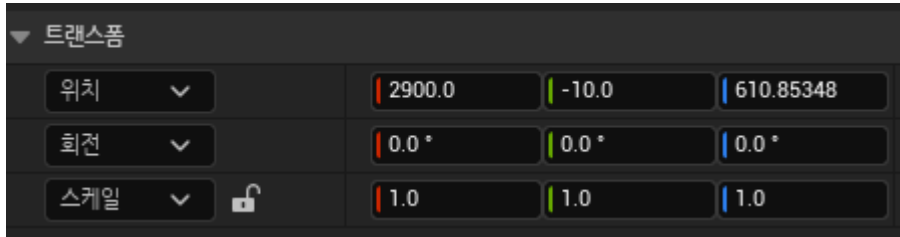
뷰포트 ⇒ 표시 ⇒ 콜리전을 체크 하거나 ALT+C로 콜리전을 활성화 해서 화면의 콜리전 영역을 보이게 설정

캐릭터의 트랜스폼을 조정해서 2D 맵의 적당한 위치를 정함





화면에 보이는 트랜스폼. 이 값은 각자의 맵에 맞게 조정한다



타일 맵 위에 올려진 Hero1



06. Paper2DCharacter 애니메이션

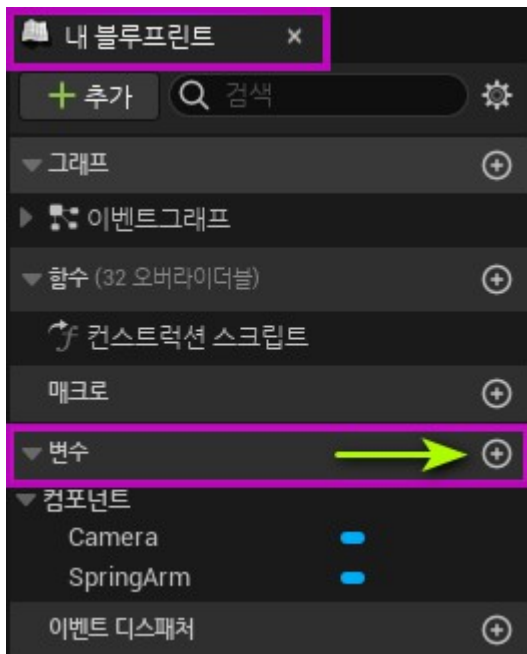
6.1 애니메이션

6.1.1 Idle 애니메이션

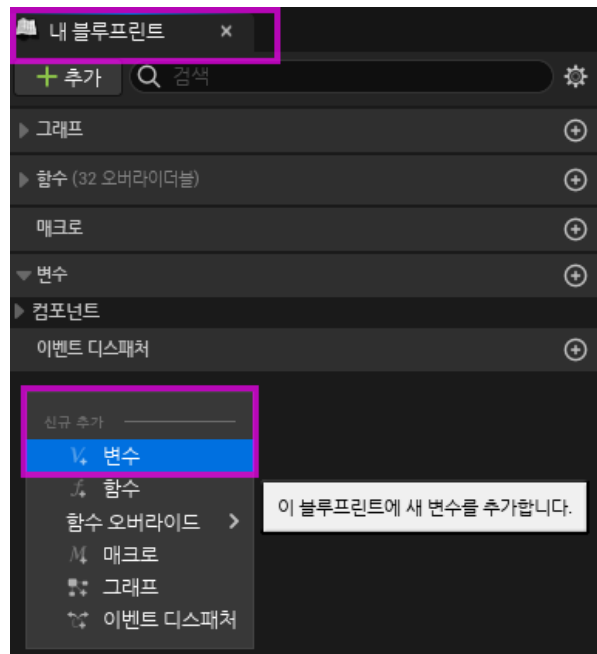
PaperFlipbookActor의 애니메이션은 스프라이트 컴포넌트의 소스 플립북을 교체하는 것이다. 이 교체를 지원하는 함수가 **SetFlipbook()** 이다. 교체 과정은 먼저 앞 과정에서 제작한 Hero의 Idle, Run JumpRise, JumpLand, Attack 동작에 해당하는 변수를 설정한다. 이 변수들을 이동이나 공격 이벤트에서 교체한다

먼저 애니메이션을 지정하는 플립북 오브젝트 레퍼런스를 추가하자

내 블루프린트 ⇒ 변수 ⇒ 추가("+") 버튼을 클릭 또는 내 블루프린트 ⇒ 빈공간 우 클릭



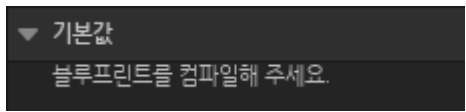
또는



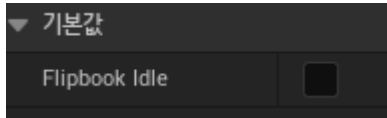
변수의 이름을 flipbookIdle로 정한다



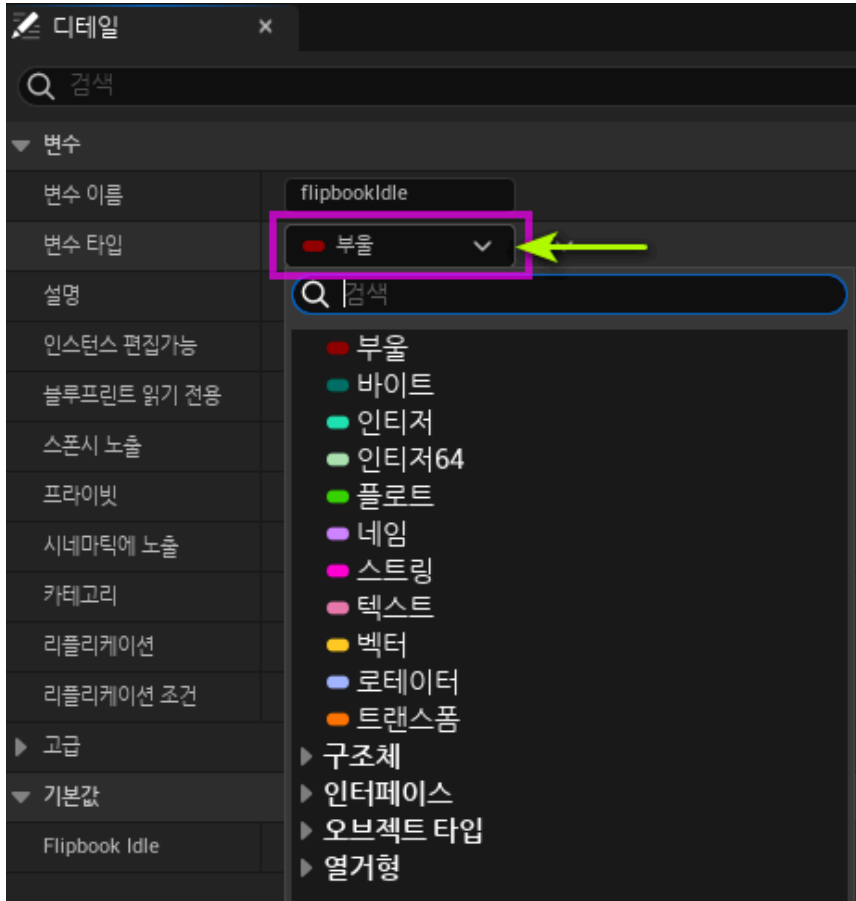
변수 타입 변경은 디테일 패널에서 하는 것이 좋다.
생성한 **flipbookIdle**을 선택하면 디테일 패널에 변수의 속성을 설정할 수 있는 메뉴들이 보인다.
속성의 마지막 기본값을 확장하면 플루프린트를 컴파일 해주세요라는 문구가 나온다. 기본값은 컴파일을 해야 설정할 있는 것이다.



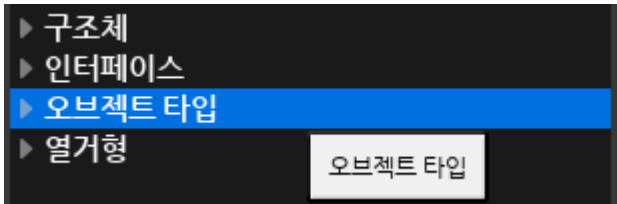
F7 키로 컴파일을 진행한다
컴파일이 완료되면 기본 값을 설정할 수 있는 항목이 보인다.



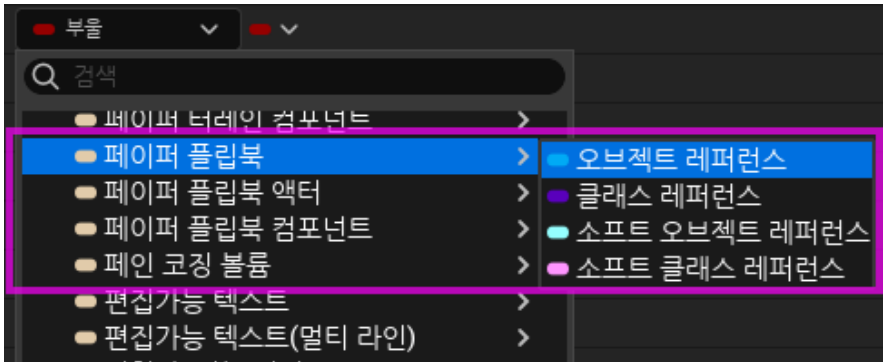
flipbookIdle 변수는 플립북 레퍼런스를 저장할 변수다. 변수 타입을 플립북 레퍼런스로 변경한다



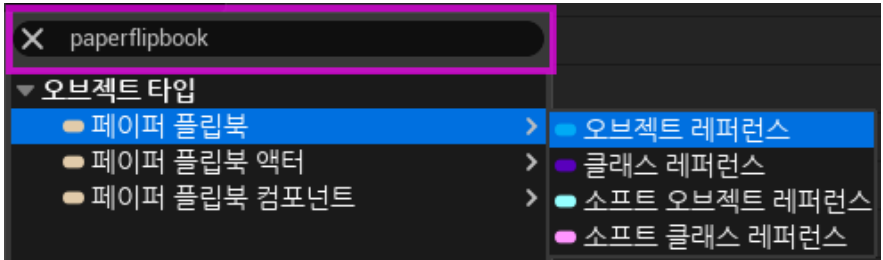
변수 타입의 콤보 박스를 클릭해서 오브젝트 타입을 선택



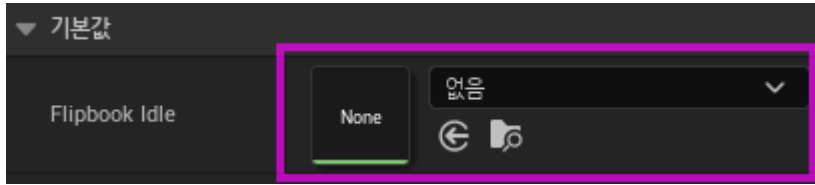
리스트에서 페이퍼 플립북을 찾는다
페이퍼 플립북 오브젝트 레퍼런스를 찾는다



검색창에서 paperflipBook을 입력해서 쉽게 찾을 수 있음

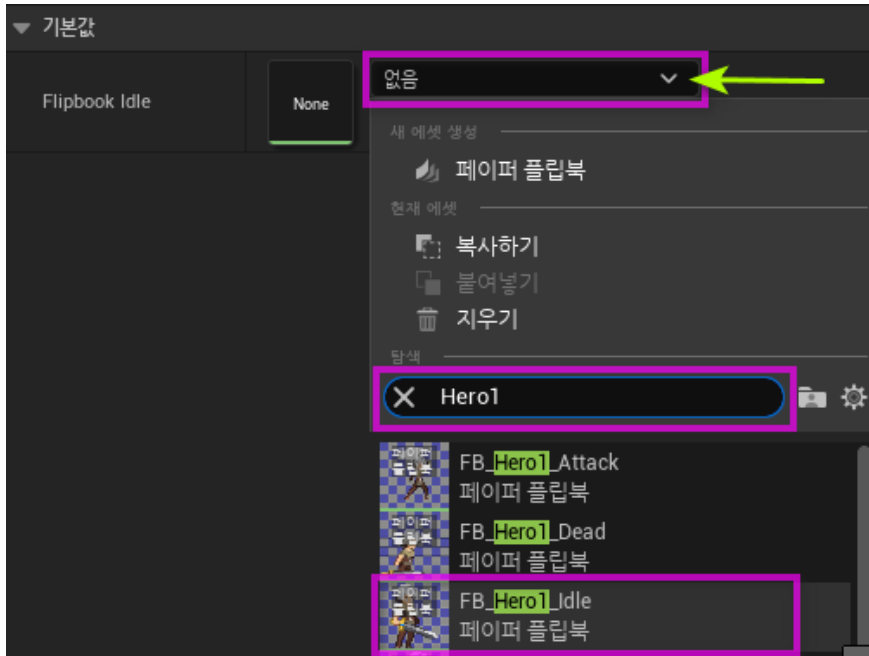


컴파일을 해야 기본 값을 정할 수 있음. F7 키를 눌러 컴파일을 실행한다.
기본값 표시



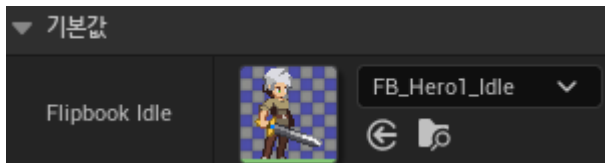
기본 값 에셋 페이퍼 플립북 설정

기본 값 설정 ⇒ 에셋 선택 상자 클릭 ⇒ 플립북 검색창에 Hero1 입력 ⇒ FB_Hero1_Idle 선택



F7 키를 눌러 컴파일을 실행한다.

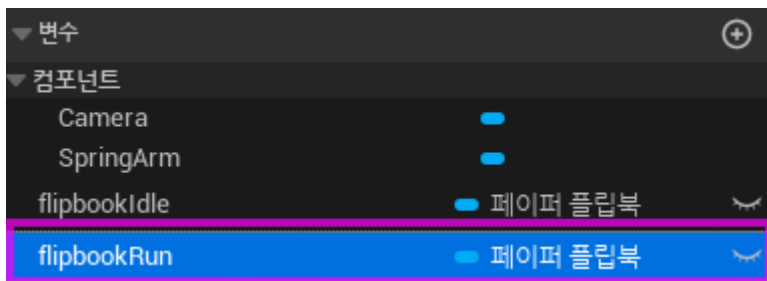
기본 값을 확인



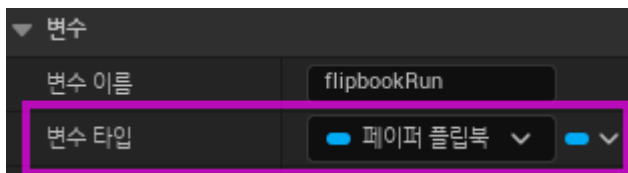
6.1.2 Run 애니메이션

Idle 애니메이션 설정과 같은 과정을 반복한다.

변수에 flipbookRun 변수를 추가



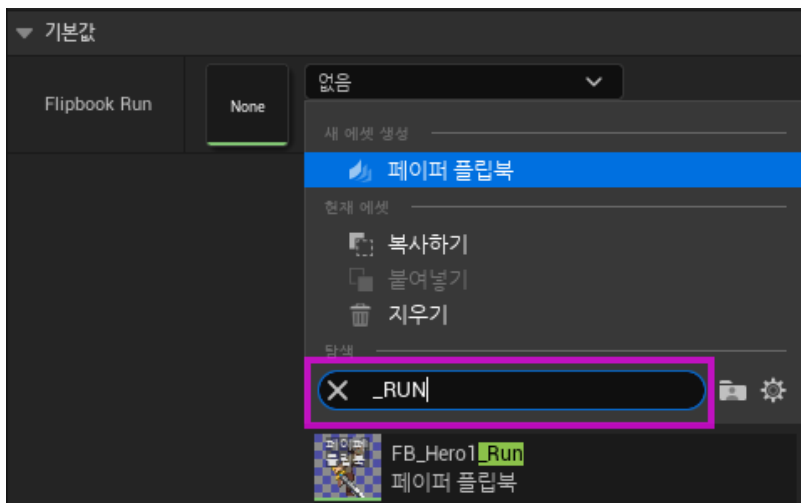
디테일 패널에서 변수 타입을 "페이퍼 플립북" "오브젝트 레퍼런스"로 변경



F7 키를 눌러 컴파일을 실행한다.

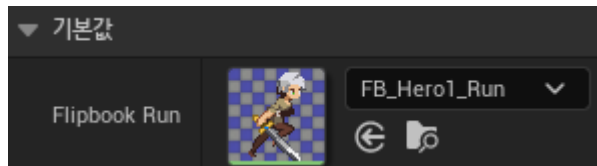
기본 값 에셋 페이퍼 플립북 설정

기본 값 설정 ⇒ 에셋 선택 상자 클릭 ⇒ 플립북 검색창에 _Run 입력 ⇒ FB_Hero1_Run 선택



F7 키를 눌러 컴파일을 실행한다.

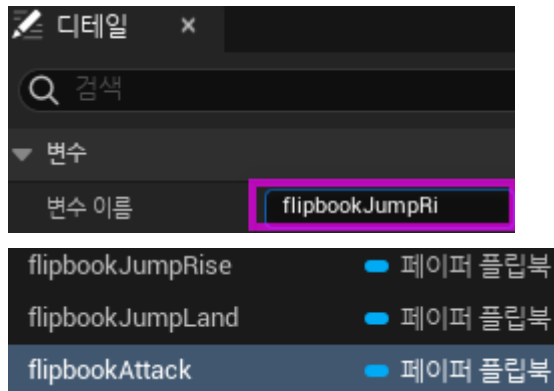
기본 값을 확인 한다.



6.1.3 Jump, Attack 애니메이션

JumpRise, JumpLand, Attack 3개의 변수를 한 번에 생성하고 진행하면 작업 속도가 빠르다.

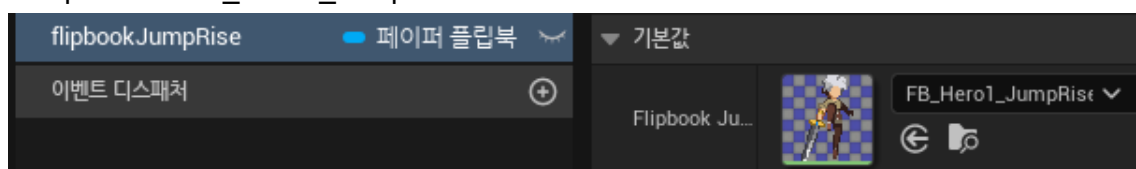
변수 "+" 버튼을 3번 눌러서 3개의 변수를 먼저 생성하고 각각 flipbookJumpRise, flipbookJumpLand, flipbookJumpAttack 으로 정한다. 아니면 flipbookRun 변수를 복사에서 붙여 넣기로 변수를 먼저 만들고 진행해도 된다.



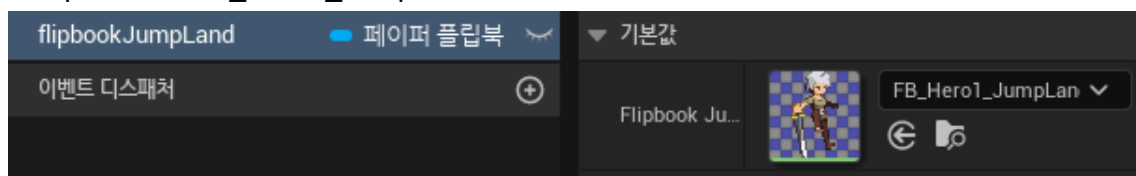
F7 키를 눌러 컴파일을 실행해서 기본 값 설정이 표시되도록 한다



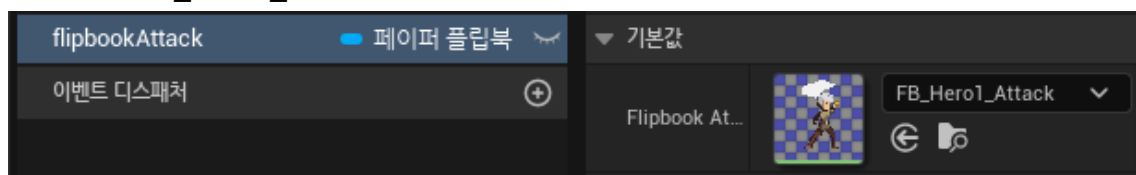
flipbookJumpRise 에 FB_Hero1_JumpRise 설정



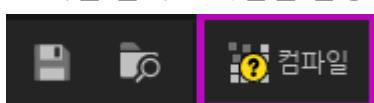
flipbookJumpLand 에 FB_Hero1_JumpLand 설정



flipbookAttack 에 FB_Hero1_Attack 설정



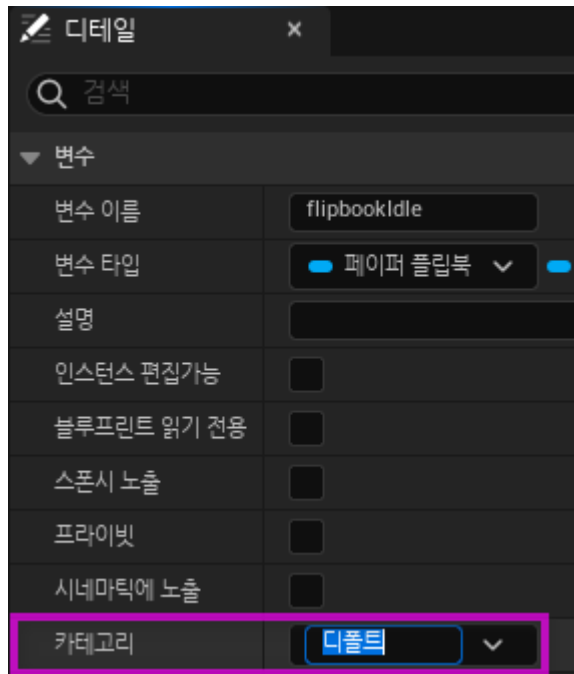
F7 키를 눌러 컴파일을 실행한다.



6.1.4 카테고리 설정

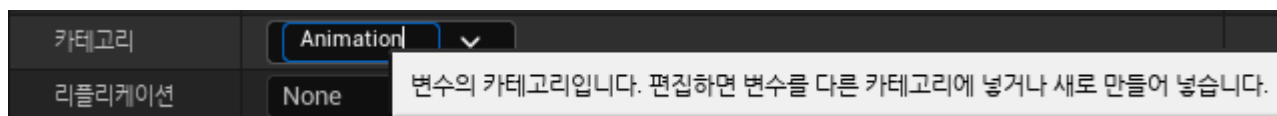
카테고리를 설정하면 디테일의 검색창에서 찾기가 쉬워진다.

변수를 추가하면 카테고리가 "디폴트" 로 정해진다. 이 값을 변경하자.

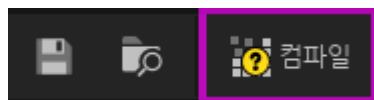


디폴트 ⇒ Animation으로 변경

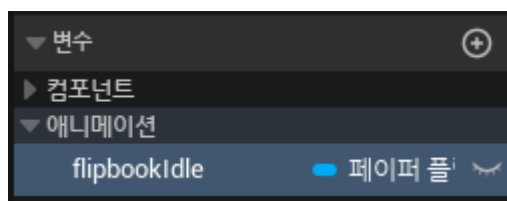
Animation 영문으로 입력해도 한글로 변경이 됨



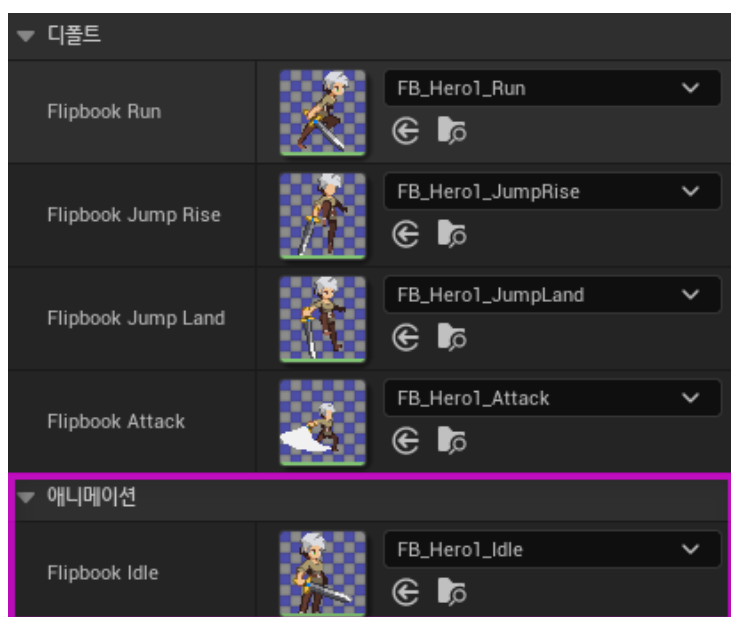
카테고리 변경에도 컴파일해야 적용이 된다.



컴파일 후에 디테일 패널 ⇒ 변수 리스트를 보면 애니메이션 카테고리 아래 변수가 표시된다

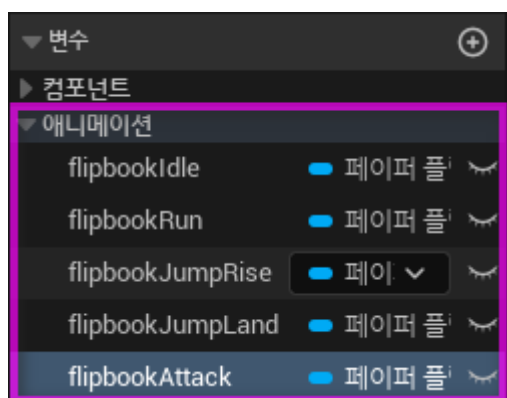


내 블루프린트 ⇒ 루트 컴포넌트 클릭 ⇒ 디테일 패널 ⇒ 애니메이션 카테고리 하위에 변수가 표시된다

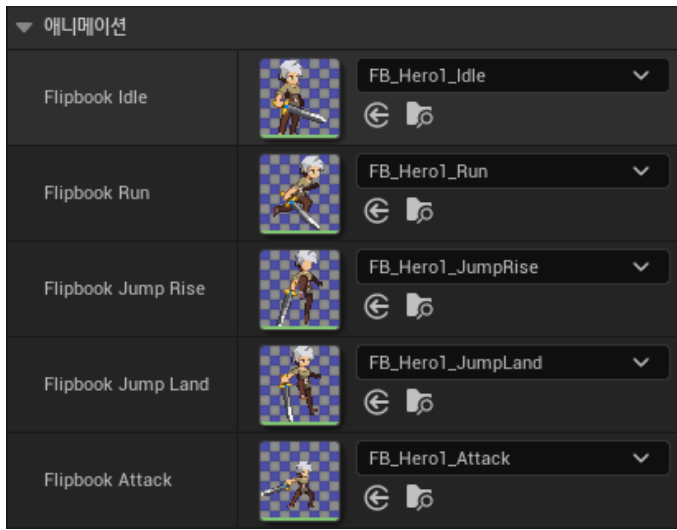


Run, Jump, Attack 변수에 카테고리를 Animation으로 적용하고 컴파일을 진행한다

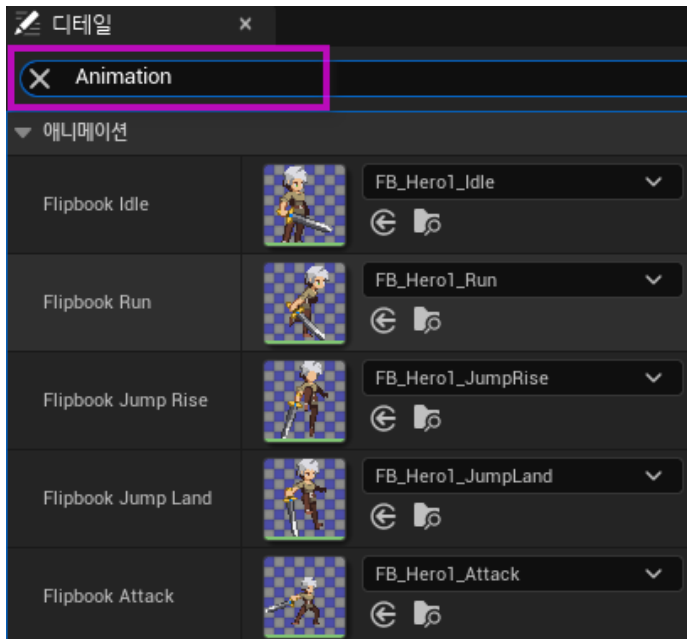
내 블루프린트 변수 리스트 확인



루트 컴포넌트 디테일 패널에서 애니메이션 카테고리 확인



디테일 검색창에서 카테고리 이름으로 검색이 되는 지 확인



지금까지 블루프린트의 정적 리소스들을 스크립트에서 활용할 **Resource**들을 애셋들을 제어할 수 있느

6.2 PaperCharacter 스크립트

6.2.1 애니메이션 변경 Run

애니메이션 변경 함수 **SetFlipbook()**

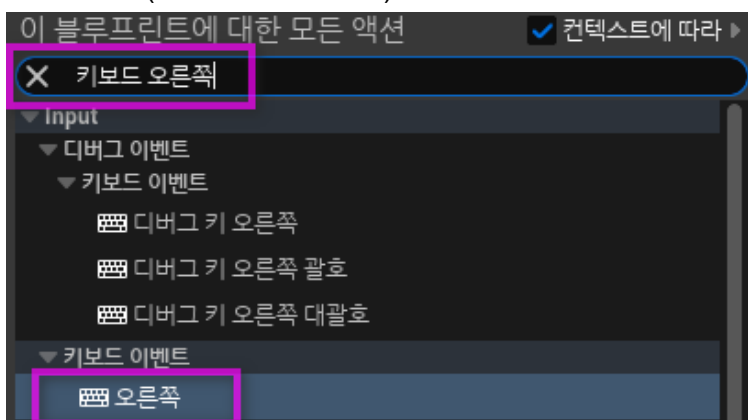
테스트로 오른쪽 방향키를 누르면 **idle** 애니메이션을 **Run** 애니메이션으로 변경하는 스크립트를 작성하자

BP2D_Hero1 블루프린트 클래스를 더블 클릭 ⇒ 이벤트 그래프 창을 연다

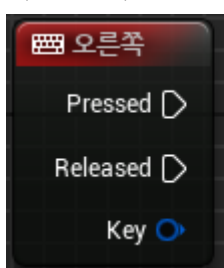
이벤트 그래프(Event Graph) 창의 빈 공간을 우클릭한다.

검색창에 "키보드 오른쪽"을 입력하면, 관련 입력 이벤트 목록이 표시된다.

"컨텍스트에 따라"(Context Sensitive) 체크 박스를 활성화하면 현재 블루프린트 클래스에서 사용 가능한 액션만 표시된다.



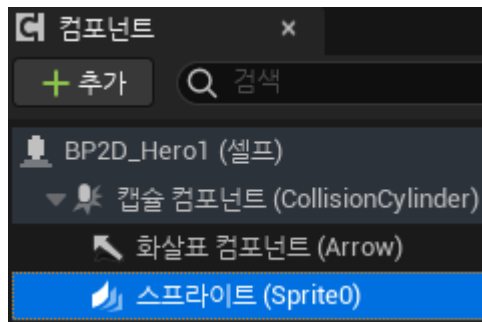
오른쪽 키보드 이벤트 노드가 추가됨



컴포넌트 패널에서 스프라이트 선택

컴포넌트 패널에서 스프라이트 컴포넌트를 선택한다.

선택한 상태로 이벤트 그래프(Event Graph) 창으로 끌어다 놓는다(Drag & Drop).
이 동작은 해당 스프라이트 컴포넌트를 Get(가져오기) 하는 것이다.



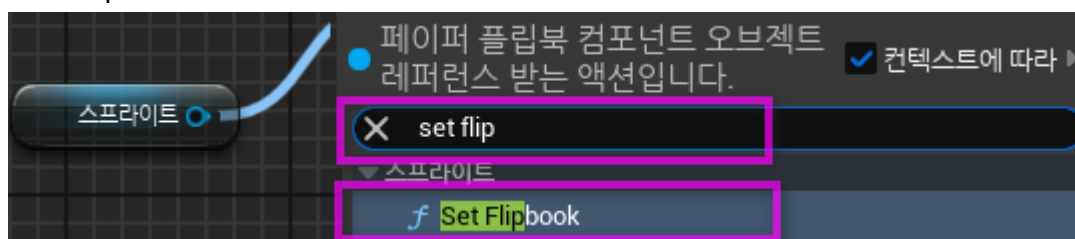
오른쪽 키보드 이벤트 노드 옆에



스프라이트 노드에서 새노드 배치



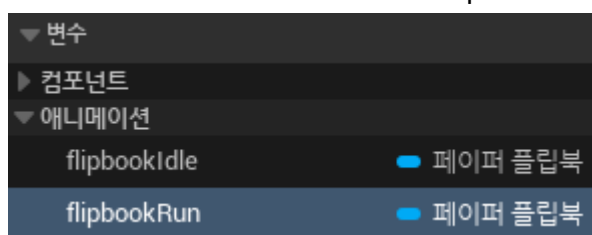
검색창에 set flipbook 입력력



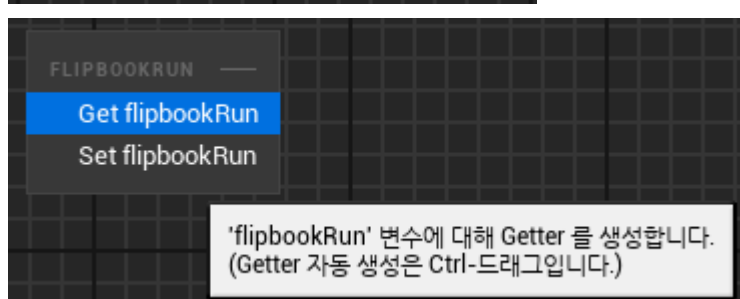
Set Flipbook 함수 노드 배치



내 블루프린트 ⇒ 변수 ⇒ 애니메이션 ⇒ flipbookRun을 끌어다 이벤트 그래프 창에 놓는다



Get / Set flipbookRun 컨텍스트 메뉴에서 Get flipbookRun 선택



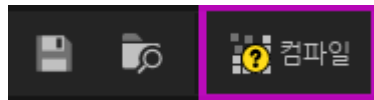
스프라이트 노드 아래로 이동



Net Flipbook에 Get 한 flipbookRun 를 연결



F7 키를 눌러 컴파일을 실행한다.



시뮬레이션을 실행하고 오른쪽 방향키를 누르면 run 동작 변경을 확인 할 수 있다.



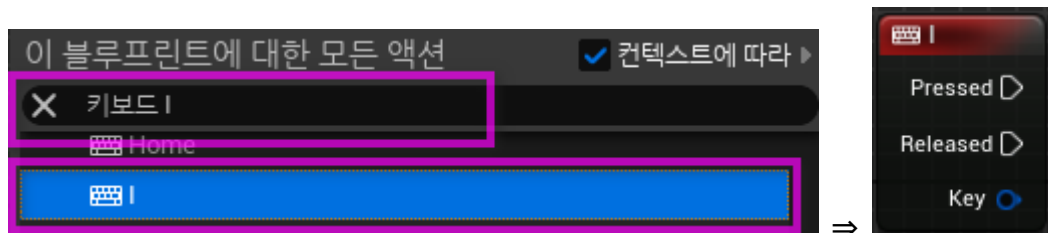
6.2.2 애니메이션 변경 Idle

"I" 키를 누르면 Idle로 환원 되는 애니메이션을 추가해보자

BP2D_Hero1 블루프린트 클래스를 더블 클릭 ⇒ 이벤트 그래프 창을 연다

이벤트 그래프(Event Graph) 창의 빈 공간을 우클릭한다.

검색창에 "키보드 I"를 입력 "I" 해당하는 키보드 이벤트 노드를 선택한다



컴포넌트 패널의 **Sprite**를 끌어다 놓아도 되지만 이벤트 그래프에 있는 스프라이트 노드를 복제해서 사용할 수 도 있다.
스프라이트 노드를 선택 ⇒ 우 버튼 복제 하기 또는 **CTRL+D**를 사용해서 복제한다.



"키보드 I" 이벤트 노드 옆으로 이동



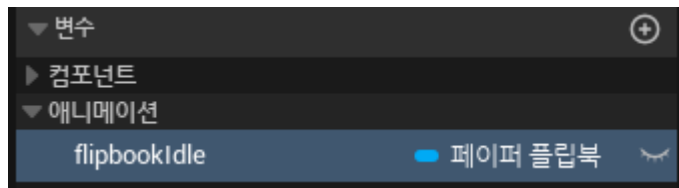
같은 스프라이트를 사용하므로 Set Flipbook 노드도 복제해서 사용한다



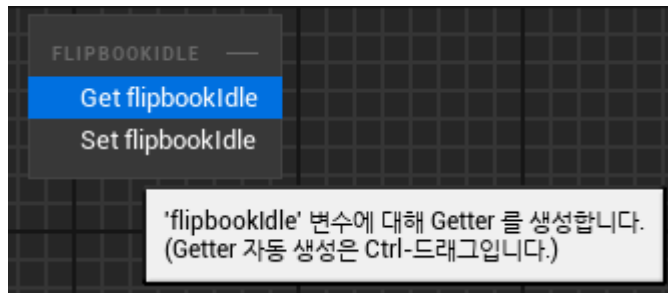
보기 좋게 다음과 같이 노드들을 정렬한다



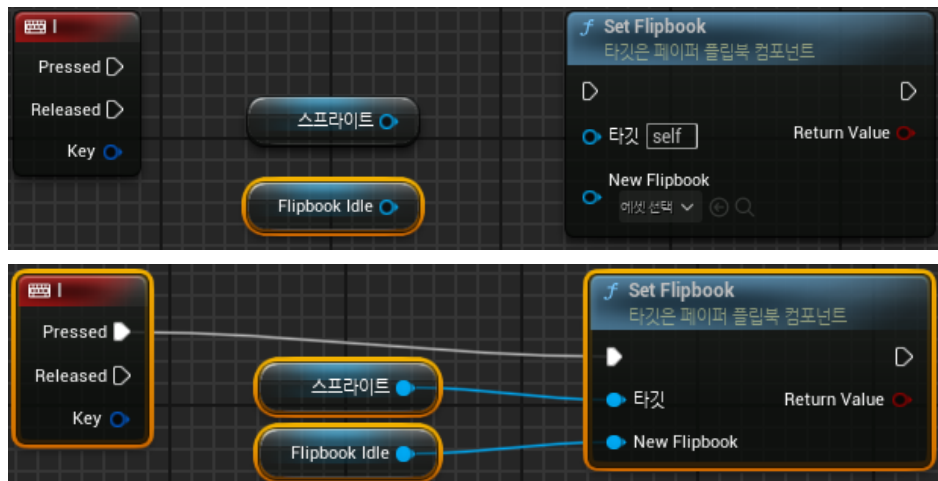
변수 섹션에서 flipbookIdle을 선택하고 그래프 창으로 끌어 놓는다



Get flipbookIdle Getter 를 선택한다.



Idle 애니메이션 제어에 해당하는 노드들을 정렬하고 연결한다



F7 키를 눌러 컴파일을 실행해서 Idle 변경을 완료한다.

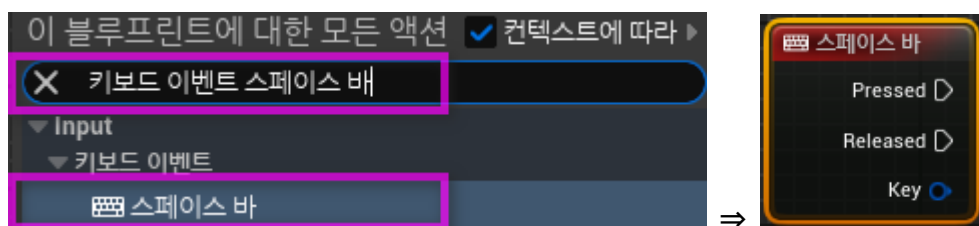
6.2.3 애니메이션 변경 JumpRise

"스페이스바" 키를 누르면 JumpRise 동작 애니메이션을 추가해보자

BP2D_Hero1 블루프린트 클래스를 더블 클릭 ⇒ 이벤트 그래프 창을 연다

이벤트 그래프(Event Graph) 창의 빈 공간을 우클릭한다.

검색창에 "키보드 I"를 입력 "I" 해당하는 키보드 이벤트 노드를 선택한다



스프라이트, Set Flipbook 노드를 동시에 선택한다

우 버튼 복제 하기 또는 CTRL+D를 사용해서 복제한다.



"키보드 스페이스 바" 이벤트 노드 옆으로 이동 한다

변수 섹션에서 flipbookJumpRise을 선택하고 그래프 창으로 끌어 놓고 Getter 노드를 선택한다

Run, Idle 노드처럼 실행 핀과 입력 노드들을 연결한다.

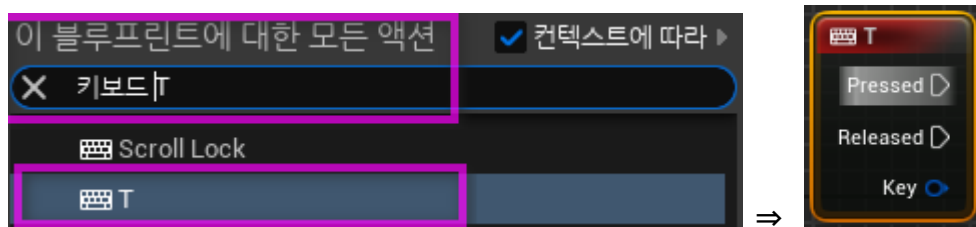


F7 키로 컴파일을 진행해서 작업을 완료한다
시뮬레이션에서 점프 애니메이션으로 변경되는지 확인한다



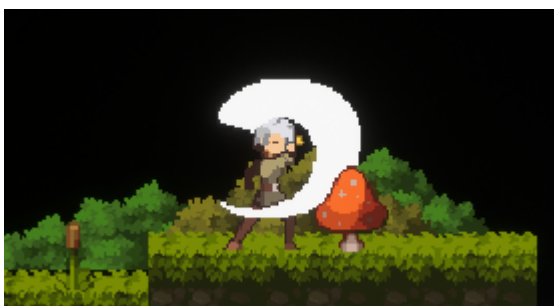
6.2.4 애니메이션 변경 Attack

지금까지 Run, Idle, JumpRise 를 구현해 보았다.
마지막 단계로 Attack을 구현해보자.
키보드 이벤트는 T 키를 사용할 것이다.
이벤트 그래프 액션 검색창에 "키보드 T" 를 입력하고 키보드 T 이벤트를 선택한다.



JumpRise 작업할 때와 같이 스프라이트, Set Flipbook 노드를 복제하고 키보드 T 이벤트 옆에 정렬한다.
내 블루프린트 변수 섹션에서 flipbookAttack을 이벤트 그래프에 끌어다 놓고 Getter를 선택한다.

시뮬레이션을 실행하고 키보드를 눌러서 애니메이션이 변경되는 지 확인한다



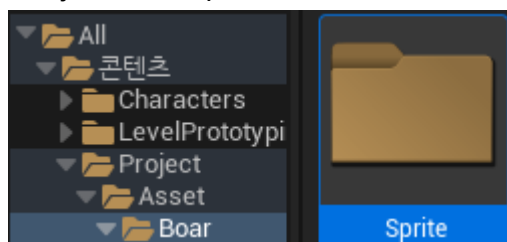
6.2.5 Mob Character

이번에는 Hero 캐릭터에서 학습했던 플립북 과정을 Mob Boar(멧돼지) 캐릭터 블루프린트를 만들어서 복습하자
애니메이션 작업의 전체 과정은 다음과 같은 순서로 진행된다.

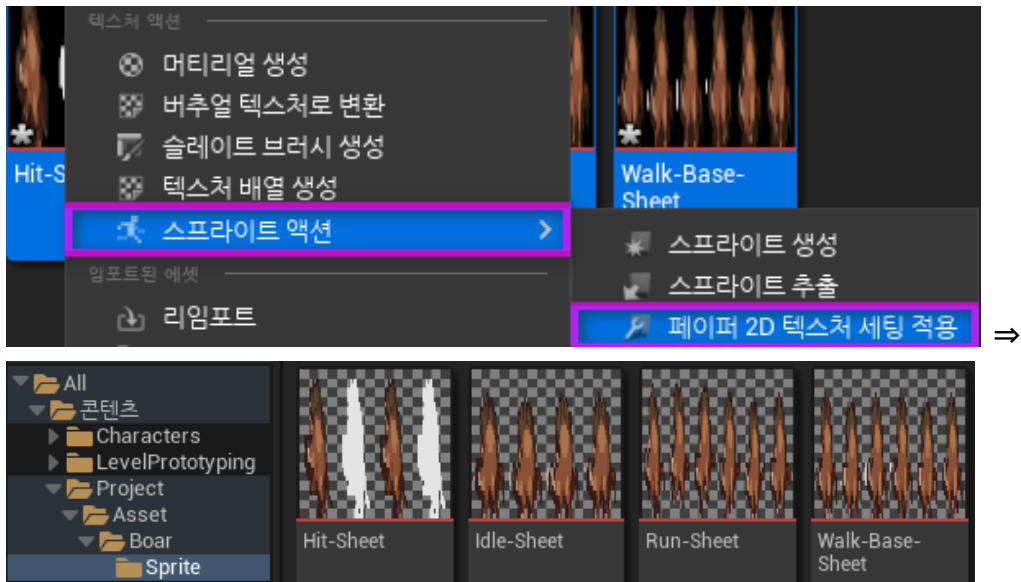
이미지 임포트 ⇒ (언리얼에서 자동으로 텍스처 변경) ⇒ 페이퍼2D 텍스처 세팅 적용 ⇒ 스프라이트 추출 ⇒ PaperCharacter
블루프린트 생성 ⇒ 스프라이트 컴포넌트에 Idle 플립북 지정 ⇒ 캡슐 컴포넌트 세이프 조정 ⇒ 애니메이션 교체용 플립북 애니메이션
변수 생성 ⇒ 컴파일 ⇒ 카테고리 "Animation" 적용 ⇒ 컴파일 완료
충돌(콜리전)과 관련된 캡슐 컴포넌트 세이프 조정 부분은 이후의 Chapter Collision 에서 자세히 다룬다.

[Mob Boar](#) 에서 몹 Boar 이미지 를 다운로드 한다. 압축을 풀면 Hit, Idle, Run, Walk 4 가지 동작 이미지 폴더가 포함되어 있다.

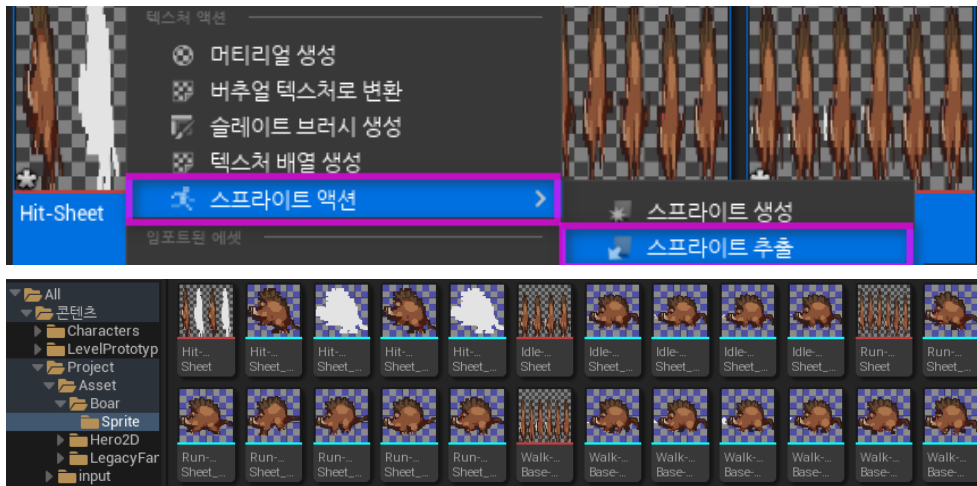
"콘텐츠/Project/Boar/Sprite" 폴더를 생성한다. 이 폴더는 Boar 캐릭터의 스프라이트 이미지를 관리하기 위한 전용 경로이다.



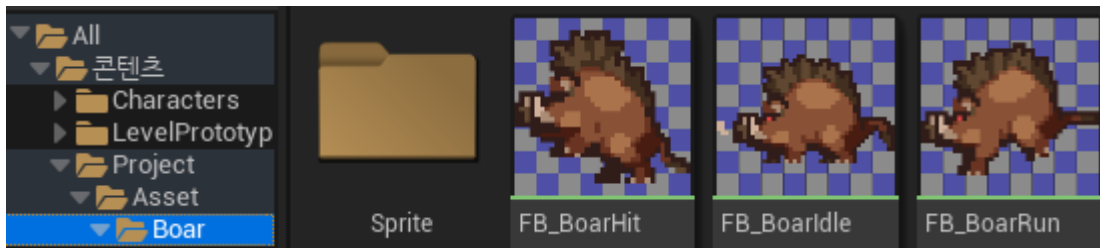
이 폴더에 Boar 이미지를 Import한다. Hit, Idle, Run, Wark 텍스처를 전부 선택하고 우클릭 ⇒ "페이퍼2D 텍스처 세팅 적용" 진행한다.



스프라이트를 추출한다.

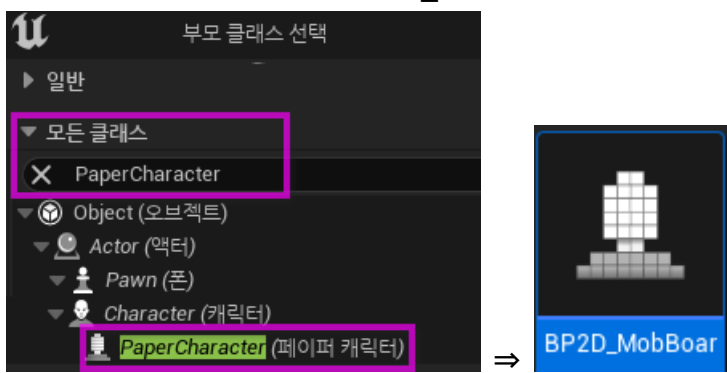


Idle, Run, Hit 3가지 동작에 대한 플립북을 생성한다.



Mob Boar에 대한 PaperCharacter 블루프린트를 생성한다. 나중에 AI를 적용하기 위해서 Mob Boar에 대한 PaperCharacter 블루프린트를 생성한다.

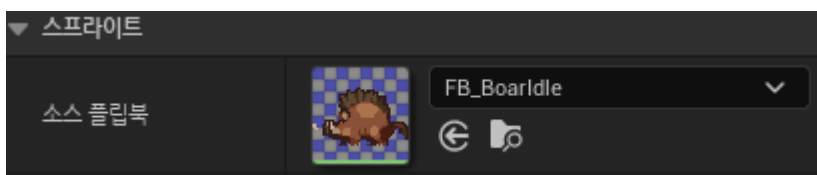
콘텐츠 브라우저 "콘텐츠/Project/Boar" 폴더 선택 ⇒ 추가 버튼(+) ⇒ 블루프린트 클래스 ⇒ 부모 클래스 선택 ⇒ PaperCharacter 선택 ⇒ 블루프린트 생성 ⇒ 이름 변경 ⇒ BP2D_MobBoar



소스 플립북 설정

BP2D_MobBoar 를 연다. 컴포넌트 패널 ⇒ 스프라이트 선택한다.

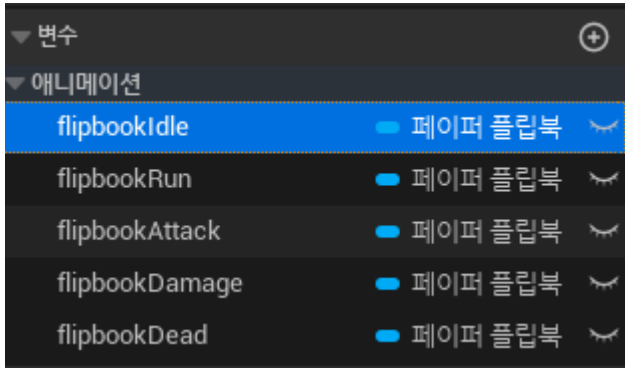
디테일 패널 ⇒ 스프라이트 ⇒ 소스 플립북 ⇒ FB_BoarIdle로 교체한다.



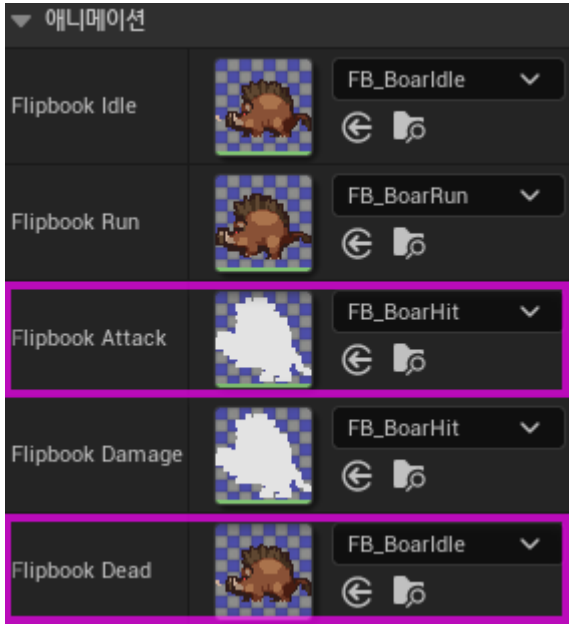
애니메이션 변수 생성

내 블루프린트에서 변수 버튼 +을 클릭하고 flipbookIdle, flipbookIdle, flipbookRun, flipbookAttack, flipbookDamage, flipbookDead 변수를 추가한다. 기본값을 설정하기 위해서 컴파일을 진행한다.

디테일 패널에서 카테고리 "디폴트"를 "Animation"으로 변경하고 컴파일을 진행한다.



Attack과 Dead 동작에 대한 애니메이션 텍스처가 없다. Attack은 Hit 플립북을, Dead에는 Idle 플립북을 설정한다.



월드 배치

레벨 에디터에서 BP2D_MobBoar을 끌어다 월드에 배치한다.
XYZ 위치를 3290, -25, 600에 배치하고 실행해본다.



07. 인풋 시스템

언리얼 향상된 입력 시스템 ⇒ [강의] 언리얼02 Unreal Enhanced Input

구성 요소

- Input Action**
단일 입력 행위를 정의 (Jump, Move, Attack 등)
- Input Mapping Context**
여러 Input Action을 키/패드/마우스에 매핑한 집합
- Trigger**
입력이 활성화될 조건 정의 (예: 눌림, 유지, 릴리즈 등)
Input Action 에서 정의
- Modifier**
입력값 보정 (예: 감도 조정, 반전, 스케일 등)
Input Action 에서 정의

블루프린트로 Input Event 전달 순서

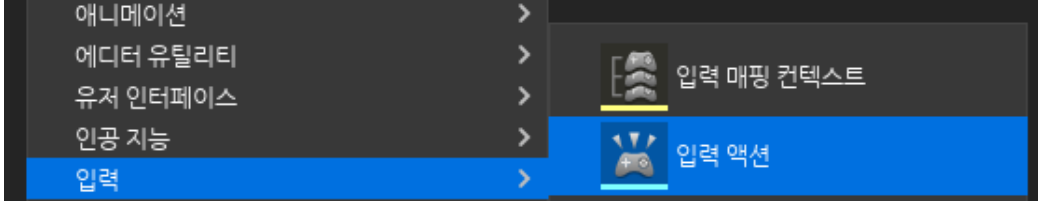


input 폴더 생성



7.1 입력 액션(Input Action) 생성

컨텐츠 브라우저 추가 버튼 또는 우 클릭



이름을 IA_2DMove



같은 방식으로 점프용 액션 생성



공격용 IA_2DAttack1 생성



전체는 3개를 확인



7.2 Input Action Value Type 과 IA_2DMove

7.2.1 Value Type

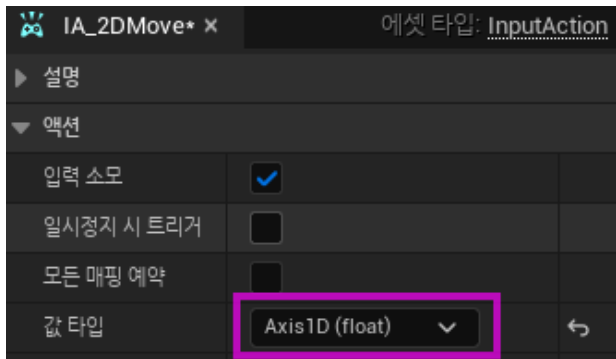
Input Action Value Type은 액션이 트리거될 때 게임 로직에 제공할 자료형을 지정

- Input Action Value Type 종류
 - Digital
 - bool: true/false
 - 키/버튼(눌림/땡)
 - Jump, Attack, Interact

- Axis1D
 - 1차원 축
 - float: -1.0 ~ +1.0
 - 마우스 휠, 트리거, 키보드 축 가상화
 - 가속/감속, 줌, 스로틀
- Axis2D 2차원 벡터
 - FVector2D
 - 게임패드 스틱, WASD(가상 2D)
 - 이동(Move), 카메라 Look
- Axis3D 3차원 벡터
 - FVector
 - 3D 컨트롤러/공간 포인터
 - 3D 이동/조작 특수 케이스

7.2.2 IA_2DMove

IA_2DJump, IA_2DAttack1은 단일 동작이라 디폴트 상태인 Digital로 설정
IA_2DMove는 좌우 이동만 설정하기에 Digital(bool) ⇒ Axis1D로 변경

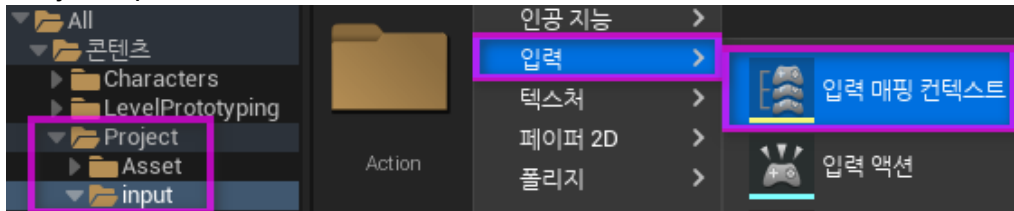


7.3 Input Mapping Context

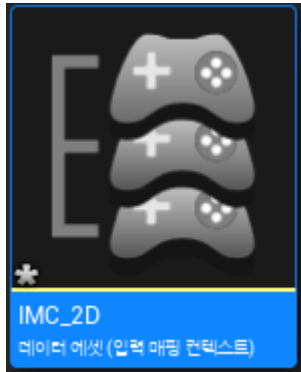
Input Mapping Context를 생성하여 이전 장에서 만든 Input Action을 키보드나 패드, 마우스 입력에 연결하자.

7.3.1 Input Mapping 생성

/콘텐츠/Project/input 폴더를 선택하고 빈곳에 우클릭 ⇒ 입력 ⇒ 입력 매핑 컨텍스트를 선택해서 입력 매핑 컨텍스트를 생성한다.

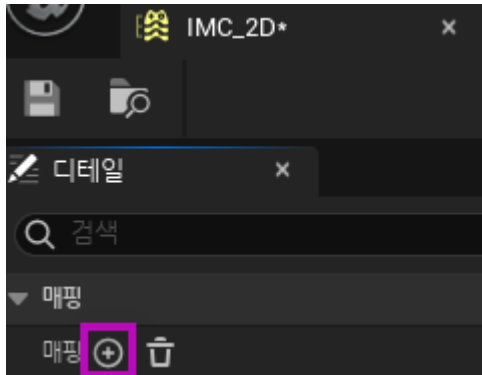


생성된 컨텍스트 이름을 IMC_2D로 변경한다.

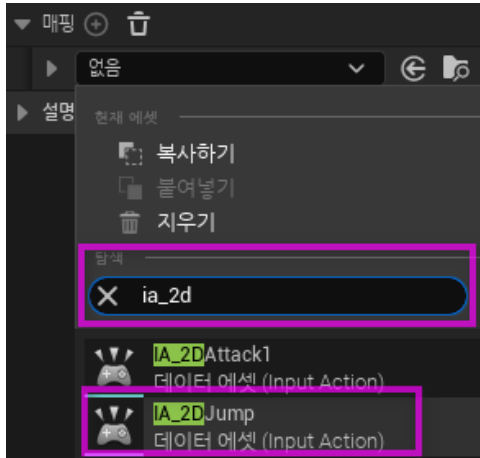


7.3.2 Input Mapping에 Action Control 설정

IMC_2D을 클릭해서 에디터로 열고 디테일 패널 ⇒ 매핑을 확장 ⇒ 매핑 옆 액션 매핑 추가 버튼(+) 클릭한다.

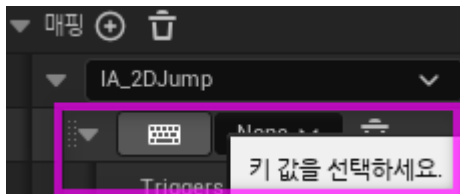


액션 매핑 컨트롤 찾기에서 ia_2d 를 입력, IA_2DJump를 선택

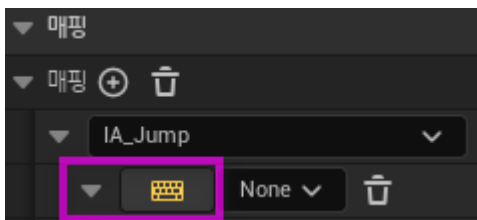


키 바인딩

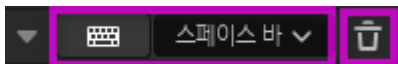
디폴트 상태로 키보드 아이콘에 마우스를 올려 놓으면 "키 값을 선택하세요" 툴팁이 표시 된다.



마우스로 클릭하면 노란색으로 변경이 되는데 노란색은 매핑 설정 상태이다. 이 상태에서 스페이스바를 누르면 키보드 아이콘이 흰색으로 변경이되고 매핑이 스페이스바로 설정이 된다.



스페이스 바로 변경



만약 필요없는 매핑이라면 휴지통을 클릭해서 삭제할 수도 있다.

7.3.3 입력 매핑 추가

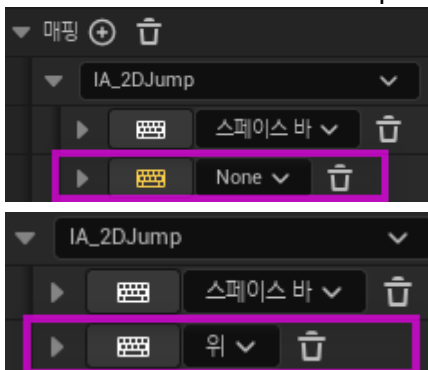
Jump에 다른 인풋을 더 추가할 수도 있다.

방향 키 Up("↑") 키를 Jump에 추가하자.

IA_2DJump 옆 추가 버튼 (+) 누른다



이전 방식과 동일하게 키보드 아이콘을 마우스로 클릭한다.
노란색 버튼으로 변경된 상태에서 Up 키를 누른다.



IA_2DJump 액션에 방향키 위("↑") 가 설정 된것을 확인한다.

7.3.4 Input Mapping Attack1 설정

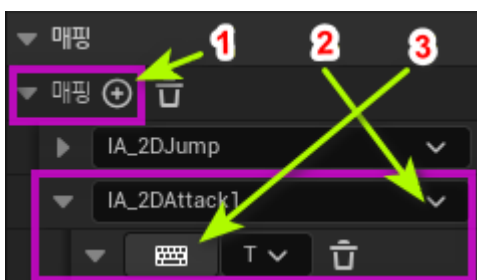
Attack1에 해당하는 입력 매핑을 추가하자.

Jump 때와 마찬가지로 액션 매핑 추가(+) 버튼을 클릭한다.

액션 매핑 컨트롤 찾기 입력란에 ia_2d를 입력하고, 목록에서 IA_2DAttack1을 선택한다.

새로 생성된 매핑의 입력 바인딩 버튼(키보드 아이콘) 을 클릭하여 노란색으로 활성화한다.

이 상태에서 키보드의 "T" 키를 눌러 매핑을 설정한다. → 버튼 색이 흰색으로 바뀌면 T 키가 Attack1 액션에 정상적으로 매핑된 것이다.



7.4 Input Mapping Context Modifiers 설정

Enhanced Input Modifier는 입력 값에 대한 보정이나 변환을 수행하며, IMC(인풋 매핑 컨텍스트) 에서 설정할 수 있다.

모디파이어의 대표적인 예로는 좌우 이동(Input Action Move) 에서 왼쪽 이동 시 값을 음수로 반전시키는 **Negate**가 있다.

7.4.1 Input Mapping Move 설정

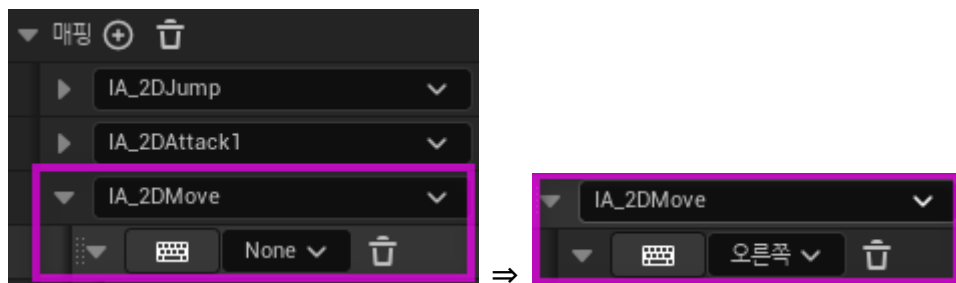
Jump 때와 동일하게 매핑 추가(+) 버튼을 클릭한다.

액션 매핑 컨트롤 찾기 입력란에 ia_2d 를 입력하고, 목록에서 IA_2DMove 를 선택한다.

새로 생성된 매핑의 Key Binding 버튼(키보드 아이콘) 을 클릭하여 노란색으로 활성화한다.

이 상태에서 키보드의 오른쪽 "→" 키를 눌러 매핑을 설정한다.

같은 방식으로 왼쪽 "←" 키에 대해서도 매핑을 추가한다.



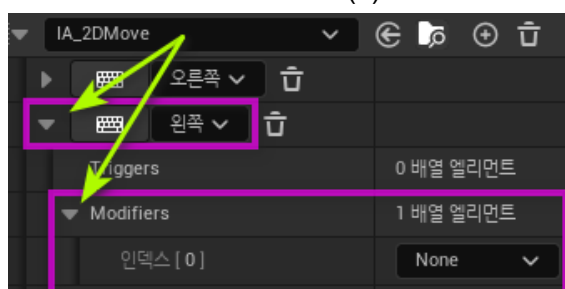
7.4.2 Negate 모디파이어 설정

왼쪽 이동은 오른쪽 이동의 반대 방향이므로, 입력 값에 마이너스(-) 를 곱하면 된다.

이 역할을 하는 모디파이어가 바로 **Negate** 이다.

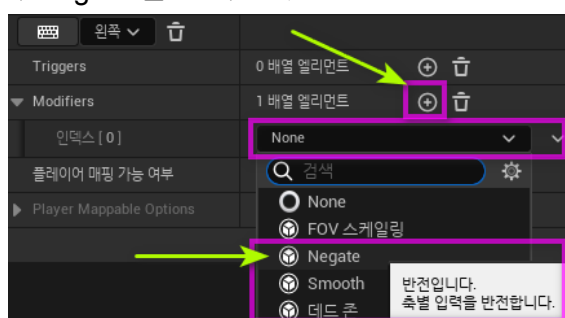
왼쪽으로 이동은 간단히 입력에 마이너스를 곱하면 되는 것이다. 이에 해당하는 것이 **Negate**이다.

왼쪽 ← 키 매핑 항목 우측의 확장(▼) 버튼을 클릭한다.



Modifiers 섹션이 열리면 모디파이어 추가(+) 버튼을 클릭한다.

목록에서 **Negate** 를 선택한다.



7.5 Enhanced Input 사용하기

순서 개요

현재 블루프린트의 Player Controller 구하기

Get Enhanced Input Local Player Subsystem 노드 호출

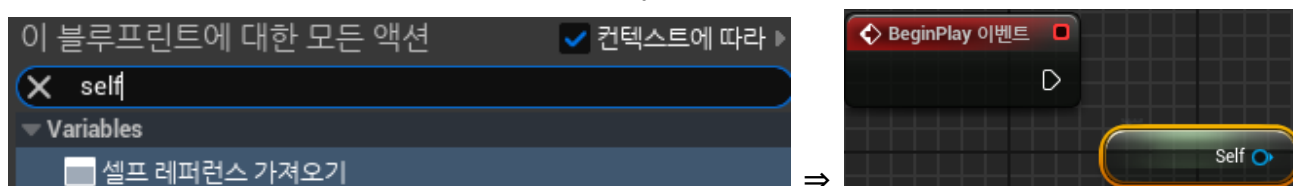
Add Mapping Context 노드 연결

IMC_2D 매핑 컨텍스트 지정

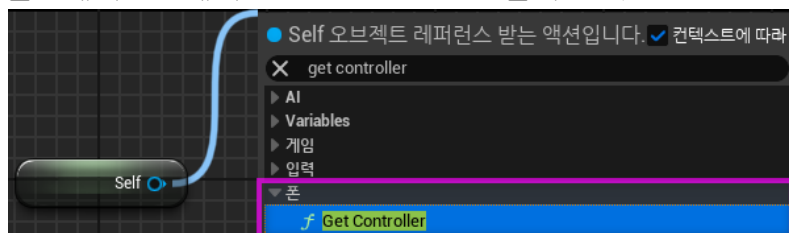
7.5.1 블루프린트 구성 단계

블루프린트 BP2D_Hero1에 입력 시스템을 연결하자

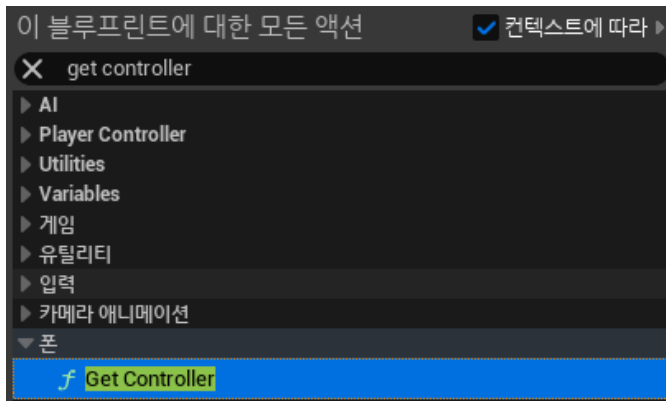
블루프린트 BP2D_Hero1를 열고 현재 블루프린트의 player controller 노드를 구하기 위해서 self 레퍼런스를 얻는다.



셀프 레퍼런스에서 Get Controller 노드를 구한다.



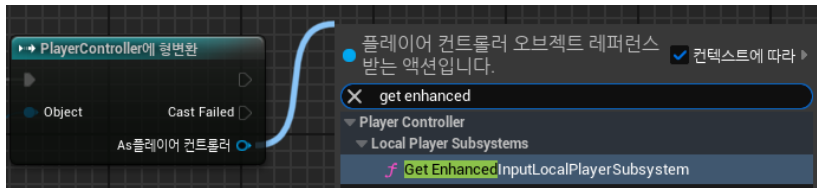
사실 self 레퍼런스 없어도 직접 자신의 player controller 노드를 구할 수 있다.



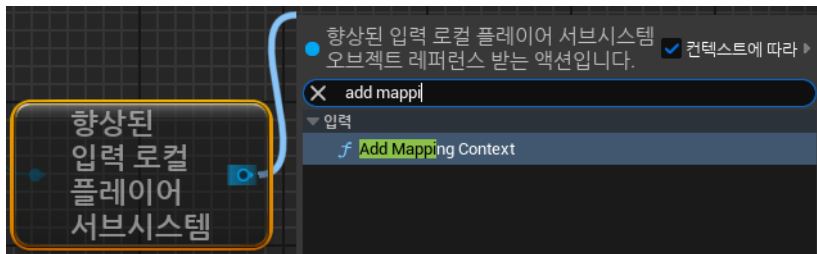
Get Controller는 PlayerController의 부모 클래스다.
 항상된 입력 시스템은 플레이어 컨트롤러에 있으므로 PlayerController로 형 변환 노드를 구성한다.



형 변환 노드의 결과인 As플레이어 컨트롤러에서 InputLocalPlayerSubsystem 노드를 얻는다.



Enhanced Input Local Player Subsystem 출력 핀에서 Add Mapping Context 노드를 배치한다.



Add Mapping Context 노드의 입력 핀 Mapping Context에 IMC_2D를 정한다.
 Mapping Context 입력 핀에 IMC_2D 에셋을 연결한다.
 Priority 값은 기본(0)으로 두어도 무방하다.
 노드 구성이 완료되면 컴파일(F7)을 눌러 저장한다.



노드를 완성하고 컴파일(F7)을 실행한다.

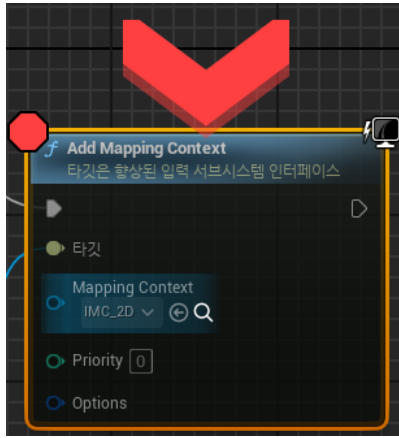


7.5.2 중단점으로 실행 확인

Add Mapping Context 노드까지 올바르게 실행되는지 확인하기 위해 중단점(Breakpoint)을 설정하자.

Add Mapping Context 노드를 선택한 뒤 F9 키를 눌러 중단점을 토글한다.

플레이 실행 시 해당 노드가 호출되면 중단점이 활성화되어 실행 흐름이 멈춘다. → 정상적으로 도달하면 입력 시스템이 연결된 것이다.

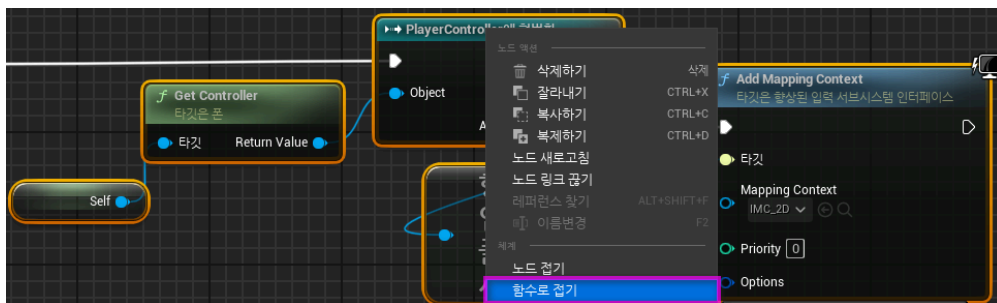


7.5.3 InitInput 노드 생성

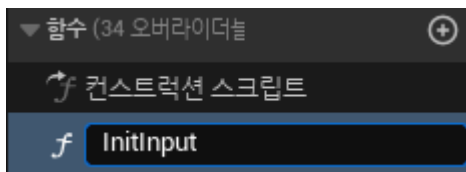
BeginPlay에는 여러 초기화 노드가 연결될 수 있으므로, 입력 시스템 초기화를 별도의 함수로 정리하자.

Get Controller, Cast to Player Controller, Get Enhanced Input Local Player Subsystem, Add Mapping Context 노드들을 모두 드래그로 선택한다.

선택된 상태에서 우클릭 → Collapse to Function (함수로 접기) 를 선택한다.



함수 목록에서 새로 생성된 함수의 이름을 NewFunction에서 InitInput으로 변경한다



완성된 InitInput 노드



정리된 BeginPlay

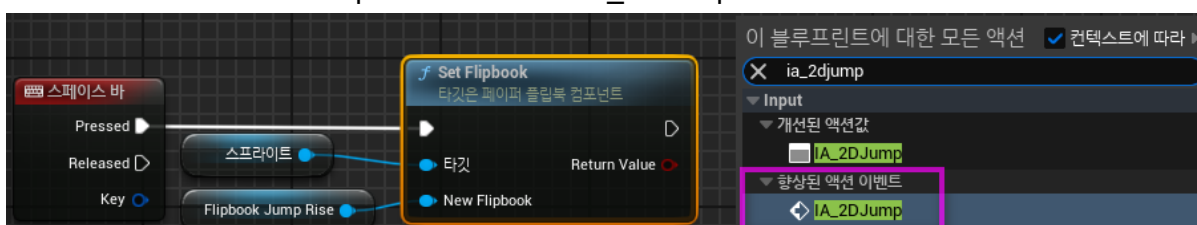


7.6 Input 에 애니메이션 연결

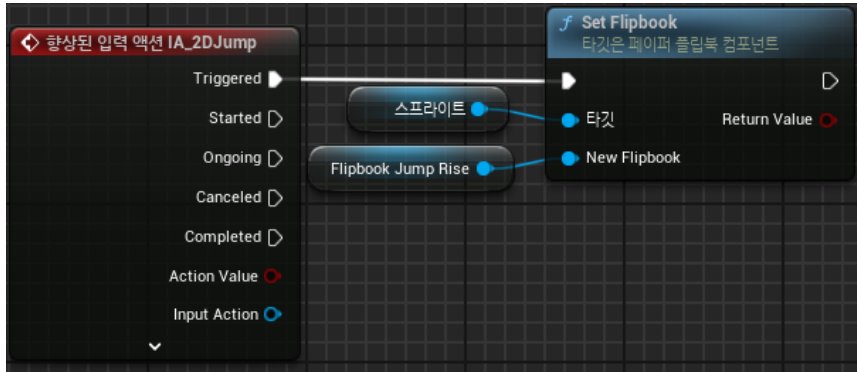
항상된 입력 시스템 초기화가 완료 되었다.

애니메이션 테스트로 사용했던 오른쪽 키 이벤트에는 IDC_2DMove, 스페이스 바 이벤트에는 IDC_Jump, T Key 이벤트에는 IDC_Attack1를 연결할 것이다.

Move 애니메이션 설정하는 Set Flipbook 노드 옆에 IA_2DJump 액션을 검색하고 항상된 액션 이벤트 IA_2DJump를 선택한다.

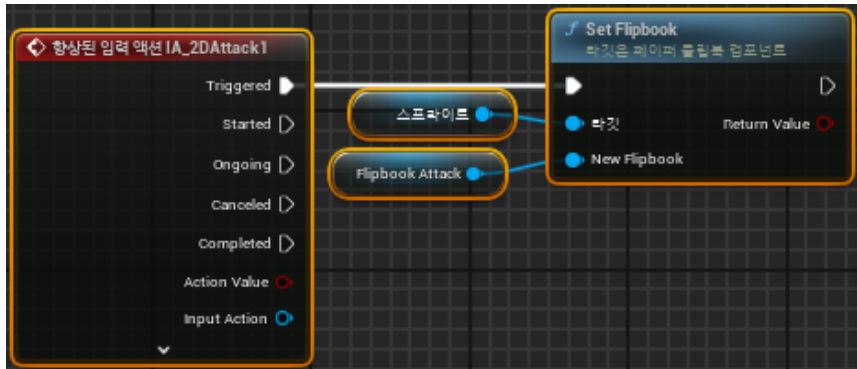


오른쪽 방향 키 대신 IA_2DJump 이벤트 노드를 연결한다.



시뮬레이터 실행으로 애니메이션이 변경되는 지 확인한다.

IA_2DJump와 같은 방식으로 T 키보드 대신 IA_2DAttack1 이벤트 노드를 연결한다.



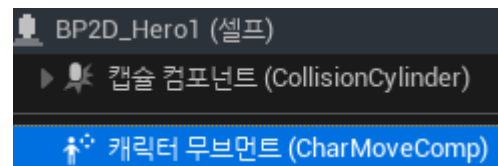
오른쪽 방향 키 대신 IA_2DMove 이벤트를 연결한다



블루프린트 컴파일(F7)로 완료하고 시뮬레이터에서 확인한다.

08. 2D Move

언리얼 character의 이동은 캐릭터 무브먼트 컴포넌트를 이용한다.



캐릭터 무브먼트 컴포넌트를 선택하고 디테일 패널을 보면 중력, 가속, 속도에 대한 다양한 값을 볼 수 있다.

물체의 순간 이동 변화량은 이동 방향 * 이동 속력 * 시간 이다.

이 값을 현재의 위치에 더해주면 된다.

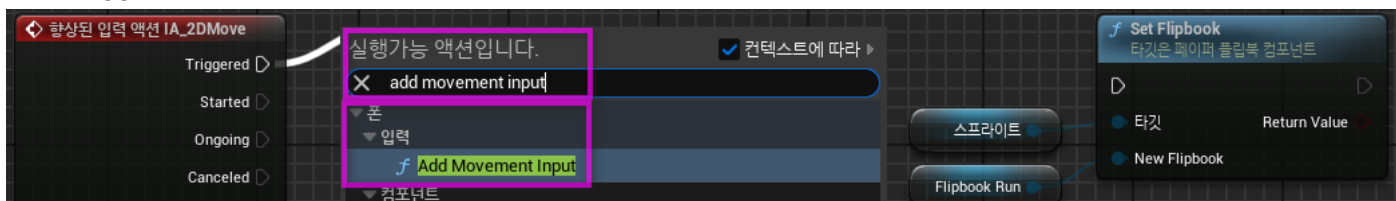
직접 계산해도 되지만 언리얼 블루프린트 노드에 이를 구현한 노드가 있다.

8.1 Add Movement Input

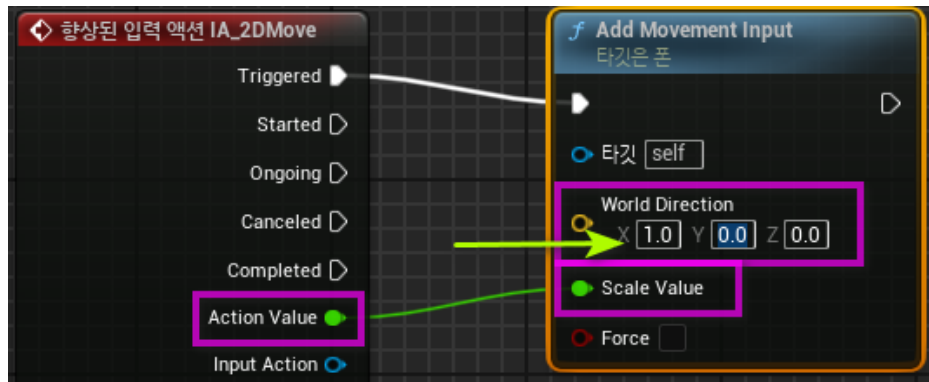
BP2D_Hero1 블루프린트를 연다

IA_2DMove 이벤트와 Set Flipbook 노드를 끊고 적당한 거리를 둔다.

IA_2DMove Triggerd 실행핀에서 AddInputMovement 노드를 인출한다.



IA_2DMove Triggerd 실행핀에서 AddInputMovement 노드를 인출한다.



World Direction은 이동 방향이다.

좌/우 이동 이므로 1.0, 0.0, 0.0 을 입력한다.

Action Value는 왼쪽 이동 버튼 앞서 Input 모디파이어 설정에서 Negate를 선택했기 때문에 -1 이 온다.

오른쪽 이동 버튼은 +1 값이 그대로 이다.

컴파일(F7)하고 시뮬레이터 실행에서 좌/우 방향키를 누르면 캐릭터가 Idle 동작에서 이동하고 있음을 볼 수 있다.

Set Flipbook 노드를 연결하면 간단하게 이동과 애니메이션을 연결 하면 될 것 같은데, 앞서 언리얼 이동은 캐릭터 무브먼트 컴포넌트를 이용한다고 했다.

8.2 캐릭터 무브먼트 컴포넌트

캐릭터 이동 애니메이션 설정을 좀 더 디테일하게 구현하자.

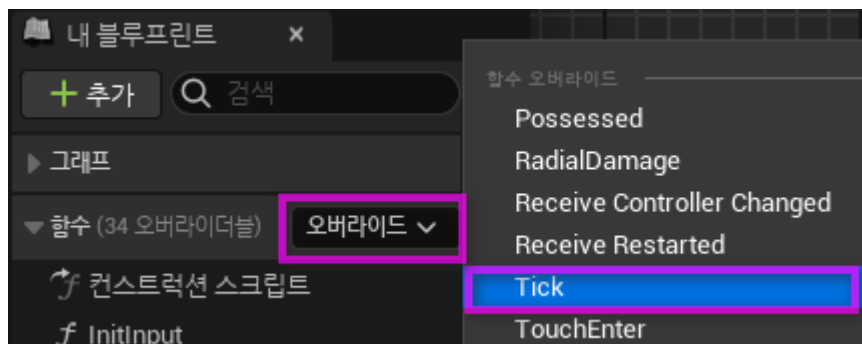
캐릭터 이동 애니메이션은 지면에 있는 경우에 적용해야한다. 지면에서 움직이고 있는지(멈춤 상태 포함), Jump 상태인지 먼저 판단한다.

지면에서 이동 상태인 경우 속도의 크기가 0보다 크면 이동 애니메이션을 설정하고 속도의 크기가 0이면 Idle 애니메이션을 동작 시킨다.

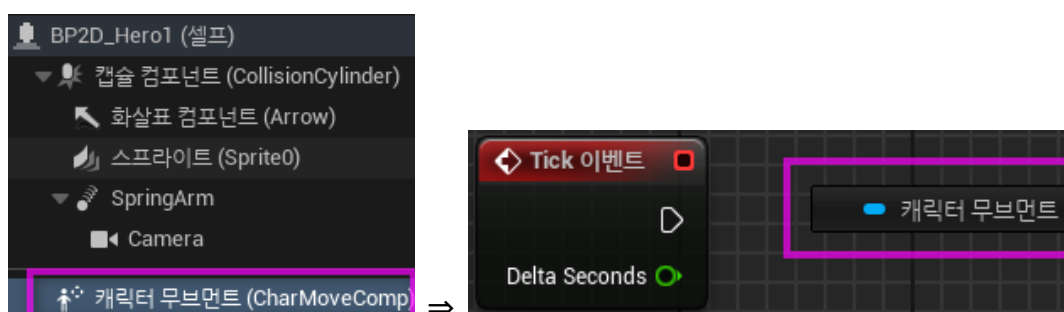
8.2.1 IsMovingOn Ground

이름 그대로 Ground에서 이동(멈춤 상태 포함)하고 있는지 확인하는 노드다. 이 노드를 Tick 연결해서 이동 중이면 Run 애니메이션을 이동 중이 아니라면 Idle 애니메이션을 연결한다.

Tick 이벤트를 지웠다면 내 블루프린트 함수 ⇒ 오버라이드 ⇒ Tick을 선택한다

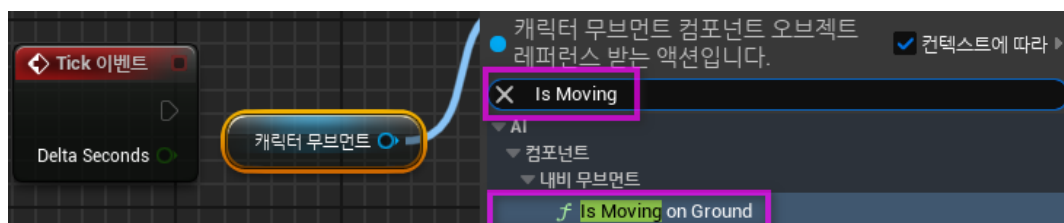


컴포넌트 패널에서 캐릭터 무브먼트를 선택하고 Tick 이벤트 옆에 끌어다 놓는다

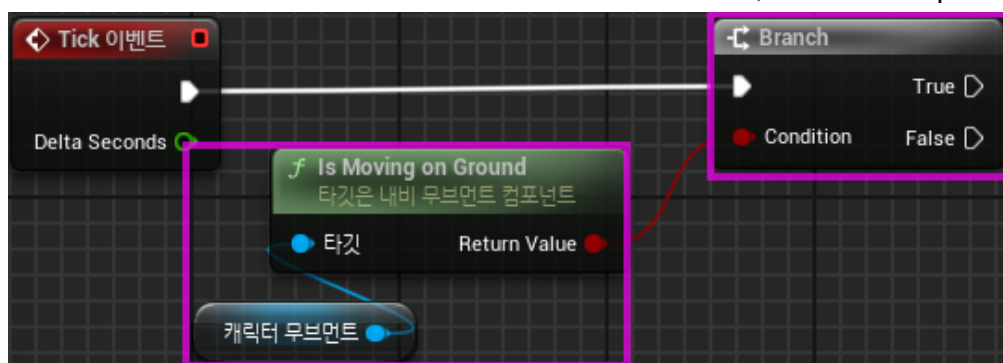


캐릭터가 그라운드에서 이동인지 확인한다. 그라운드 이동이 아니라면 jump Rise, jump Landing 상태이다.

캐릭터 무브먼트 노드에서 Is Move On Ground를 찾는다.

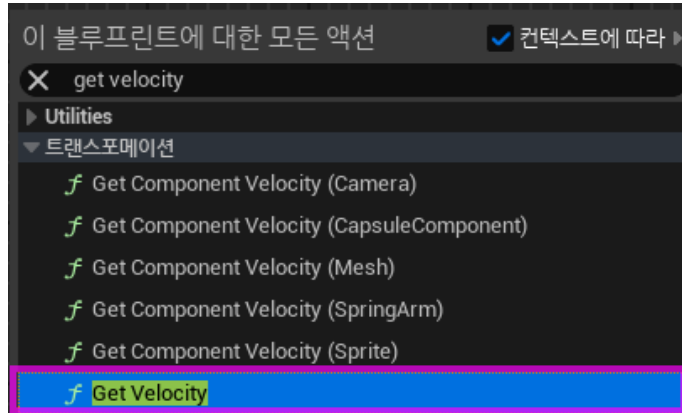


Return value에 Branch 노드를 붙여서 True는 이동 애니메이션에, False는 Jump에 연결할 수 있도록 구성한다.



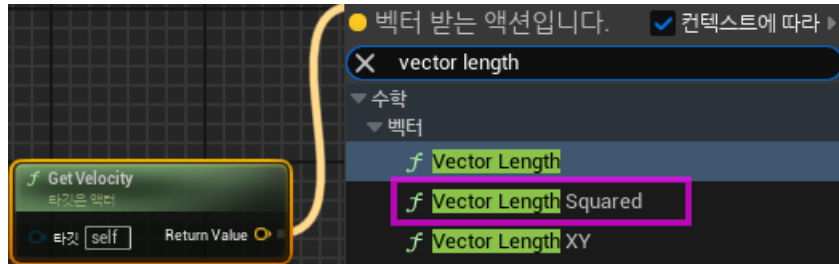
8.2.2 Vector Length Squared

빈 공간에 우 클릭 하고 Get Velocity 노드를 얻는다.

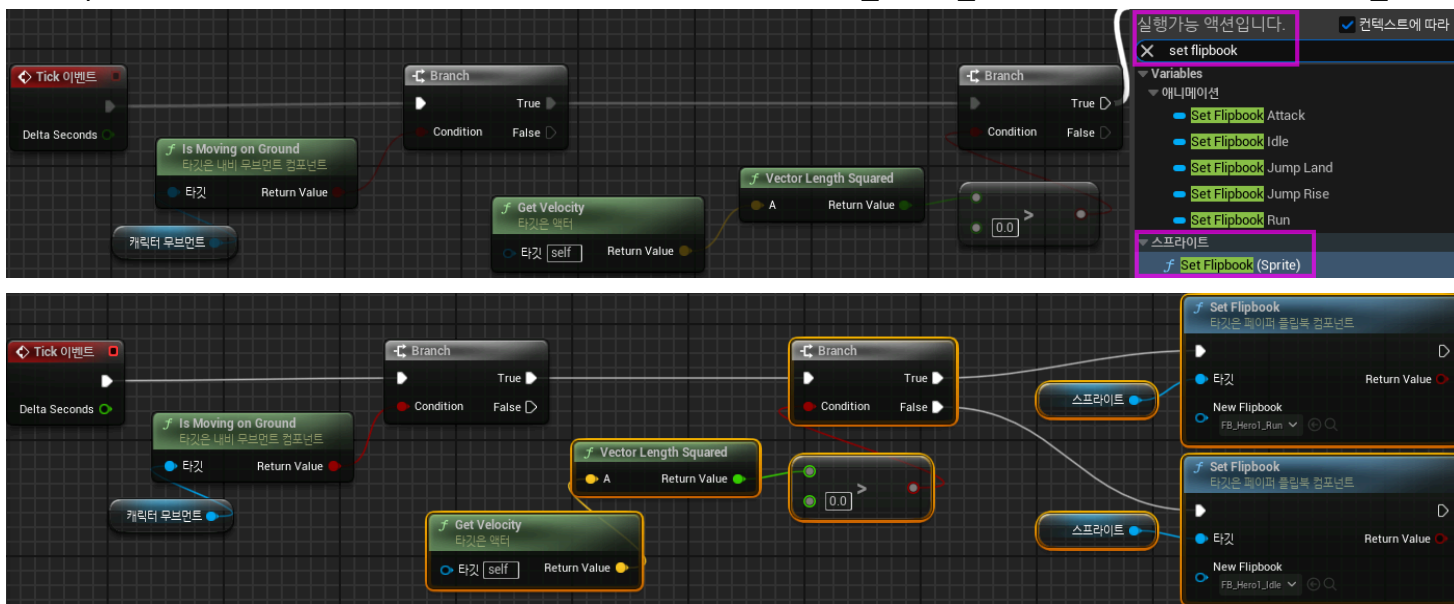


속도의 크기가 0 이 아닌 것만 판단하기 때문에 Vector Length Squared를 선택한다.

Vector Length Squared는 Vector 길이의 제곱 값을 반환하기 때문에 Vector Length 보다 처리 속도가 빠르다.



Set Flipbook 노드를 사용해서 속도 제곱의 크기가 0 보다 크면 FB_Hero1_Run 플립북을, 0과 같으면 FB_Hero1_Idle 를 설정한다.



컴파일(F7)을 하고 시뮬레이터에서 실행해본다.

8.3 좌/우 반전

우측을 바라보는 캐릭터가 좌측으로 이동할 때는 좌우 반전이 필요하다.

캐릭터의 오른쪽이 X축, 위쪽이 Z축인 환경에서 단순히 Z축(Yaw) 회전을 0°(오른쪽) ↔ 180°(왼쪽)로 바꾸면 이론상 반전이 되지만, 캐릭터(또는 Player)에 카메라가 종속되어 있으면 카메라까지 함께 회전한다.

이를 피하는 간단한 방법은 Set Control Rotation 을 사용해 컨트롤러의 제어 회전(Control Rotation) 만 바꾸어 캐릭터의 '논리적 바라보는 방향' 을 전환하는 것이다.

다만 이때 Spring Arm이 부모 회전을 따르거나 상대 회전이면, 컨트롤러/폰 회전 변화가 카메라에 전파되어 배경이 뒤집힌 것처럼 보일 수 있다.

따라서 캐릭터의 Spring Arm을 '절대 회전(Absolute Rotation)'으로 설정한다. 그러면 Spring Arm(및 카메라)은 항상 월드 기준 회전을 유지하고, Set Control Rotation 으로 방향만 바뀐 캐릭터만 좌우 반전된다.

8.3.1 Compare Float Node

Compare Float 노드는 두 개의 float 값을 비교하여 결과에 따라 'A > B', 'A == B', 'A < B' 의 세 가지 실행 경로로 분기하는 노드이다.

향상된 입력 시스템(Enhanced Input)의 Action Value에서는 -1.0 ~ +1.0 범위의 값을 가져올 수 있다. 이 값이 0보다 크면 오른쪽 이동, 0보다 작으면 왼쪽 이동을 의미한다.

이를 이용해 캐릭터의 Z축 회전을 제어할 수 있다.

오른쪽 이동일 때 → Z축 0° 회전

왼쪽 이동일 때 → Z축 180° 회전

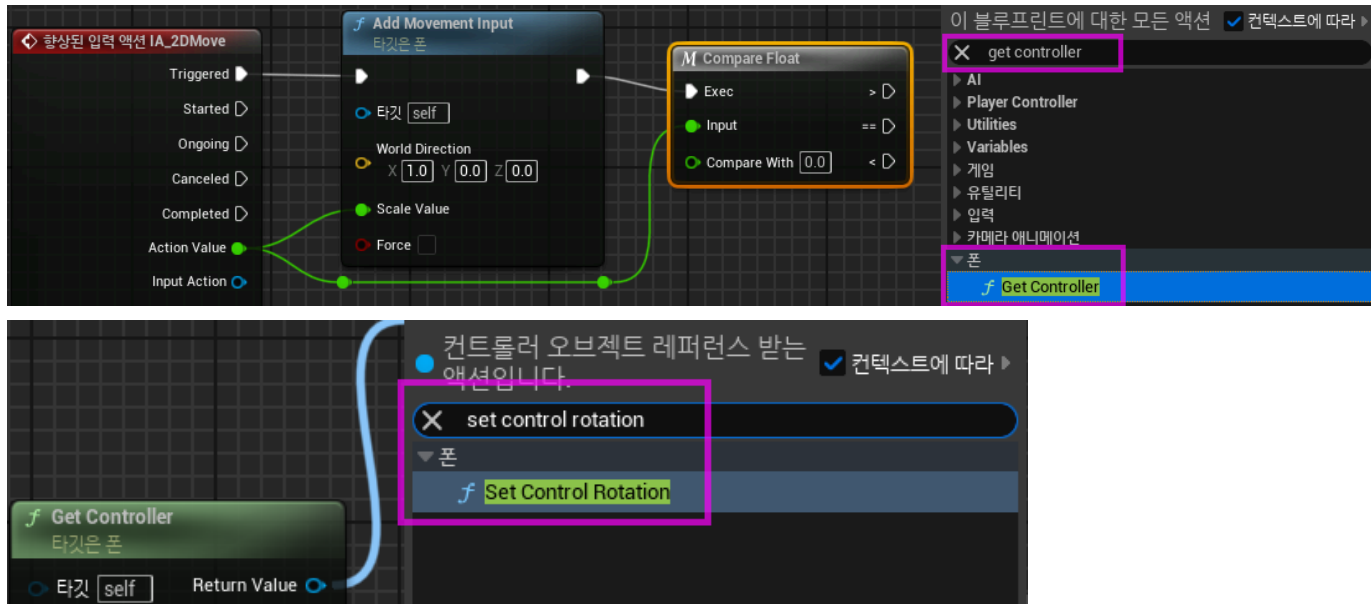
즉, 캐릭터가 바라보는 방향을 입력값에 따라 자동으로 반전시킨다.

Add Movement Input 노드의 실행 출력 핀(Exec Out)에 Compare Float 노드를 연결한다.



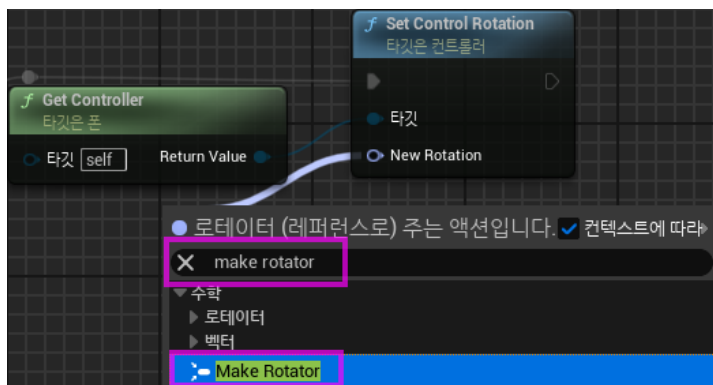
8.3.2 Set Control Rotation

이벤트 그래프의 빈 공간을 우클릭하여 **Get Controller** 노드를 추가한다.
Get Controller의 출력 핀에서 Set Control Rotation 노드를 호출한다.

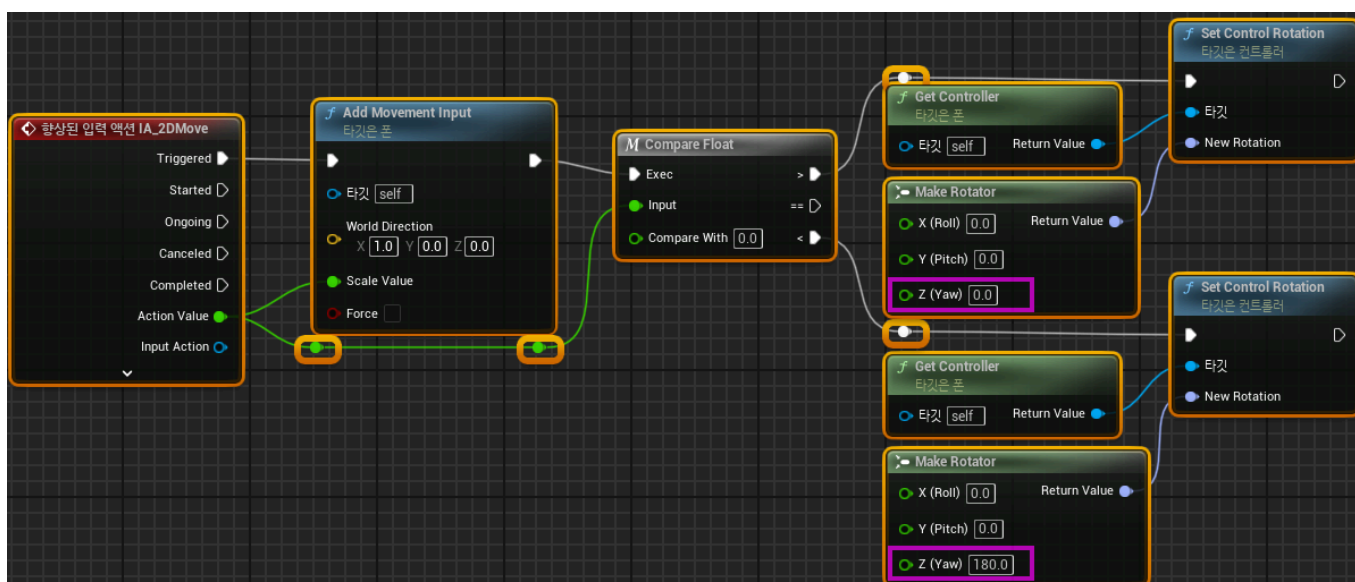


8.3.3 Make Rotator

Set Control Rotation의 입력 핀 New Rotation에서 액션 make rotator를 검색하고 Make Roator 노드를 연결한다.



지금까지 오른쪽 방향 이동을 구현했다.
이제 왼쪽 방향 이동은 Compare Float 노드의 A < B 실행 핀에서 시작한다.
오른쪽 방향과 동일한 과정을 반복하되,
Make Rotator 노드의 Z 축 회전 값을 180°로 설정한다..



컴파일을 완료하고 시뮬레이터서 실행한다.
실행하면 다음 그림처럼 왼쪽으로 이동할 때 카메라가 **180°** 회전하며 이동하는 모습을 확인할 수 있다.



이 문제를 올바르게 수정하려면 카메라의 회전 기준을 상대 회전(Relative Rotation)에서 절대 회전(Absolute Rotation)으로 변경해야 한다.

이렇게 설정하면 캐릭터가 좌우로 이동할 때, 카메라는 고정된 상태로 유지되며 캐릭터만 반전되어 이동하는 모습을 확인할 수 있다. 컴포넌트 Spring Arm 을 선택한다. 디테일 패널에서 트랜스폼 회전을 클릭한다. 월드를 선택하면 회전을 절대 회전으로 변경된다.



8.4 직교(Orthographic) 카메라

지금까지는 Perspective(투영) 카메라를 사용해 왔다. 투영 카메라는 3D 연출용 카메라로, 원근감이 표현된다.

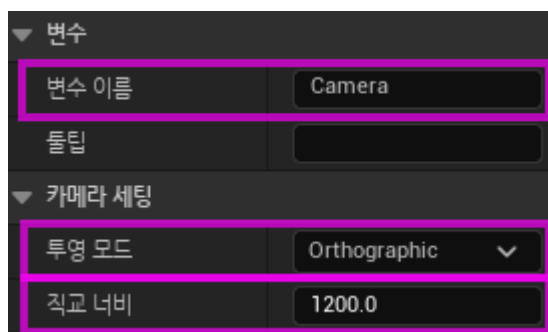
하지만 완전한 2D 연출을 위해서는 직교(Orthographic) 카메라를 사용하는 것이 적합하다.

직교 카메라는 카메라의 거리와 관계없이 동일한 비율로 화면을 출력하므로, 원근 왜곡 없이 2D 그래픽을 정확하게 표현할 수 있다.

Camera Component를 선택하고, Details 패널 → Camera Settings → Projection Mode를 Orthographic으로 변경한다.

이어서 Ortho Width(직교 너비) 값을 조정하여 원하는 화면 크기를 설정한다.

직교너비 값을 조정해서 적당한 값을 찾는다. 지금은 1200을 이용할 것이다.



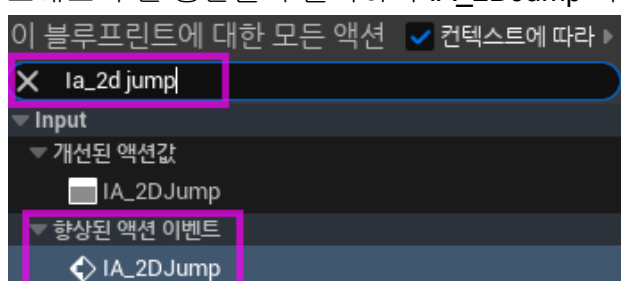
8.5 Jump

8.5.1 Jump / Stop Jump

점프(Jump) 구현은 Jump 노드를 이용한다. Jump 노드를 호출하면 Character Movement Component의 Is Moving On Ground 값이 false로 변경되어, 캐릭터가 공중 상태임을 나타낸다.

이 점을 기억하고, 먼저 향상된 입력 시스템(Enhanced Input)에서 Jump 노드를 호출하여 점프 동작을 실행하자.

이벤트 그래프의 빈 공간을 우클릭하여 IA_2DJump 이벤트 노드를 추가한다.



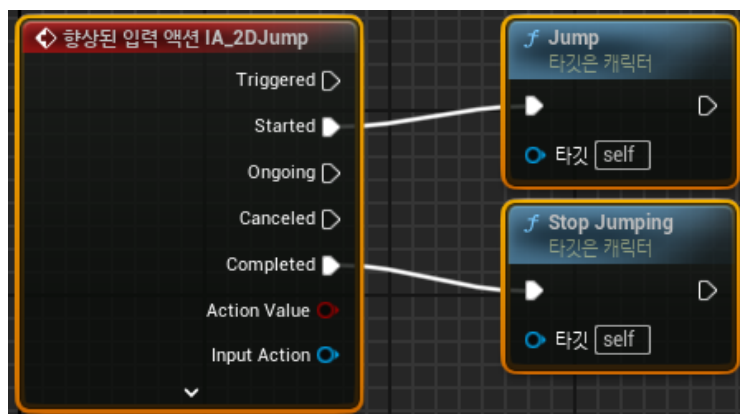
Started 실행핀에서 Jump 노드를 호출 한다.



Canceled 실행 핀에서 stop jump 노드를 호출 한다.



완성된 노드



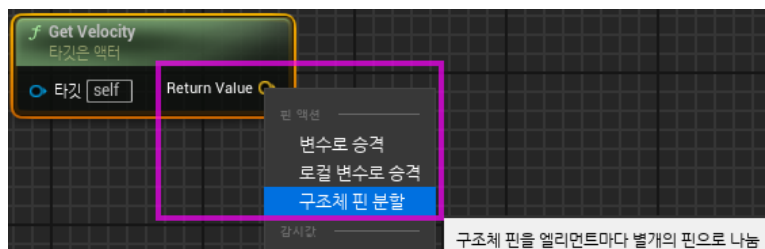
8.5.2 Jump Animation

앞서 Is Moving On Ground 노드의 True 반환 시점에서 이동 애니메이션을 재생하도록 설정했었다.

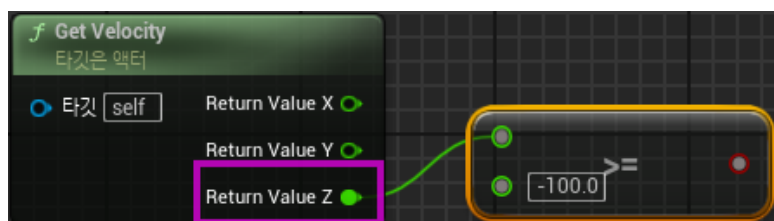
Is Moving On Ground 노드가 False를 반환하면, 캐릭터가 지면에서 떨어져 공중에 떠 있는 상태임을 의미한다.

캐릭터가 점프하여 상승하는 상태는 이동 속도(Velocity)의 Z 값이 양수(+) 일 때이며, 반대로 Z 값이 음수(-) 인 경우는 낙하 중임을 의미한다. 따라서 적절한 Z 값의 임계값을 기준으로 분기하여 상승 시에는 JumpRise 애니메이션, 하강 시에는 JumpLand 애니메이션을 재생하도록 설정한다.

이벤트 그래프의 빈 공간을 우클릭하여 Transform → Get Velocity 노드를 추가한다. Get Velocity 노드를 구하는 방법은 앞에서 설명한 것과 동일하다.

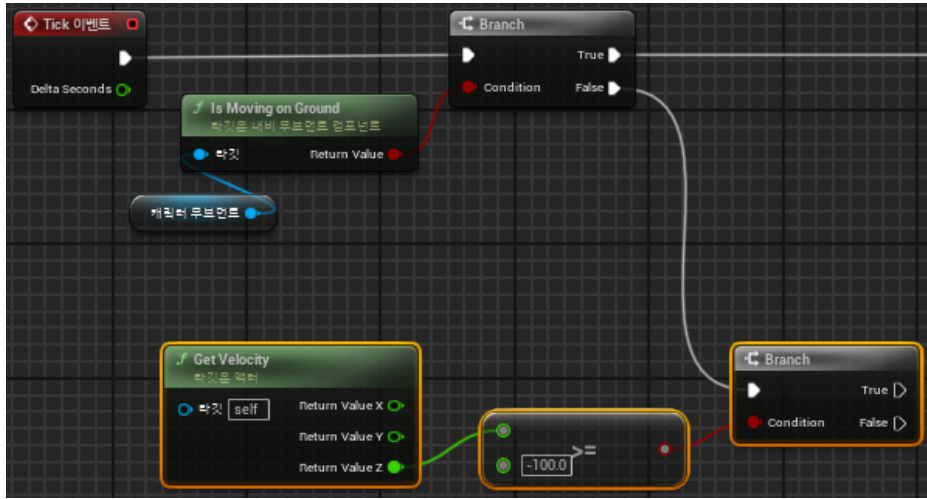


Get Velocity 노드의 Return Value(노란색 출력 핀)에 마우스를 올리고 우클릭 → 구조체 핀 분할(Split Struct Pin) 을 선택한다. 이렇게 하면 X, Y, Z 세 축의 속도 값을 각각 개별로 사용할 수 있다.

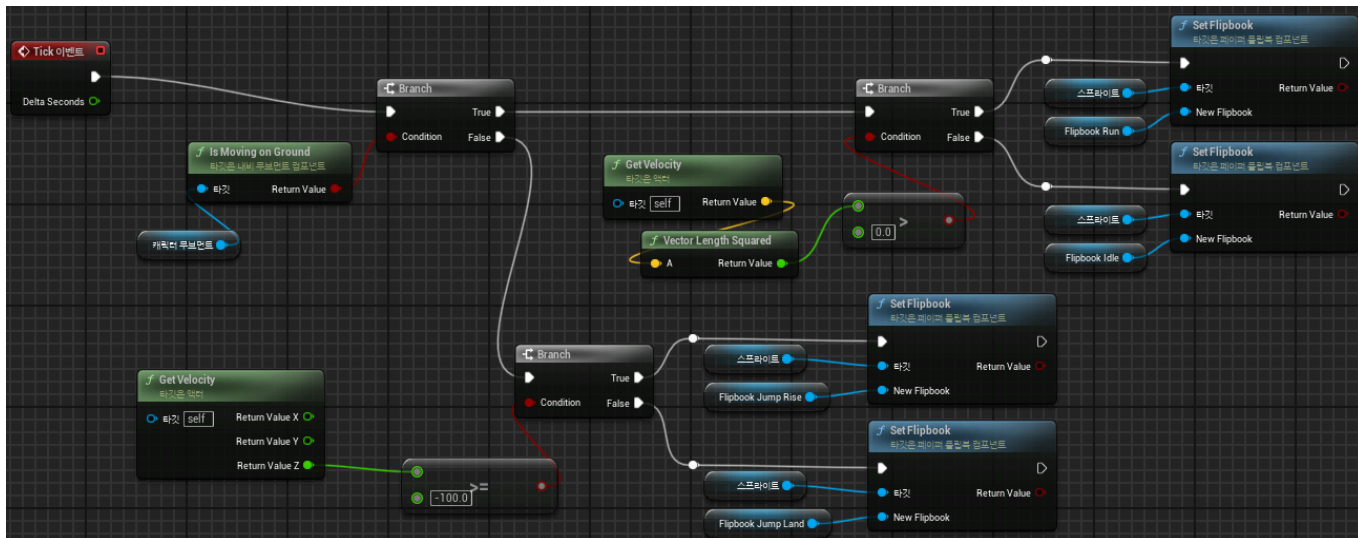


분할된 Z 값을 기준으로 Compare Float 노드를 사용하여 $Z > -100.0$ 인 경우에는 JumpRise 애니메이션을 재생하도록 설정한다.

-100.0 값을 선택한 이유는 캐릭터가 떨어지기 시작하자마자 바로 Jump Land 애니메이션으로 전환되지 않도록 약간의 낙하 여유 구간을 두기 위함이다. 즉, 일정 높이 이상 떨어졌을 때만 Jump Land 동작이 자연스럽게 적용되도록 하기 위해서다



Set Flipbook 노드를 이용해서 전체 노드를 완성한다.



Tick 이벤트에는 다른 노드들이 계속 추가될 수 있다. 복잡해지는 그래프를 정리하기 위해, Tick 이벤트의 실행 핀(Exec Out) 부터 연결된 모든 노드를 선택한 후 우클릭 → "함수로 접기(Collapse to Function)" 를 선택한다. 이렇게 생성된 함수의 이름을 UpdateAnimation 으로 정한다.

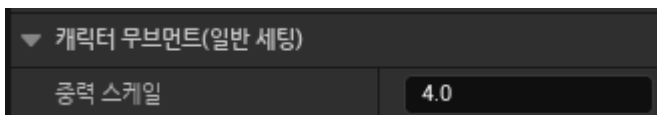


8.6 캐릭터 무브먼트

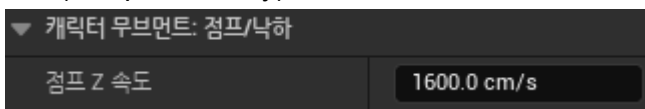
실행해 보면 캐릭터의 움직임이 풍선처럼 가볍게 느껴진다. Character Movement의 여러 물리 값을 조정하여, 무게감 있고 타격감이 살아 있는 액션감을 만들어 보자.

컴포넌트 패널에서 캐릭터 무브먼트(Character Movement Component)를 선택하고 Details 패널에서 다음 값을 조정한다.

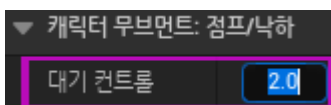
중력 스케일(Gravity Scale) → 4.0



점프 Z 속도(Jump Z Velocity) → 1600



대기 컨트롤(Air Control)을 크게 하면 점프 중에서도 좌우 이동이 가능하다. 디폴트 0.25 ⇒ 1.0~2.0으로 변경하면 공중에서도 좀 더 자유롭게 이동이 가능하다



값을 조정한 뒤 실행해 보면, 무게감이 느껴지면서도 속도감 있는 점프 액션을 확인할 수 있다



09. Collision / Attack

9.1 Collision

Attack은 다른 액터(**Actor**)와의 상호 작용이 필요한 과정이다.

가장 단순한 아이디어는 공격 애니메이션이 재생되는 동안, 특정 영역 안에 존재하는 액터에게 ****Damage(피해 수치)****를 가하는 것이다. 이때 "특정 영역 안에 다른 액터가 존재하는가?"를 판단하기 위해 콜리전(**Collision**) 기능을 사용한다.

언리얼 엔진의 콜리전 동작 방식은 크게 **Block**과 **Overlap** 두 가지로 구분된다.

- **Block** 충돌은 두 액터가 실제로 물리적으로 맞닿는(**contact**) 경우 발생한다.
예를 들어, 탄환이 벽에 부딪히는 상황에서 **Block** 충돌이 발생하며, 이때 엔진은 **Hit** 이벤트를 발생시킨다.
- **Overlap** 충돌은 두 액터의 충돌 영역이 서로 겹쳤을 때(**overlap**) 발생한다.
예를 들어, 공격 범위 안에 적이 들어온 경우가 이에 해당하며, 이때 엔진은 **BeginOverlap** 및 **EndOverlap** 이벤트를 발생시킨다.

공격 판정(**Attack Detection**)에서는 실제 물리 충돌보다는 영역 판정이 더 중요하기 때문에, 일반적으로 **Overlap** 이벤트를 사용한다.

9.1.1 충돌 판정을 위한 컴포넌트

언리얼 엔진에서는 액터 간의 ****충돌 판정(Collision Detection)****을 처리하기 위해 여러 형태의 ****콜리전 컴포넌트(Collision Component)****를 제공한다.

이들은 액터가 물리적으로 상호작용하거나, 특정 영역 안에 다른 액터가 들어왔는지를 감지하는 데 사용된다.

1. 캡슐 콜리전 (Capsule Collision)

- 클래스: **UCapsuleComponent**
- 형태: 중심축을 기준으로 한 원통 + 양쪽 반구 형태
- 특징:
 - 캐릭터의 몸통 형태에 적합하다.
 - 점프나 회전 시 모서리에 걸리지 않아 부드러운 이동이 가능하다.
 - 언리얼의 기본 **Character** 클래스는 캡슐 콜리전을 사용한다.
- 사용 예: 플레이어, NPC, 몬스터 등 사람형 캐릭터의 충돌 영역

2. 박스 콜리전 (Box Collision)

- 클래스: **UBoxComponent**
- 형태: 축 정렬된 직육면체
- 특징:
 - 사각형 또는 직사각형 형태의 영역을 표현하기 좋다.
 - 회전 및 위치 조정이 용이하고, 공격 판정 영역으로 자주 사용된다.
- 사용 예: 검의 휘두름 범위, 타일형 구조물, 상자, 벽

3. 스피어 콜리전 (Sphere Collision)

- 클래스: **USphereComponent**
- 형태: 중심점을 기준으로 한 구형 영역
- 특징:
 - 모든 방향에서 동일한 반경을 가지므로 범위 감지용으로 적합하다.
 - 수학적 계산이 단순하여 가장 성능이 우수한 형태이다.
- 사용 예: 폭발 반경, 근접 탐지, 주변 AI 감지

4. 커스텀 콜리전 (Custom Collision)

- 클래스: **UStaticMeshComponent**, **USkeletalMeshComponent**
- 형태: 메시(**Mesh**)에 포함된 콜리전 데이터
- 특징:
 - 모델의 형상을 그대로 반영할 수 있어 정밀한 충돌 판정이 가능하다.
 - 그러나 계산량이 많아 성능 부담이 크다.

- 사용 예: 복잡한 구조물, 지형, 보스 캐릭터의 정밀 피격 영역

9.1.2 PaperCharacter와 콜리전

PaperCharacter 클래스는 언리얼 **Character**를 상속받아 만들어진 **2D** 전용 캐릭터 클래스이다. 따라서 언리얼 **Character** 클래스가 기본적으로 사용하는 캡슐 콜리전(**UCapsuleComponent**)을 그대로 포함하고 있다.

즉, **Paper2DCharacter** 역시 캡슐 형태의 충돌 컴포넌트를 기본 콜리전으로 사용하며, 이로 인해 캐릭터는 물리적인 이동이나 점프, 중력 등의 처리를 안정적으로 수행할 수 있다.

필요에 따라, 추가적인 박스(**Box**) 또는 스피어(**Sphere**) 콜리전을 자식 컴포넌트로 붙여서 공격 판정(**Attack Collision**)이나 감지 센서(**Detection Sensor**)로 활용할 수도 있다.

Hero1 블루프린트는 **PaperCharacter**를 상속받아 만들어진 캐릭터 클래스이다. 기본적으로 **PaperCharacter**는 **Capsule Collision**을 포함하고 있으며, 이는 캐릭터의 물리적 충돌과 이동 처리에 사용된다.

Hero1 블루프린트에는 추가적으로 **Spring Arm**과 **Camera** 컴포넌트가 포함되어 있다. 그러나 이 두 컴포넌트는 단지 ****시점 제어(View Control)****를 위한 요소일 뿐이므로, 충돌 기능을 적용할 필요가 없다.

또한 **PaperCharacter** 내부의 **Sprite** 컴포넌트는 화면상 캐릭터의 시각적 표현만 담당한다. 이 스프라이트는 단순한 렌더링 요소이므로, 충돌 기능을 활성화하면 오히려 불필요한 충돌 이벤트가 발생하여 게임 로직을 복잡하게 만들 수 있다.

따라서, **Hero1** 캐릭터에서는 오직 캡슐 콜리전만 충돌 판정에 사용하면 충분하다.

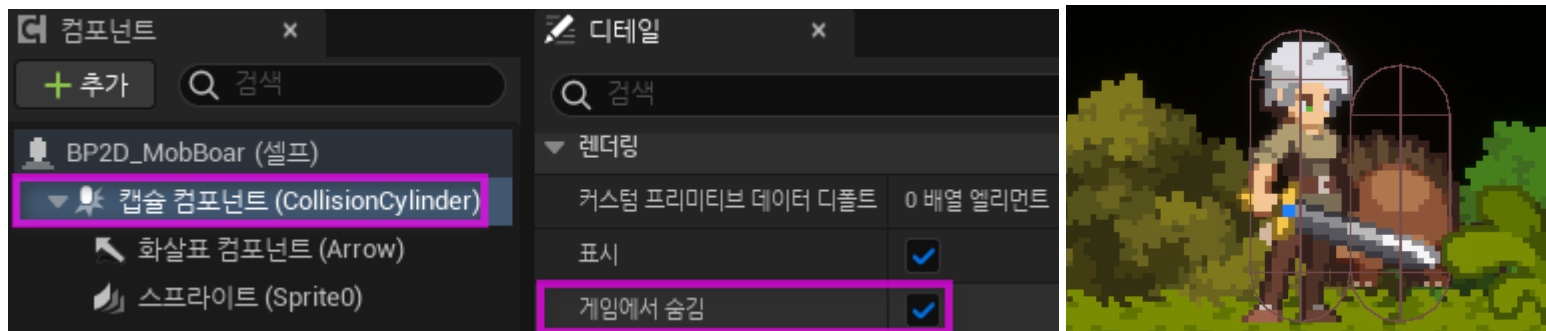
나머지 컴포넌트(**Spring Arm**, **Camera**, **Sprite**)는 충돌을 비활성화하여 게임 코드의 안정성과 단순성을 유지한다.

9.1.3 콜리전 설정

앞서 말했듯이, **PaperCharacter**로 파생된 클래스에서 캡슐 컴포넌트만 충돌 처리하고 나머지는 무시하는 것을 설정하자.

06장의 **Sample**를 실행하면 **Boar**와 캐릭터가 충돌해서 진행이 안되는 것을 볼 수 있다.

만약 충돌 영역을 플레이에서 확인 하고 싶다면 캡슐 컴포넌트 선택, 디테일 패널 ⇒ 렌더링 ⇒ 게임에서 숨김 항목이 있는데 이 항목은 당연히 디폴트가 꺼져 있다. 활성화 하면 게임 플레이에서 충돌 영역이 표시된다.



9.1.4 충돌 이벤트

언리얼의 충돌 이벤트는 크게 **Hit**(접촉/충돌) 과 **Overlap**(겹침/진입/이탈) 두 종류가 있다.

이 두 이벤트는 충돌 방식이 완전히 다르기 때문에, 정확히 이해하고 적절한 곳에 사용해야 한다.

Hit 이벤트 (OnHit, Hit Result)

Hit 이벤트는 두 충돌체가 물리적으로 '부딪혔을 때' 발생한다.

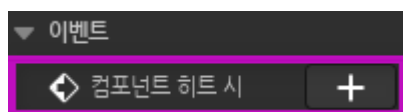
Overlap 이벤트 (BeginOverlap / EndOverlap)

Overlap은 충돌체가 겹치기만 해도(비물리 충돌) 발생하는 이벤트이다.

Block이 아니라 **Overlap** 응답에서 동작한다.

9.1.5 Hit Event

BP2D_Hero1의 블루프린트 컴포넌트 패널에서 캡슐 컴포넌트를 선택한다. 디테일 패널 하단에 보면 이벤트가 있고 "컴포넌트 히트 시"가 있는데 "+" 버튼을 눌러서 이벤트를 생성한다.

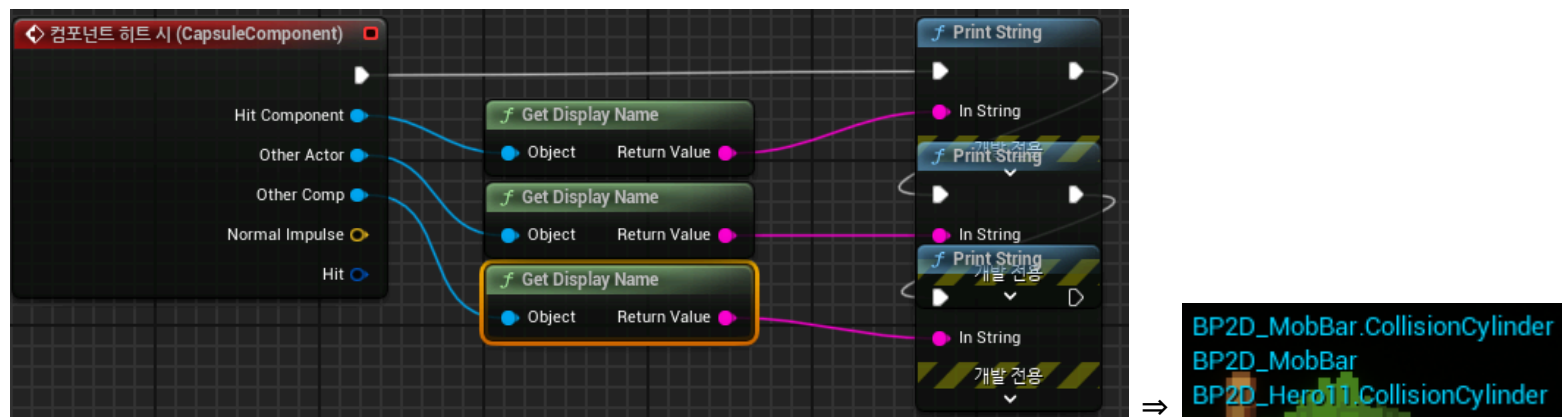


이벤트 노드의 출력 핀에는 **Hit Component**, **Other Actor**, **Other Comp**, **Normal Impulse**, **Hit** 등이 있다.

- **Hit Component**
충돌을 감지한 자신의 컴포넌트이다.
(예: **BP2D_Hero1**에서는 **Capsule Collision**이 이에 해당한다.)
- **Other Actor**
자신과 충돌한 상대 액터이다.
- **Other Comp**
충돌이 발생한 상대 액터의 특정 컴포넌트이다.
(어떤 부위·어떤 콜리전이 부딪혔는지 확인할 때 사용한다.)
- **Normal Impulse**
물리 엔진이 계산한 충돌 임펄스 값으로, 충돌 시 가해진 힘의 방향과 크기가 포함된다.
- **Hit (HitResult)**
충돌 위치, 표면 노멀 방향, 충돌 깊이, 피벗 등 매우 상세한 충돌 정보를 담고 있는 구조체이다.
-

2D 게임의 일반적인 충돌 처리에서는 **Other Actor**와 **Other Comp** 두 핀만으로도 대부분의 판정을 처리할 수 있다.

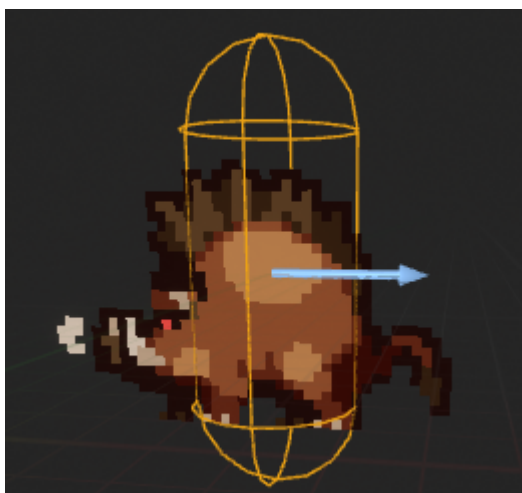
디버그 용도로 **PrintString** 노드를 연결하여 각 출력 핀에 어떤 값이 전달되는지 직접 확인해 보자. 충돌 순간에 출력되는 로그를 보면, 어떤 액터와 어떤 콜리전 컴포넌트가 부딪혔는지 빠르게 파악할 수 있다.



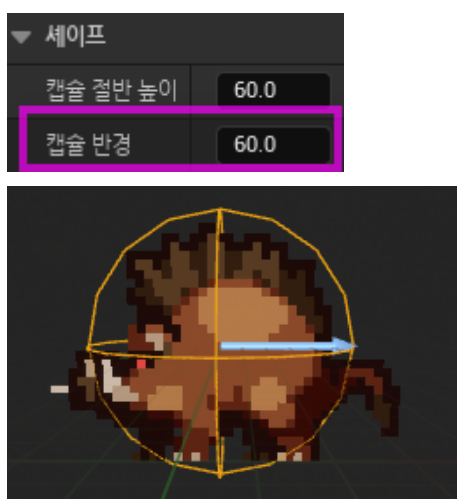
9.2 Mob Collision

9.2.1 Box Collision

Enemy로 사용할 BP2D_MobBoar 블루프린트를 열고 뷰포트에서 보면, 스프라이트가 캡슐 콜리전 영역 밖으로 빠져나와 있는 것을 확인할 수 있다.



이 상태에서 **Capsule Component**를 선택한 뒤, 디테일 패널의 **Shape(세이프)** 항목을 보면 **Capsule Radius(캡슐 반경)** 값이 **Capsule Half Height(캡슐 절반 높이)** 보다 더 크게 입력되지 않는다는 점을 알 수 있다. 즉, 언리얼의 캡슐 콜리전은 반경 $\leq$ 절반 높이를 만족해야 하며, 이 제한 때문에 가로로 너무 넓은 캡슐 모양을 만들 수는 없다.



즉, 언리얼 엔진의 캡슐 충돌체는 반경 $\leq$ 절반 높이 라는 고정 규칙을 가지고 있기 때문에 가로 방향으로 지나치게 넓은 형태의 캡슐을 만들 수 없다.

바로 이 제한 때문에 가로로 긴 형태의 2D 캐릭터(예: Boar, Ship, Dragon 등)는 캡슐만으로 정확한 충돌 영역을 만들기 어렵다.

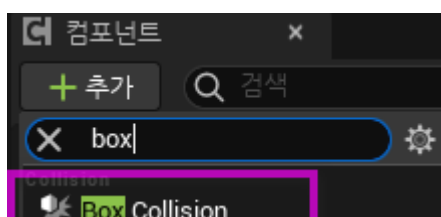
따라서 이러한 캐릭터들은 다음과 같은 방식으로 충돌 구조를 구성한다.

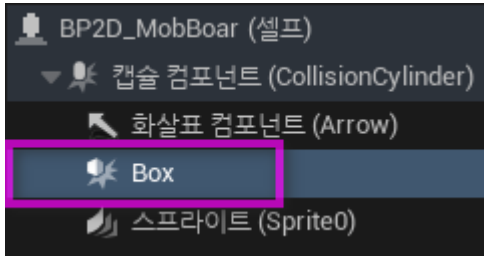
CapsuleComponent → 이동 / 지면 충돌 전용

BoxCollision → 실제 몸체의 넓이, 공격/피격 판정 등 세밀한 충돌 처리

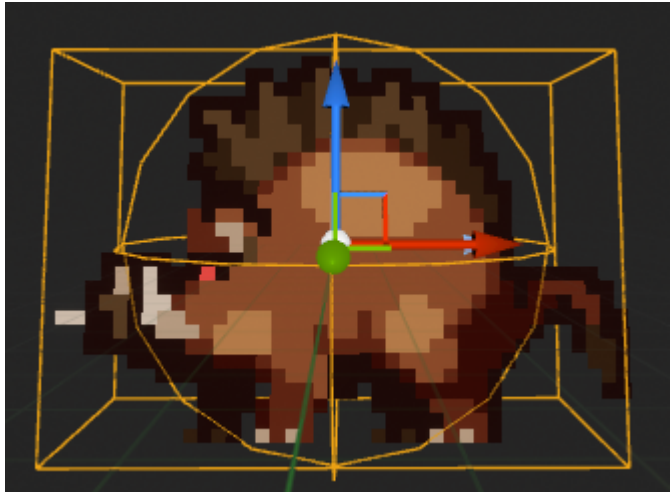
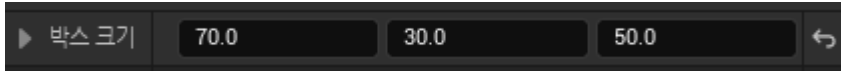
캡슐은 캐릭터 이동을 담당하는 "기본 충돌체" 역할만 수행하고 실제 공격·방어·몸체 영역과 관련된 충돌은 **BoxCollision** 같은 별도의 콜리전으로 처리하는 것이 언리얼에서 2D 캐릭터를 구성하는 가장 안정적이고 일반적인 방식이다.

이제 정확한 충돌 영역을 만들기 위해 **BoxCollision**을 추가해 보자. 컴포넌트 패널에서 "+추가" 버튼을 클릭한 뒤, 검색창에 "Box"를 입력한다. 목록에서 "Box Collision"을 선택하면 새로운 박스 충돌체가 캐릭터에 추가된다





Sprite 트랜스폼 위치를 10, 0, 10 으로 설정하고 Box의 크기는 70, 30, 50으로 설정한다.



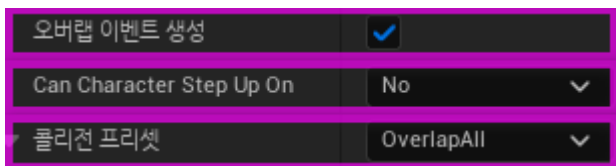
이동과 관련한 Pawn의 충돌을 Block으로 설정하게 되면 2D 게임은 플레이어가 몸이나 NPC에 갇혀서 이동을 할 수 없게 될 수 도 있다. 게임의 진행을 원활하게 하기 위해서 **Overlap** 충돌을 설정하는 경우가 많다.

Character의 캡슐 컴포넌트나 Mob에 추가한 Box collision은 충돌의 디폴트가 Block이다. 이를 **Overlap**으로 변경하자.

컴포넌트 패널에서 BoxCollision 선택

오버랩 이벤트 생성(Generate Overlap Events) 체크 → **Overlap** 이벤트를 발생시키도록 설정

콜리전 프리셋(Collision Presets) → **OverlapAll** (또는 **OverlapAllDynamic**) ← 모든 채널에 대해 Block 없이 **Overlap** 만 발생



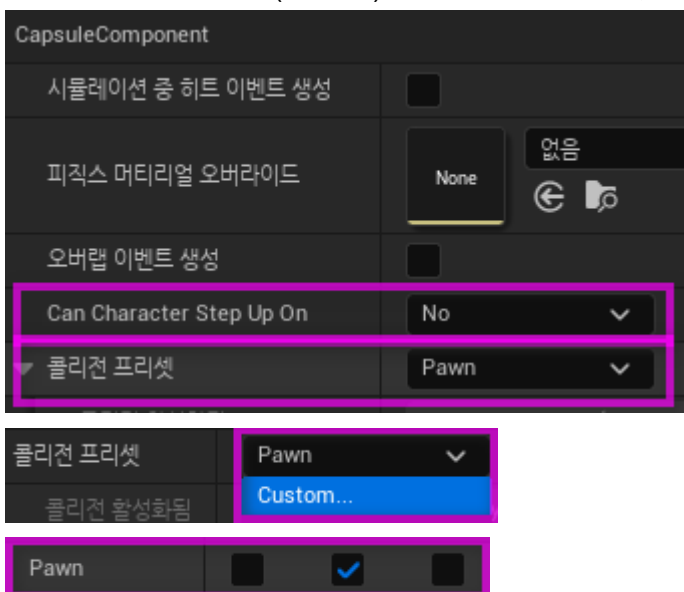
9.2.2 캡슐 콜리전 설정

컴포넌트 패널에서 루트 컴포넌트를 선택한다. BP2D_MobBoar 블루프린트의 경우 루트는 BP2D_MobBoar(셀프)가 해당된다. 이 상태에서 디테일 패널의 검색창에 "Collision" 또는 "콜리전"을 입력하면, 현재 클래스가 기본으로 가지고 있는 **CapsuleComponent**와 스프라이트에 적용된 **Sprite Collision** 관련 속성들을 한눈에 확인할 수 있다.

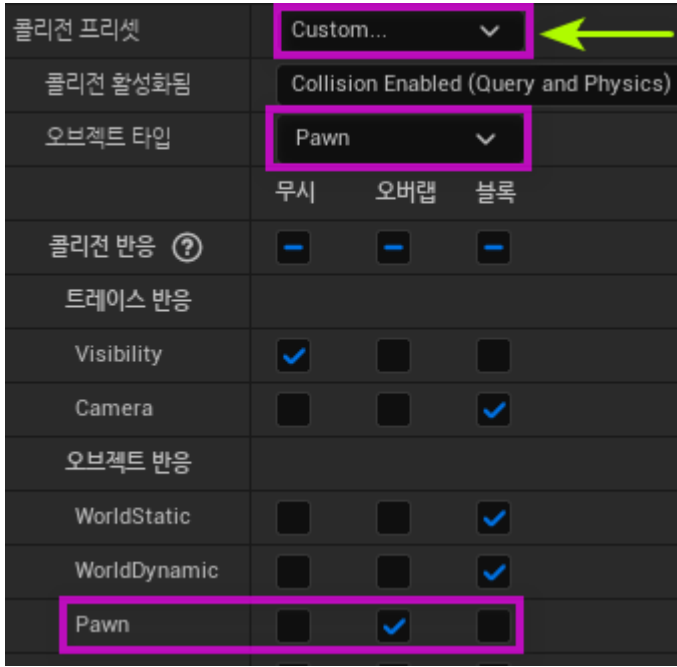
디테일 패널 검색창에 "콜리전"을 두고서 컴포넌트 패널의 캡슐 컴포넌트를 클릭한다. 디테일 패널에서는 캡슐 컴포넌트의 콜리전 프로퍼티만 보일 것이다.

Can Character Step Up on 은 No, 콜리전 프리셋의 리스트에 있는 Pawn 대신 Custom을 선택하면 상속 받은 콜리전 프리셋을 편집할 수 있다.

CapsuleComponent 는 이동과 지면(타일맵) 충돌을 담당하므로, 다른 Pawn 과의 충돌은 **Overlap** 으로만 처리하도록 변경한다.



Capsule Component 선택 ⇒ Collision Preset 은 기존처럼 Pawn 유지. Pawn 채널만 **Overlap** 으로 변경 이렇게 하면 지면/벽과는 Block, 다른 캐릭터(Pawn)끼리는 **Overlap** 이벤트만 발생한다.

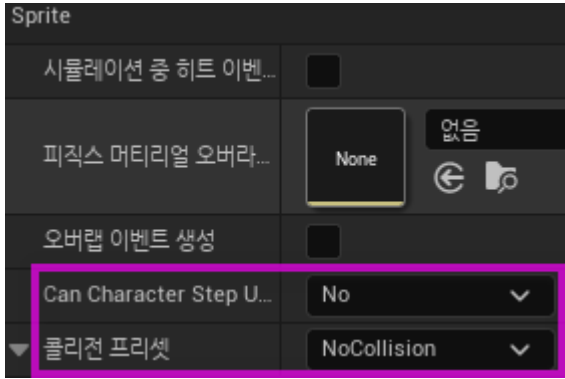


9.2.3 Sprite 컴포넌트 충돌 해제

Sprite 컴포넌트의 충돌은 CapsuleComponent 와 BoxCollision 에서 처리할 것이므로 스프라이트 자체의 충돌 기능은 비활성화하여 사용하지 않도록 한다.

Sprite 컴포넌트 선택

Collision Enabled: NoCollision



9.2.4 최종 Collision 구조 요약

CapsuleComponent

이동, 지면, 벽 충돌 전담

Pawn ↔ Pawn 은 Overlap 처리

BoxCollision

공격/피격/센서 영역 전담 (Overlap 전용)

Sprite Component

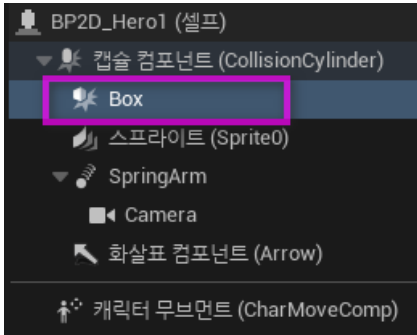
시각적인 표현만 담당

충돌 비활성화

9.3 Character Collision

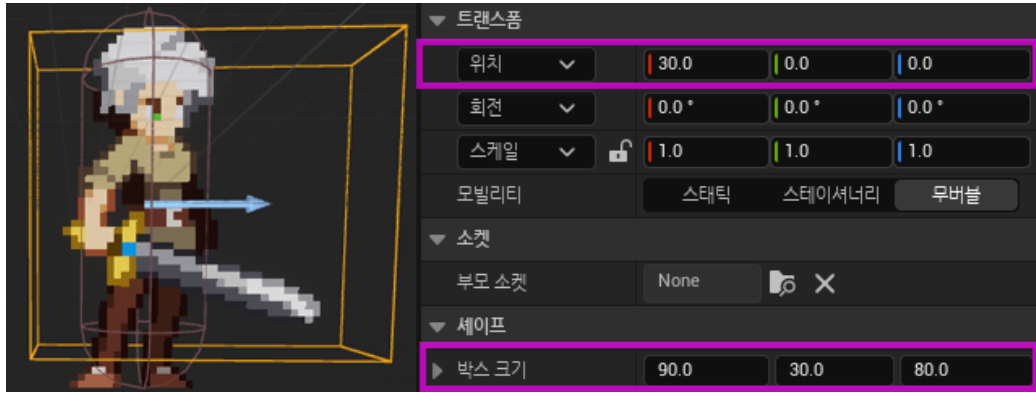
9.3.1 Box Collision

Mob에서 플레이어 캐릭터와 충돌하는 설정을 완료했다. 플레이어 캐릭터의 충돌을 설정하자. Mob 과 동일하게 컴포넌트 패널에서 추가 버튼을 누르고 검색창에 Box 를 입력, Box Collision을 선택한다.

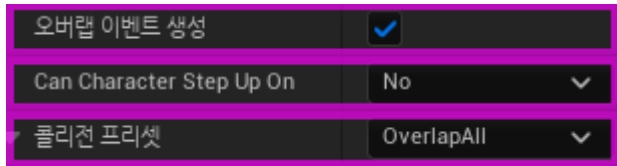


컴포넌트 Box를 선택하고 디테일 패널에서 트랜스폼 위치를 30, 0, 0을 입력한다.

박스의 크기는 90, 30, 80으로 설정한다.



Box 콜리전은 공격을 포함 **Overlap** 판정용으로만 사용할 것이므로 콜리전 프리셋(Collision Presets)을 **OverlapAll**로 변경한다.

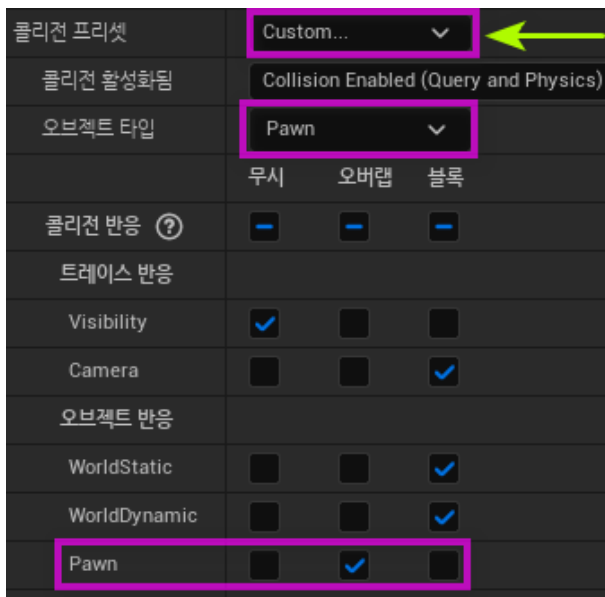


9.3.2 캡슐 콜리전 설정

Can Character Step Up on 은 No로 설정한다.

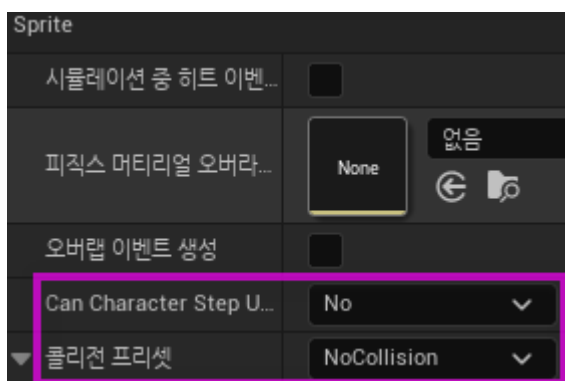


Collision Preset 은 기존처럼 Pawn 유지. Pawn 채널만 **Overlap** 으로 변경해서 Mob 과 동일하게 유지한다.



9.3.3 Sprite 컴포넌트 충돌 해제

플레이어의 **Sprite** 또한 Mob 과 동일하게 비주얼만 담당하므로 충돌을 비활성화한다.



9.4 ActorBeginOverlap / ActorEndOverlap

언리얼의 **Overlap** 이벤트는 크게 액터 단위 **Overlap** 이벤트와 컴포넌트 단위 **Overlap** 이벤트 두 가지로 나뉜다.

액터 단위 **Overlap** 이벤트는 해당 액터에 포함된 모든 충돌 컴포넌트 중 하나라도 겹치면 이벤트가 실행된다.

어떤 컴포넌트에서 발생한 **Overlap**인지 구분하기 어렵다는 단점이 있지만, 단순히 "**Overlap**이 발생했는가?" 만 확인할 때는 매우 간편하다.

컴포넌트 단위 **Overlap** 이벤트는 특정 콜리전 컴포넌트 하나에서 발생하는 **Overlap**만 개별적으로 감지한다.

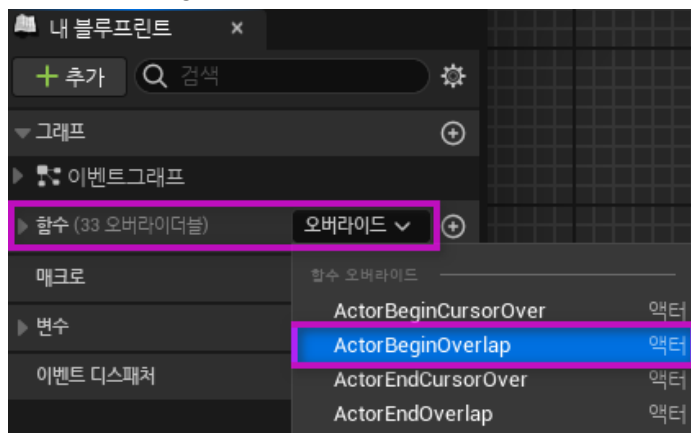
여러 컴포넌트에 대해 각각 **Overlap**을 설정해야 한다는 번거로움은 있지만, 어떤 컴포넌트가 겹쳤는지 명확하게 파악할 수 있어 디테일한 작업이 가능하고 버그를 크게 줄일 수 있다.

현재 학습 중인 기본적인 2D 과정에서는 액터 기반 **Overlap** 이벤트만으로도 충분히 원하는 충돌 처리를 구현할 수 있다.

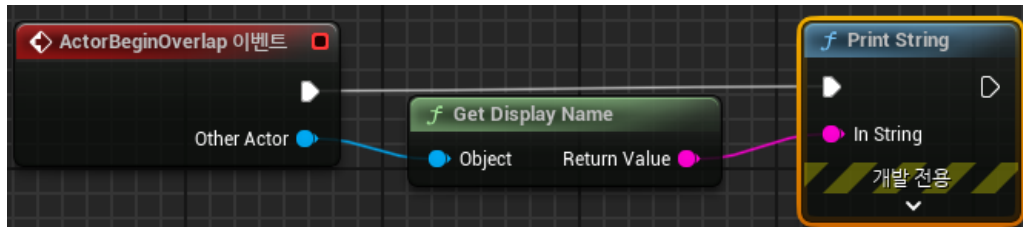
앞서 테스트한 **Capsule Component** 이벤트 블루프린트는 지운다.

내 블루프린트의 함수 섹션 추가 버튼(+)에 마우스를 올려 놓으면 + 왼쪽에 오버라이드(Override) 드롭다운 버튼이 표시 되고, 클릭하면 오버라이드할 수 있는 함수, 이벤트 목록이 표시된다.

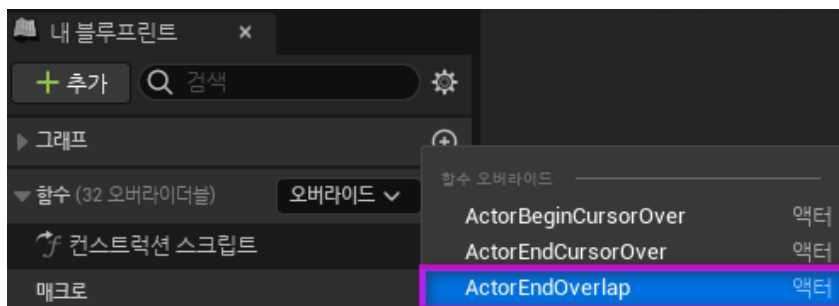
이 목록에서 **ActorBeginOverlap** 을 선택하여 액터 단위의 **Overlap** 이벤트를 블루프린트에 추가한다.



이벤트가 발생하는 지 **Print String**으로 확인하자.



ActorEndOverlap 도 추가한다.



타일 맵에 충돌하거나 **Mob**과 충돌하면 **Overlap** 이벤트가 발생하는 것을 확인 할 수 있다.



9.4.1 Attack Target List

Mob Boar의 **Box Collision** 을 처음에는 **OverlapAll** 로 설정해서, 씬에 있는 모든 충돌 가능한 액터들과 **Overlap** 이벤트가 발생하도록 만들 수 있다.

하지만 전투에서는 플레이어와의 충돌만 감지하면 되므로, **Box** 컴포넌트의 **Collision** 설정을 수정하여 **Pawn** 채널만 **Overlap** 되도록 변경한다. 이때 플레이어 캐릭터와 충돌 대상(**Mob Boar**)의 설정이 서로 맞아야 **ActorBeginOverlap** / **ActorEndOverlap** 이벤트가 정상적으로 호출된다.

즉, 예를 들어 플레이어의 **CapsuleComponent**를 **Object Type = Pawn**으로 설정하고, **Mob Boar**의 **Box Collision**에서 **Pawn** 채널만 **Overlap** 으로 지정하면, 플레이어가 **Mob Boar**의 **Box** 영역에 진입할 때만 **Overlap** 이벤트가 발생하게 된다. (주의: 반드시 "**Object Type**이 완전히 같아야 한다"는 뜻은 아니고, 한 쪽의 **Object Type**에 대해 다른 쪽이 **Overlap** 응답을 하도록 채널 설정이 서로 맞아야 한다. 여기서는 둘 다 **Pawn** 타입을 기준으로 맞춘 경우이다.)

콜리전 프리셋	Custom...
콜리전 활성화됨	Query Only (No Physics)
오브젝트 타입	Pawn
	무시 오버랩 블록
콜리전 반응 ?	☐ ☐ ☐
트레이스 반응	
Visibility	☒ ☐ ☐
Camera	☒ ☐ ☐
오브젝트 반응	
WorldStatic	☒ ☐ ☐
WorldDynamic	☒ ☐ ☐
Pawn	☐ ☒ ☐
PhysicsBody	☒ ☐ ☐
Vehicle	☒ ☐ ☐
Destructible	☒ ☐ ☐

이제 **Attack Target**을 관리하기 위한 리스트를 설정해 보자. 블루프린트의 변수 타입 중 **Set**과 **Map**은 '키(Key)'를 기반으로 항목을 구분하는 자료구조이다. **Set**은 저장되는 인스턴스 자체가 키 역할을 하므로, 대상 액터를 중복 없이 관리해야 할 때 특히 적합하다. 공격 범위 안에 들어온 적 액터들을 추적하기 위해 **Actor** 타입의 **Set** 변수 **targetMob**을 추가하여 **Attack Target** 목록으로 사용하자. 앞서 설명했듯이, 변수를 추가한 후에는 반드시 컴파일을 해야 이후 설정 변경이 가능하다는 점을 잊지 말자. 변수 타입 오른쪽 "컨테이너 선택 버튼(Container Type Selector)"을 클릭해서 단일을 세트로 변경한다.



9.4.1.1 Target List 추가

이제 **Overlap**으로 들어온 적들을 **Target List(Set)**에 등록해 보자.

1. **ActorBeginOverlap** 이벤트 노드 옆에 **targetMob** 변수를 끌어다 놓고, 나오는 메뉴에서 **Get targetMob**을 선택한다.
2. **ActorBeginOverlap** 노드의 **Other Actor** 핀은 **Actor** 타입이므로, **targetMob Set**에 추가할 수 있도록 플레이어가 공격 대상으로 사용할 액터 타입 **PaperCharacter**으로 캐스팅을 진행한다. (예: **BP2D_MobBoar**나 공통 부모 클래스에 **Cast To** 노드를 사용)
3. 캐스팅에 성공한 이후, **targetMob** 출력 핀에서 드래그하여 **Add** 노드를 생성하고, **Add** 노드의 **Item** 입력에 캐스팅된 액터를 연결한다.
이렇게 하면 공격 범위 안으로 겹쳐 들어온 액터가 **targetMob Set**에 추가된다.
4. 위 과정을 따라 노드를 배치하면, **ActorBeginOverlap** → **Cast** → **targetMob.Add**로 이어지는 **Target List** 추가용 블루프린트 구성이 완성된다.

ActorBeginOverlap 이벤트 노드 옆에 **targetMob** 변수를 끌어다 놓고, 나오는 메뉴에서 **Get targetMob**을 선택한다. **ActorBeginOverlap**의 출력 **Other Actor**를 **targetMob** 변수에 추가할 수 있도록 형 변환을 진행한다.

targetMob 출력 핀에서 드래그하여 **Add** 노드를 생성해 겹쳐 들어온 액터를 **Set**에 추가하도록 연결한다.

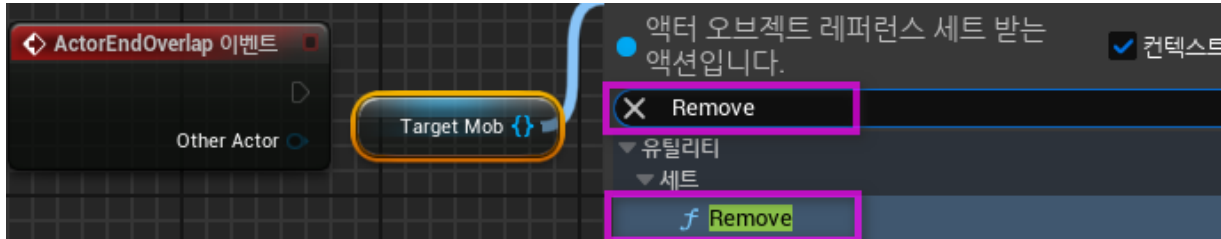


다음과 같이 노드를 완성한다.



9.4.1.2 Target List 제거

Target List 추가와 동일한 과정을 거친다. Set 자료 구조에서 Item 제거는 Remove 노드를 사용한다. ActorEndOverlap 이벤트 노드 옆에 targetMob 변수를 끌어다 놓고, 나오는 메뉴에서 Get targetMob 을 선택한다. targetMob 출력 핀에서 드래그하여 Remove 노드를 생성해 겹쳐 들어온 액터를 Set에 추가하도록 연결한다.



노드를 완성하고 컴파일 한다.



9.5 Attack

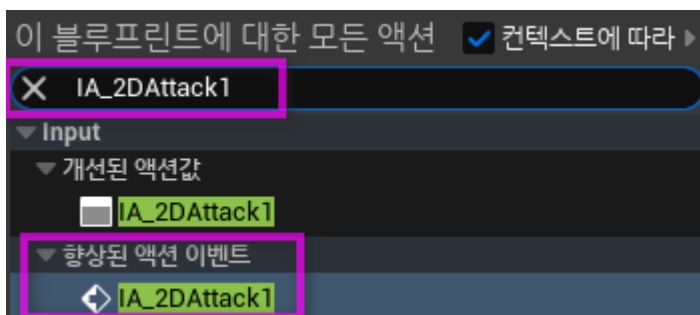
ActorBeginOverlap 이벤트를 이용해 target 리스트(Set) 를 구성했다면, 이제 Attack 애니메이션에서 이 리스트를 활용하여 공격 처리를 진행할 수 있다.

루프가 없는 스프라이트 애니메이션(예: Attack 플립북)은 재생이 완료되면 애니메이션 종료 이벤트를 발생시키는데, 이 시점에 targetMob Set에 저장된 액터들을 순회하며 각 대상에게 ApplyDamage()를 호출해 대미지를 적용한다.

물론 공격을 받은 액터 쪽에서는 Event AnyDamage(또는 필요한 경우 PointDamage, RadialDamage)를 이용하여 전달된 대미지 값을 기반으로 체력 감소와 관련된 로직을 처리하게 된다.

향상된 입력 액션 IA_2DAttack 이벤트를 추가하자

"키보드 T" 로 테스트한 Attack 애니입력하고 키보드 T 이벤트를 선택한다.



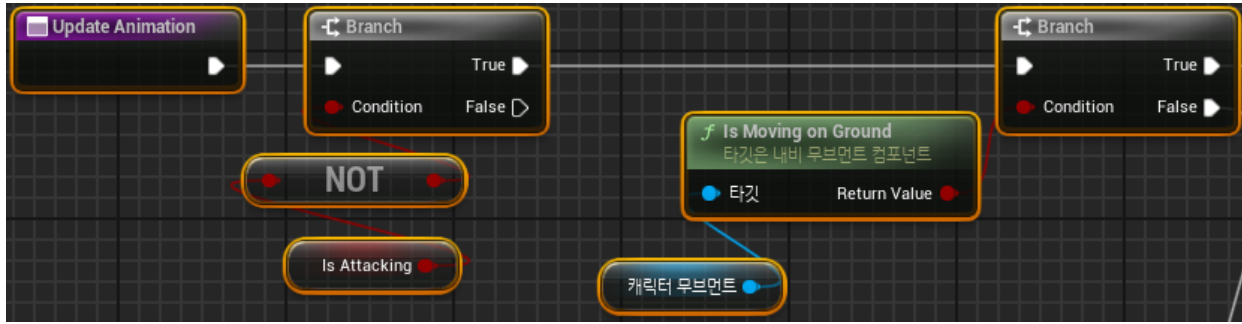
IsAttacking 변수를 추가하고 디폴트는 false로 설정하고 IA_2DAttack1 이벤트가 발생하면 IsAttacking 를 true로 설정한다. 재생이 종료되면 IsAttacking은 false로 설정한다.



Set Flipbook 함수를 호출해서 공격 애니메이션으로 변경한다.

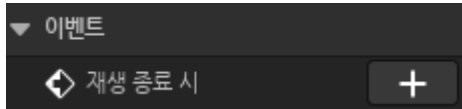


실행하면 공격 애니메이션으로 변경이 되나 화면에는 여전히 Idle 애니메이션이 진행 될 것이다. 원인은 Tick 이벤트 노드가 프레임 진행마다 UpdateAnimation 함수 호출 하는데 이 함수에서는 이동과 속도만을 가지고 애니메이션을 정하고 있기 때문이다. IsAttacking 변수의 값을 가져오는 노드를 Is Moving Ground 노드 앞에 추가해서 간단히 해결할 수 있다.



9.5.1 재생 종료

컴포넌트 패널에서 스프라이트를 선택, 디테일 패널 ⇒ 이벤트를 보면 재생 종료 시가 있다. + 버튼을 눌러서 이벤트를 추가하자.



애니메이션 재생이 완료되면 IsAttacking 변수 값을 false 로 설정한다.



9.5.2 Looping

실행하면 Attack 동작이 연속해서 나온다.

이것은 스프라이트 컴포넌트의 bLooping 디폴트 값이 1로 설정되어 있기 때문이다.

Set Flipbook 노드 뒤에 Set Looping : false를 연결하고 Play from Start 노드를 연결해서 Attack 동작을 실행하자.



9.5.3 SetupFlipbook 함수 생성

Attack 동작은 한 번만 재생되고 끝나는(Non-Loop) 애니메이션이다. 반면 Idle, Run 같은 기본 동작들은 계속 반복 재생(Loop) 되어야 한다. 지금까지 작업을 하고 나면, Idle 이나 Run 애니메이션을 재생할 때마다 매번 Set Flipbook → Set Looping → Play From Start 이 세 개의 노드를 연달아 호출해야 한다는 사실을 자연스럽게 깨닫게 된다.

같은 패턴을 반복해서 쓰지 않기 위해, "애니메이션용 플립북"과 "루프 여부(Looping)"를 인자로 받는 함수를 하나 생성하고, 이 함수 안에서 다음 세 동작을 한 번에 처리하도록 만들자.

- Set Flipbook(어떤 플립북을 재생할지 설정)
- Set Looping(Loop 여부 설정: Idle/Run은 true, Attack은 false 등)
- Play From Start(항상 처음 프레임부터 재생 시작)

이렇게 함수로 묶어 두면, Idle/Run/Attack 같은 애니메이션 전환 시에 플립북 애니메이션 교체를 단 한 번의 함수 호출로 처리할 수 있게 된다.

내 블루프린트 함수 섹션의 추가 버튼(+) 을 클릭하여 새 함수를 생성한다. 함수의 이름은 SetupFlipbook 으로 정한다.

이제 디테일 패널에서 입력(Input) 영역의 추가 버튼(+) 을 눌러 인수를 추가한다.

첫 번째 변수는 이름을 flipbook, 타입을 Paper Flipbook Object Reference 로 설정한다.

입력 채널의 "+" 버튼을 한 번 더 눌러 두 번째 인수를 추가한다.

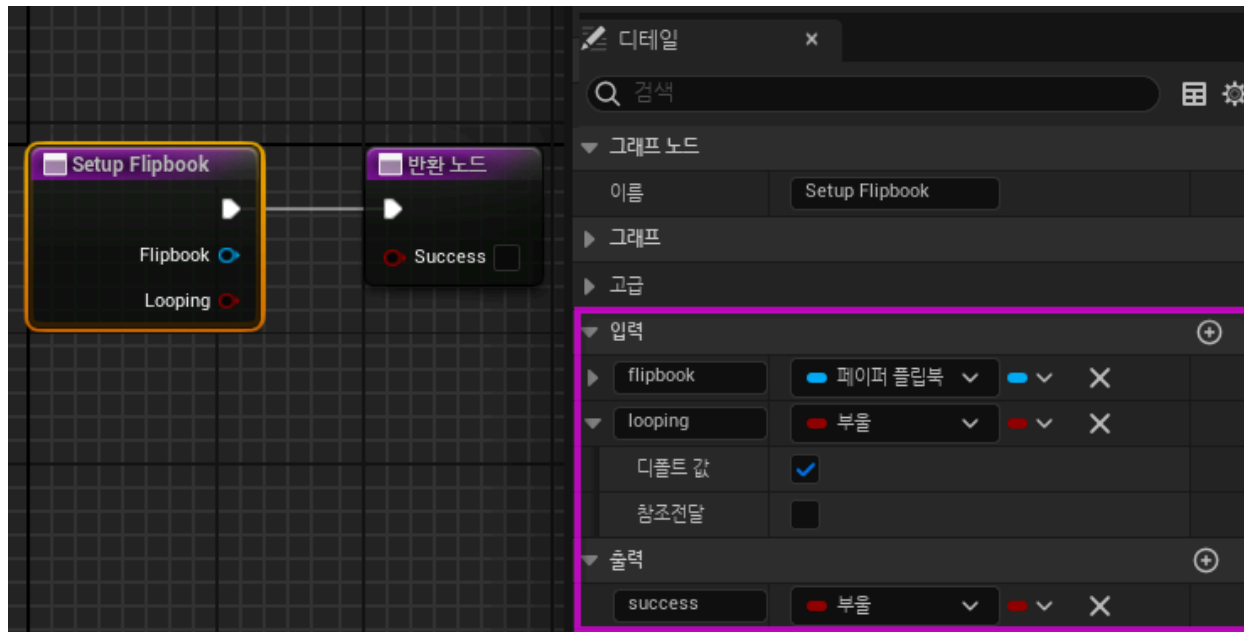
변수 이름은 looping, 타입은 Boolean 으로 정한다.

looping 변수를 확장하여 기본값(Default Value) 을 true 로 설정해 준다.

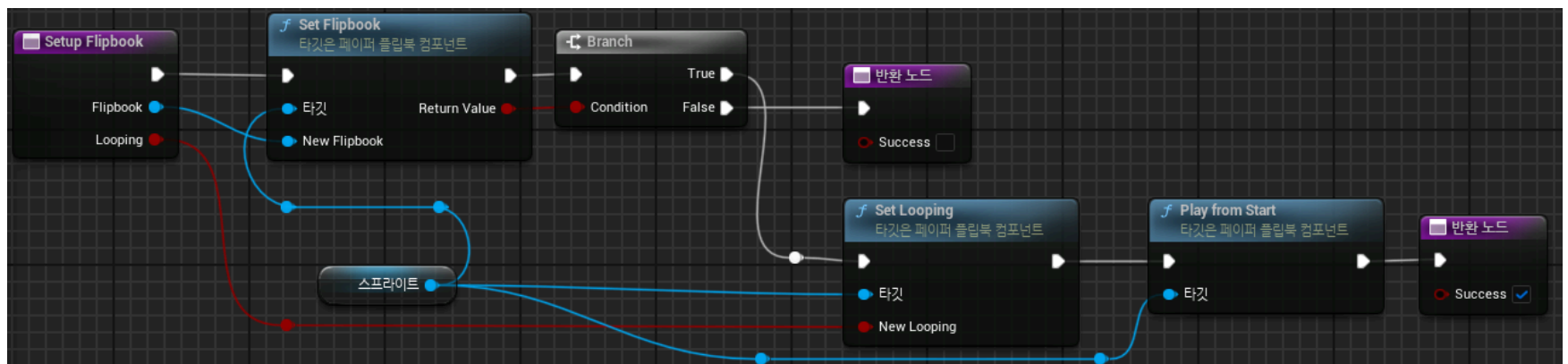
다음으로, 출력(Output) 채널의 추가 버튼(+) 을 클릭하여

이 함수의 반환 값을 나타낼 변수를 추가한다.

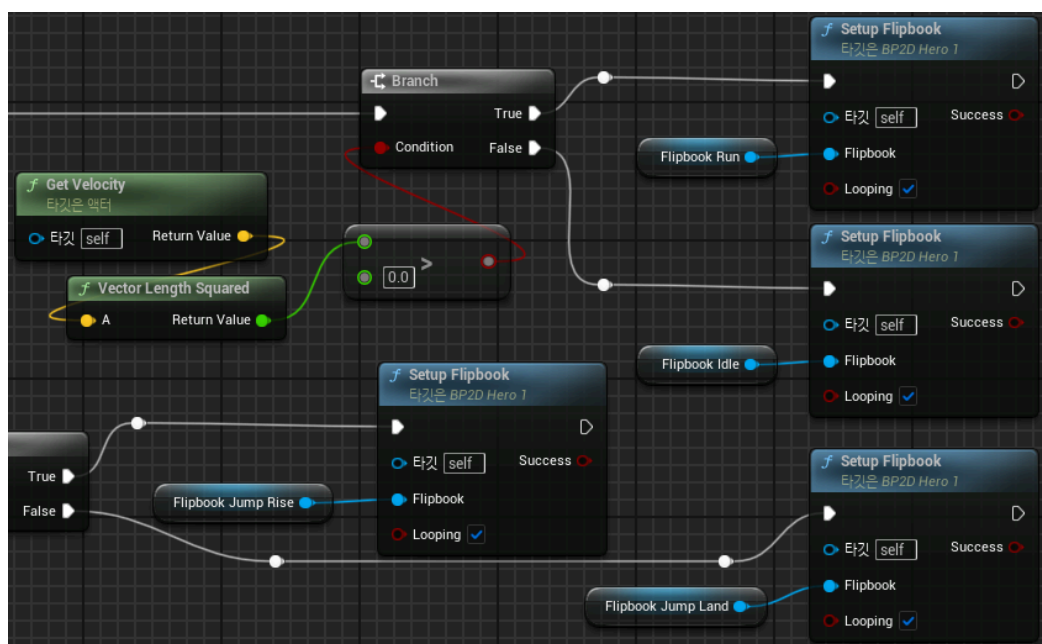
변수 이름은 success, 타입은 Boolean 으로 설정한다.



IA_2DAttack1 입력 액션에서 사용했던 그래프를 참고하여 **SetupFlipbook** 블루프린트를 완성하자. 이 함수는 플립북 설정에 변경이 있을 때만 **true** 를 반환하도록 동작한다.
 이전에는 매번 함수가 호출될 때마다 항상 플립북을 교체했지만, 이제는 현재 재생 중인 플립북과 루프 설정을 확인한 뒤, 변경이 필요할 때만 **Set Flipbook / Set Looping / Play From Start**를 실행하도록 노드를 구성한다.
 만약 이미 같은 플립북이 같은 **Loop** 설정으로 재생 중이라면, 아무 작업도 수행하지 않고 **success = false**를 반환하도록 처리한다.



SetupFlipbook 함수를 사용해서 **Attack, Idle, Run/ Jump** 동작 애니메이션들을 정리하자.

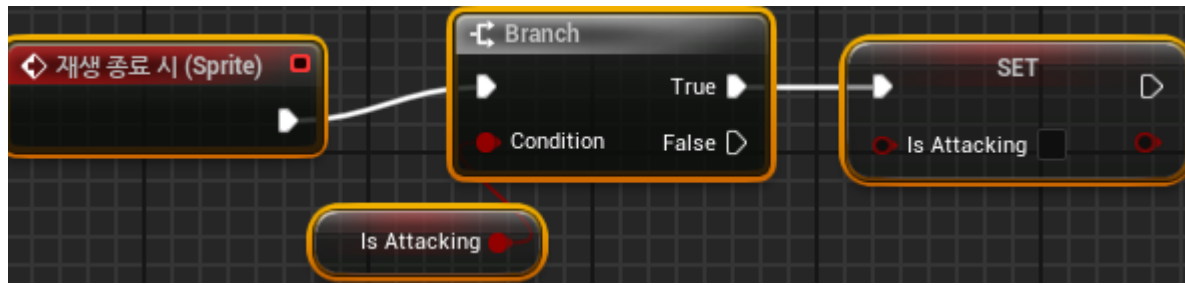


9.5.4 Apply Damage

공격 애니메이션과 **Target List** 구성이 모두 완료되었다.

이제 **Attack** 동작이 재생을 마치는 시점에, **Target List**에 들어 있는 액터들에게 데미지 이벤트를 보내도록 구현해 보자.

데미지는 공격 상태일 때만 전달되어야 하므로, **IsAttacking**이 **True**인 경우에만 **ApplyDamage**를 호출하도록 흐름을 제한해야 한다. 이를 위해 공격 애니메이션 재생 종료 지점에서, **IsAttacking** 변수를 검사하는 **Branch** 노드를 추가하고, **IsAttacking == True**일 때만 다음 노드(**ForEach**로 **Target List**를 순회하며 **Apply Damage** 호출)가 실행되도록 블루프린트를 다음과 같이 수정한다.



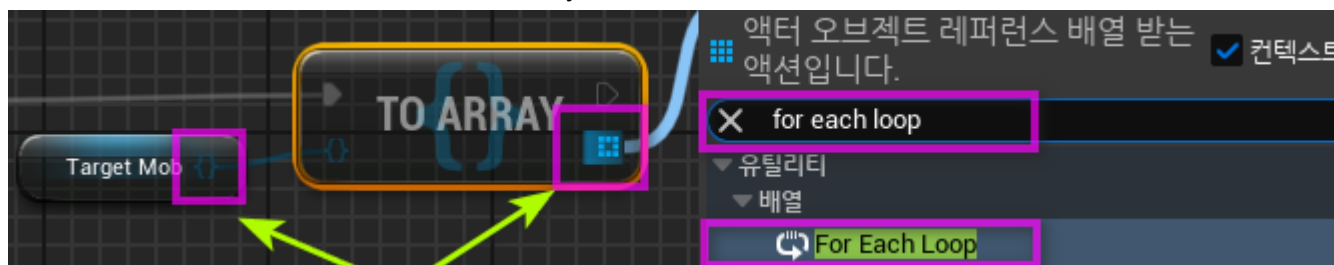
블루프린트에서 **Set**을 직접 **For Loop**로 순회하는 기능은 제공되지 않는다.

따라서 **Set**에 저장된 타겟들을 순회하려면, 먼저 **Set**을 **Array**로 변환한 뒤 **Array**를 기준으로 **For Each Loop**를 사용해야 한다.

재생 종료 시 노드 옆에 **targetMob** 변수를 끌어다 놓고 **Get targetMob**을 선택한다.



targetMob 노드의 출력 핀에서 드래그하여 **To Array** 노드를 호출한다. 이렇게 하면 **Set**에 들어 있던 액터들이 임시 **Array**로 변환된다.



To Array 노드의 출력 핀에서 드래그하여 **For Each Loop** 노드를 생성한다.

For Each Loop 노드의 **Array Element** 핀에는 **Target List**에 저장된 각 액터(공격 대상)가 순서대로 넘어온다.

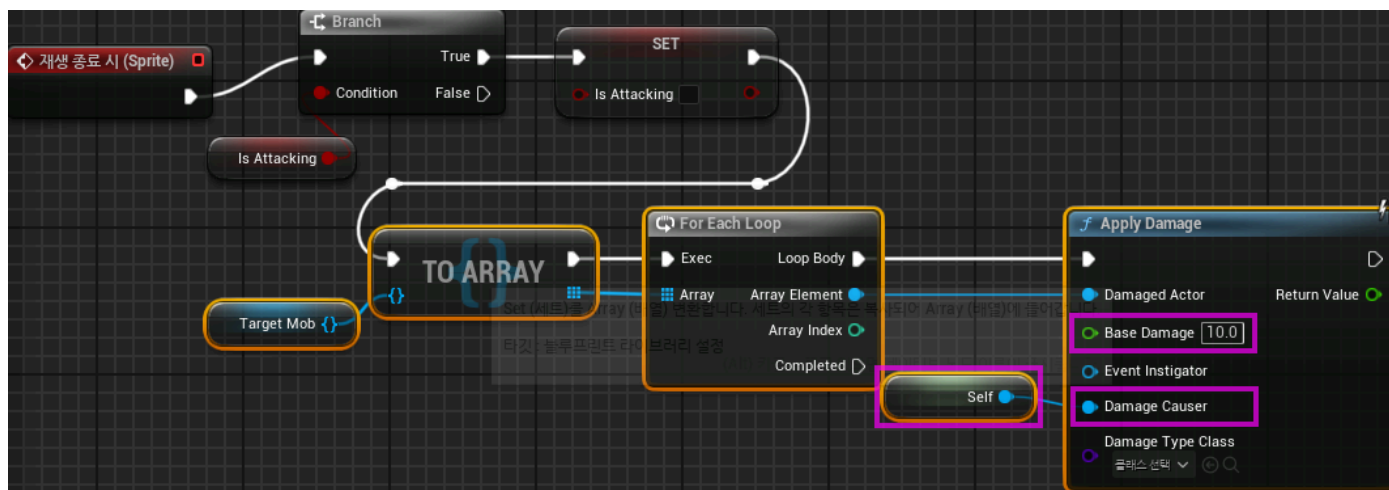
Array Element 핀에서 드래그하여 **Apply Damage** 함수를 호출하고, **Damaged Actor** 입력에 **Array Element**를 그대로 연결한다.

Base Damage 값에는 실제로 줄 데미지 값을 지정한다.

이 때 주의할 점은, **Base Damage** 값이 0.0이면 **AnyDamage** 이벤트가 호출되지 않아 상대방이 데미지를 전혀 받지 못한다는 것이다.

따라서 최소한 0.0 보다 큰 값을 설정해야 피격 측 **Actor**의 **Event AnyDamage**가 정상적으로 실행된다.

Damage Cause에 **Self**를 연결해서 송신 주체가 누구인지 정한다.



For Each Loop Completed 핀에 앞서 끊었던 **Set IsAttacking** 노드를 연결한다. 컴파일하고 노드를 정리한다.

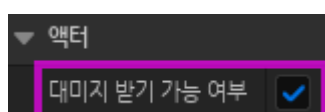
9.5.5 AnyDamage 이벤트

ApplyDamage를 수신할 액터에서 이 이벤트를 처리하기 위한 **AnyDamage** 이벤트를 구현하자.

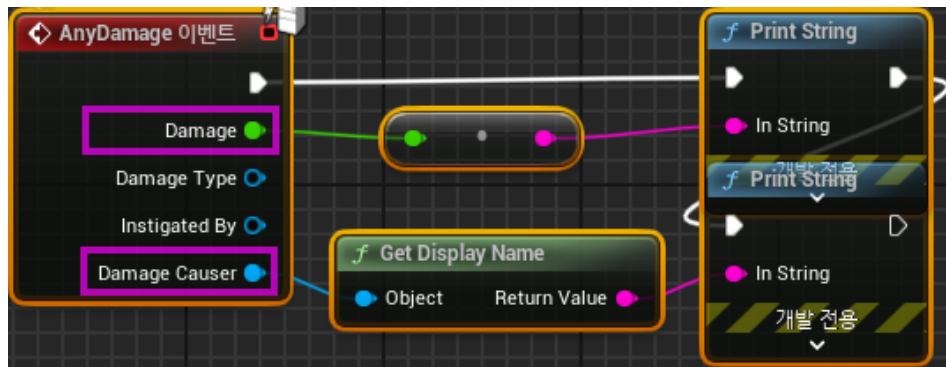
BP2D_MobBoar 블루프린트를 열고, 내 블루프린트(My Blueprint) 패널에서 함수 추가 버튼(+) 옆에 있는 오버라이드(Override) 드롭다운 버튼을 클릭한 뒤 목록에서 **AnyDamage**를 선택하여 데미지 수신 이벤트를 추가한다.



데미지 이벤트를 설정했음에도 이벤트가 안온다면 디테일 패널에서 액터 ⇒ 데미지 받기 기능 여부가 **true**인지 확인 한다.



Print String으로 AnyDamage 이벤트가 오는지 확인한다.



10. Effect

이전 장에서는 플레이어의 공격 애니메이션과 **Target** 액터의 **Damage** 이벤트 처리 과정을 구현하였다. 이 단계까지 완료했다면, **Mob AI**를 제외한 전투의 기본 구조는 대부분 갖춰진 상태이다.

이제부터는 플레이어가 공격을 수행할 때와 몹 **Boar**가 피해를 입을 때, 그 상황에 맞는 애니메이션으로 자연스럽게 전환되도록 애니메이션 교체 과정을 구현해 보자.

타격 이펙트(**Hit Effect**)는 별도의 이펙트 제작 없이도, 엔진에서 제공하는 **Content/StarterContent/Blueprints/Blueprint_Effect_Explosion** 을 활용해 간단히 구현할 수 있다.

10.1 Spawn Actor

Spawn Actor 노드는 게임 실행 중 새로운 액터(**Actor**)를 월드에 생성하는 기능을 제공한다.

블루프린트와 **C++** 모두에서 매우 자주 사용되는 핵심 기능 중 하나로, 탄환, 이펙트, 적 캐릭터, 상호작용 오브젝트처럼 런타임에 동적으로 생성해야 하는 모든 객체에 활용된다.

Spawn Actor 노드는 다음과 같은 세 가지 주요 구성 요소가 있다.

1) SpawnActor Class

생성할 액터의 종류(클래스)를 지정한다.

예를 들어 **Blueprint_Effect_Explosion**, **BP_Bullet**, **BP_Energy**, **BP_Item** 등이 이에 해당한다.

2) Spawn Transform

생성되는 액터의 위치(**Location**), 회전(**Rotation**), 크기(**Scale**) 를 정의한다.

보통 **Make Transform** 노드 또는 기존 액터의 위치·회전을 사용해 값을 구성한다.

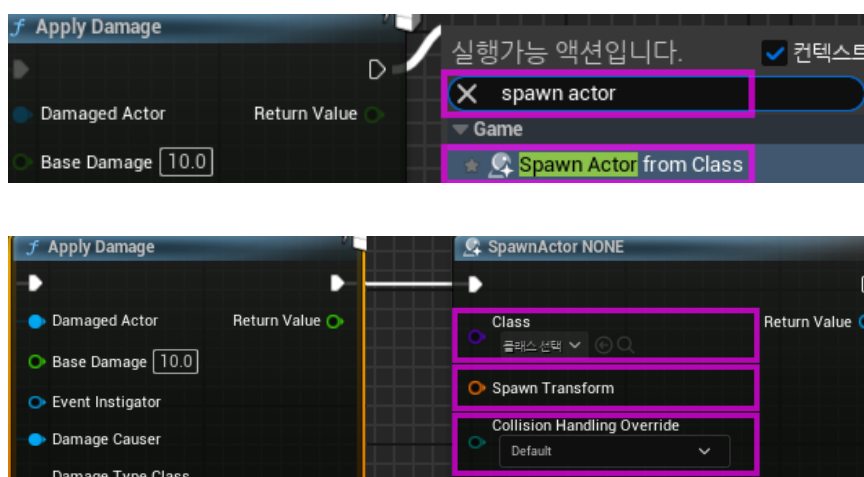
3) Collision Handling Override

액터를 스폰할 때 충돌이 발생하면 어떻게 처리할지 결정하는 옵션이다.

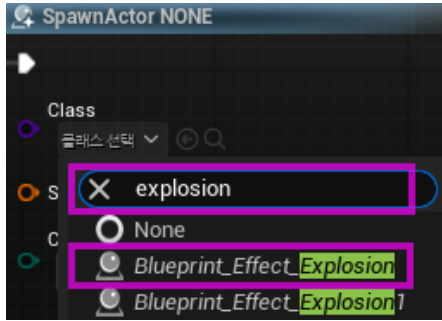
- **Always Spawn**
충돌 여부와 관계없이 무조건 생성한다. 이펙트처럼 충돌이 필요 없는 경우에 자주 사용된다.
- **Adjust if Possible but Always Spawn**
충돌이 발생하면 가능한 위치로 조정하여 생성한다.
- **Do Not Spawn if Colliding**
생성 위치에서 충돌이 감지되면 스폰하지 않는다.
캐릭터 생성처럼 부정확한 위치 생성이 위험한 경우에 적합하다.

엔진에서 제공하는 **Content/StarterContent/Blueprints/Blueprint_Effect_Explosion** 을 활용해 간단하게 타격 이펙트를 구현해보자.

Apply Damage 노드의 출력 실행 핀에서 마우스로 드래그해서 새 노드 배치를 실행하고 액션을 검색창에 **Spawn Actor**에서 새로운 노드를 생성한다.



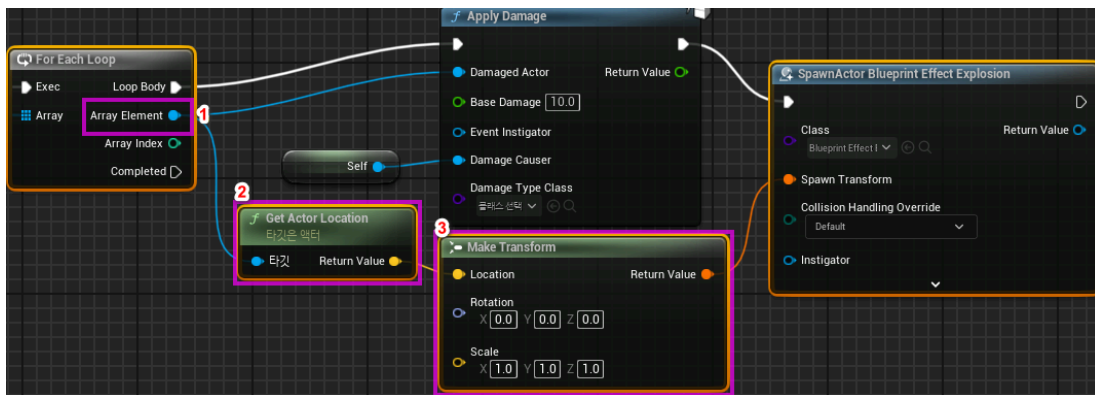
Spawn Actor Node의 **Class** 검색창에 **explosion**을 입력하면 빠르게 **Blueprint_Effect_Explosion**를 선택할 수 있다.



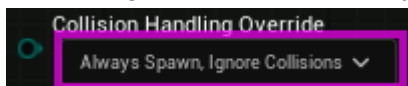
Transform이 없으면 실행해도 에러를 발생한다. Target의 위치만 필요하므로 액터로 구성된 Array Element에서 위치(Locaiton)를 구한다.



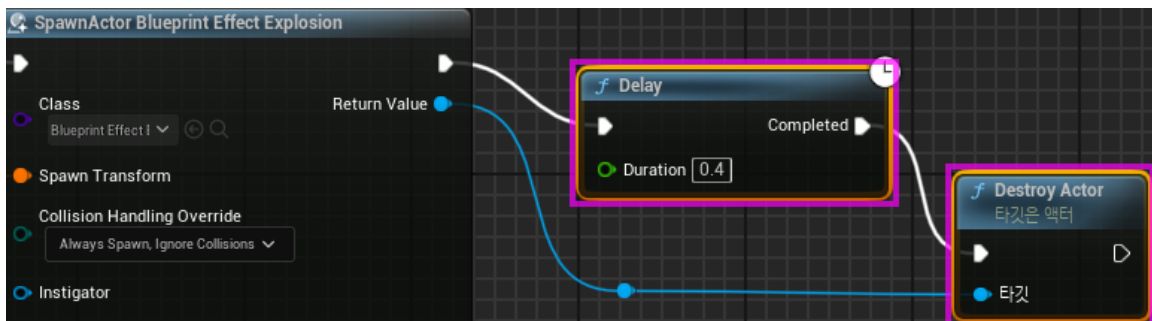
Make Transform 함수를 사용해서 앞서 구한 액터의 위치와 함께 새로운 Tranform을 만들고 Spawn 액터에 연결한다.



Collision Handling Override 값을 Always Spawn 으로 변경하여, 충돌 여부와 관계없이 이펙트가 항상 생성되도록 설정한다.



또한, 재생된 Explosion 이펙트가 메모리에 계속 남아 있지 않도록 일정 시간이 지나면 자동으로 제거되도록 처리해야 한다. 이를 위해 Delay 노드를 사용해 지정한 시간만큼 대기한 뒤, Destroy Actor 노드를 사용하여 Spawn한 이펙트를 소멸시키도록 구성한다. 이렇게 하면 Hit 이펙트는 생성 후 일정 시간이 지나 자연스럽게 사라지며, 불필요한 액터가 누적되는 문제도 예방할 수 있다.



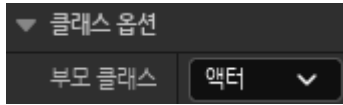
컴파일을 진행하고 실행해 보면, 공격 애니메이션 재생이 끝나는 시점에 이펙트가 나타나는 것을 확인할 수 있다.



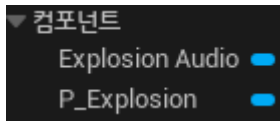
10.2 Explosion 분석

2D 스프라이트 기반 이펙트를 직접 만들어 볼 것이다. 앞에서 타격 이펙트로 임시 사용했던 Blueprint_Effect_Explosion 블루프린트를 먼저 분석한 뒤, 이를 참고하여 비슷한 구조로 구현해 보자.

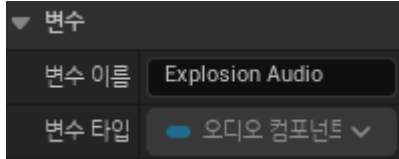
Blueprint_Effect_Explosion 을 열어 보면, 부모 클래스가 Actor 기반 클래스인 것을 알 수 있다.



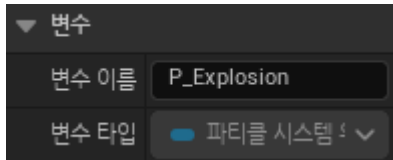
내 블루프린트의 변수 섹션을 보면 사운드를 담당하는 **Explosion Audio** 와 비주얼을 담당하는 **P_Explosion** 두 변수가 정의되어 있다.



디테일 패널에서 각 변수의 타입을 확인해 보면, **Explosion Audio** 의 타입은 오디오 컴포넌트이고,



P_Explosion 의 타입은 파티클 시스템 컴포넌트로 설정되어 있음을 알 수 있다.

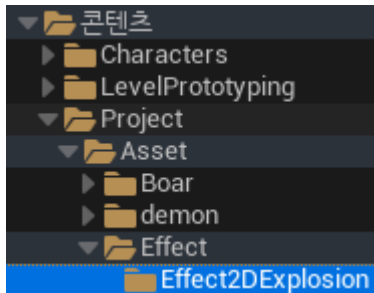


여기서 사용하는 파티클 표현을 스프라이트(또는 플립북) 로 교체하면, 동일한 구조를 활용하여 **2D 전용 스프라이트 이펙트**를 구현할 수 있다.

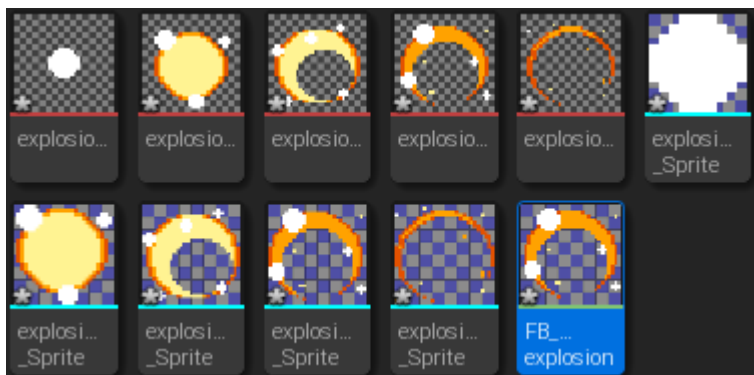
10.3 2D Explosion 액터

Blueprint_Effect_Explosion 을 응용해서 2D 스프라이트 기반 **Effect** 을 생성하자.

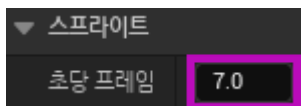
먼저 2D Effect를 저장할 "콘텐츠/Project/Asset/Effect/Effect2DExplosion" 폴더를 생성한다.



지금까지의 과정을 충실히 따라왔다면, 플립북을 만드는 과정은 어렵지 않을 것이다. 이제 **Explosion Effect**를 위한 플립북을 만들어 보자. [Effect_ExplosionSprite.zip](#) 파일을 **Effect_ExplosionSprite.zip** 파일을 다운로드한 뒤, 압축을 풀고 포함된 **PNG** 파일들을 언리얼 프로젝트에 임포트한다. 임포트한 텍스처에 **Paper2D**용 텍스처 세팅을 적용한 다음, 각 텍스처로부터 스프라이트를 생성한다. 이제 생성된 스프라이트들을 이용해 **FB_Explosion** 이라는 이름의 플립북(**Paper Flipbook**) 에셋을 만들고, 연속적으로 재생되도록 프레임을 배치하여 폭발 이펙트용 플립북을 완성한다.

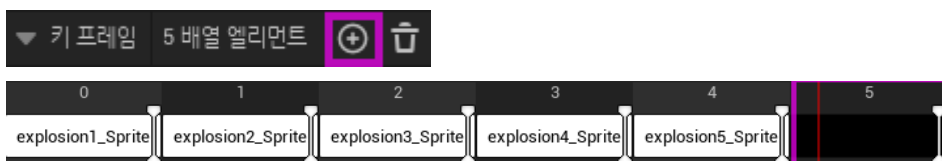


FB_explosion을 열고 디테일 패널에서 스프라이트의 초당 프레임을 **6.0~10.0** 중에 적당한 값을 선택한다.

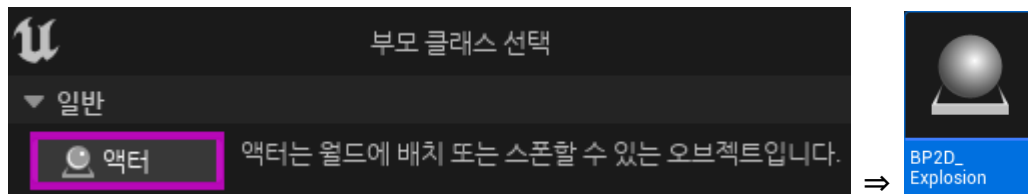


Explosion처럼 1회성으로 재생되고 소멸하는 액터는, 제거되기 직전에 아주 짧은 지연이 발생할 수 있다. 이로 인해 플립북 재생이 끝난 뒤, 마지막 스프라이트가 잠시 화면에 남아 보이는 문제가 생길 수 있다. 이 문제를 간단히 해결하는 방법은 플립북의 마지막에 빈 프레임을 하나 추가하여 화면에 아무것도 표시되지 않도록 만드는 것이다.

FB_Explosion 플립북을 연 뒤, 디테일 패널에서 키프레임 추가(+) 버튼을 눌러 새 프레임을 하나 넣고, 해당 프레임을 빈 상태(스프라이트 없음) 로 둔다. 이렇게 하면 재생 마지막 순간에 화면이 비워져, 이펙트가 자연스럽게 사라지게 된다.



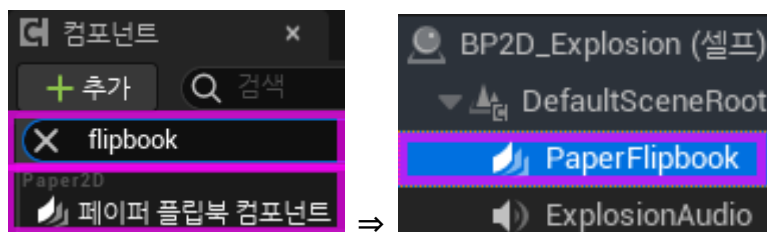
Effect2DExplosion 폴더의 빈 공간을 우클릭하여 **Blueprint Class** 를 선택하고, 일반 카테고리에서 **Actor** 를 선택한다. 이 블루프린트의 이름은 **BP2D_Explosion** 으로 설정한다.



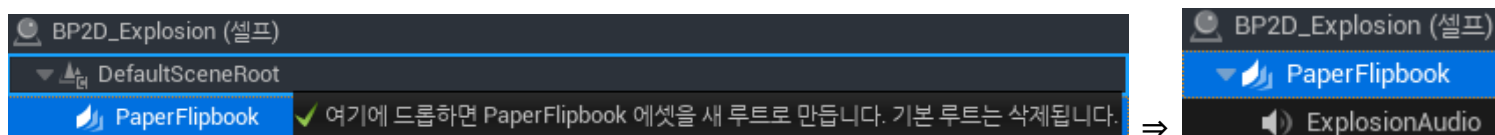
이제 BP2D_Explosion 을 열어 컴포넌트를 구성하자. 먼저 컴포넌트 패널에서 추가("+") 버튼을 누르고, 검색창에 **audio** 를 입력하여 **Audio Component** 를 선택한다. 이 컴포넌트의 이름은 **ExplosionAudio** 로 변경한다.



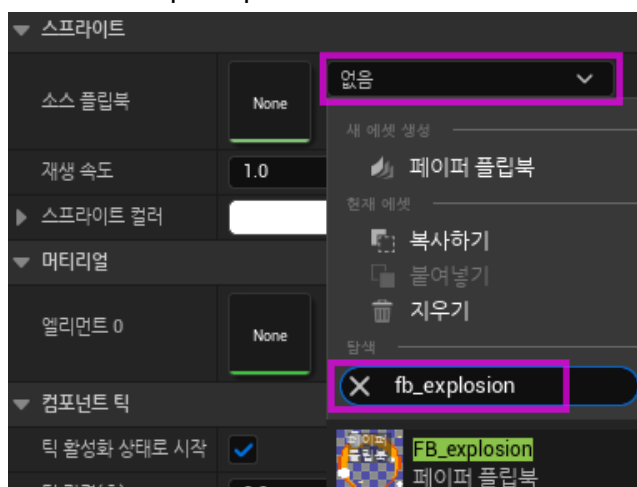
다음으로 플립북 컴포넌트를 추가한다. 마찬가지로 컴포넌트 패널에서 추가("+") 버튼을 눌러 검색창에 **flipbook** 을 입력하고, **Paper Flipbook Component** 를 선택한다. 컴포넌트 이름은 **PaperFlipbook** 으로 정한다.



참고로 **DefaultSceneRoot**를 사용할 필요가 없다고 판단될 수도 있다. 추가한 **PaperFlipbook** 컴포넌트를 마우스로 드래그하여 **DefaultSceneRoot** 위로 옮기면, **DefaultSceneRoot**가 자동으로 제거되고 **PaperFlipbook**이 새로운 루트 컴포넌트(**Root Component**) 가 된다.



컴포넌트 패널의 **PaperFlipbook**을 선택하고 디테일 패널의 스프라이트의 소스 플립북을 **FB** 새로운 루트 컴포넌트(**Root Component**) 가 된다.

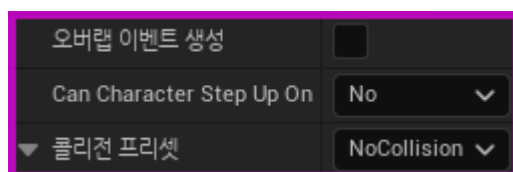


이펙트 액터는 충돌 처리가 전혀 필요 없다.

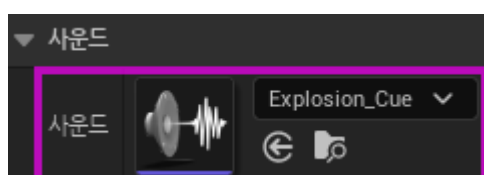
따라서 **PaperFlipbook** 컴포넌트의 충돌 관련 설정을 모두 비활성화해 주어야 한다.

- Generate Overlap Events(오버랩 이벤트 생성) : False
- Can Character Step Up On : No
- Collision Preset(콜리전 프리셋) : NoCollision

이렇게 설정하면 이펙트는 물리나 충돌에 관여하지 않고, 화면에 표시되는 역할만 수행하게 된다.



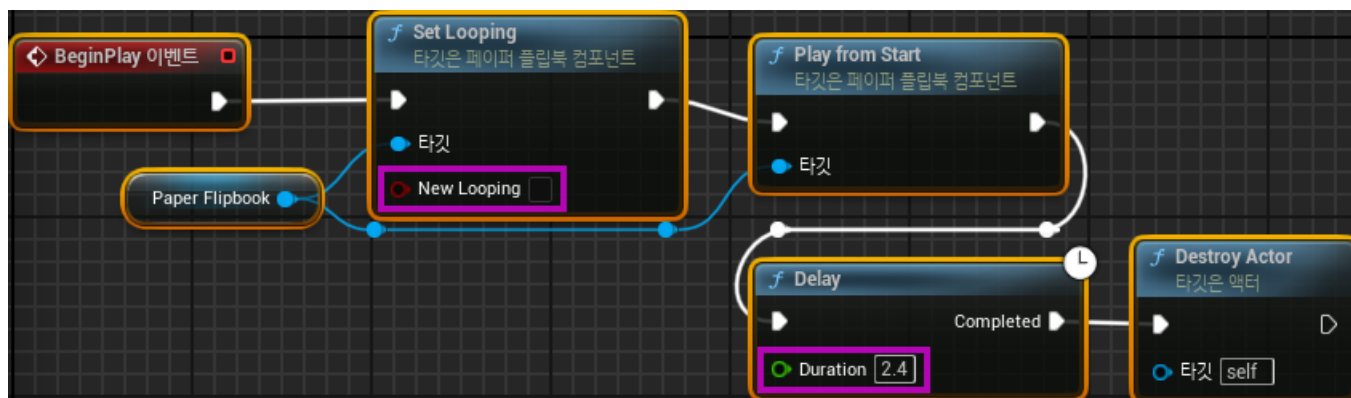
ExplosionAudio 설정은 매우 간단하다. 언리얼 엔진의 **Content/StarterContent/Audio/Explosion_Cue** 에 이미 적절한 폭발 효과음이 준비되어 있으므로 이를 사용하면 된다. **BP2D_Explosion** 블루프린트를 연 상태에서 컴포넌트 패널에서 **ExplosionAudio** 컴포넌트를 선택한 뒤, 디테일 패널의 **Sound** 항목에서 **Explosion_Cue** 를 검색하여 선택한다. 이렇게 하면 이펙트가 재생될 때 **ExplosionAudio** 컴포넌트를 통해 폭발 사운드가 함께 출력된다.



이펙트 용 블루프린트에 스크립트를 추가하여 동작을 마무리하자. 이전에는 **Explosion** 이펙트를 사용할 때마다, **Spawn → Delay → Destroy Actor** 를 호출하는 흐름을 매 번 구성해야 했다. 지금 구성하는 이펙트 블루프린트는 연출 전용 액터이므로, 블루프린트 내부에서 스스로 재생 후 소멸하도록 만들어 두면, 어디에서 스폰하더라도 같은 로직을 반복해서 작성할 필요가 없다.

- 1) **BeginPlay**에서 모든 것을 처리할 예정이라 **BP2D_Explosion** 블루프린트에서 **BeginPlay** 이벤트만 남기고 나머지 이벤트는 모두 삭제한다.
- 2) 컴포넌트 패널의 **PaperFlipbook** 을 그래프에 끌어다 놓아 **Get** 노드를 만든 뒤, 이를 타겟으로 **Set Looping** 함수를 호출하고 **New Looping** 값을 **false** 로 설정한다.
- 3) 같은 **PaperFlipbook**을 타겟으로 **Play from Start** 함수를 호출하여, 이펙트가 시작과 동시에 한 번만 재생되도록 한다.
- 4) 2.4 초 후에 파괴도록 **Delay** 노드를 추가하고 **Duration** 값을 2.4로 설정한다.
- 5) **Delay**의 **Completed** 핀에서 **Destroy Actor** 노드를 연결하고, 타겟은 **self**로 둔다. 컴파일을 진행하고 작업을 완료 한다.

이렇게 구성하면, 이펙트는 스폰되는 순간 한 번 재생된 뒤 2.4초 후 자동으로 파괴되어, 여러 곳에서 이펙트를 호출하더라도 별도의 **Delay/Destroy** 설정을 반복할 필요가 없다.



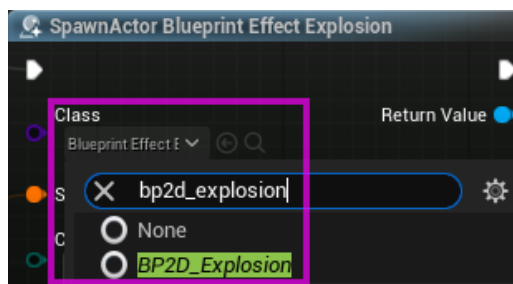
10.3.1 2D Explosion 적용

BP2D_Hero1의 블루프린트 스크립트에서는 스프라이트 재생이 종료될 때 **SpawnActor Blueprint Effect Explosion** 노드를 통해 이펙트를 생성했었다. 이제 임시로 사용했던 **Explosion** 대신, 우리가 만든 2D 이펙트를 사용하도록 변경해 보자.

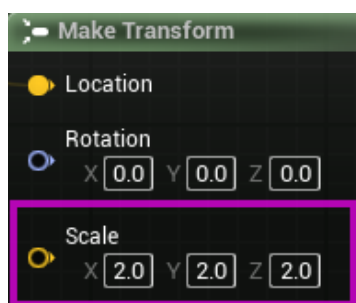
SpawnActor 노드의 **Class** 입력 핀을 클릭하고 검색창에 **BP2D_Explosion** 을 입력한 뒤, 리스트에서 **BP2D_Explosion** 을 선택한다.

이렇게 하면 공격 애니메이션 종료 시 기존 **Explosion** 이펙트가 아닌, 직접 제작한 2D 폭발 이펙트가 스폰되어 재생되게 된다.

Spawn Actor 다음에 있던 "Delay" 와 "Destroy Actor" 함수는 **BP2D_Explosion** 내부에서 구현 되었으므로 삭제한다.



컴파일하고 실행하면 이펙트의 크기가 작다고 생각되면 **Make Transform** 함수의 **Scale**을 올리면 간단히 해결된다.



10.3.2 Effect 동기화

공격 이펙트를 플립북 재생이 모두 끝난 뒤에 실행하다 보니, 실제 공격 동작과 이펙트의 타이밍이 맞지 않는 모습을 확인할 수 있다.

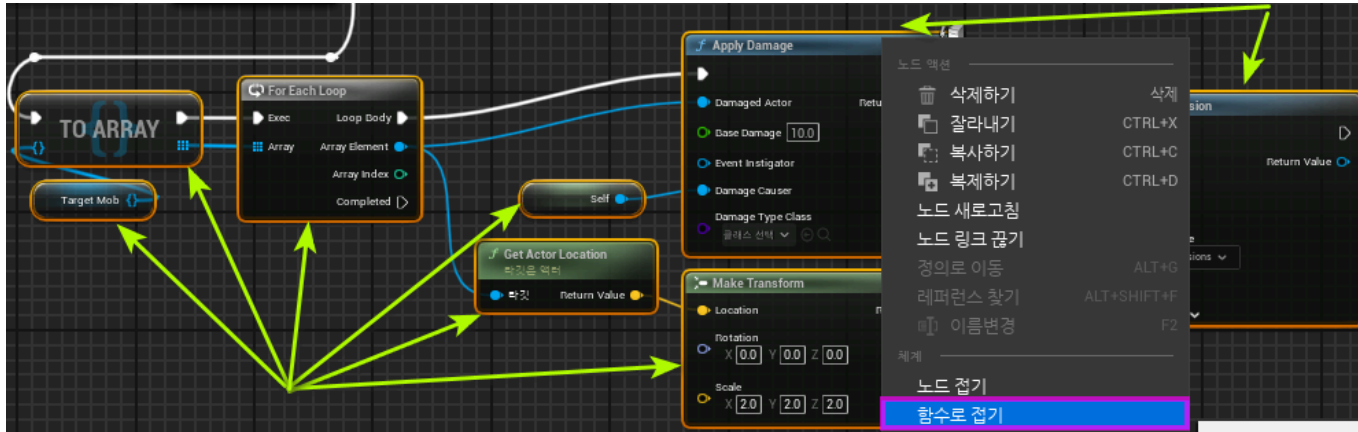
현재 사용하는 **Paper2D Flipbook** 자체로는 스프라이트 애니메이션 트랙의 특정 프레임에서 이벤트를 직접 발생시키는 기능이 없다.

따라서 이 구조에서는 공격 애니메이션을 전환할 때 지연 시간(**Delay**)을 수동으로 조절하여, 원하는 타이밍에 이펙트와 대미지 처리가 실행되도록 맞추는 수밖에 없다.

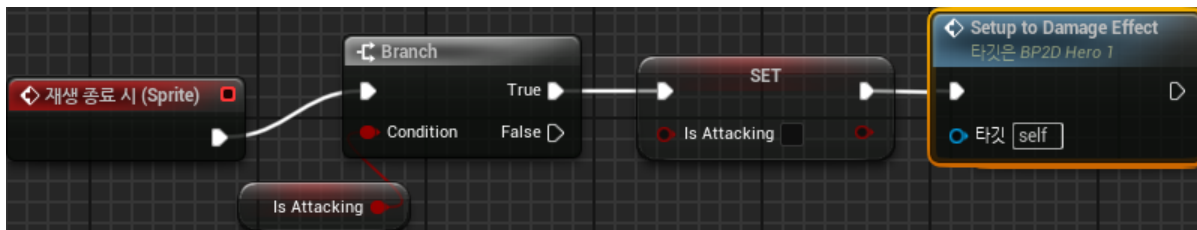
이후에 배울 **PaperZD**는 애니메이션 트랙에서 **노티파이(Notify)**를 직접 설정하고 발생시킬 수 있는 기능을 제공하므로, 공격 타이밍과 이펙트, 사운드, 대미지를 보다 정확하게 동기화할 수 있는 보다 적절한 도구가 될 것이다.

Damage 관련 노드를 **IA_2D_Attack1** 입력 액션에 직접 연결해도 되지만, 이미 구현된 노드가 많아 그래프가 복잡해진다. 그래프를 정리하기 위해 이 부분을 함수로 분리하자.

애니메이션 관련 노드들을 선택할 때는 **Shift** 또는 **Ctrl** 키를 누른 상태에서 마우스로 클릭하면 여러 노드를 동시에 선택할 수 있다. 이렇게 원하는 노드들을 모두 선택한 뒤, 선택된 노드 중 하나를 마우스 오른쪽 버튼으로 클릭하면 “함수로 접기(**Convert to Function**)” 메뉴가 나타난다.



생성된 함수의 이름은 "SetupToDamageEffect" 로 정한다. 이 과정에서 중간에 있던 Reroute Node(리루트 노드) 들도 함께 정리되면서, 그래프가 다음과 같이 훨씬 깔끔하게 정돈된다.

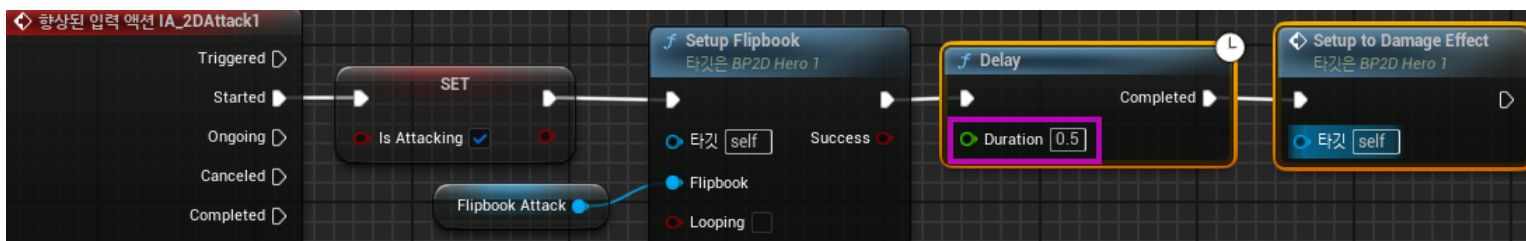


이제 공격 애니메이션의 재생 종료 시점에 연결되어 있던 SetupToDamageEffect 노드를 분리하자.

Alt 키를 누른 상태에서 SetupToDamageEffect 노드의 입력 실행 핀을 마우스로 클릭하면, 해당 실행 경로가 손쉽게 끊어진다. 현재 샘플 프로젝트의 Hero1 공격 애니메이션은 초당 10프레임(10 FPS) 으로 재생된다. 이를 시간으로 바꾸면 1초 / 10프레임 = 0.1초/프레임 이 된다. 공격 타이밍을 5프레임 지점으로 두고 싶다면, 0.1초 × 5프레임 = 0.5초 가 되므로 Delay 시간은 0.5초가 된다.

IA_2D_Attack1 입력 액션 처리 흐름의 끝 이었던 Setup Flipbook 노드에서 먼저 Delay 노드를 배치하고 Duration = 0.5 로 설정한 뒤 Delay의 완료 핀(Completed)을 "SetupToDamageEffect" 노드에 연결한다.

이렇게 구성하면 공격 입력 후 0.5초(5프레임 경과) 시점에 이펙트 및 데미지 처리 로직이 실행되어, 공격 애니메이션과 이펙트 타이밍을 보다 자연스럽게 맞출 수 있다.



컴파일한 뒤 실행해 보면, 이전 방식보다 공격 애니메이션과 이펙트의 싱크가 훨씬 더 자연스럽게 맞아 들어가는 것을 확인할 수 있다.

10.3.3 Mob Damage Animation 동기화

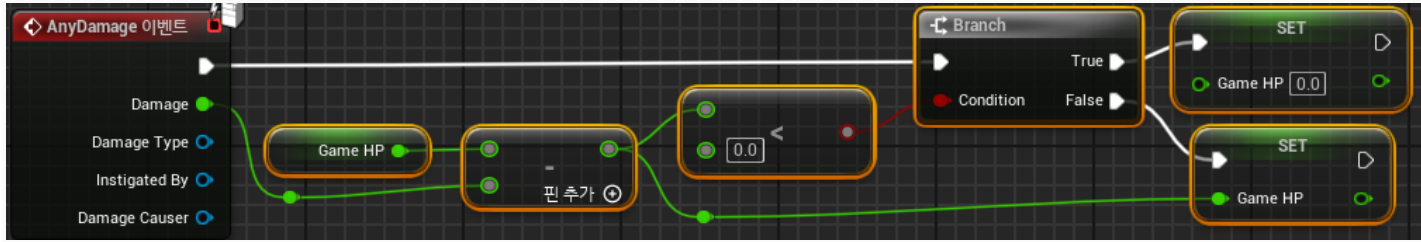
남아 있는 BP2D_MobBoar의 AnyDamage 이벤트를 구현하자. HP가 0이 되면 Destroy Actor를 호출 해서 사라지도록 구성할 것이다. 내 블루 프린트 패널에 변수 채널에 gameHP 를 추가한다. 변수 타입은 플로트로, 기본값은 100.0으로 설정한다.



AnyDamage 이벤트에서 다음과 같은 코드를 블루 프린트로 적용한다.

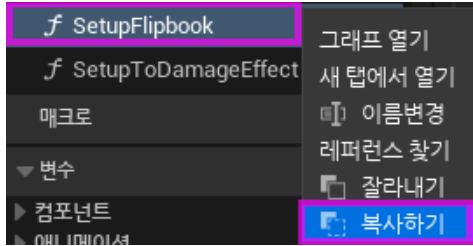
```
gameHP = gameHP - Damage;
if(0 > gameHP)
{
    gameHP = 0.0;
}
SetFlipbook(Idler);
SetLoop(False);
PlayFromStart();
```

블루프린트로 구현하면 다음과 같다.

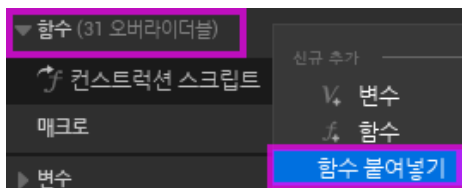


이제 플레이어와 같은 **SetupFlipbook** 함수를 적용한다. 플레이어와 몹은 같은 **PaperCharacter**를 상속 받아서 사용하고 있고, 플레이어의 **SetupFlipbook** 함수는 플레이어만의 변수 없이 만들어서 내 블루프린트에서 **SetupFlipbook**를 복사해서 **BP2D_MobBoar**에 붙여 놓고 컴파일 해도 문제가 없다.

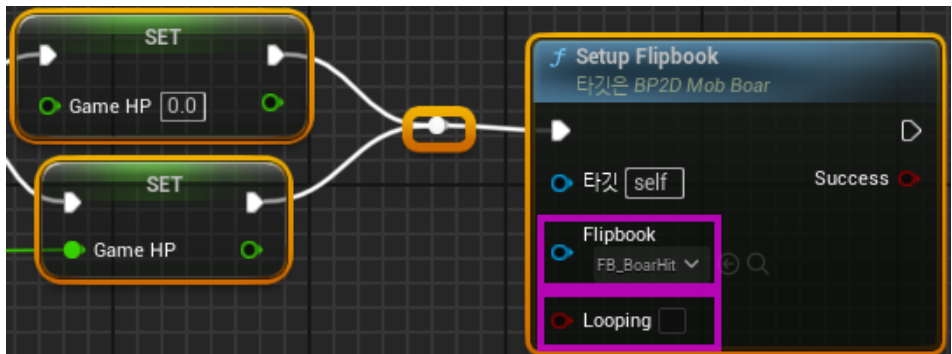
BP2D_Hero1의 내 블루프린트 함수 섹션에서 **SetupFlipbook** 함수를 복사한다.



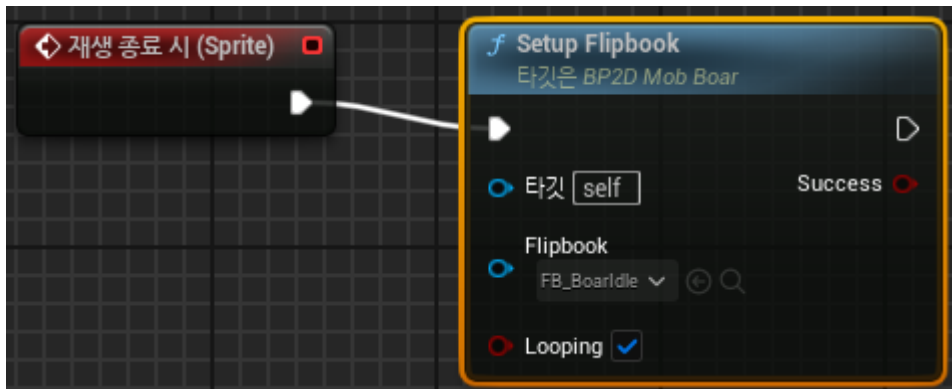
BP2D_MobBoar의 내 블루프린트 함수 섹션에 붙여 넣는다.



컴파일을 실행해서 함수가 문제 없는지 확인한다. **AnyDamage** 이벤트에서 **gameHP** 값을 설정한 뒤, 피격 애니메이션이 한 번만 재생되도록 **SetupFlipbook** 노드를 배치한다.



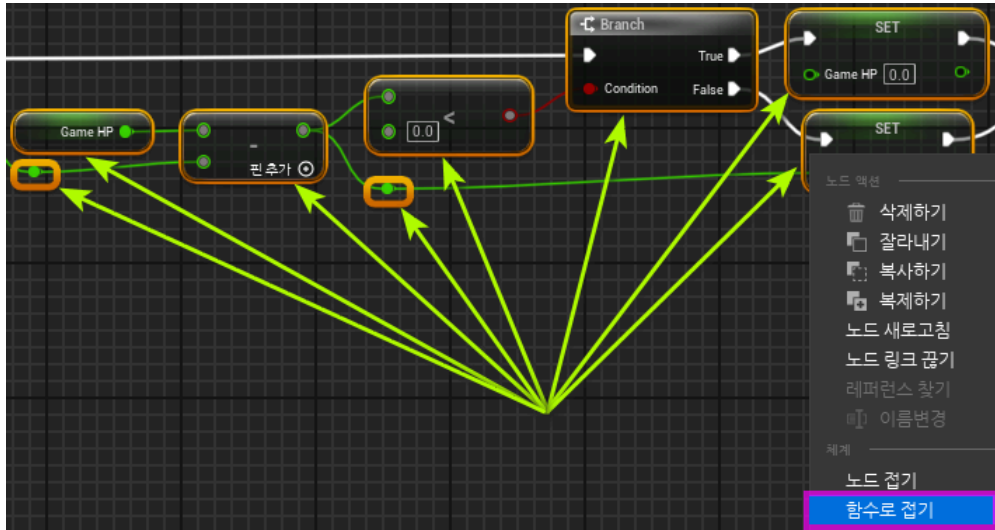
Hit 애니메이션이 끝나면 **Idle** 애니메이션이 루프로 진행되어야 한다. 컴포넌트 패널에서 **Sprite**를 선택하고 디테일 패널의 이벤트에서 재생 종료 시 추가 버튼(+)를 클릭해서 "재생 종료 시(Sprite)" 이벤트를 생성한다.



FB_BoarHit의 프레임 숫자가 적고 초당 프레임이 **15.0**으로 구성되어 있어서 **Hit** 애니메이션이 안보일 수 있다. 초당 프레임을 **5.0**정도로 수정하면 애니메이션이 보일 것이다.

데미지 과정은 먼저 **Hit** 애니메이션을 재생하고, 이후 **gameHP**가 **0.0**이면 몹의 액터를 소멸시키는 흐름이어야 한다. 그러나 현재처럼 데미지 적용 노드와 애니메이션 노드의 순서가 뒤섞여 있으면 수정과 관리가 어려운 구조가 된다.

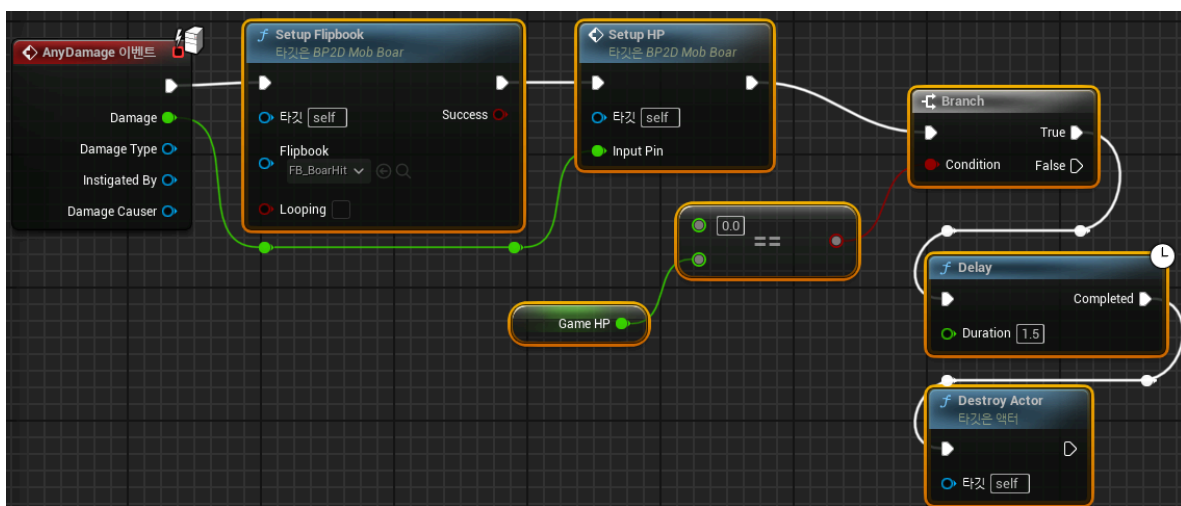
gameHP를 계산하는 스크립트는 이미 완성된 형태이므로, 이 부분을 하나의 함수로 묶어서 순서를 재구성하는 것이 좋다. **AnyDamage** 이벤트에서 **gameHP** 값을 처리하는 모든 노드를 선택하여 함수로 변경한다. 이 함수의 이름은 **SetupHP**로 정한다.



이제 다음 순서로 데미지 처리를 정리한다.

1. SetupFlipbook 으로 Hit 애니메이션을 1회 재생한다.
2. SetupHP 함수를 호출하여 체력을 계산한다.
3. gameHP 값이 0.0이면 DestroyActor를 호출하여 몸을 소멸시킨다.

이와 같은 순서로 블루프린트를 구성하면 피격 처리 흐름이 명확해지고 유지 관리가 쉬워진다.



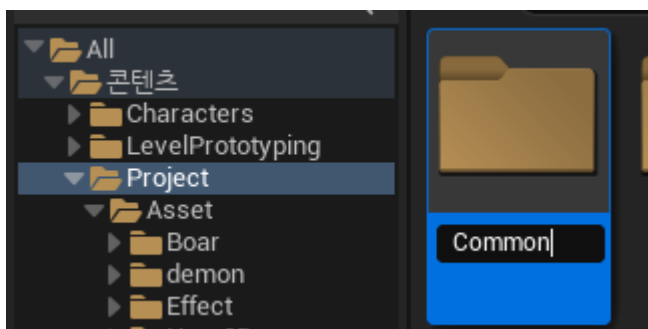
11. Game Instance

플레이어의 체력과 점수는 맵 전환, 게임 상태 변환에서도 유지해야 한다. 따라서 플레이어 액터에 저장하면 데이터를 잃어 버릴 수 있어서 게임 시작부터 게임 종료할 때까지 유지가 되는 객체에 저장해야 한다. 가장 좋은 객체가 **Game Instance**이다.

언리얼 엔진에서 **Game Instance** 는 게임이 실행되는 동안 단 한 번 생성되어 게임 종료 시점까지 계속 유지되는 전역 객체이다. 레벨이 바뀌거나 맵을 이동해도 파괴되지 않기 때문에 프로젝트 전체에서 공통으로 활용할 수 있는 데이터를 보관하기에 적합한 구조이다. 플레이어의 체력, 점수, 스테이지 진행 정보처럼 맵 전환과 게임 상태 변화에서도 유지되어야 하는 값은 플레이어 액터가 아니라 **Game Instance**에 저장하는 것이 안전하다. 플레이어 캐릭터는 맵 로딩 시 다시 생성되지만 **Game Instance**는 파괴되지 않기 때문이다. **Game Instance**는 다음과 같은 용도로 자주 사용된다.

- 플레이어의 체력, 점수, 경험치 같은 지속 데이터 보관
- 현재 스테이지 번호, 진행 상황, 난이도 정보 유지
- 게임 전체 옵션(음량, 그래픽 설정) 저장
- 여러 레벨에서 공유되는 시스템 객체 관리

게임 전반에서 공통으로 사용할 블루프린트를 정리하기 위해, 콘텐츠/Project 폴더 안에 **Common** 폴더를 하나 생성하자.

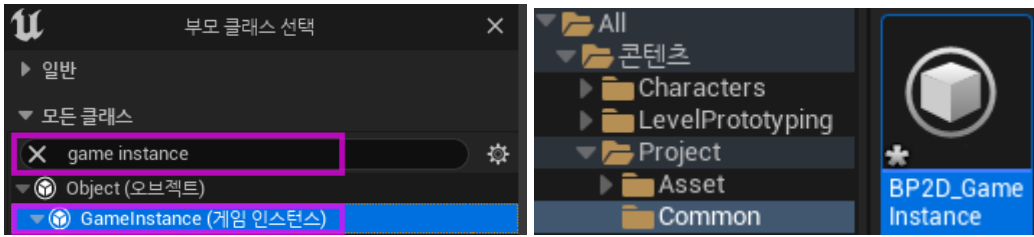


11.1 Game Instance 프로젝트 설정

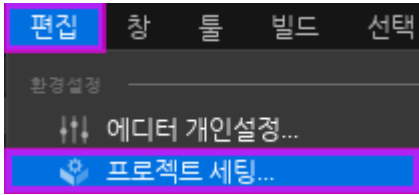
이제 **GameInstance** 블루프린트를 생성하고 프로젝트에 설정하자.

Common 폴더의 빈 공간을 우클릭하여 **Blueprint Class**를 선택하고, 부모 클래스 선택 창의 검색창에 "game instance" 를 입력한다.

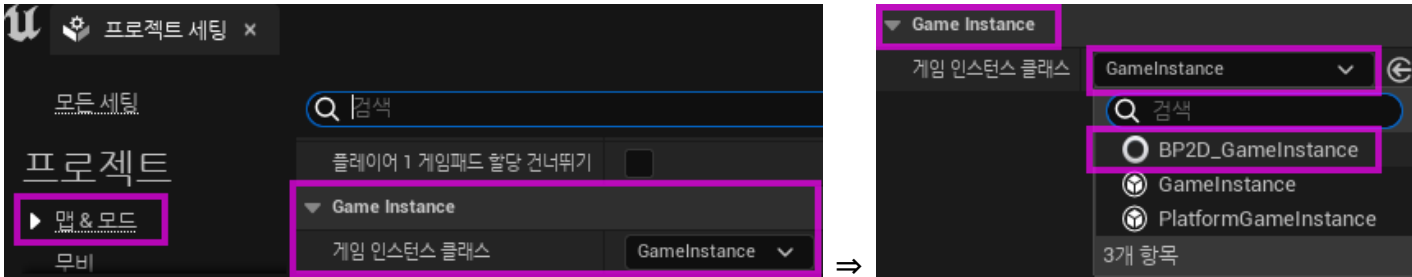
목록에서 **GameInstance**를 선택한 뒤 블루프린트 이름을 **BP2D_GameInstance**로 정한다.



Game Instance는 해당 프로젝트 전 구간에서 사용하기 때문에 프로젝트 설정에서 지정한다. 메뉴 편집(**Edit**) → 프로젝트 세팅...(Project Settings)을 클릭한다.



"프로젝트 세팅" 창에서 왼쪽 패널의 프로젝트(Project) ⇒ 맵& 모드(Maps & Mods) 를 클릭한다.
우측 패널 "프로젝트 - 맵 & 모드" 하단에 **Game Instance** 설정이 있다.



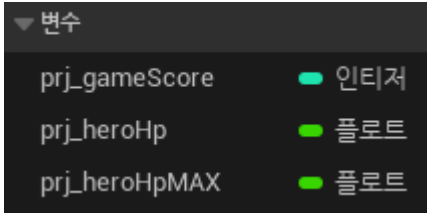
GameInstance 를 **BP2D_GameInstance**로 교체하고 저장한다.

11.2 Game Project 변수

이 **BP2D_GameInstance**에는 점수나 체력처럼 레벨을 이동해도 유지되어야 하는 게임 플레이 데이터를 저장하게 된다.
BP2D_GameInstance 블루프린트를 열어 보면, 비주얼 구성 요소를 담는 컴포넌트 패널과 뷰포트가 보이지 않는다. **Game Instance** 블루프린트는 화면에 배치되는 액터가 아니라, 오직 게임 진행에 필요한 변수와 함수 같은 논리적 요소만을 다루는 전역 관리 객체라는 점을 한눈에 알 수 있다.

내 블루프린트 패널의 변수 섹션의 추가(+) 버튼을 클릭해서 **prj_gameScore**, **prj_heroHp**, **prj_heroHpMAX** 3개의 변수를 추가한다.

	타입	기본 값
prj_gameScore	integer	100
prj_heroHp	float	900.0
prj_heroHpMAX	float	1000.0



체력(**HP**) 변수는 정수형 대신 소수점을 다룰 수 있는 **float** 타입을 사용하는 것이 일반적이다. 그 이유는 피격 시 대미지를 퍼센티지(%)로 계산할 수 있고,
체력 바(**HP Bar**)와 같은 **UI** 요소에서 나눗셈 과정에서 값이 손실되지 않도록 보다 정밀한 계산을 수행할 수 있기 때문이다.

11.3 Game Project 함수

게임 진행과 HUD 표시를 위해 점수, 체력, 최대 체력 변수를 설정했다. 다음 단계로는 이 변수들의 값을 매번 직접 **Get** 해서 설정해 줄 수도 있다.

하지만 체력의 경우 다음과 같은 조건이 붙었을 때

- 체력이 최대 체력을 넘지 않아야 하고,
- 데미지를 입었을 때도 0보다 작아지지 않아야 한다

이 같은 조건을 매번 체크해야 한다. 이 로직을 매번 노드로 구성하면, 나중에 조건이 바뀔 때마다 모든 곳을 찾아서 수정해야 하는 문제가 생긴다. 이러한 반복을 줄이기 위해, 체력·점수에 관련된 처리를 함수로 묶어 두는 것이 좋다. 조건이 바뀌더라도 함수 내부만 수정하면 이 함수를 사용하는 블루프린트들은 그대로 재사용할 수 있다.

여기서는 다음 네 가지 함수를 추가할 것이다.

- 1) 플레이어가 아직 살아 있는지 확인하는 함수
- 2) 점수를 추가하는 함수
- 3) 체력을 추가(또는 감소)하는 함수
- 4) UI에서 사용할, 현재 체력과 최대 체력의 비율을 반환하는 함수

11.4 IsPlayerAlive

먼저, "플레이어가 살아 있는지"를 확인하는 함수를 만든다.

간단히 C++ 코드로 표현하면

```
bool IsPlayerAlive() {  
    if(prj_heroHp > 0.0)  
        return true;  
    else  
        return false;  
}
```

이것을 블루 프린트로 구현하자.

- 1) BP2D_GameInstance 블루프린트를 연다.
- 2) 내 블루프린트(My Blueprint) 패널에서 함수 섹션의 추가(+) 버튼을 클릭하고, 새 함수 이름을 **IsPlayerAlive** 로 정한다.
- 3) 한 번 컴파일한 뒤, 함수가 선택된 상태에서 디테일 패널의 출력 섹션(Outputs)에서
 - 타입을 **Boolean**(불리언)으로 지정
 - 이름을 **Alive** 로 하는 출력을 하나 추가
- 4) 그래프에서는 앞서 보여 준 의사 코드와 동일하게,
 - **prj_heroHp** 값을 가져와서
 - **0.0** 과 비교(> 비교)한 뒤
 - 그 결과를 **Alive** 출력 핀에 연결해 주면 된다.

이렇게 만들어 두면, 다른 블루프린트에서 **IsPlayerAlive** 함수를 호출하는 것만으로 “플레이어가 살아 있는가?”라는 조건을 항상 동일한 규칙으로 판단할 수 있다.



11.5 AddScore

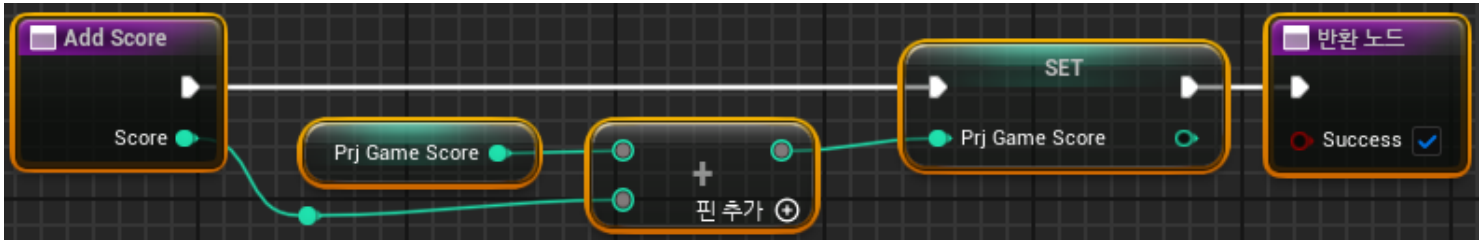
점수를 추가하는 함수는 매우 단순하다. 전달받은 점수를 현재 총 점수에 더해 주기만 하면 된다. 간단히 C++ 코드로 표현하면 다음과 같다.

```
bool AddScore(int score) {  
    prj_gameScore = prj_gameScore + score;  
    return true;  
}
```

또는 반환 값이 필요 없다면 다음과 같이 작성할 수도 있다.

```
void AddScore(int score) {  
    prj_gameScore = prj_gameScore + score;  
}
```

반환 값은 기능적으로 반드시 필요하지는 않지만, 앞서 만든 다른 함수들과의 일관성을 유지하기 위해 **Boolean** 타입의 반환 값을 추가한다. 블루프린트로 구현할 때는 함수 생성 과정이 **IsPlayerAlive**와 동일하며, 출력 섹션(Outputs)에 **Boolean** 타입의 **success** 변수를 하나 추가한다. 노드 구성은 점수를 더한 뒤, 반환 노드(Return Node)의 "success" 값을 **true** 로 설정해 마무리한다.

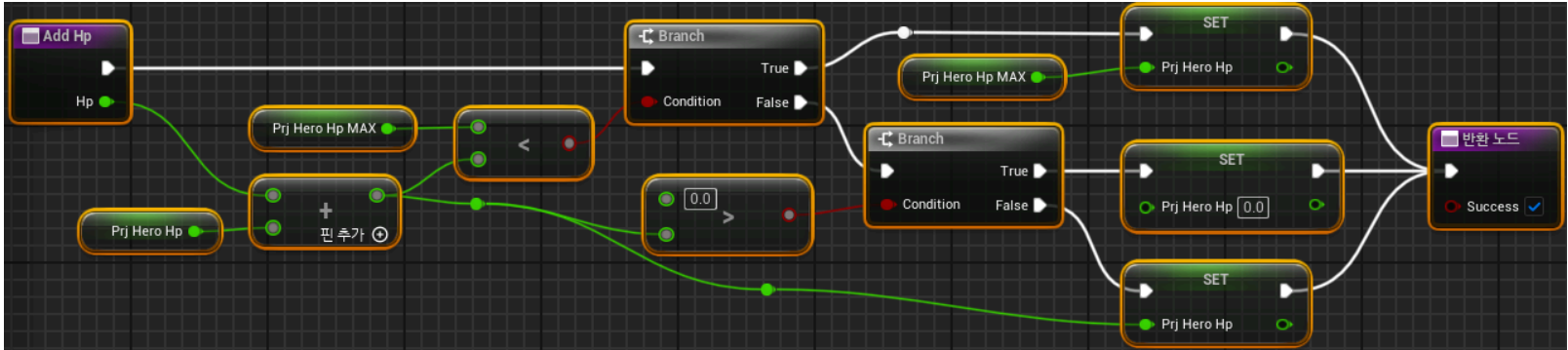


11.6 AddHp

체력을 증가시키는 함수는 점수를 추가하는 함수와 유사하게 간단하게 만들 수 있다. 그러나 체력을 올릴 때 최대 체력을 넘어서는 안 된다는 조건이 필요하다면, 이 조건을 함수 내부에서 처리하도록 구성해야 한다. 또한 대미지를 입는 경우에도 별도의 **AddDamage** 함수를 만들 필요 없이, **AddHp** 함수에 음수 값을 전달하는 방식으로 동일하게 처리할 수 있다. 이 때는 체력이 **0** 미만으로 내려가지 않도록 하는 조건이 추가된다. 이 두 조건(최대 체력 제한, 최소 체력 제한)을 만족하도록 코드를 작성하면 체력 값을 계산한 뒤 이를 최대·최소 범위에 맞게 보정하는 과정에서 자연스럽게 두 개의 **Branch**(조건 분기)가 필요하다는 것을 직관적으로 알 수 있다. C++ 형태로 표현하면 다음과 같다.

```
bool AddHp(float hp) {
    prj_heroHp = prj_heroHp + hp;
    if( prj_heroHpMax < prj_heroHp ) {
        prj_heroHp = prj_heroHpMax;
    }
    else {
        if( 0.0 > prj_heroHp ) {
            prj_heroHp = 0.0;
        }
        else {
        }
    }
    return true;
}
```

블루프린트로 구현하면 다음과 같다.



11.7 GetHeroHpPercent

UI에서 필요한 체력과 최대 체력의 비율을 반환하는 함수는 아주 쉽다. 단순히 **prj_heroHp / prj_heroHpMax** 를 반환 하면 된다. C++ 형태로 표현하면 다음과 같다.

```
float GetHeroHpPercent() {
    return prj_heroHp / prj_heroHpMax;
}
```

블루프린트에서도 구현 방법은 동일하다. 함수를 생성하고 이름을 **GetHeroHpPercent** 로 정한다. 컴파일한 뒤, 디테일 패널의 출력 섹션(Outputs)에 **Float** 타입 **ret** 변수를 추가 한다. 그래프에서 **prj_heroHp** 값을 **prj_heroHpMax** 로 나누어 반환 노드(Return Node)의 **ret**에 연결하면 끝이다.



12. Game Ui

12.1 UMG

UMG(Unreal Motion Graphics: 언리얼 모션 그래픽) 은 언리얼 엔진에서 게임 **UI(User Interface:사용자 인터페이스)**를 제작하기 위한 비주얼 UI 저작 툴이자 UI 시스템이다.

게임의 체력 바, 점수 표시, 메뉴, 아이콘, 대사창 등 게임 화면에 나타나는 대부분의 사용자 인터페이스를 **UMG**로 구성한다.

UMG의 장점

- 시각적 디자인: **UI** 요소를 드래그 앤 드롭 방식으로 배치하여 코드를 작성하지 않고도 직관적으로 **UI**를 구성할 수 있다. **UI** 작업 속도가 빠르고 수정·반복 작업에 유리하다.
- 데이터 바인딩(Data Binding): 게임 변수(**C++** 혹은 블루프린트)와 **UI** 속성을 직접 연결할 수 있다. 예를 들어, 플레이어의 체력 값을 **ProgressBar**의 퍼센트 값과 연결하면 게임 내 체력 변화가 **UI**에 자동 반영된다.
- 블루프린트·C++ 연동 유연성: **UI** 이벤트는 블루프린트와 **C++** 양쪽에서 제어할 수 있다. 규모가 작은 **2D** 게임부터 시스템 기반의 프로젝트까지 폭넓게 활용 가능하다.

UMG의 역할

UMG는 게임 내에서 플레이어가 확인하거나 조작하는 모든 그래픽 **UI** 요소를 제작하는 데 사용된다.

- HUD(체력, 점수)
- 메뉴 화면
- 인벤토리
- 대화 UI
- 대미지 표시
- 버튼 기반 인터페이스

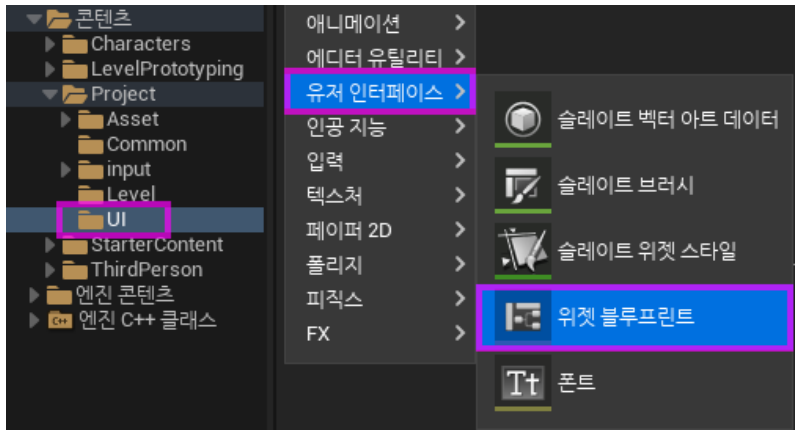
2D 게임에서는 특히 HUD와 간단한 메뉴 UI가 주로 활용된다.

12.2 UMG 구성 요소

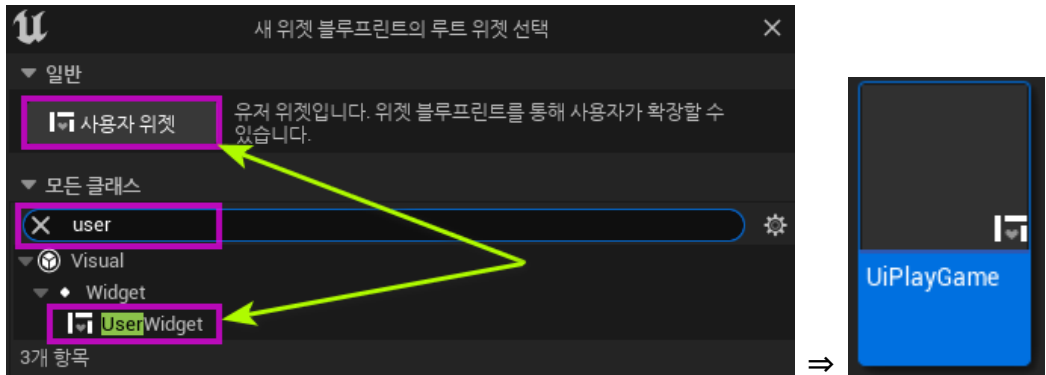
위젯 블루프린트(Widget Blueprint)는 **UMG** 시스템 안에서 실제 **UI** 화면을 제작하는 **UI** 전용 블루프린트 에셋이다. **Game HUD**용 UI를 위젯 블루프린트로 만들어 가면서 **UMG**의 다양한 구성 요소들을 함께 확인해 보도록 한다.

먼저 위젯 블루프린트 저장할 전용 폴더로 "/콘텐츠/Project/UI" 폴더를 생성한다.

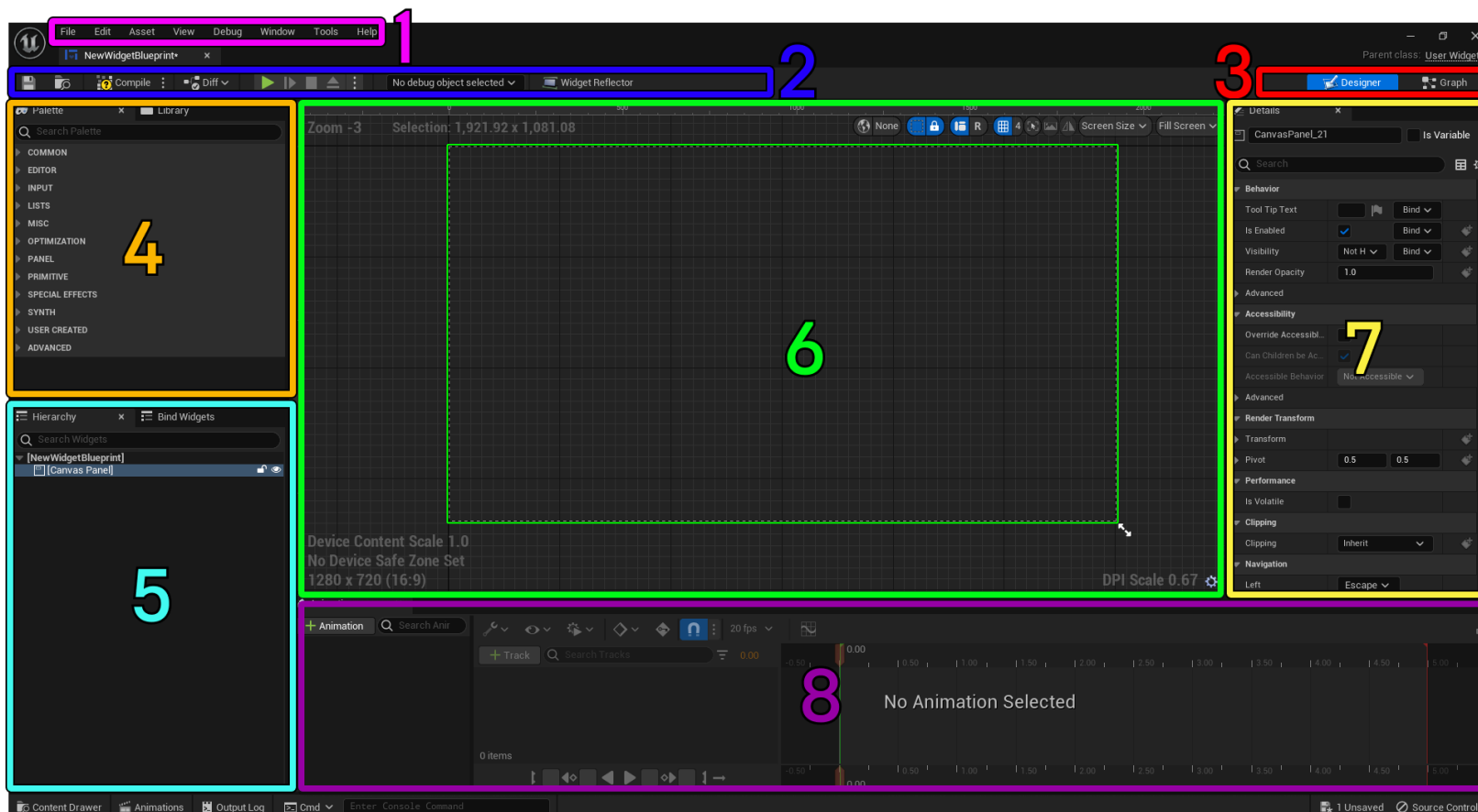
UI 폴더 빈 공간에 우 클릭, 유저 인터페이스 ⇒ 위젯 블루프린트를 선택한다.



새 위젯 블루프린트의 루트 위젯 선택 창에서 "사용자 위젯" 버튼을 바로 클릭하거나 모든 클래스의 검색창에서 **user**를 입력하고 **UserWidget**을 선택한다.



생성된 위젯의 이름을 "UiPlayGame" 로 지정한 후에 클릭해서 위젯 편집창(Widget Editor)을 연다.



위젯 편집창의 각 영역은 언리얼 엔진 문서 "[언리얼 엔진의 UMG에서의 위젯 블루프린트](https://dev.epicgames.com/documentation/ko-kr/unreal-engine/widget-blueprints-in-umg-for-unreal-engine)" (<https://dev.epicgames.com/documentation/ko-kr/unreal-engine/widget-blueprints-in-umg-for-unreal-engine>) 에서 다음과 같이 설명된다.

- 1 ⇒ 메뉴 바
일반적인 메뉴 바
- 2 ⇒ 툴 바
컴파일, 저장, 실행(Play) 등 자주 사용하는 UMG/블루프린트 관련 기능들이 배치
- 3 ⇒ 에디터 모드
UMG 위젯 블루프린트 에디터의 두 가지 모드를 전환.
 - 디자이너(Designer) : UI 레이아웃과 시각적인 구성 작업.
 - 그래프(Graph) : UI 이벤트 및 로직 스크립트 작성.
- 4 ⇒ 팔레트
드래그 앤 드롭으로 배치할 수 있는 UWidget 기반 위젯 목록 제공.
버튼, 텍스트, 이미지, 패널 등 다양한 UI 요소 포함.
- 5 ⇒ 계층구조
현재 위젯의 부모-자식 관계(트리 구조) 표시
위젯을 이 창에 드래그하여 계층 구조를 재정렬할 수도 있음.
- 6 ⇒ 비주얼 디자이너
레이아웃의 시각적 표현.
팔레트에서 끌어 온 위젯을 배치, 크기·정렬 등을 그래픽 방식으로 조작.
- 7 ⇒ 디테일
현재 선택된 위젯의 프로퍼티 표시, 위치·정렬·색상·텍스트·이벤트 바인딩 등 다양한 값을 설정.
- 8 ⇒ 애니메이션
UMG 애니메이션 트랙 관리
위젯의 위치 변화, 투명도 변화 등 애니메이션을 위한 키프레임을 추가하고 편집

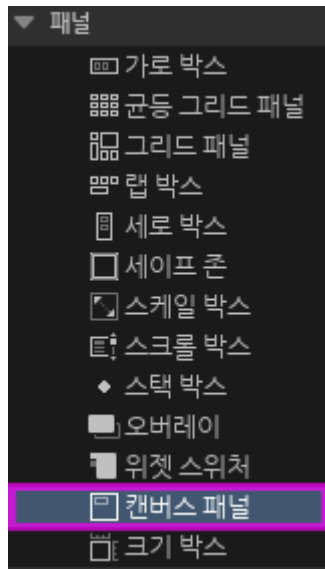
12.3 HUD 화면 디자인

12.3.1 캔버스 패널

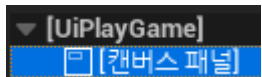
이 번 HUD에서는 “점수(Score)”와 “체력 바(HP Bar)”만 화면에 표시할 예정이다. 위젯 블루프린트의 에디터 모드에서 디자이너(Designer) 탭을 선택한다.



화면의 모든 UI 위젯은 캔버스 패널(Canvas Panel) 아래에서 배치되어야 한다. 팔레트(Palette) 패널에서 패널(Panel) 섹션을 펼치고 캔버스 패널(Canvas Panel) 을 선택한다.

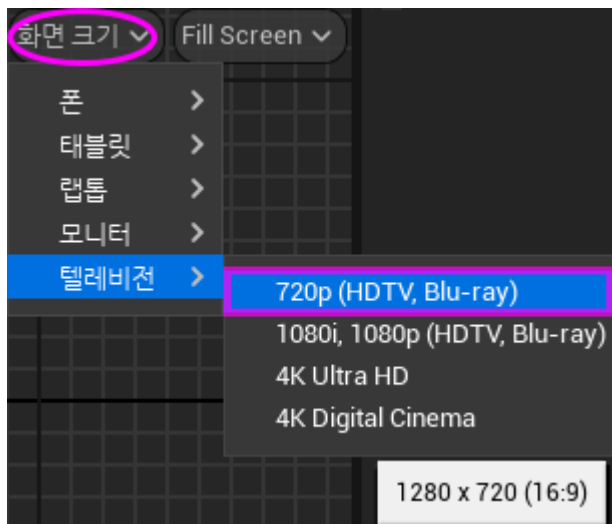


계층 구조(Hierarchy) 패널을 보면, 루트 위젯인 "UiPlayGame" 아래에 방금 추가한 "캔버스 패널"이 생성된 것을 확인할 수 있다.



캔버스 패널은 HUD 배치의 기준이 되는 화면 해상도(Design Resolution)를 설정할 수 있다. 이를 변경하는 방법은 두 가지가 있다.

- 1) 비주얼 디자이너(Visual Designer) 우측 상단의 "화면 크기(Screen Size)" 버튼을 눌러 원하는 해상도를 선택한다.



- 2) 비주얼 디자이너 우측 하단의 Screen Size Handle을 마우스로 드래그하여 캔버스 크기를 직접 조정한다.

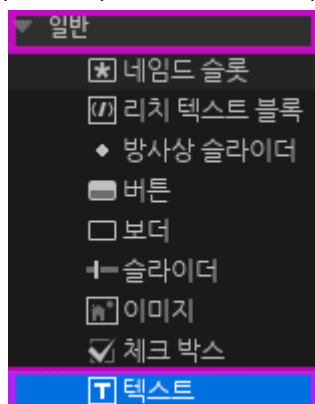


이 번 HUD에서는 언리얼 엔진에서 널리 사용되는 16:9 비율의 1280 × 720 해상도를 기준으로 설정한다.

12.3.2 텍스트

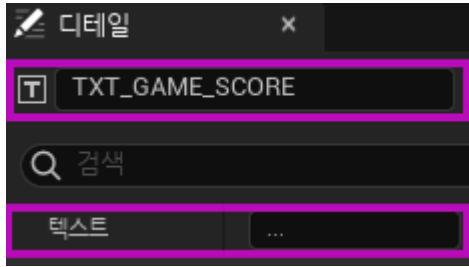
점수를 표현할 텍스트를 추가할 차례다

팔레트(Palette) 패널에서 일반(Common) 섹션을 펼치면 텍스트(Text) 위젯을 확인할 수 있다.

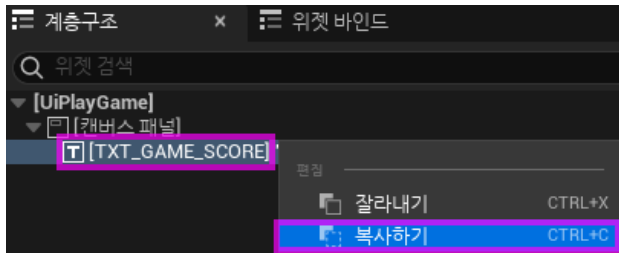


텍스트를 추가하고 이름을 "TXT_GAME_SCORE"로 변경한다.

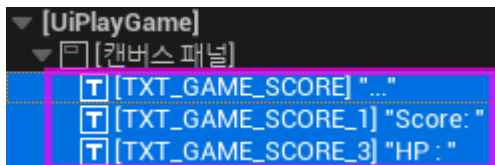
디테일(Details) 패널의 텍스트(Text) 항목에서 기본 값(Default Text)을 "..."으로 입력한다.



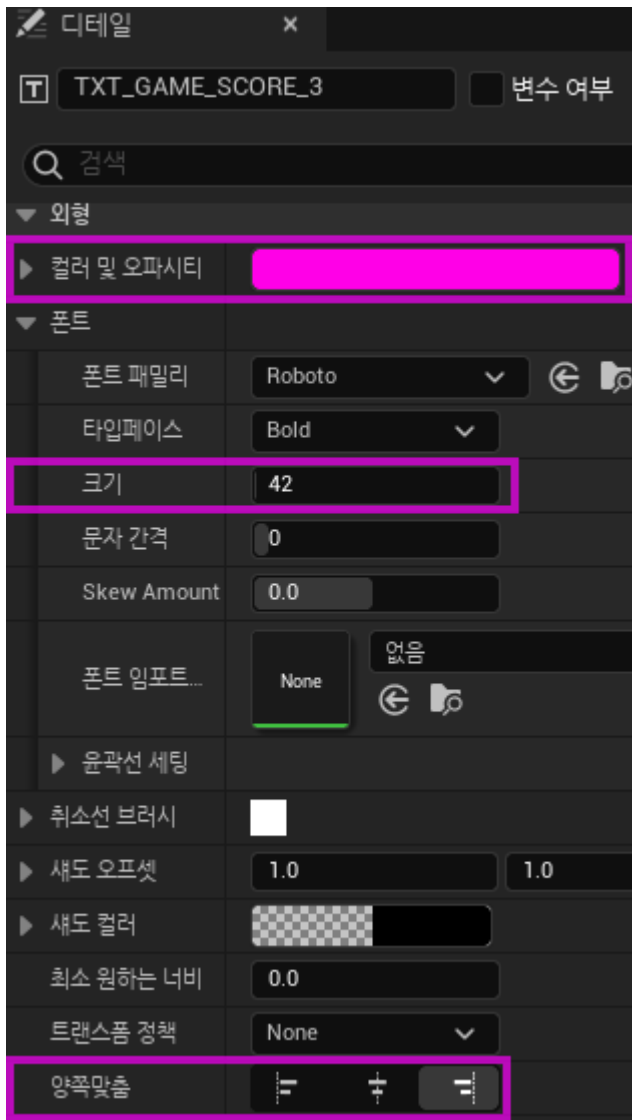
TXT_GAME_SCORE 를 복사하고, 붙여넣기(CTRL+V)를 하면 텍스트 위젯이 추가가 된다.



붙여놓은 텍스트 위젯의 텍스트를 "Score: ", "HP: " 변경한다.



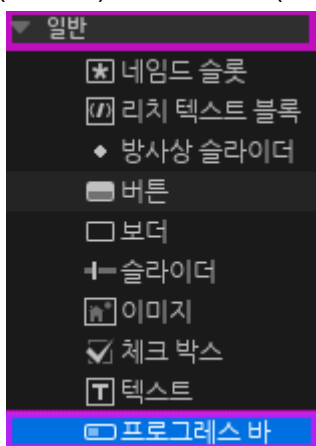
텍스트 위젯을 선택한 상태에서 디테일(Details) 패널의 "컬러 및 오파시티(Color and Opacity)", 폰트(Font)의 크기, 정렬 옵션을 조정해서 화면에 표시될 글자의 색상, 크기, 정렬 방식을 변경할 수 있다.



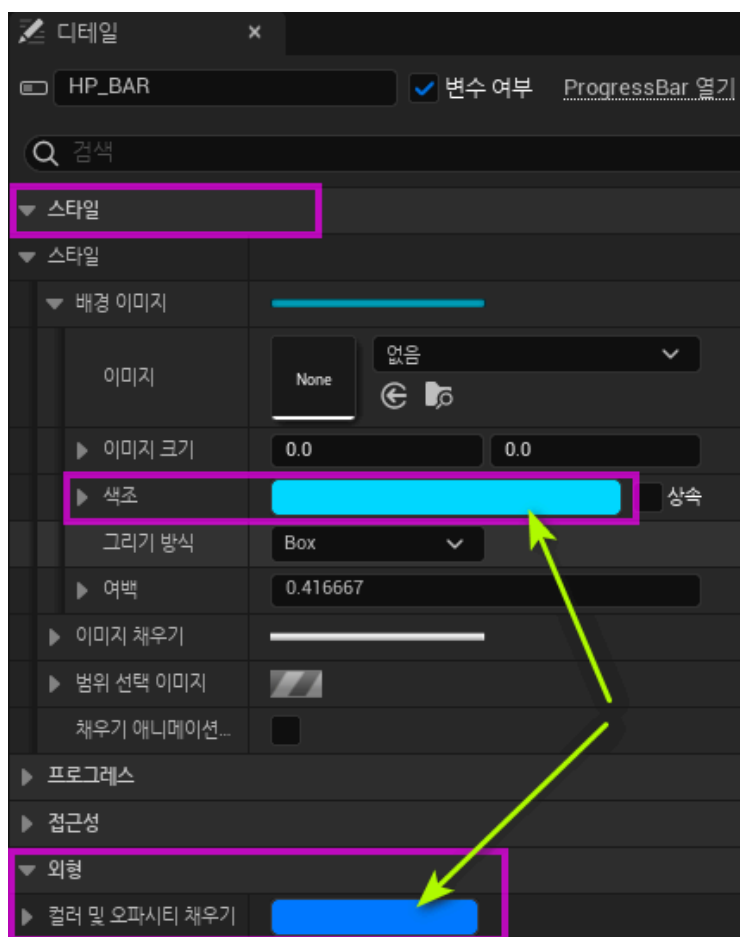
12.3.3 프로그레스 바

다음으로 체력 바(HP Bar)를 추가한다.

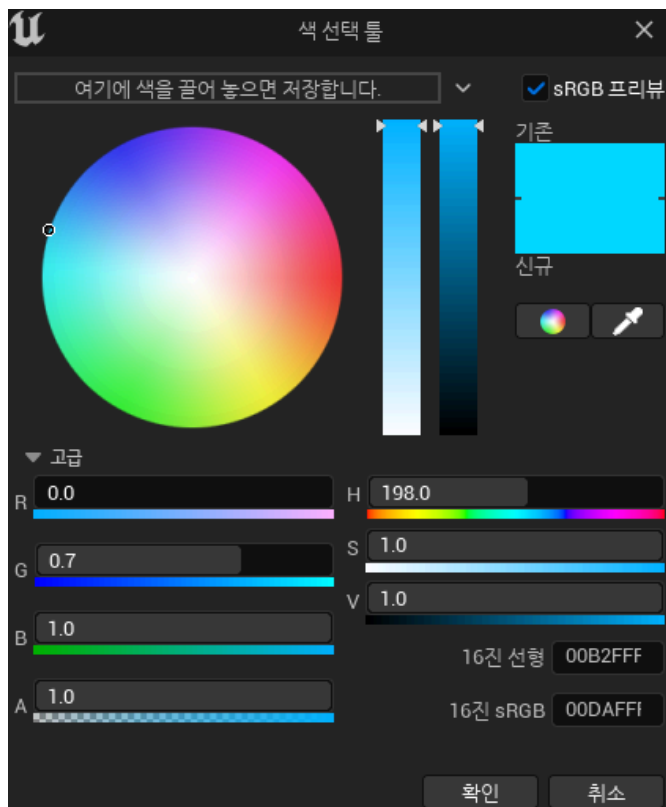
팔레트(Palette) 패널의 일반(Common) 섹션 아래를 보면 프로그레스 바(Progress Bar) 위젯이 있다.



이를 캔버스에 추가한 뒤 이름을 HP_BAR 로 변경한다.
 디테일 패널에서 배경 스타일과 외형을 적당한 색을 설정한다.



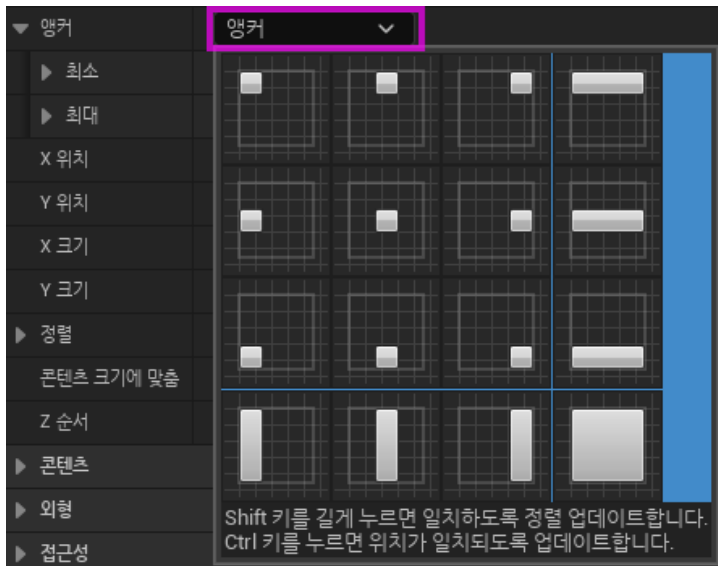
색상 변경은 색조 버튼이나 컬러 및 오파시티 채우기의 색상 버튼을 클릭하면 색 선택 툴이 나온다.



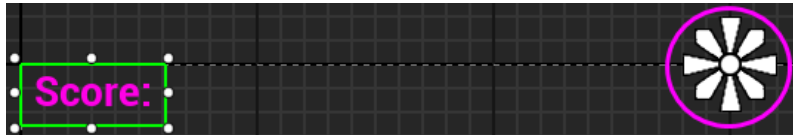
12.3.4 앵커 및 위치

앵커(Anchor) 는 화면에 표시되는 위젯의 기준점(Reference Point) 을 의미한다. 디자이너(Designer) 창에 배치된 모든 위젯은 각각 자신의 앵커를 가진다.

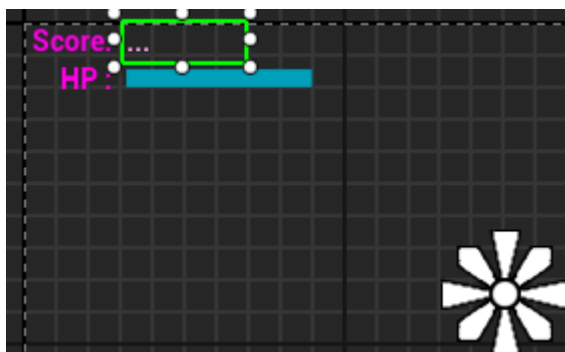
위젯을 선택한 후 디테일(Details) 패널의 앵커(Anchors) 섹션에서 드롭다운 버튼을 클릭하면 기준점으로 사용할 앵커를 선택할 수 있다. 새로운 앵커를 선택하면, 위젯의 위치 값은 선택된 앵커를 기준으로 다시 계산되어 재배치되며 화면에 표시되는 앵커 메달리온(Anchor Medallion) 위치도 변경이 된다.



변경된 앵커 메달리온.



지금은 복잡한 UI 배치는 아니므로, 앵커는 기본값인 '중앙(센터)' 상태로 두겠다. 이제 네 개의 UI 위젯을 마우스로 드래그하여 원하는 위치에 배치한다.



12.5 HUD 화면 그래프

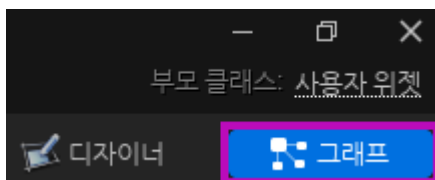
12.5.1 Score

"TXT_GAME_SCORE"에 `GameInstance`의 점수를 연결하는 방법을 설명한다.

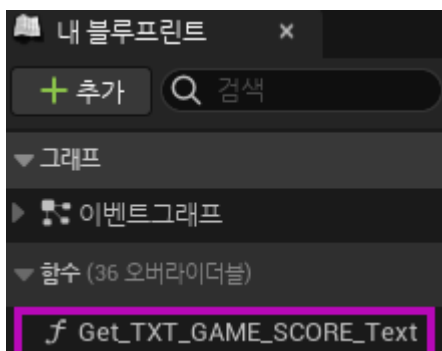
"TXT_GAME_SCORE" 위젯을 선택하면 디테일(Details) 패널의 콘텐츠 텍스트(Content → Text) 영역에 바인드(Bind) 드롭다운 버튼이 있다. 이 버튼을 클릭하고 "바인딩"을 선택하면 언리얼이 자동으로 `Get_TXT_GAME_SCORE_Text` 라는 함수를 생성되고 텍스트 문자열은 함수로 대체된다.



디자이너 버튼 옆에 "그래프" 버튼을 누르면 디자인의 위젯들을 프로그래밍 할 수 있는 에디터가 나온다.



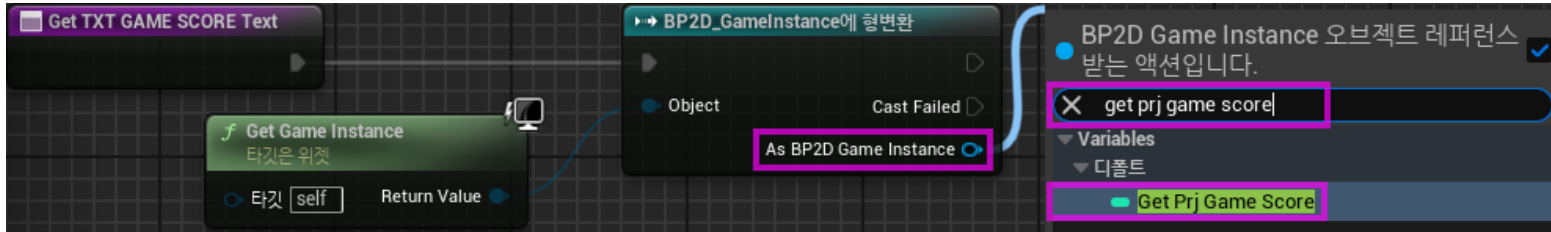
내 블루프린트의 함수 섹션에서 방금 새로 생성한 함수 `Get_TXT_GAME_SCORE_Text` 가 있다.



`Get_TXT_GAME_SCORE_Text` 함수의 반환(Return Value) 값이 텍스트 위젯에 표시될 실제 문자열이 된다.



GetGameInstance 노드는 현재 프로젝트에 설정된 Game Instance를 반환한다. 우리는 프로젝트 설정에서 BP2D_GameInstance 를 Game Instance로 지정해 두었으므로, GetGameInstance 노드를 가져온 후 이를 BP2D_GameInstance로 캐스팅(Cast To BP2D_GameInstance) 한다. 캐스팅된 BP2D_GameInstance 노드에서 검색창에 "get prj game score"을 입력하면 BP2D_GameInstance에 만들어 둔 prj_GameScore 변수를 가져올 수 있다.



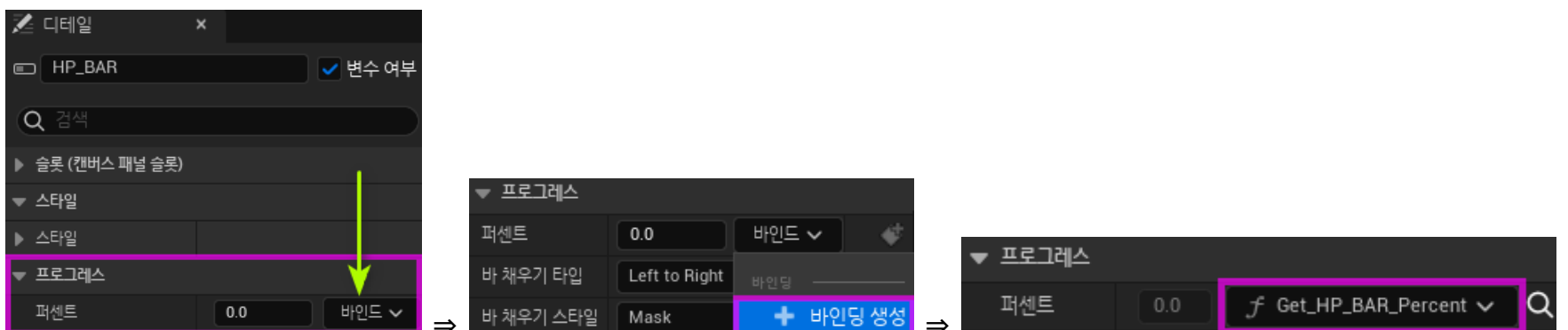
이 점수 값은 정수 또는 실수(Float/Int) 이므로 UMG 텍스트에 출력하려면 ToText 노드를 사용해 문자열로 변환해야 한다. 완성된 Get_TXT_GAME_SCORE_Text 는 다음과 같다.



12.5.2 체력 바

체력 바의 진행 정도는 디테일(Details) 패널의 프로그레스(Progress) 섹션에 있는 "퍼센트(Percent)" 값으로 결정된다. 퍼센트 항목의 바인드 버튼을 클릭하면 "+ 바인딩 생성" 메뉴가 나타난다.

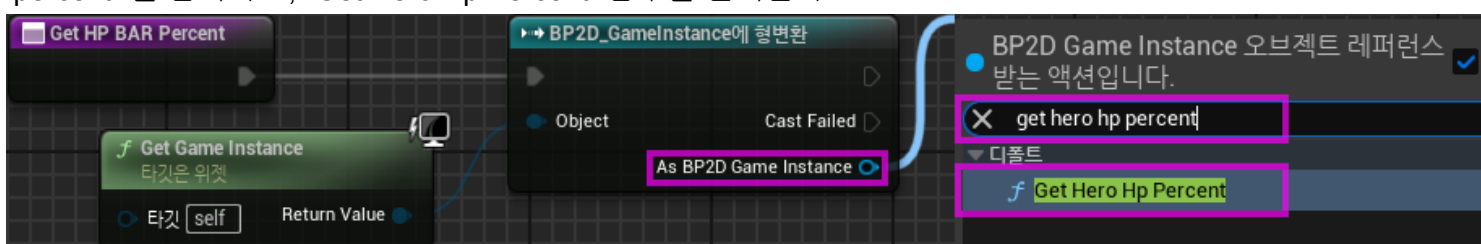
이 항목을 선택하면 TXT_GAME_SCORE의 경우와 동일하게 자동으로 Get_HP_BAR_Percent 함수가 생성되며, 체력 바의 퍼센트 값은 이 함수의 반환 값으로 대체된다.



Get_HP_BAR_Percent 함수가 생성되면, 반환 노드의 "Return Value" 에 BP2D_GameInstance가 제공하는 체력 비율 함수 "GetHeroHpPercent" 를 연결하여 Ui에 반영할 수 있다.



먼저 GetGameInstance 노드를 가져온 후에, BP2D_GameInstance 로 캐스팅한다. 캐스팅된 BP2D_GameInstance 노드에서 검색창에 "get hero hp percent" 를 입력하고, "Get Hero Hp Percent" 함수를 선택한다.



선택된 "Get Hero Hp Percent" 노드의 반환 핀(Ret)을 반환 노드 "Return Value" 입력 핀에 연결한다.



그래프를 컴파일하고 저장하면 UI 구성이 완료된다.

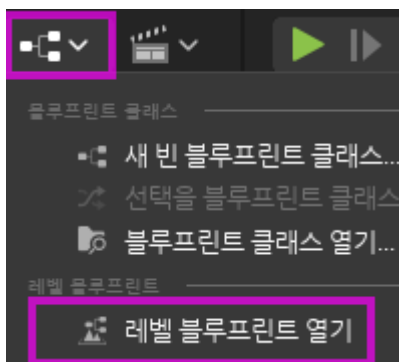
12.6 위젯 블루프린트 게임 연결

작업을 완료한 **UiPlayGame** 위젯을 실제 게임 화면에 표시하려면, 런타임에 위젯 인스턴스를 생성한 뒤 뷰포트에 추가해야 한다. 과정을 간단히 요약하면

Create Widget ⇒ Class에 생성할 위젯 블루프린트 선택 ⇒ Add to Viewport

12.6.1 위젯 노드 생성

툴바의 블루프린트 메뉴 버튼을 눌러서 현재 레벨의 블루프린트를 연다

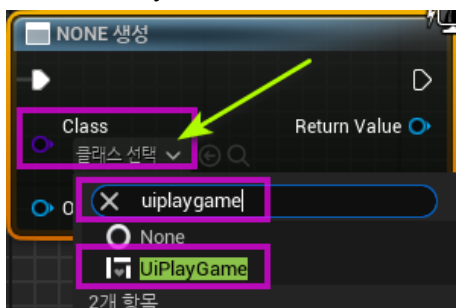


레벨 블루프린트의 **BeginPlay** 이벤트에서 다음과 같이 구성한다.

Begin Play 이벤트에서 이전의 완료 지점에 실행 핀을 끌어서 실행 가능 액션 검색창에 "create widget" 을 입력하고 위젯 생성을 선택한다



생성된 위젯 노드는 "NONE 생성"으로 되어 있다. **Class** 입력 핀의 "클래스 선택" 메뉴 버튼을 클릭한다. 검색창에 앞서 작업한 "uiplygame" 을 입력하고 "UiPlayGame" 블루프린트를 선택한다.



생성된 위젯 노드는 "NONE 생성"에서 "Ui Play Game 위젯 생성" 으로 변경이 된다.

마지막으로 "Add to Viewport" 함수를 호출하자.



컴파일하고 저장해서 작업을 완료한다.

전체 노드 구성은 다음과 같다.

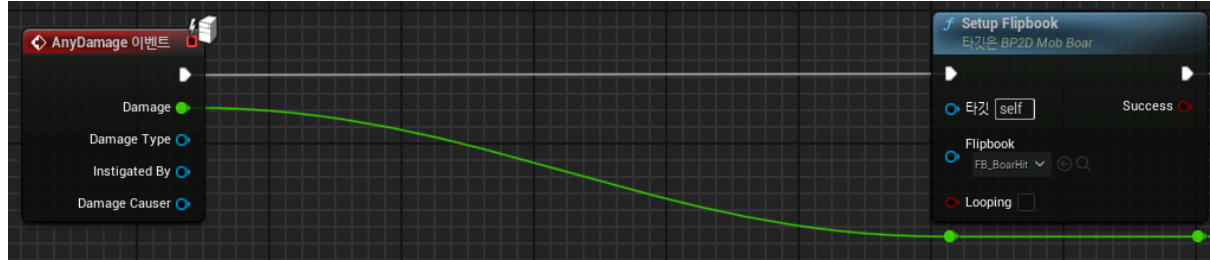


참고로 플레이어나 플레이어 컨트롤러에서도 위젯을 생성하고 **Add to Viewport**로 화면에 연결할 수 있다. 하지만 플레이어(**Player**) 또는 **Mob**은 **BP2D_GameInstance** 값만 설정하는 ‘생산자(Producer)’ 역할, **UiPlayGame HUD**는 그 값을 읽어서 화면에 표시하는 ‘소비자(Consumer)’ 역할로 분리하면 **UI**와 게임 로직이 서로 독립적인 모듈로 유지되어 구조가 더 견고해진다.

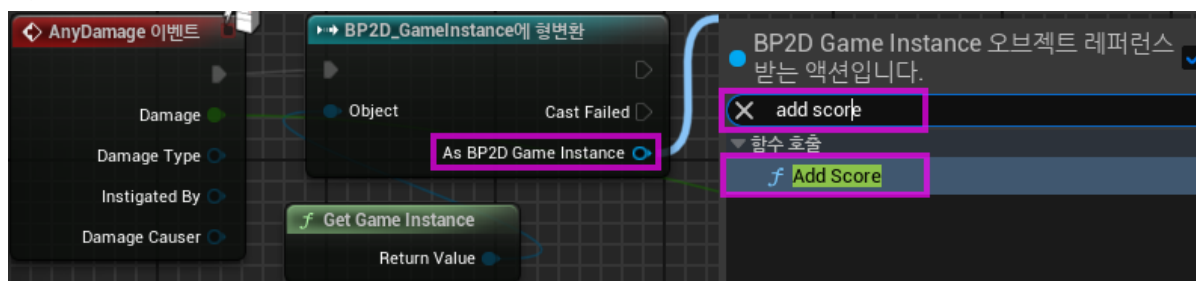
12.7 Mob의 AnyDamage에 점수 연결

몸이 받는 **Damage**에 점수를 입력하자. 테스트로 몸이 받는 데미지 값에 **100**을 곱해서 점수로 설정할 것이다.

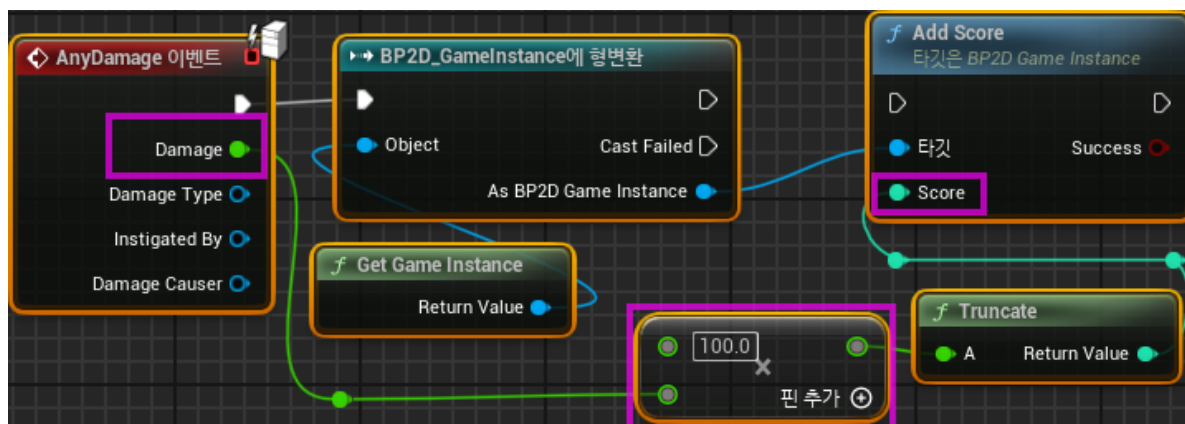
BP2D_MobBoar 블루프린트에서 **"AnyDamage 이벤트"**를 선택한다. **"AnyDamage 이벤트"**와 **"Setup Flipbook"** 노드 사이에 점수를 추가하기 위해서 간격을 늘린다.



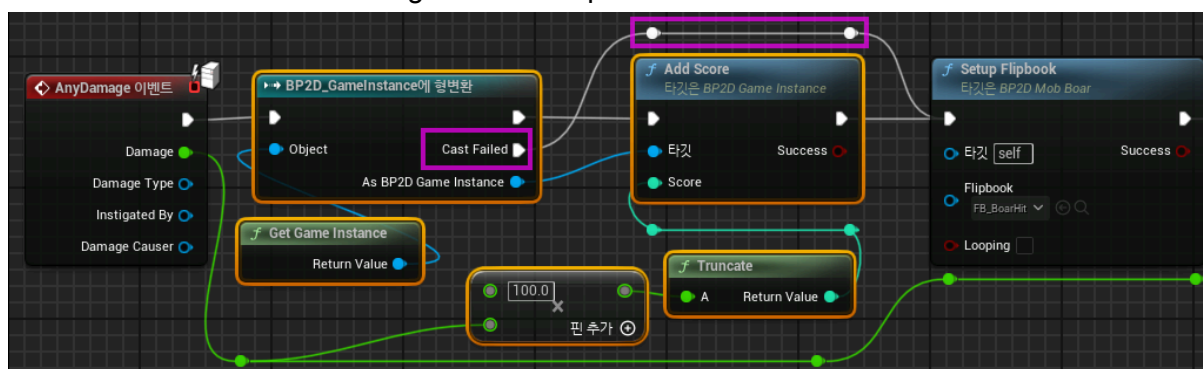
BP2D_GameInstance의 **"AddScore"** 함수를 호출 하기 위해서 위젯 작업처럼 **GetGameInstance** 노드를 가져온 후 이를 **BP2D_GameInstance**로 캐스팅(Cast To **BP2D_GameInstance**) 한다. 캐스팅된 **BP2D_GameInstance** 노드에서 검색창에 **"add score"**을 입력하고, **"Add Score"** 함수를 선택한다.



"AnyDamage 이벤트" **Damage** 출력 핀에 **100**을 곱한다. 이 노드를 **"Add Score"** 노드의 **Score**에 연결하면 자동으로 **Truncate** 노드가 추가되어 **int** 타입으로 변환된 값이 **Score** 입력에 연결된다.



실행 흐름과 나머지 노드들을 전부 연결한다. **Game Instance** 캐스팅이 실패하더라도 다음 흐름이 진행 될 수 있도록 **"Cast Failed"**도 **Setup Flipbook** 노드에 연결한다. 물론 **Damage** 핀도 **"Setup HP"**에 연결한다.



실행하면 **Score**와 **Hp**가 화면에 표현된다.



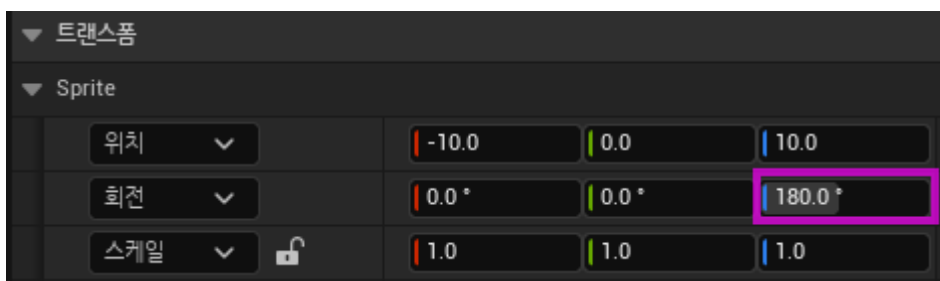
13. 기본 AI 행동(Basic AI Behaviors)

AI Controller를 사용하기 전에 게임에서 사용하는 가장 간단한 4가지 몸의 기본 행동 무작위 이동, 플레이어 탐지, 공격 거리 체크, 근접 공격 실행 등을 구현해볼 것이다.

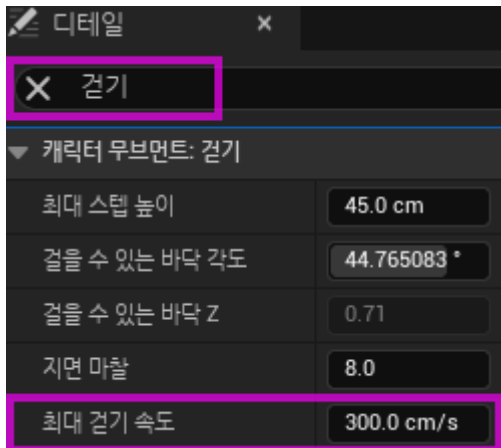
13.1 Transform

기본 행동을 구현하기 앞서서 **Transform**과 이동 속도를 조정하자. **Moar** 캐릭터는 디폴트 상태가 **-X** 방향을 바라보고 있다. 이렇게 캐릭터의 방향이 제각각이면 블루프린트 스크립트를 따로 작성해야 한다.

트랜스폼의 회전을 수정해서 플레이어 캐릭터와 같이 **+X** 방향을 바라보도록 수정해서 다른 몸이 있을 경우에도 스크립트를 공유할 수 있도록 한다.



최대 걷기 속도도 캐릭터와 비교해서 약간 느리게 적용해서 몸이 플레이어를 추월하지 않도록 하자



13.2 Ai 이동 / 공격 관련 변수

무작위(Random) 이동을 구현하려면 우선 이동의 목적지가 필요하다. 이에 해당하는 변수를 **"targetPosition"**으로 정하자. **"targetPosition"**은 3D 좌표를 사용하므로 타입은 **Vector**로 설정한다.

플레이어를 타겟으로 공격하기 위해서는 공격 대상이 될 플레이어 액터를 저장할 변수가 필요하다. 이 변수 이름은 **"targetActor"**로 정하고, 타입은 **Actor**로 설정한다. 플레이어와 몸은 모두 **PaperCharacter**를 부모로 생성한 클래스이다. **PaperCharacter**는 **Actor** → **Pawn** → **Character**로 이어지는 상속 계층을 따른다는 점을 기억하자. 가능한 한 구체적인 자식 타입 대신 상위 클래스 타입을 사용하면, 다양한 몸 클래스에서 더 유연하게 재사용할 수 있다.

플레이어를 감지하는 방법에는 시야각(원뿔 형태), 직사각형 영역, 원형 범위, X축 거리만 확인하는 방법 등 여러 방식이 있다. 여기서는 몸을 중심으로 한 원(Circle)을 사용하고, 이 원 안에 플레이어가 들어왔는지를 검사하는 방식을 사용한다. 이때 몸이 플레이어를 감지할 수 있는 최대 반경을 나타내는 변수를 **"targetActorFindRange"**로 정한다.

근접 공격 가능 여부도 마찬가지로 원형 범위를 사용한다. 근접 공격이 닿는 거리를 나타내는 변수는 **"attackRange"**로 정한다.

여기서 잠깐, “플레이어 감지와 근접 공격 판정을 충돌(콜리전)으로 처리하면 더 정확하지 않을까?” 라는 생각이 들 수 있다. 실제로 충돌을 사용하면 더 디테일한 판정이 가능하다. 다만, 이 단원에서는 원형 범위를 이용한 방식이 구현과 이해 측면에서 더 단순하므로, 원을 이용한 거리 기반 판정을 사용한다.

변수 초기화와 관련해서, 액터 타입 변수는 기본값을 **None**(없음)으로 두면 유효한지(**IsValid**) 여부를 쉽게 검사할 수 있다. 하지만 무작위 이동에 사용할 목적지 위치(**Vector**)는 **None**으로 표현할 수 없다. 이 책에서는 “아직 목적지가 설정되지 않았다”는 의미를 나타내기 위해 (-1000.0, -1000.0, -1000.0) 같은 특수한 값을 초기값으로 사용한다.

플레이어에게 줄 데미지 값을 표현하기 위해 공격력 변수 **"attackPower"**도 추가한다. 지금까지 설명한 내용을 정리하면, 몸 에 필요한 변수와 초기값은 다음과 같다.

변수	타입	초기값	설명
gameHP	float	100.0	캐릭터 체력 값
targetPosition	vector	-1000.0,-1000.0,-1000.0	무작위 이동의 목적지
targetActor	actor	없음	공격할 타겟 액터(플레이어 캐릭터)
isAttacking	boolean	false	"공격 중" 인지 상태 값
targetActorFindRange	float	300.0	플레이어 위치를 찾는 최대 반경
attackRange	float	90.0	공격으로 데미지를 줄 수 있는 플레이어와의 최대 거리
attackPower	float	20.0	공격력. 플레이어에게 주는 데미지

추가된 변수들

gameHP	플로트
targetPosition	벡터
targetActor	액터
isAttacking	부울
targetActorFindRange	플로트
attackRange	플로트
attackPower	플로트

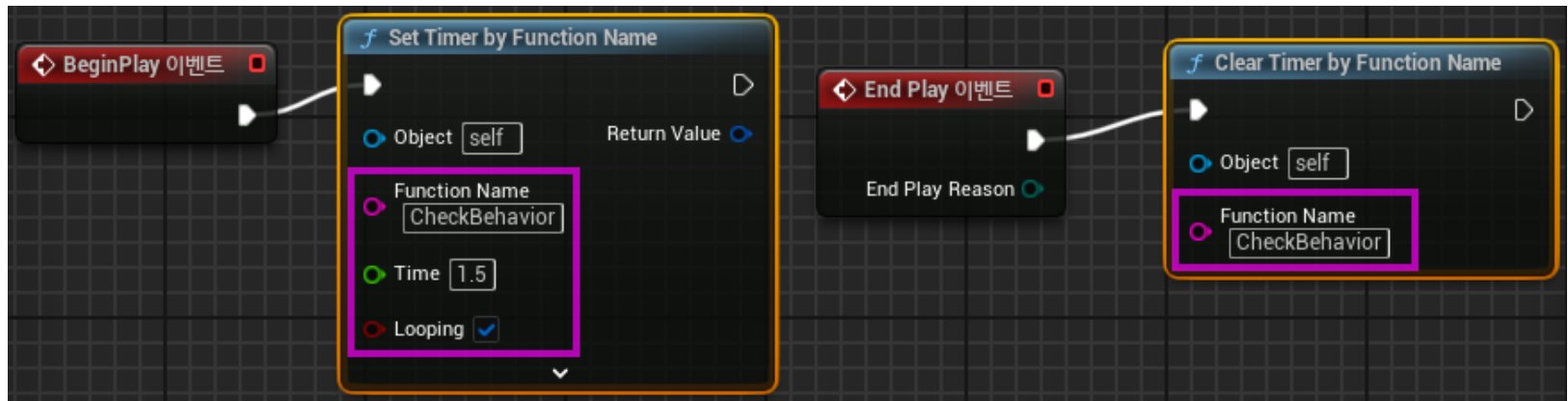
13.3Begin 이벤트

게임에서 사용하는 몸의 AI는 궁극적으로 이동과 플레이어 타겟팅, 그리고 공격이다. 이것을 간단히 그래프로 그리면 다음과 같이 플레이어를 찾고, 찾은 플레이어가 있다면 플레이어 위치로 이동하고 플레이어를 공격한다. 플레이어를 못찾으면 무작위 이동 위치를 찾아서 이동한다.



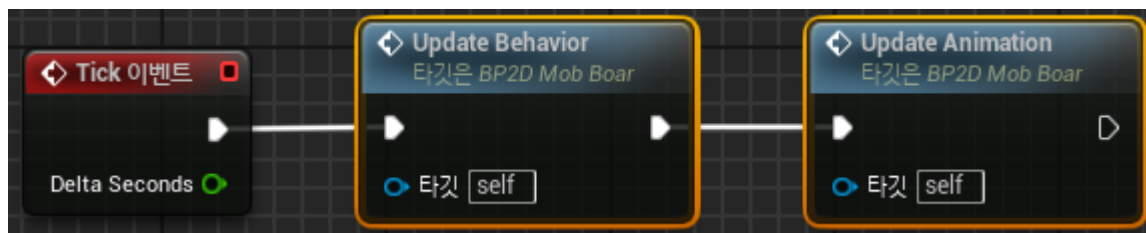
그래프는 간단하지만 프로그램으로 구현하려면 몇개의 블루프린트 함수를 알고 있어야 한다. 먼저 주기적으로 앞서 보인 그래프를 실행해야 하는데 여기에 알맞는 함수가 **"Set Timer by Function Name"** 함수다. 이함수는 특정 시간(초) 마다 **Function Name**으로 주어진 함수를 실행한다. 타이머는 시스템 자원을 사용한다. 따라서 **End Play**에서 해제해야 메모리 누수를 방지할 수 있다.

"Set Timer by Function Name" 함수에 적용할 함수를 이름으로 전달하기에 주기적으로 호출 하는 함수를 구현 하지 않아도 컴파일에는 문제 없다. 주기적으로 실행할 함수를 **"CheckBehavior"**로 정하자.



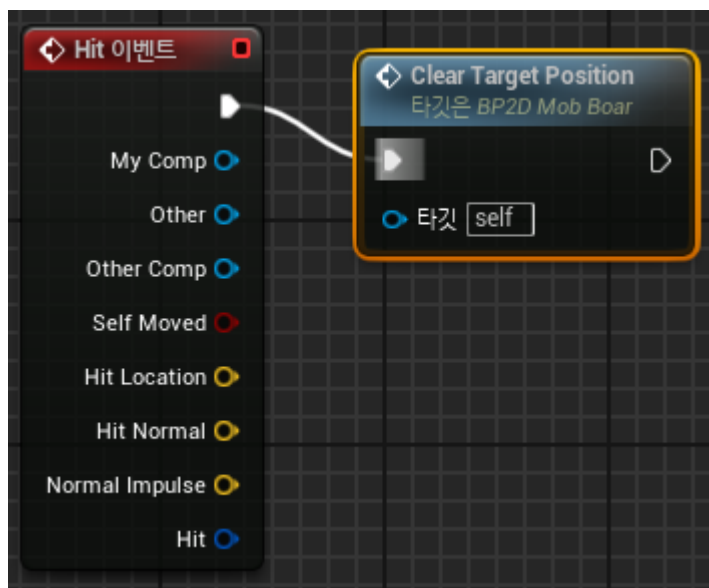
13.4 Tick 이벤트

Tick 이벤트에서 몸의 행동을 업데이트 하는 함수와 애니메이션을 업데이트 하는 함수의 노드를 구성한다. 내 블루프린트의 함수 섹션 추가 버튼(+)을 눌러서 "UpdateBehavior" 함수를 추가하고 노드를 완성하자. UpdateBehavior 함수 구현은 뒤에서 자세히 할 것이다.



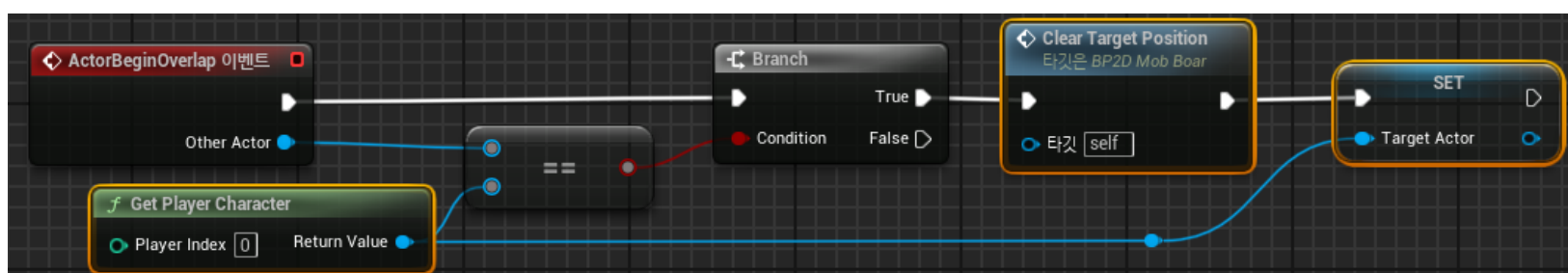
지금까지 과정을 충실히 따라왔다면 몸의 Block 충돌은 타일 맵이나 static, 또는 Dynamic static에서만 발생할 것이다. 이것은 이동 관련 충돌이고, Hit 이벤트가 발생하는데, Hit 이벤트가 발생하면 이동을 멈추도록, 무작위 이동 위치를 클리어 한다.

Tick에서처럼 내 블루프린트의 함수 섹션 추가 버튼(+)을 눌러서 "ClearTargetPosition" 함수를 추가하고 노드를 완성하자. "ClearTargetPosition" 함수 구현도 뒤에서 자세히 할 것이다.



13.5 BeginOverlap 이벤트

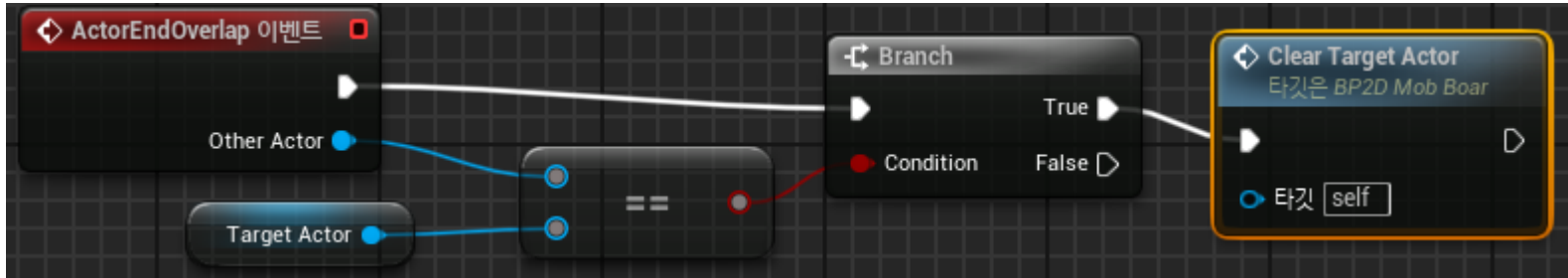
Overlap 이벤트는 Pawn만 발생하도록 했었다. 플레이어 캐릭터와 Mob 캐릭터 모두 PaperCharacter 기반으로 만들어져 있어서 Overlap 이벤트가 발생할 때 플레이어 인지 비교하고, 플레이어만 targetActor에 저장한다. 이 때 이동 중에 발생할 수도 있기 때문에 "ClearTargetPosition" 함수를 호출 해서 랜덤 이동이 아닌 플레이어를 추적하거나 플레이어를 공격하는 상태가 되도록 한다.



13.6 EndOverlap 이벤트

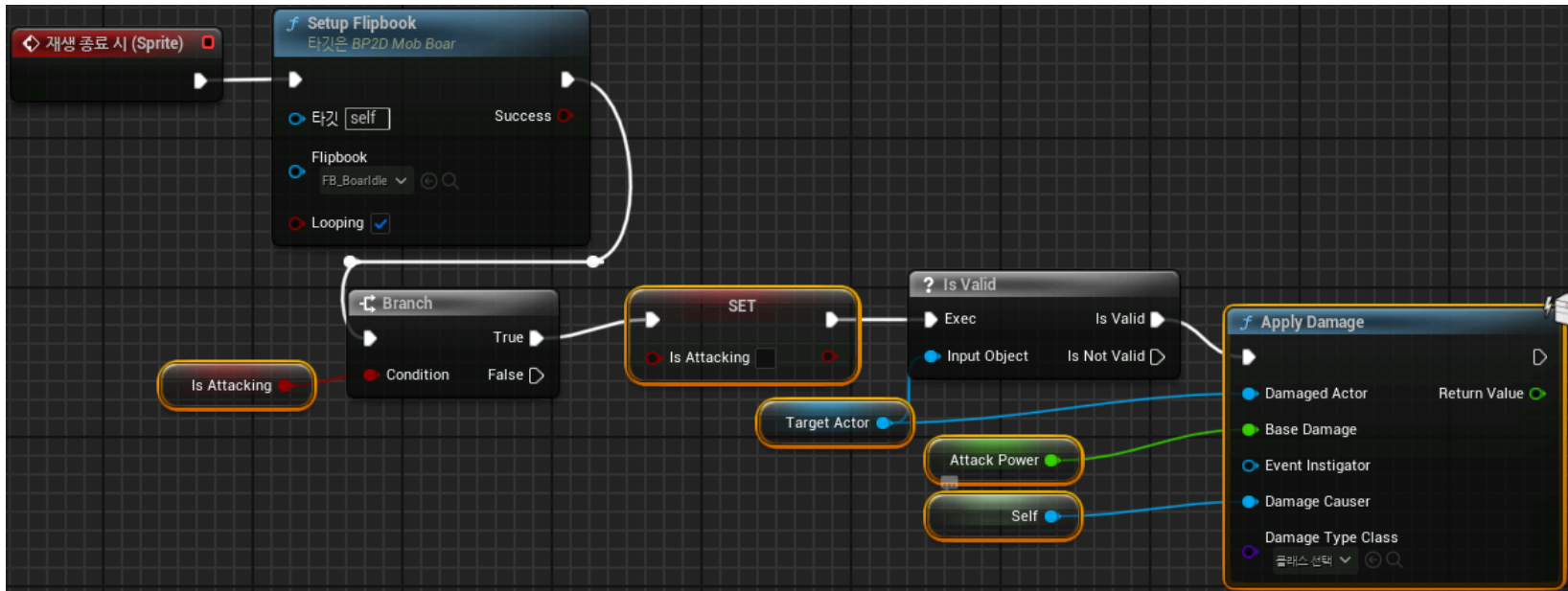
End Overlap 이벤트에서도 저장된 targetActor와 비교해서 타겟 액터를 해제한다.

앞서 Tick, Hit 에서처럼 "ClearTarget" 함수를 생성하고 노드를 완성한다. "ClearTarget" 함수 또한 뒤에서 구현할 것이다.



13.7 재생 종료 이벤트

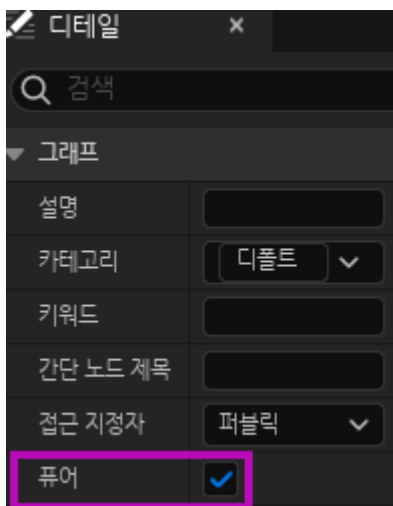
"재생 종료 시(Sprite)" 이벤트는 루프가 아닌 애니메이션이 실행 완료 후 발생하는 이벤트이다. 여기에 해당하는 이벤트는 공격, 데미지, Dead 등이 있다. **isAttacking** 이라는 변수를 사용해서 공격인 애니메이션만 적용하는도록 **Branch**를 사용하고, **targetActor**에 데미지를 주는 함수 **ApplyDamage**에 공격력을 입력한다. **ApplyDamage** 함수 사용은 **Hero** 캐릭터에서 구현해봤음으로 생략한다. **isAttacking** 는 **false**가 되도록 설정한다.



13.8 Pure 함수

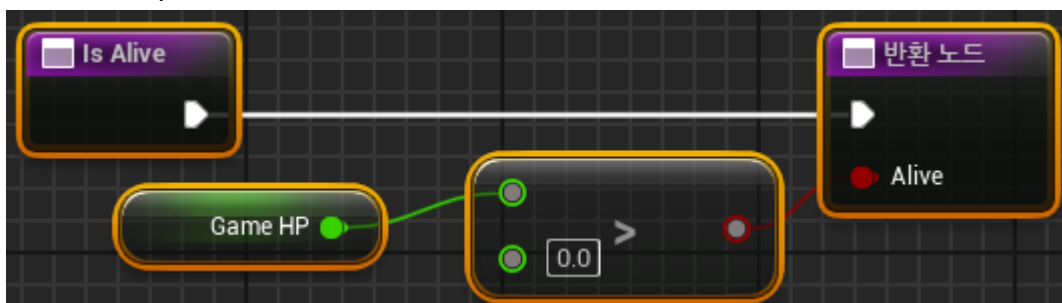
언리얼의 **Pure** 함수는 실행 핀(**Execution Pin**)이 없는 함수로, 입력값을 기반으로 계산된 결과만을 반환하는 함수이다. **Pure** 함수는 클래스의 상태나 멤버 변수를 변경할 수 없으며(**Side Effect**가 없음), 그래프 내에서 필요할 때마다 자동으로 계산된다. 이러한 특성 덕분에 그래프가 단순해지고, 수학적 연산이나 조건 판정처럼 부작용이 없는 데이터 처리에 유용하게 사용된다.

Pure 함수로 만들려면, 함수 에디터의 디테일 패널에서 '**Pure**' 속성을 활성화하면 된다. 이 옵션을 켜면 해당 함수는 실행 핀 없이 값을 계산하는 **Pure** 함수로 동작하게 된다.



13.9 IsAlive 함수

Is Alive는 간단히 **Hp**가 0보다 크기만 하면 된다.



13.10 HasTargetPostion 함수

무작위 이동에서 목적지가 있는 지 확인 함수이다.

벡터는 "없음(NONE)"으로 처리할 수 없기에 캐릭터가 도달할 수 없는 특정한 위치를 지정하고 비교한다. 현재 프로젝트는 -1000, -1000, -1000을 유효하지 않은 디폴트 위치로 정했고, 이 위치와 같으면 캐릭터가 무작위 이동에서 목적지가 없는 것으로 간주한다.



13.11 HasTargetActor 함수

IsValid 노드로 간단히 구현할 수 있다.



13.12 CanAttack 함수

CanAttack 함수는 몬스터(Mob)가 플레이어를 공격할 수 있는지를 판단하는 함수이다.

판정 기준은 단순하다. 몸과 플레이어 사이의 거리가 몸이 가진 '공격 가능 거리(attackRange)'보다 작은지 확인하면 된다.

2D 게임(Paper2D)은 보통 X-Y 평면을 사용하므로 거리 계산은 다음과 같다.

플레이어와 몸의 거리 D

$$D = \sqrt{(Player.x - Mob.x)^2 + (Player.z - Mob.z)^2}$$

그러나 sqrt(root를 계산하는 함수) 함수는 비용이 크기 때문에, 실제 게임 코드에서는 제곱 근사를 사용하여 연산을 줄인다.
거리 비교

$$\sqrt{(Player.x - Mob.x)^2 + (Player.z - Mob.z)^2} \leq attackRange$$

이것은 제곱 비교와 동일한 의미를 가진다

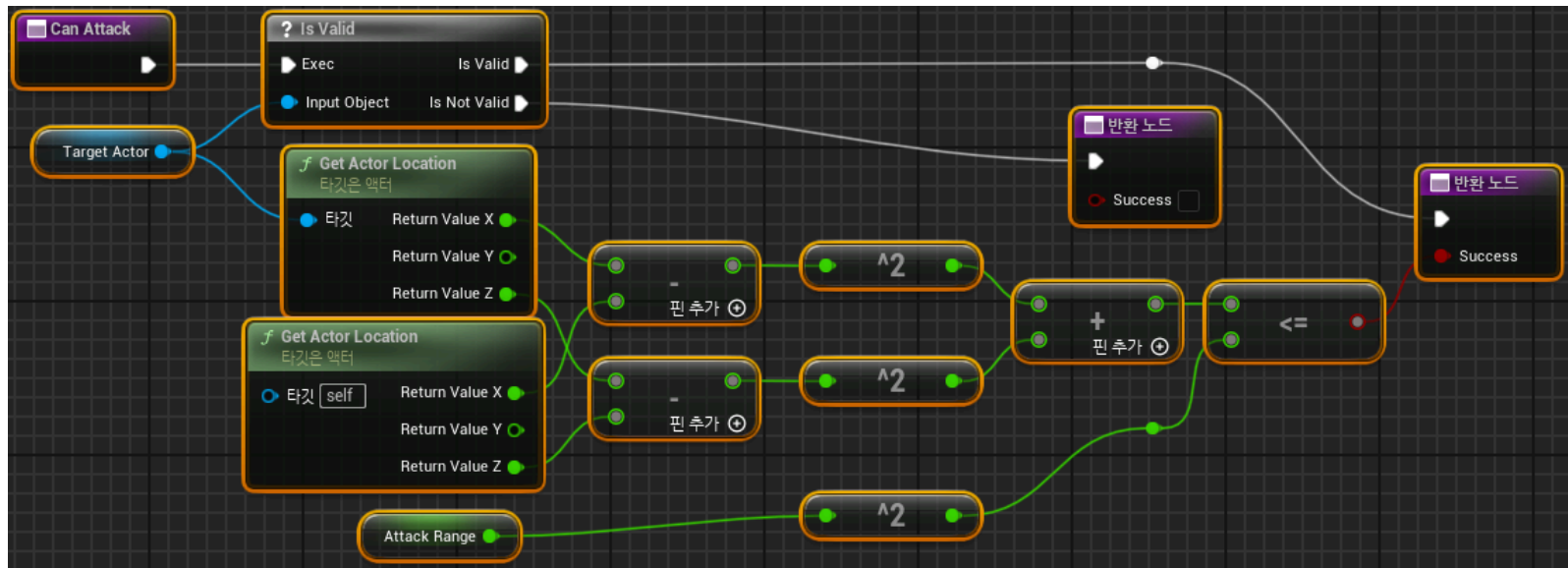
$$(Player.x - Mob.x)^2 + (Player.z - Mob.z)^2 \leq attackRange^2$$

두 식은 같은 의미를 가지므로, 다음과 같이 제곱 값끼리 비교하면 성능이 향상된다.

C++ 코드로 표현하면 다음과 같이 만들 수 있을 것이다.

```
bool CanAttack() {
    if( false == IsValid(targetActor))
        return false;
    float distSquare = (targetActor.GetActorLocation().x - self.GetActorLocation().x)^2 +
                      (targetActor.GetActorLocation().z - self.GetActorLocation().z)^2;
    return ( distSquare <= attackRange^2);
}
```

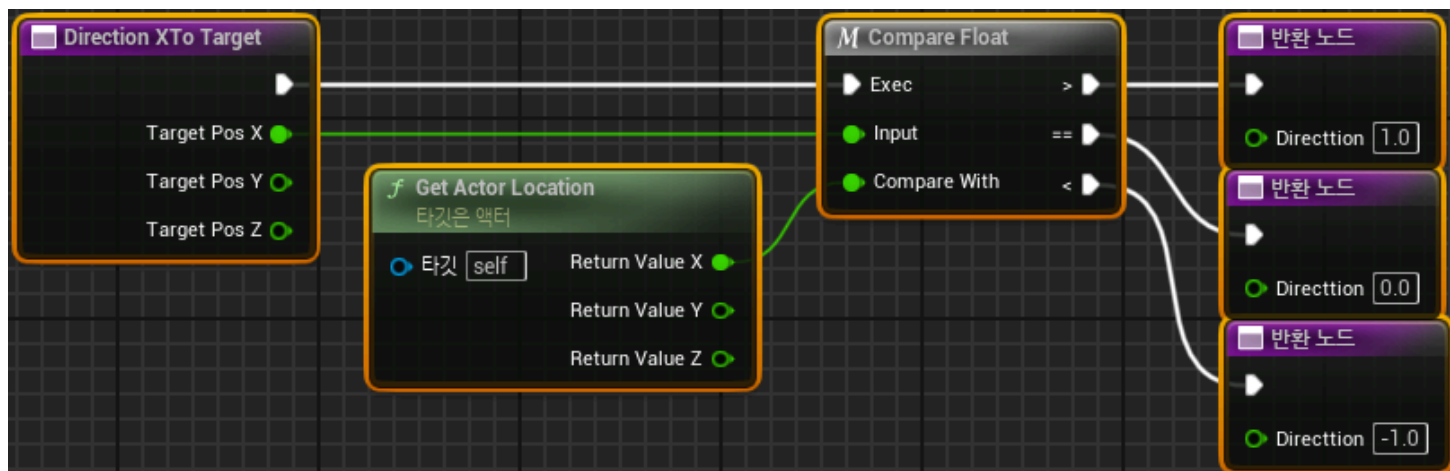
수식은 간단한데 제곱으로 비교하는 블루 프린트가 복잡해 보인다. 주의 깊게 작성하자.



13.13 DirectionXToTarget 함수

DirectionXToTarget 함수는 몸 기준에서 플레이어가 어느 방향에 있는지를 판단하는 함수이다. 플레이어와 몸의 X좌표 차이를 기준으로 왼쪽(-1), 같은 위치(0), 오른쪽(1)을 반환한다. 가장 단순한 구현은 다음과 같다.

- $PlayerX > MobX \rightarrow 1$ (플레이어가 몸의 오른쪽에 있음)
- $PlayerX == MobX \rightarrow 0$ (정확히 같은 X에 있음)
- $PlayerX < MobX \rightarrow -1$ (플레이어가 몸의 왼쪽에 있음)



13.14 IsInAttackRange 함수

IsInAttackRange 함수는 플레이어가 공격 가능 범위 안에 있는지를 판정하는 함수이다. 몸과 플레이어 사이의 거리를 계산한 뒤, 그 거리가 몸의 공격 범위(targetActorFindRange)보다 작거나 같은지 확인하여 true 또는 false를 반환한다.

CanAttack 함수와 유사하게 성능을 위해 거리의 제곱 값을 이용한다.

- $"\text{플레이어와 몸 사이 거리}^2 \leq \text{targetActorFindRange}^2$

간단히 C++ 코드로 만들 수 있다.

```
bool IsInAttackRange() {
    FVector playerPosition = GetPlayerCharacter(0).GetActorLocation();
    FVector selfPosition = self.GetActorLocation();
    float distSquare = (playerPosition.x - selfPosition.x)^2 +
                      (playerPosition.z - selfPosition.z)^2;
    float targetActorFindRangeSquare = targetActorFindRange^2;
    if( distSquare <= targetActorFindRangeSquare)
        return true;
    else
        return false;
}
```

수식 때문에 복잡해 보이지만 구현 하기는 어렵지 않다.



13.15 IsInTargetPosition

IsInTargetPosition 함수는 무작위 이동(Random Move)에서 몸이 목적지에 도달했는지를 판단하는 함수이다. 이동은 프레임 단위로 이루어지기 때문에, 이동 속도와 **DeltaTime**에 따라 목적지 좌표와 정확히 일치하지 않는 경우가 자주 발생한다. 이 상태에서 ‘목적지와 정확히 같은 좌표인지’를 계속 확인하면, 몸이 끝없이 이동을 반복하는 문제가 생길 수 있다. 따라서 일정 허용 범위(Tolerance)를 두고, 목적지에 충분히 가까워지면 도달한 것으로 판단하여 **true**를 반환한다. 이렇게 함으로써 몸이 목적지 주변에서 불필요하게 이동을 반복하지 않도록 한다.

또한 이 함수는 **CanAttack**, **IsInAttackRange**와 마찬가지로 ‘거리 비교’를 수행하는 함수이므로 구현 구조가 매우 유사하다. 간단히 **C++** 의사 코드로 다음과 같이 만들 수 있다.

```
bool IsInTargetPosition(float tolerance, FVector targetPosition) {
    FVector selfPosition = self.GetActorLocation();
    float distSquare = (targetPosition.x - selfPosition.x)^2 +
                      (targetPosition.z - selfPosition.z)^2;
    float toleranceSquare = tolerance^2;
    return ( distSquare <= targetActorFindRangeSquare);
}
```

IsInTargetPosition 함수는 **Blueprint**에서도 동일하게 구현할 수 있으며, 목적지 위치와 현재 위치의 차이를 이용하여 거리의 제곱을 계산한 뒤 허용 범위(Tolerance)와 비교한다.



13.16 SetupTargetPosition

SetupTargetPosition 함수는 몸이 무작위(Random) 이동을 하기 위해, 새로운 목적지(Target Position)를 설정하는 기능을 담당한다. 이 함수는 **Impure** 함수이므로 실행 핀을 사용하며, 내부에서 멤버 변수인 **targetPosition** 값을 직접 변경한다.

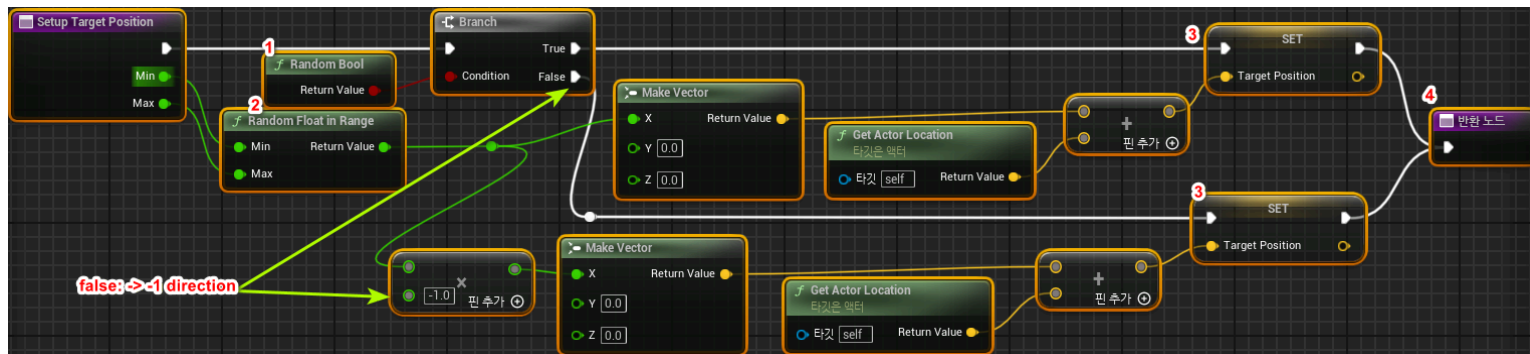
전체 동작 흐름은 다음과 같다.

- 1) 왼쪽 또는 오른쪽 중 어느 쪽으로 이동할지 결정
 - **Random Bool** 노드가 **true / false** 중 하나를 반환한다.
 - **true** → 오른쪽
 - **false** → 왼쪽
- 2) X축 이동 거리(random offset) 생성
 - **Random Float in Range** 노드는 설정된 범위(Min~Max) 안에서 무작위 값을 하나 생성한다.
 - 이 값은 몸이 한 번에 얼마나 이동할지를 결정하는 거리이다.
 - 왼쪽으로 이동할 경우, X값에 -1을 곱해 반대 방향의 값으로 만든다.
- 3) 새로운 목적지 벡터 생성
 - 현재 몸의 위치 (**GetActorLocation**)에 무작위 이동 거리 벡터를 더해서(+) 최종 목적지(Target Position) 좌표를 계산한다.

- Y와 Z값은 0으로 두었는데, 2D 게임(Paper2D)에서는 X축 이동만 필요하기 때문이다.
- 새로운 목적지 = 현재 캐릭터의 위치 + Random으로 얻은 위치

4) 계산된 위치를 Target Position 변수에 저장

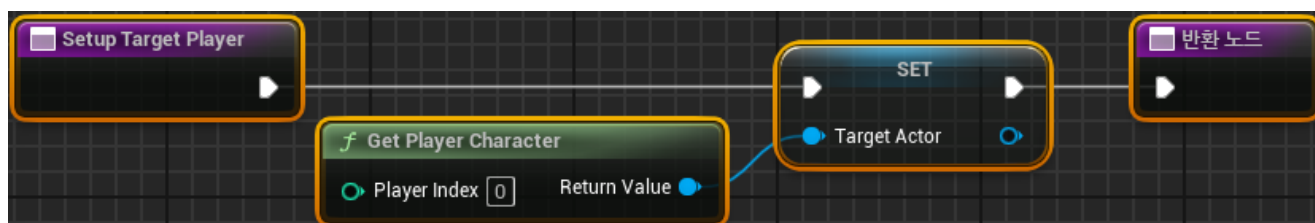
- 계산된 Vector 값을 SET Target Position 노드로 저장한다.



13.17 SetupTargetPlayer

SetupTargetPlayer 함수는 몸이 공격할 플레이어를 지정하는 함수이다.

플레이어의 위치를 찾거나 공격 가능한지 판단하는 모든 AI 로직은 이 변수(targetActor)를 기반으로 동작한다. 언리얼의 플레이어 캐릭터는 기본적으로 Player Index 0번에 배치되어 있으므로, Get Player Character 노드를 사용하여 해당 플레이어의 캐릭터를 가져온 뒤, 몸이 참조할 변수인 targetActor에 저장한다.



13.18 ClearTargetPosition

ClearTargetPosition 함수는 targetPosition 값을 “유효하지 않은 위치”로 초기화하는 함수이다. 무작위 이동이 끝났거나, 다른 동작으로 전환할 때 기존 목적지 값을 그대로 두면 이후 로직에서 잘못 사용될 수 있다. 이를 방지하기 위해 (-1000, -1000, -1000)처럼 게임에서 절대 도달할 수 없는 특수한 좌표를 넣어 목적지를 초기화한다. 이 값은 “현재 이동할 목적지가 없음”을 의미한다.



13.19 ClearTargetActor

ClearTargetActor 함수는 몸의 타겟 액터(targetActor)를 초기화하는 함수이다.

이 함수는 targetActor 값을 ‘없음(None)’ 상태로 설정하여, 현재 몸이 추적하거나 공격해야 할 플레이어가 존재하지 않는 상황을 만들 때 사용된다.

이를 통해 몸은 더 이상 플레이어를 타겟으로 삼지 않고, 이후 로직에서 무작위 이동 또는 새로운 타겟 탐색을 수행할 수 있게 된다.



13.20 MovePositionX

MovePositionX 함수는 입력으로 받은 TargetPos 위치를 기준으로, 몸을 X 방향으로만 이동시키는 함수이다. TargetPos는 무작위 이동의 목적지(targetPosition)가 될 수도 있고, 플레이어의 현재 위치가 될 수도 있다.

이 함수의 전체 동작 순서는 다음과 같다.

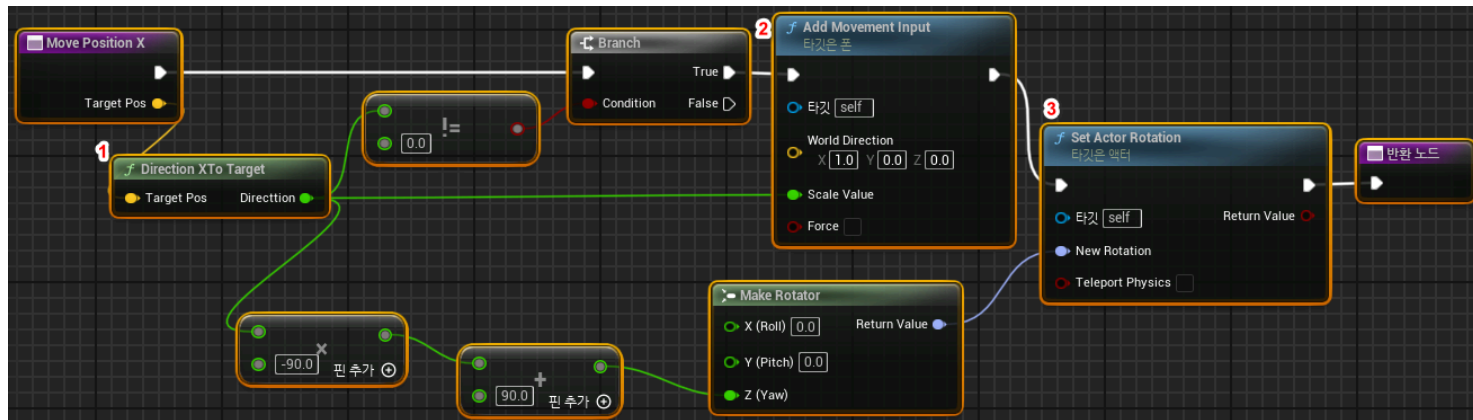
1) DirectionXToTarget 함수를 호출하여 타겟이 몸의 오른쪽에 있는지(1), 왼쪽에 있는지(-1), 같은 X 위치에 있는지(0) 확인 하고 반환되는 Direction 값이 0이면, 이동할 필요가 없으므로 Branch에서 바로 종료한다.

2) Direction 값이 1 또는 -1일 경우, 타겟이 몸의 왼쪽 또는 오른쪽에 있다는 의미이므로 이동을 시작한다. Add Movement Input 노드를 통해 몸을 X축 방향(±1.0, 0.0, 0.0)으로 이동한다.

3) 2D 게임에서는 이동 방향에 따라 좌우 Flip 또는 회전이 필요하다. Direction이 1 → 오른쪽 -1 → 왼쪽 이므로, 이를 기반으로 다음 값을 만든다.

- Direction +1.0 (오른쪽 보기) ⇒ Yaw 0°
- Direction -1.0 (왼쪽 보기) ⇒ Yaw 180°

블루프린트에서는 Make Rotator 노드로 회전 값을 생성하고 SetActorRotation으로 적용한다.

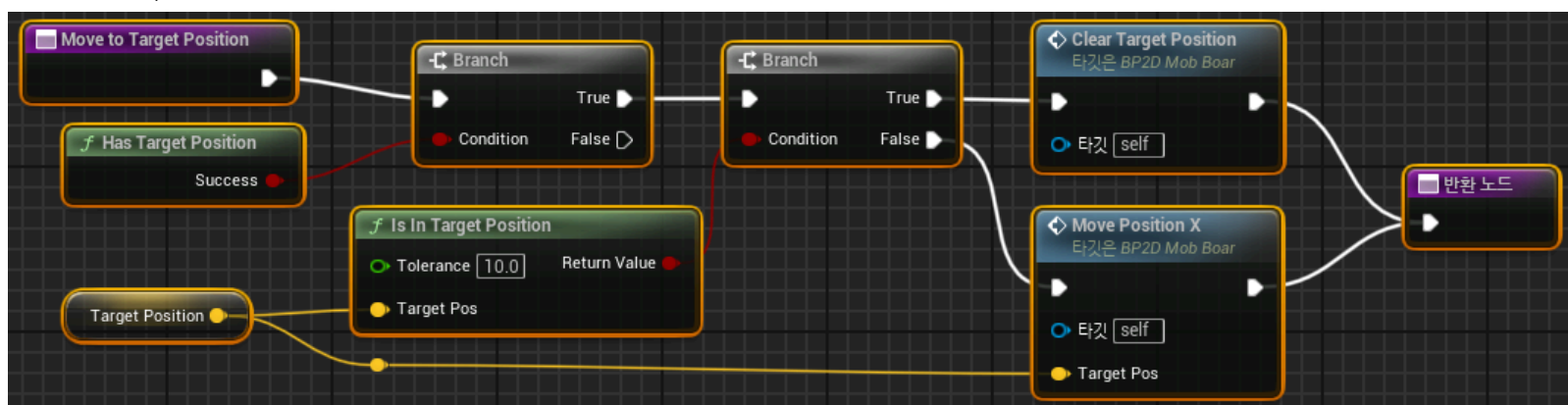


13.21 MoveToTargetPosition

캐릭터는 설정된 **targetPosition**(이동 목적지)로 이동한다. 이동 과정에서는 **Tolerance** 값을 사용하여, 목적지에 정확히 일치하지 않아도 일정 범위 안에 들어오면 도착한 것으로 판단한다. 캐릭터가 목적지에 도달한 것으로 판정되면, **targetPosition** 값을 무효화하여 다음 이동을 준비한다.

이 함수의 전체 동작 순서는 다음과 같다.

- 1) Has Target Position 함수를 호출해 목표 위치가 유효한지 검사한다.
 - Success = True → 다음 단계로 진행
 - Success = False → 즉시 종료 (목표 위치가 없으므로 이동할 필요 없음)
- 2) Is In Target Position 함수는 현재 몸의 위치가 Target Position에 Tolerance(허용 오차) 안에 들어왔는지 검사한다.
 - Tolerance: 10.0
 - Return Value = True → 목표 지점에 도착한 것으로 판단
- 3) 목표 지점에 도착한 경우 Clear Target Position 함수를 호출해 현재 Target Position 값을 초기화한다. 다음 이동을 위해 Target Position을 비워둔다.
- 4) 아직 목표 지점에 도착하지 않은 경우 Move Position X 함수를 호출하여 몸을 Target Position 방향으로 1프레임 이동시킨다. 이 함수는 X축 이동만 처리하므로, 2D 횡스크롤 몸 이동에 적합하다.



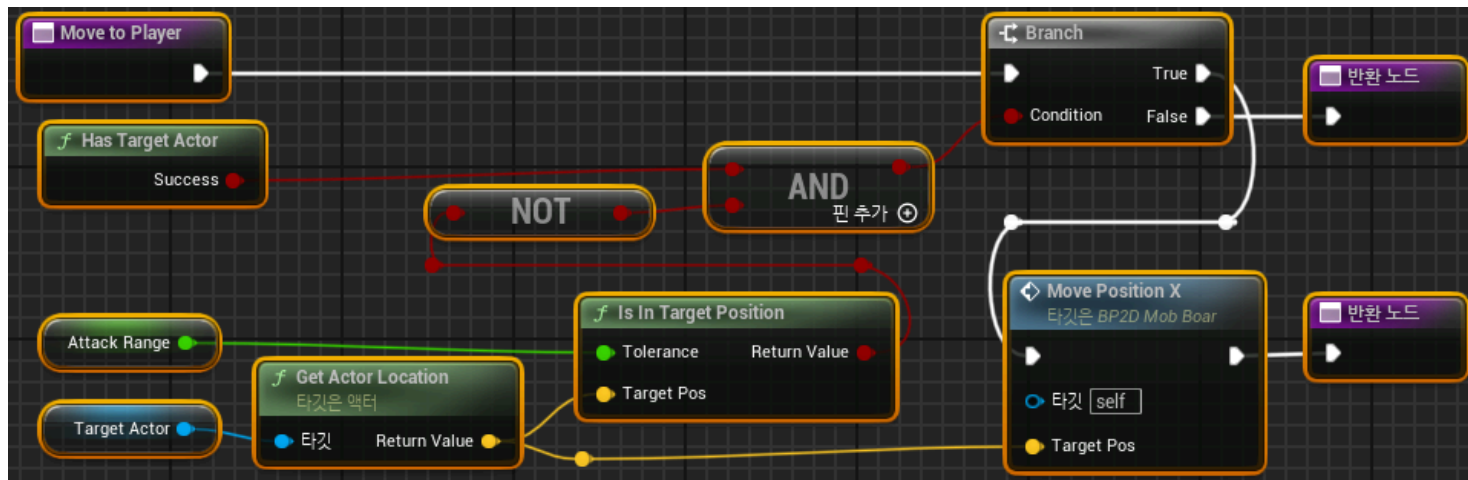
13.22 MoveToPlayer

이 그래프는 몸이 플레이어(Target Actor) 쪽으로 이동하도록 처리하는 함수이다. 공격 범위 밖에 있고, 아직 목적지에 도달하지 않았을 때만 이동을 수행한다.

이 함수의 전체 동작 순서는 다음과 같다.

- 1) Has Target Actor: 먼저 Has Target Actor 함수를 호출하여 현재 목표(플레이어)가 존재하는지 검사한다.
 - Success = False → 이동을 하지 않고 즉시 반환
 - Success = True → 다음 조건 판단으로 진행
- 2) 플레이어(Target Actor)의 위치를 Get Actor Location으로 가져오고, 자신(몸)과의 거리 기반으로 공격 범위 안에 있는지 판단한다.
- 3) Get Actor Location으로 가져온 플레이어 위치를 Target Pos로 전달한다. 이 Target Pos는 아래의 이동 처리에서 직접 사용된다.
- 4) Is In Target Position 함수에서 Tolerance(허용 오차) 값 안에 진입했는지 검사한다.
 - 도착한 경우 → 이동을 멈추고 반환
 - 아직 도달하지 않은 경우 → 아래 이동 로직으로 넘어감

5) Move Position X 함수를 사용해서 플레이어 위치(Target Pos)를 향해 X축으로 이동한다.



13.23 AttackPlayer

AttackPlayer 함수는 캐릭터의 상태를 ‘공격 상태’로 전환하는 역할을 한다. 이 함수에서 상태만 변경하면, 이후 애니메이션 업데이트 과정(UpdateAnimation)에서 자동으로 공격 애니메이션이 선택되어 재생된다. 즉, 공격 동작의 실제 실행은 **UpdateAnimation**이 담당하고, **AttackPlayer** 함수는 그 출발점을 만드는 트리거 역할을 한다.



13.24 CheckBehavior

CheckBehavior 함수는 행동 선택을 결정하는 함수로 “지금 타깃이 있는가? → 없으면 플레이어가 공격 거리 안인가? → 그렇지 않으면 무작위 목적지로 이동하자”라는 순서로 몸의 행동을 선택한다.

함수의 구성은 다음과 같다.

1) Has Target Position + Has Target Actor (OR)

- 현재 무작위 이동 목표 위치가 있거나, 혹은 플레이어 타겟(Actor)이 이미 설정되어 있다면 True가 된다.
- 이 값이 True이면, 첫 번째 Branch에서 곧바로 반환 노드로 이어져 지금 진행 중인 행동을 그대로 유지하고 함수를 종료한다. ⇒ 이미 “할 일”이 있는 상태이므로, 새로운 행동을 다시 계산하지 않는다.

2) Is in Attack Range

- 아직 타깃이 없을 때에만 실행된다.
- 플레이어가 공격 범위 안에 있는지 검사한다.
- True이면 “플레이어를 공격 대상으로 삼을 수 있는 상황”으로 판단한다.

3) Clear Target Position

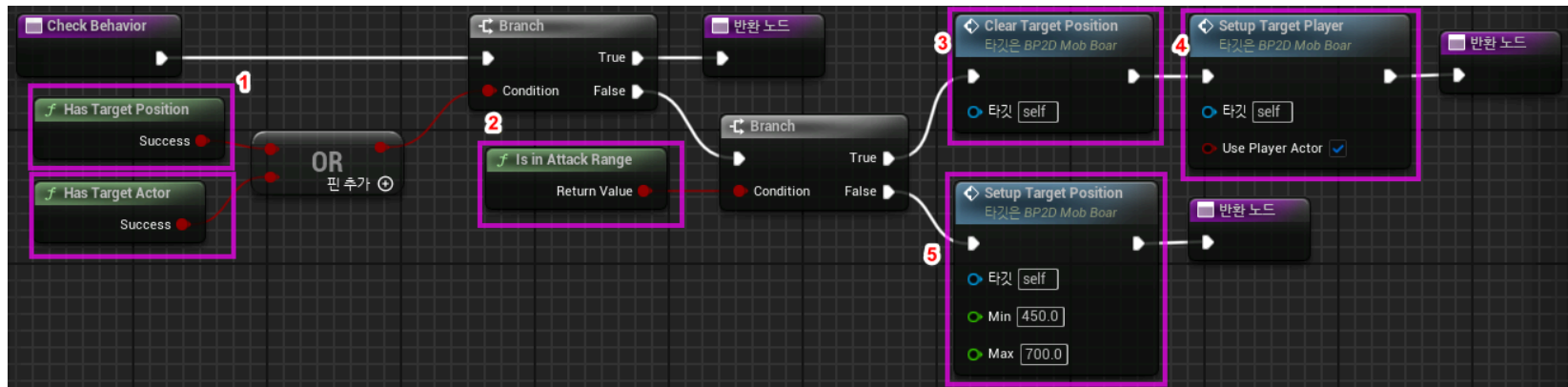
- 공격 범위 안에 플레이어를 발견한 경우, 이전에 사용하던 무작위 이동용 **Target Position**을 모두 지운다.
- 더 이상 랜덤 이동은 필요 없고, 이제 플레이어만 추적하면 되기 때문이다.

4) Setup Target Player (Use Player Actor 체크)

- 플레이어 액터를 ****새로운 타겟(공격 대상)****으로 설정한다.
- 이후 다른 함수(예: **Move to Player, AttackPlayer**)에서 이 타겟 정보를 사용하여 이동·공격을 수행하게 된다.

5) Setup Target Position

- 플레이어가 공격 범위 안에 없고, 타깃도 설정되어 있지 않은 경우 실행된다.
- Min = 450.0, Max = 700.0 범위 안에서 "무작위 이동 목적지(Target Position)" 를 생성한다.
- 이 값은 이후 Move to Target Position 함수에서 사용되어 몸이 주변을 배회(Wander)하거나 순찰하는 것처럼 보이게 만든다.



13.25 UpdateBehavior

UpdateBehavior 함수는 CheckBehavior 함수의 결과와, 충돌 이벤트 및 Overlap 이벤트에서 설정된 플레이어 액터·무작위 목적지 정보를 종합하여 현재 캐릭터가 어떤 행동을 수행할지 결정한다.
함수의 구성은 다음과 같다.

1) IsAlive

- 체력 값(HP)을 사용해서 캐릭터가 현재 살아 있는지를 검사한다. 죽은 상태에서는 어떤 행동도 하지 않아야 하므로 첫 단계에서 필터링한다.

2) HasTargetPosition

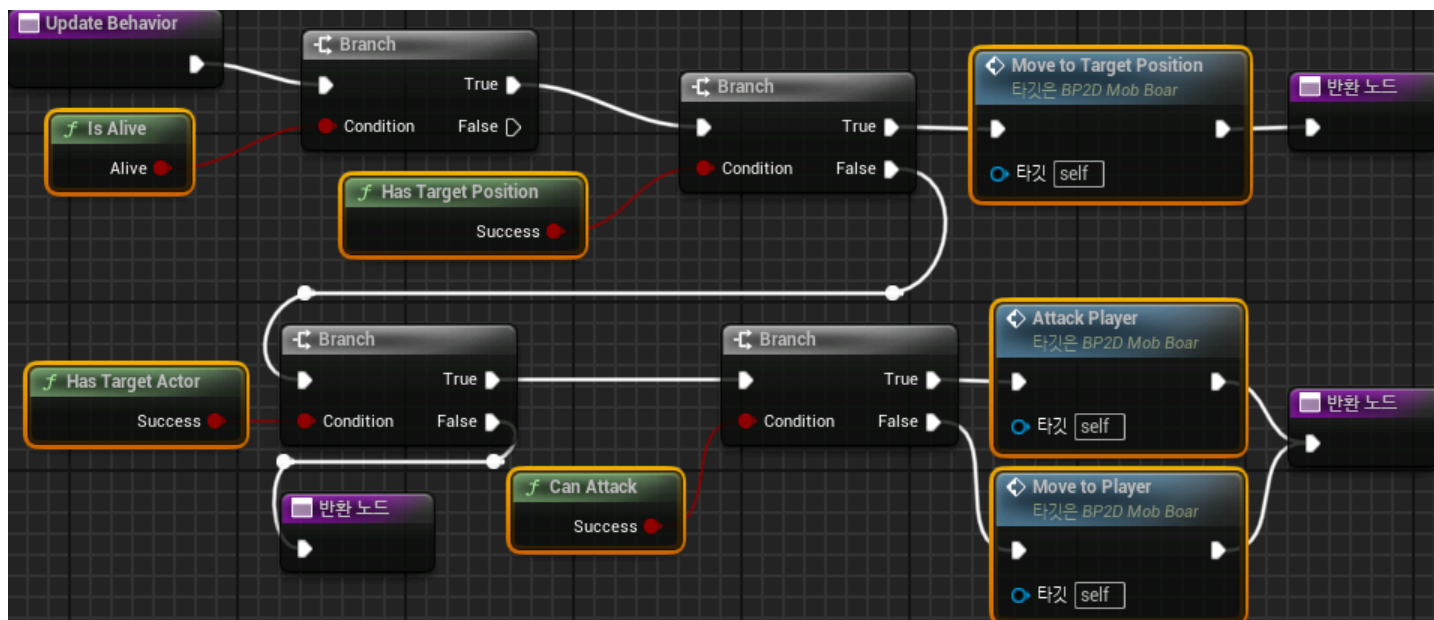
- 이동 목적지(Target Position)가 설정되어 있는지 검사한다. 목적지가 있으면 해당 방향으로 이동한다.

3) Has Target Actor

- 플레이어를 현재 추적 대상으로 가지고 있는지를 검사한다. 플레이어가 공격 대상으로 설정되어 있지 않으면 함수를 종료한다.

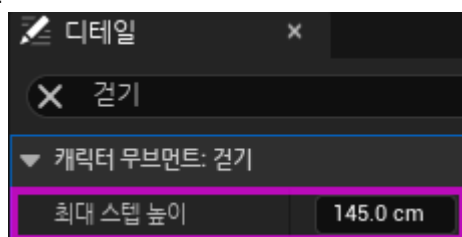
4) Can Attack

- 플레이어가 공격 범위 안에 있는지를 판단한다.
- 공격거리 안에 있으면 즉시 공격 상태로 변경하고, 밖이면 공격할 대상위치로 이동한다.



13.26 플레이어 설정

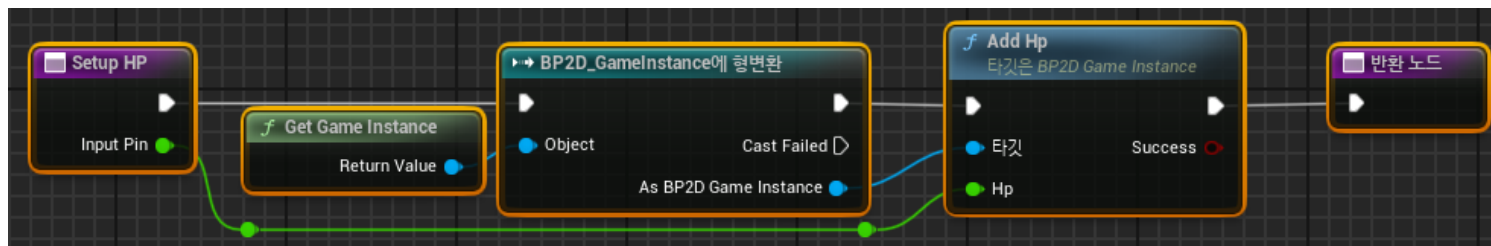
BP2D_Hero1의 이동을 원활하게 하기 위해서 문턱을 자유롭게 넘을 수 있도록 최대 스텝 높이를 올린다.



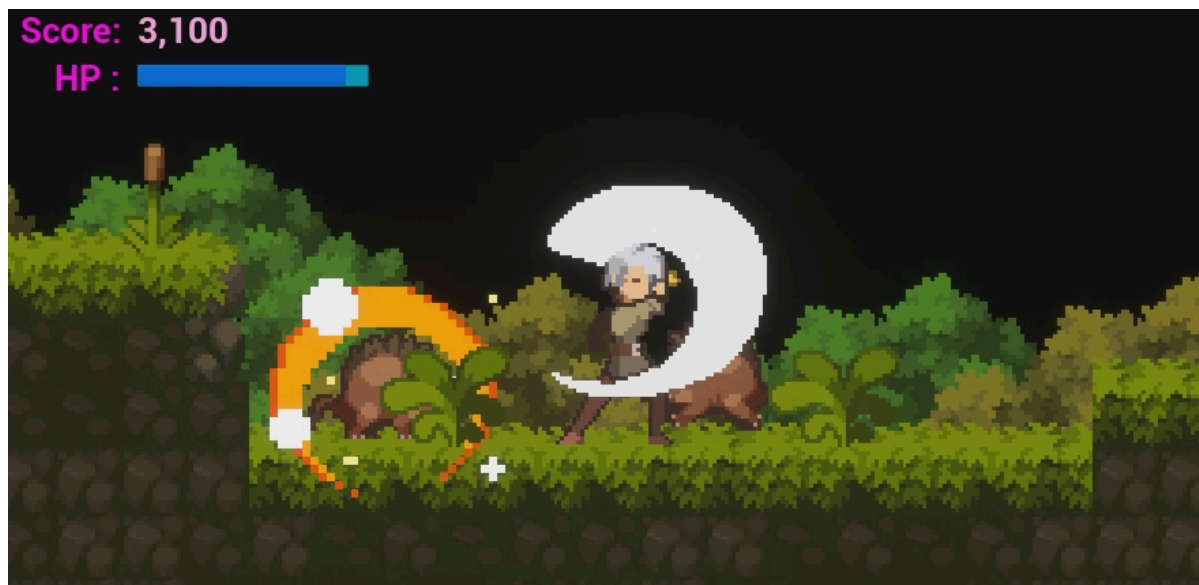
플레이어가 Damage 을 받는 로직을 추가한다. Mob에서 사용한 AnyDamage부분과 유사하게 구현 한다. Damage는 +값이 오는데, -1을 곱해서 AddHp함수 사용에서 문제가 없도록 한다.



몹과 유사하게 SetupHP 함수를 구현한다.
GameInstance의 AddHp 함수를 호출한다.



실행 결과 화면이다.



14. Ai Controller

AI Controller(인공지능 컨트롤러)는 Pawn(특히 Character)을 조종하는 전용 컨트롤러 클래스이다. 게임에서는 자동 사냥, NPC 이동, 몬스터(Mob) AI 등과 같이 규칙 기반의 행동을 구현할 때 주로 사용한다.

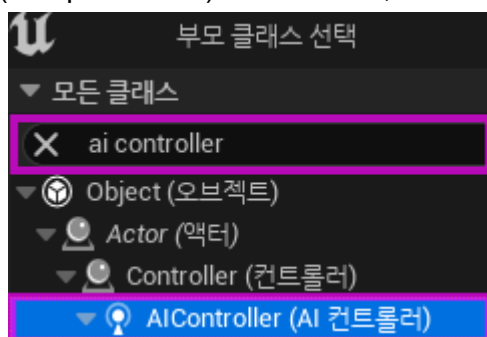
NPC나 Enemy의 행동이 단순한 경우에는 AI Controller 없이 캐릭터 블루프린트의 Tick 이벤트에서 직접 처리해도 충분하다. 하지만 AI 행동을 Character 내부에 계속 추가하다 보면 흐름이 복잡해지기 쉽다.

AI Controller를 사용하면 행동 로직을 분리하여 보다 정돈된 구조를 유지할 수 있다. 이러한 경우 행동 로직을 AI Controller로 분리하면 유지 보수와 확장성이 크게 개선된다.

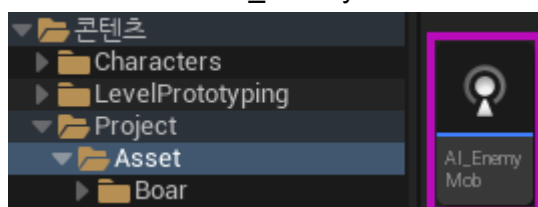
이 단원에서는 몸의 이동을 캐릭터 블루프린트에서 구현하고 AI Controller를 생성해서 그대로 옮길 것이다.

14.1 AIController 생성 및 Character에 연결

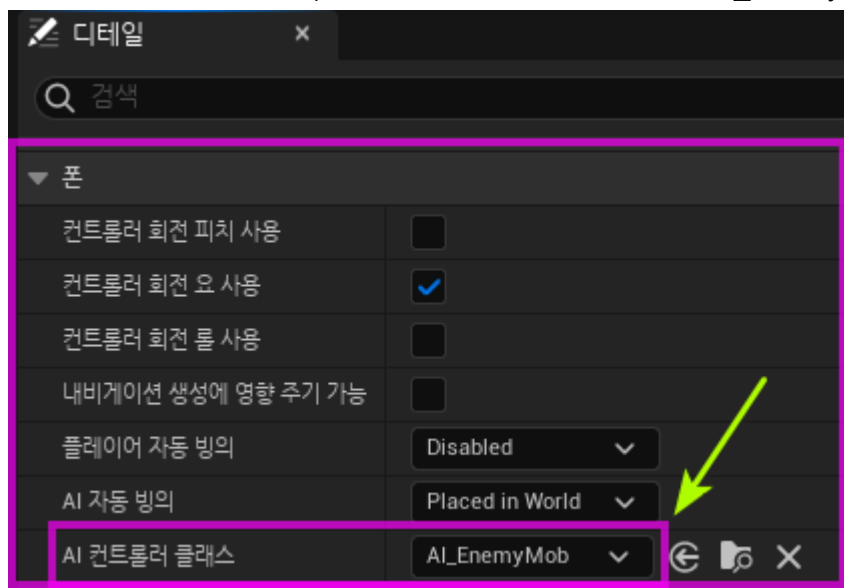
먼저 AI Controller를 생성한다. 콘텐츠 브라우저에서 "/콘텐츠/Project/Asset" 폴더를 선택한 뒤, 폴더의 빈 공간을 우클릭한다. "블루프린트 클래스(Blueprint Class)"를 선택하고, 부모 클래스 선택 창의 상단 검색창에 "ai controller"를 입력한 다음 목록에서 "AIController"를 선택한다.



새 블루프린트 이름은 "AI_EnergyMob"으로 정한다.



Pawn 계열 클래스는 모두 AI Controller를 사용할 수 있다는 점을 기억해 두자. 이제 몸의 행동을 제어할 AI Controller를 몸 캐릭터에 연결해 보자. 몸 블루프린트를 연 상태에서, 캐릭터(또는 루트) 컴포넌트를 선택한 뒤 디테일 패널에서 "Pawn" 섹션을 찾는다. "AI Controller Class(또는 AI 컨트롤러 클래스)" 프로퍼티에서 방금 생성한 "AI_EnergyMob"을 선택한다.



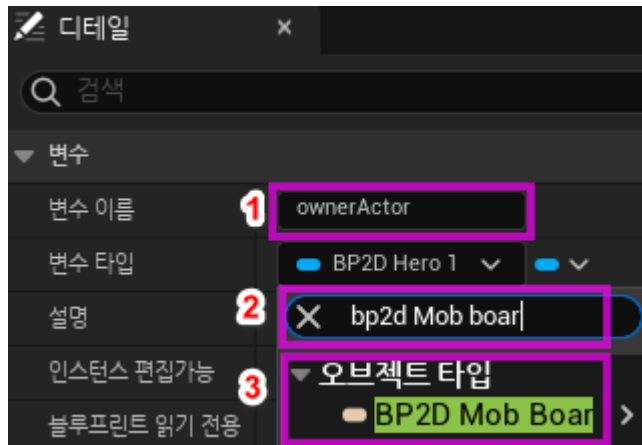
이렇게 하면 몸 캐릭터에 AI_EnergyMob 컨트롤러가 간단히 연결되고, 플레이 중에 몸은 해당 AI Controller를 통해 제어되게 된다.

14.2 OnPossess

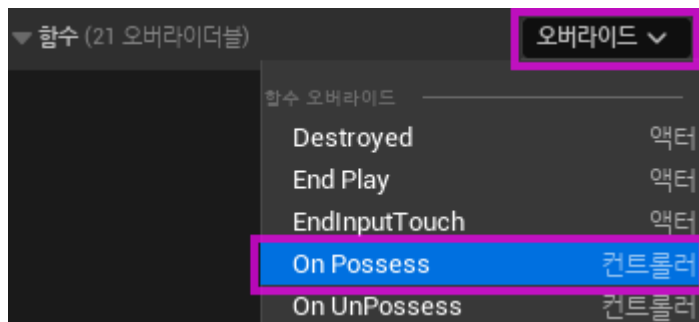
AIController "AI_EnergyMob" 블루프린트를 구성할 차례다.

플레이어를 감지할 수 있는 최대 반경, 근접 공격 가능 범위는 컨트롤러가 아닌 Mob의 특성이다. 이러한 변수를 가져오려면 "Get Controlled Pawn"을 필요할 때 마다 "BP2D_MobBoar"로 캐스팅해서 사용해도 되지만 변수에 추가해 놓고 사용한다면 좀 더 편리할 것이다.

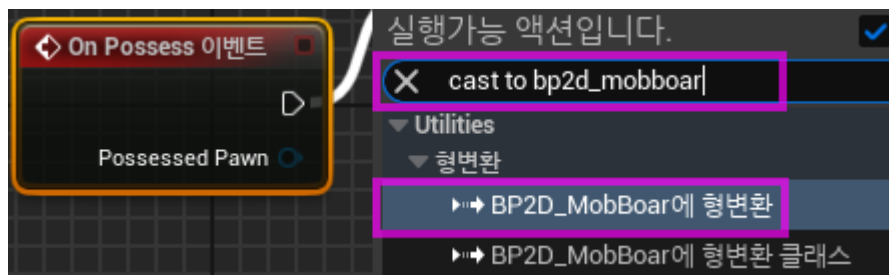
"AI_EnergyMob"의 내 블루프린트 변수 섹션에 ownerActor를 추가한다. 디테일 패널의 검색창에서 "bp2d Mob boar"를 입력하고 "BP2D_MobBoar"를 선택한다.



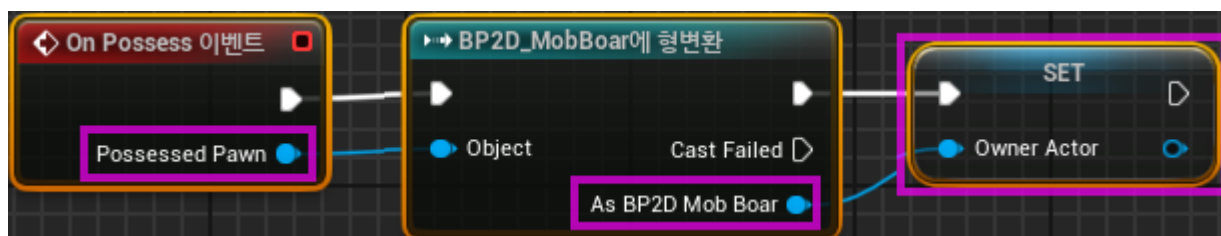
OnPosses 이벤트는 **AIController**에 **Pawn** 클래스가 바인딩 될 때 호출 되는 함수다. **BeginPlay**보다 먼저 호출되며, 바인딩되는 클래스의 오브젝트 레퍼런스를 알려준다.
함수 섹션에서 오버라이드 메뉴 버튼을 클릭하고 "On Posses"를 선택한다.



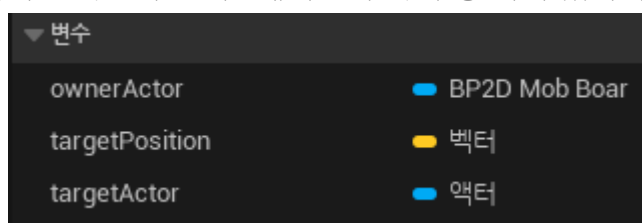
실행 핀의 실행 가능 액션 검색창에서 "cast to bp2d_mobboar"를 입력하고 "BP2D_MobBoar"에 형변환을 선택한다.



"On Posses" 이벤트의 Possesed Pawn 을 형변환 노드에 연결한다.
형변환 노드의 결과인 "As BP2D Mob Boar"를 이전에 생성한 OwnerActor"로 설정한다.



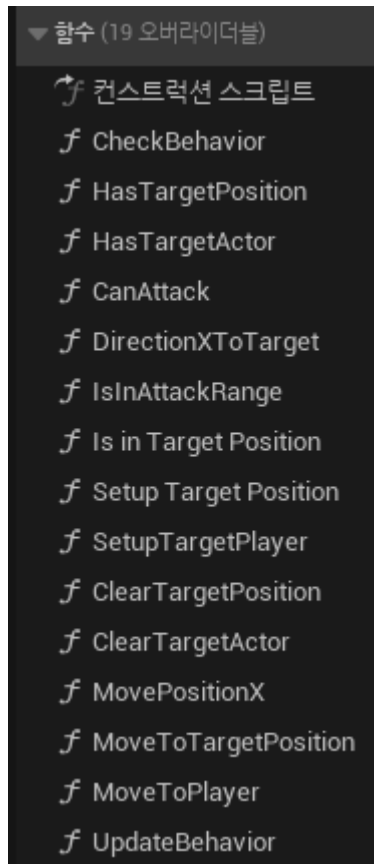
순수 **AIController**에서만 처리할 무작위 이동, 플레이어 추적에 관한 변수 **targetPosition**, **targetActor** 변수를 **BP2D_MobBoar**에서 "CTRL+C" 로 복사해서 내 블루프린트 변수 섹션에 붙여 놓는다.(CTRL+V)
변수 목록은 다음과 같이 3개의 변수가 구성 되어 있어야 한다.



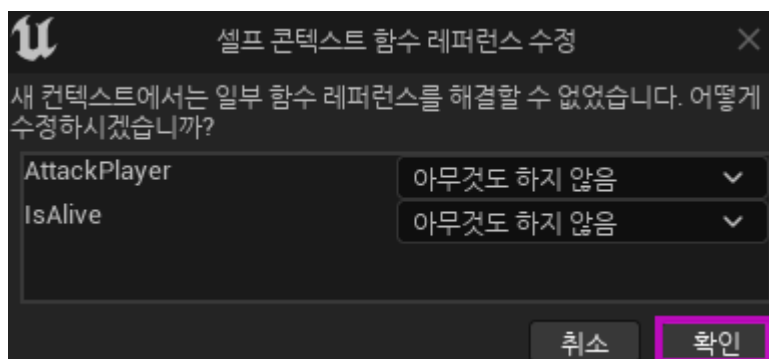
14.3 AIController 변수/함수 설정

14.3.1 함수 복사

변수와 마찬가지로 함수들 또한 복사가 가능하고 해서 붙여 넣기를 진행하면된다.



함수를 복사하다보면 존재하지 않는 함수로 인해서 레퍼런스를 찾을 수 없다는 팝업창이 나온다.



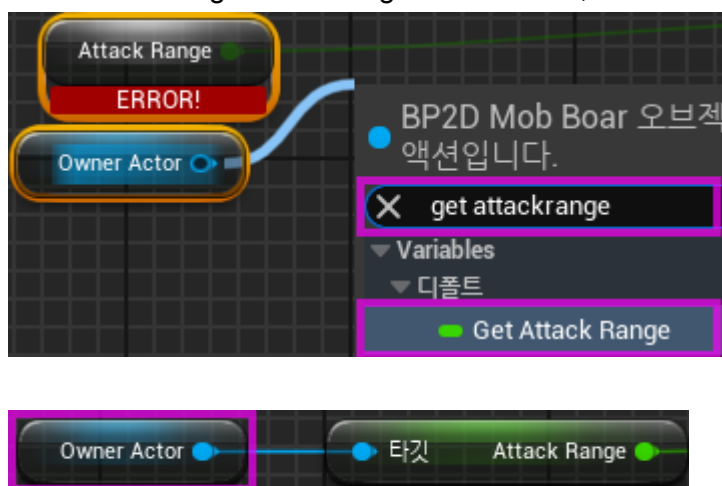
확인을 누르고 컴파일을 하면 해당 함수나 변수가 누락되어 에러가 발생한다. 이 에러들은 BP2D_MobBoar 오브젝트 레퍼런스를 저장하고 있는 변수 **ownerActor**에서 호출 하면 된다.

13.3.2 에러 수정

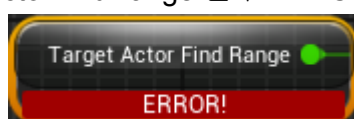
다음과 같이 **attackRange** 변수는 BP2D_MobBoar에만 있고, AIController에는 존재하지 않는다.



OwnerActor 노드에서 "get attackrange"를 검색하고, 목록에서 "Get Attack Range"를 선택하여 연결하면 에러를 해결할 수 있다.



targetActorFindRange 변수도 AIController에 없는 변수다. **attackRange**와 같은 방식으로,

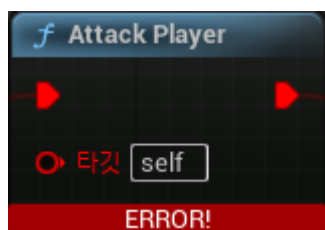


OwnerActor 노드에서 "get target actor find"를 검색하고, "Get Target Actor Find Range" 함수를 선택하여 연결한다.



ownerActor 노드에서 "get target actor find"를 검색하고 "Get Target Actor Find Range"를 선택하면 된다.

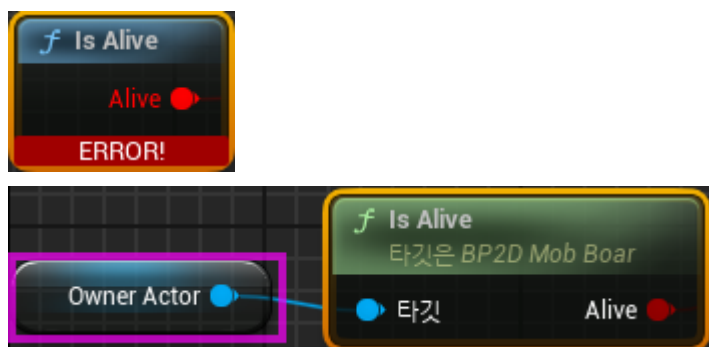
변수뿐만 아니라, AIController에 존재하지 않는 함수를 호출해서도 에러가 발생할 수 있다.



변수 에러를 수정했던 것과 동일한 방식으로, OwnerActor 노드에서 에러가 발생 중인 함수 이름 "attack player"를 검색하고, 목록에서 "Attack Player" 함수를 선택해 연결하면 된다.



IsAlive 함수 호출 에러도 Attack Player 함수와 같은 방법으로, OwnerActor 노드에서 "Is Alive"를 검색해 올바른 함수를 선택해 연결하면 해결된다.

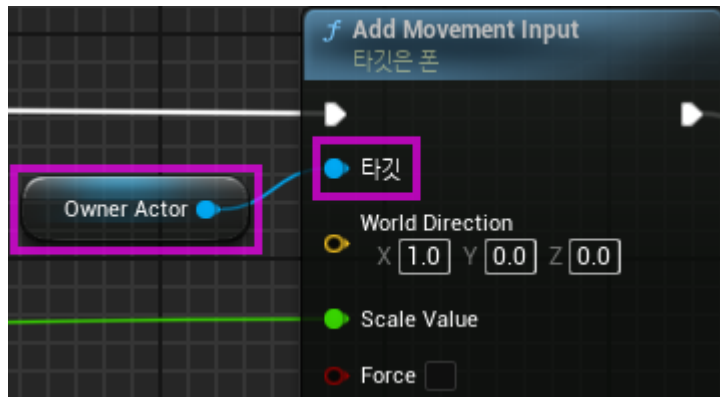


13.3.3 Add Movement Input 타겟 수정

Add Movement Input은 Character의 MovementComponent에 이동 입력을 전달하는 함수다. 따라서 AIController에서 이 노드를 사용하면서 Target을 Self로 그대로 두면, AIController 자신을 대상으로 호출하게 되어 에러가 발생한다.



이 경우 Target 입력 핀에 Self 대신 OwnerActor 노드를 연결해야 한다. 이렇게 하면 실제 몸 캐릭터에 이동 입력이 전달된다.



13.3.4 SetActorRotation 타겟 수정

SetActorRotation 함수는 AIController에서 호출해도 컴파일 에러는 발생하지 않는다. 하지만 기본 Target이 AIController 자신이기 때문에, 회전이 컨트롤러에만 적용되고 캐릭터는 회전하지 않는다.

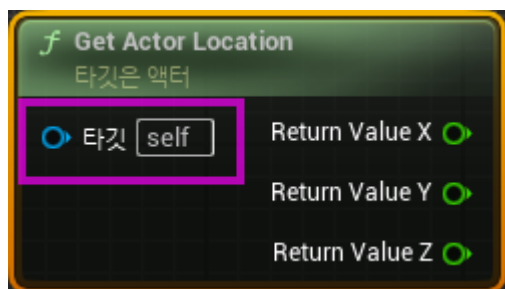
몸의 좌우 반전을 위해서는 SetActorRotation의 Target 핀을 OwnerActor(또는 Get Controlled Pawn) 노드에 연결하여, 실제 캐릭터 액터에 회전 값을 적용해야 한다.



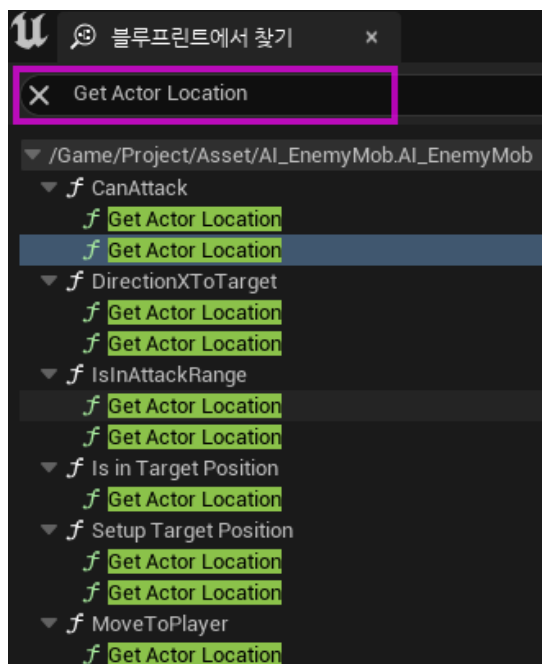
13.3.5 Get Actor Location 수정

함수를 복사해서 AIController로 가져온 뒤 붙여 넣고 컴파일을 완료하면, 수정해야 할 부분이 몇 가지 있다. 그중 대표적인 것이 Get Actor Location 함수 호출 부분이다.

Character 블루프린트에서 사용하던 스크립트를 그대로 옮겨오면, 대부분 Get Actor Location의 Target이 Self로 되어 있다. 그러나 AIController에서 Self는 컨트롤러 자신을 가리키며, AIController는 캐릭터와 함께 움직이지 않을 수 있다. 이 상태로 두면 Get Actor Location은 캐릭터의 위치가 아니라 AIController의 위치를 반환하게 되어, 위치 기반 계산이 모두 잘못된다.



Ctrl + Shift + F를 눌러 블루프린트 전역 검색을 열고 "Get Actor Location"을 검색한다.



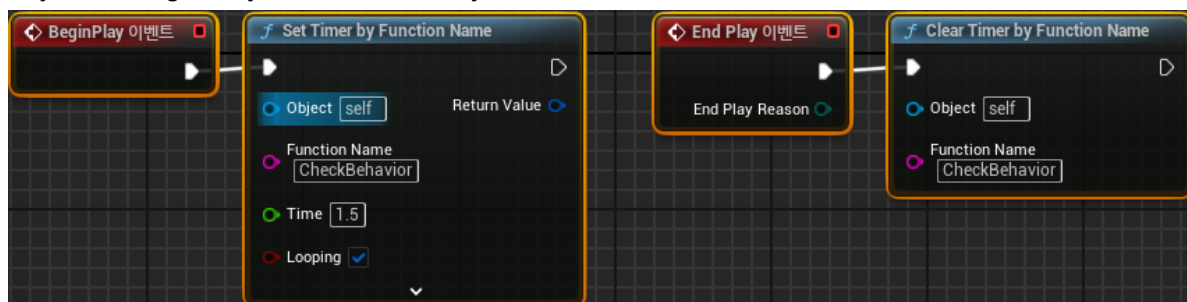
AIController 블루프린트에서 Get Actor Location의 Target 핀에 Get Controlled Pawn 노드를 연결한다.



Character 인스턴스를 저장한 ownerActor 변수를 OwnerActor 변수를 사용하지 않고 Get Controlled Pawn을 사용하는 이유는, 좀 더 범용적인 함수를 사용하여 다른 몸이나 Pawn에도 그대로 이식할 수 있도록 범용성과 재사용성을 높이기 위해서다.

13.3.6 BeginPlay 이벤트

"AI_EnergyMob" BeginPlay 이벤트, EndPlay 이벤트와 동일하게 노드를 구성한다.



당연하게 "AI_EnergyMob"의 Begin 이벤트와 End Play에 있던 당연히 Set Timer by Function Name과 Clear Timer by Function Name 노드는 삭제한다.

13.3.7 Tick 이벤트

Tick 이벤트는 Update Behavior만 처리한다.

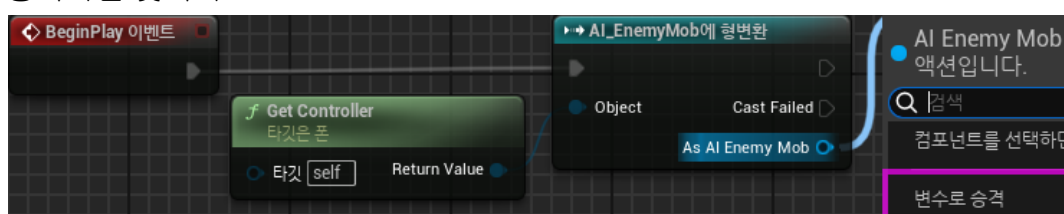


14.4 몸 캐릭터 클래스

14.4.1 controllerAi 변수 추가

지금까지 내 블루프린트 변수 섹션에서 변수를 추가하고 타입을 설정했으나 많이 번거로웠다.

AIController 변수를 쉽게 추가하는 방법은 "Get Controller" 얻은 AIController를 "AI_EnergyMob" 으로 캐스팅하고, 캐스팅된 노드를 액션에서 변수로 승격하는 것이다.



변수의 이름을 controllerAi로 변경하고 노드 구성을 완료한다.



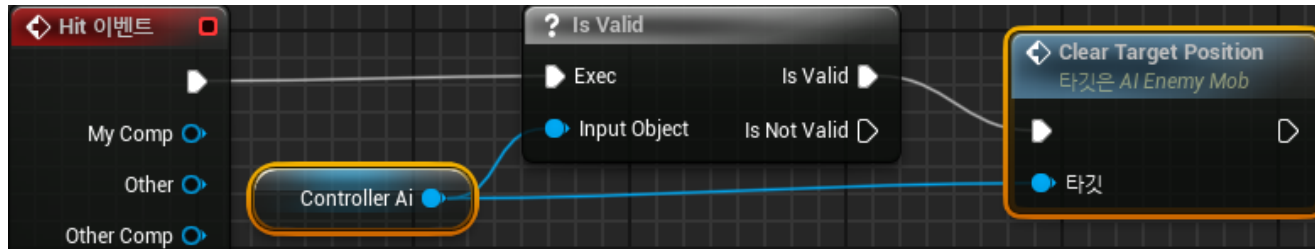
14.4.2 Tick 이벤트 정리

Tick에 붙어있던 AI_EnergyMob로 복사된 Update Behavior 함수를 떼어낸다.



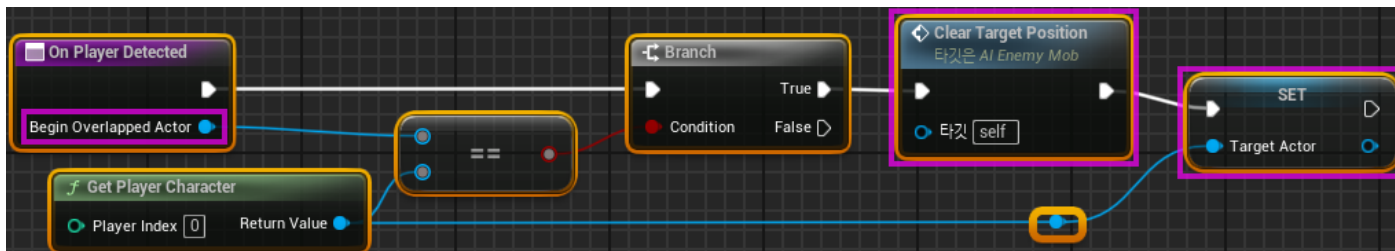
14.4.3 Hit 이벤트 정리

AI_EnergyMob로 복사된 "ClearTargetPosition" 함수를 controllerAi 노드에서 호출한다

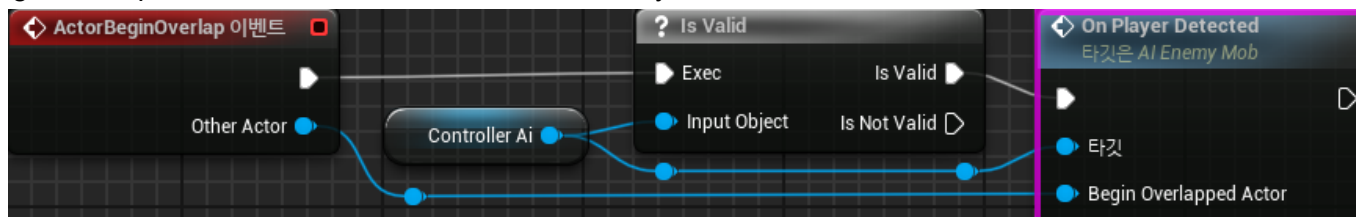


14.4.4 ActorBeginOverlap 이벤트

AI_EnergyMob에 동일한 기능을 하는 OnPlayerDetected 함수를 생성하고, OnPlayerDetected 함수에서 ActorBeginOverlap에 있던 노드들을 그대로 옮겨와서 다음과 같이 구성한다.

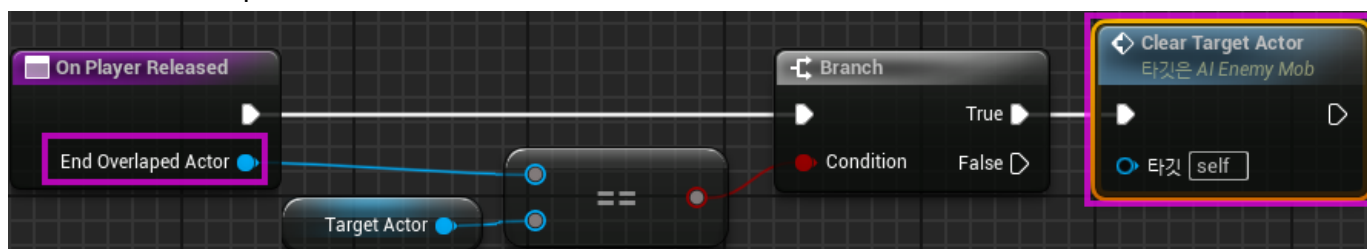


ActorBeginOverlap 이벤트는 controllerAi 노드에서 OnPlayerDetected를 호출 하면 된다.

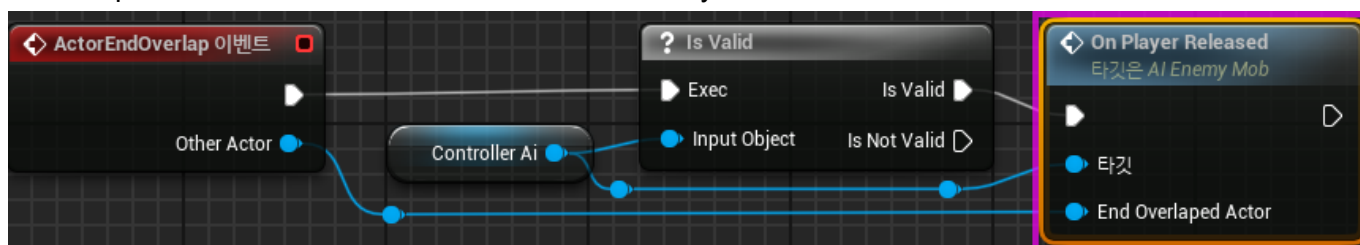


14.4.5 ActorEndOverlap 이벤트

ActorBeginOverlap와 동일한 방법으로 AI_EnergyMob에 동일한 기능을 하는 OnPlayerReleased 함수를 생성하고, OnPlayerReleased 함수에서 ActorEndOverlap에 있던 노드들을 그대로 옮겨와서 다음과 같이 구성한다.



ActorEndOverlap 이벤트 노드에서 controllerAi 노드의 OnPlayerReleased 함수를 호출한다.



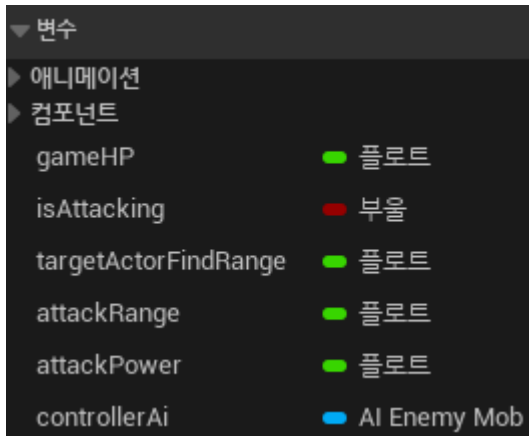
14.4.6 AttackPlayer 함수

이전 단계의 AttackPlayer 함수는 TargetActor가 유효한 경우에만 공격 상태로 전환하도록 구성되어 있었다. 하지만 현재 구조에서는 CanAttack 함수가 이미 TargetActor의 유효성을 선행 검사하고 있다. 따라서 AttackPlayer 함수의 입력 조건에서 TargetActor 검사 로직은 더 이상 필요하지 않으며, 함수 입력핀에서 TargetActor를 제거하고 단순히 캐릭터의 상태를 공격 상태로 변경하는 역할만 수행하도록 수정한다.

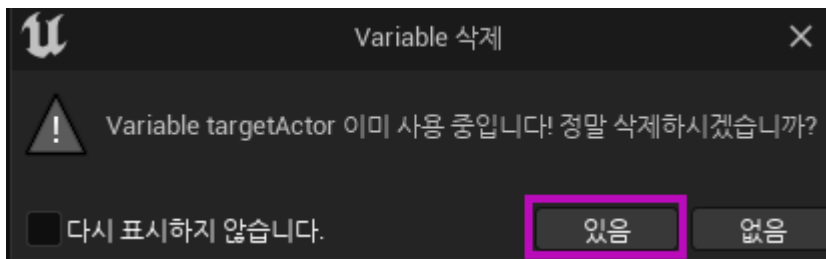


14.4.7 AI_EnergyMob 으로 복사된 변수 및 함수 삭제

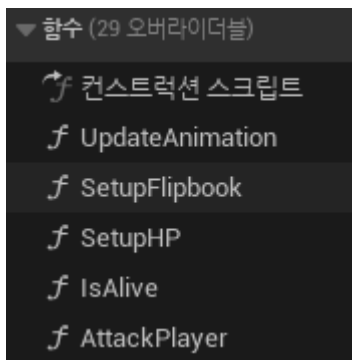
몸의 애니메이션 관련 변수(isAttacking), 체력, 플레이어 탐지 범위, 근접 공격 범위, 공격력 등의 변수만 남기고 삭제한다.



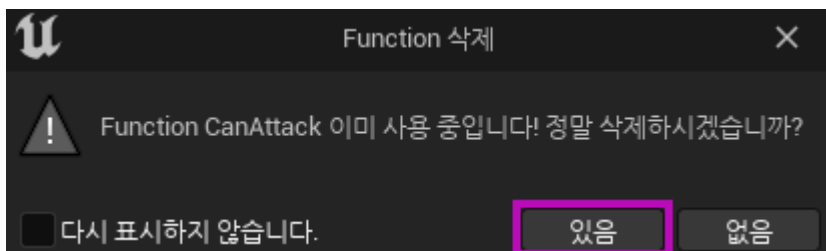
삭제하게 되면 변수를 참조하는 함수가 있어서 팝업 창이 뜬다. 있음 선택한다.



몸의 애니메이션 함수와 체력 설정, 플레이어 공격관련 함수만 남기고 전부 삭제한다.



레퍼런스 관련 팝업창이 뜨면 있음을 선택한다.



지금까지 **Character** 블루프린트에서 구현했던 **AI** 관련 로직을 **AIController** 클래스에 분리하여 옮겨오는 과정을 살펴보았다. 앞서 설명했듯이 **AI**의 동작이 단순하다면 **Character** 클래스 안에 그대로 작성해도 문제는 없다. 그러나 범용성과 재사용성, 그리고 확장성을 고려한다면 **AIController**를 별도로 구성하는 편이 훨씬 유리하다.

이러한 범용성과 재사용성이 제대로 발휘되기 위해서는, 단순히 **AIController**만 만드는 것이 아니라 캐릭터 클래스 구조를 명확하게 설계하는 것이 중요하다. 이것은 여러 게임을 제작하고 다양한 상황을 경험하면서 자연스럽게 터득되는 영역이기도 하다.

그리고 마지막으로, **AIController**를 활용할 때 반드시 기억해야 할 함수가 있다.



Get Controller, Get Controlled Pawn 이 두 함수는 **AI**가 캐릭터를 제어할 때 핵심적인 역할을 하므로 잘 기억해 두자.

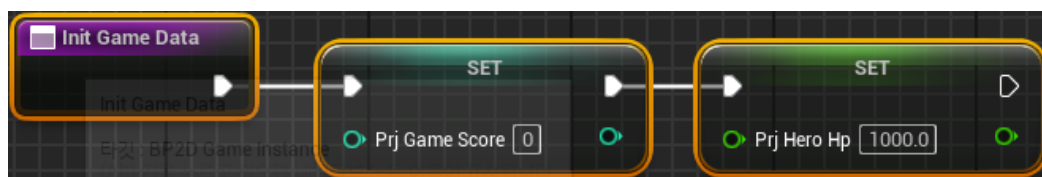
15. 게임 완성

지금까지 우리는 2D 게임에서 핵심이 되는 요소들을 하나씩 만들어 보았다. 배경 맵 구성, 캐릭터의 이동 및 공격, 몹의 행동 AI, 연출을 위한 이펙트, 그리고 점수와 플레이어 정보를 저장하는 **GameInstance**까지 다뤄보며 기본적인 게임 메커니즘을 완성할 수 있는 기반을 갖추었다.

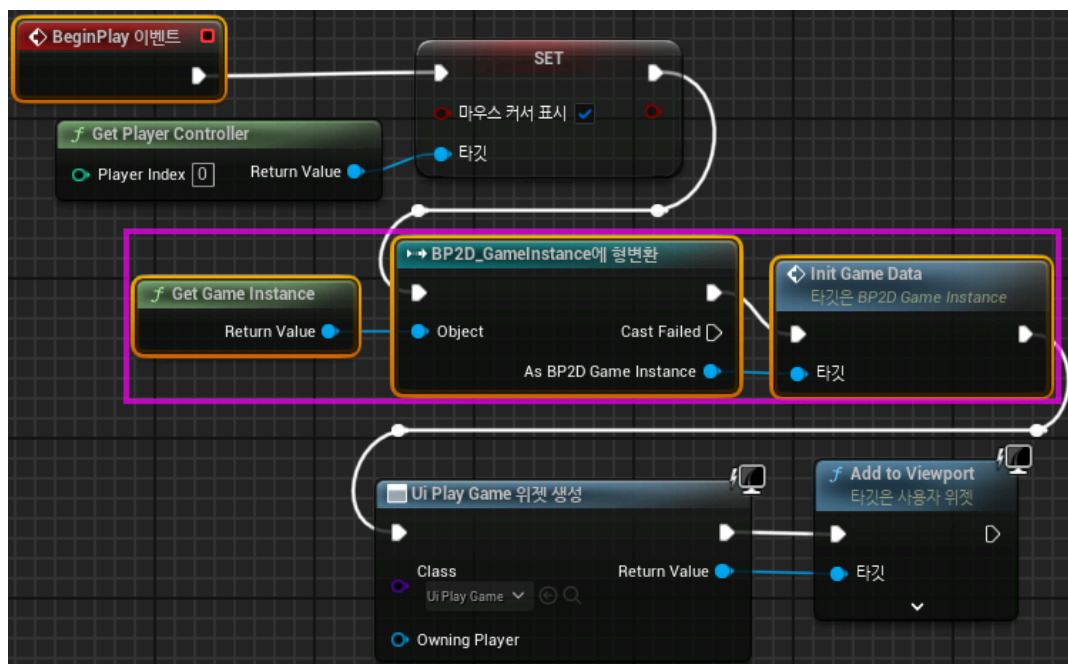
이제 지금까지 배운 요소를 바탕으로 전체 게임의 흐름을 조합하여 하나의 완성된 2D 게임을 만들어 보겠다.

15.1 Game Data 초기화

게임 플레이를 시작하기 전에 점수와 플레이어 체력 같은 기본 데이터를 초기 상태로 만들어야 한다. 이 값들은 **GameInstance**에 저장하여 전역적으로 사용하고 있으므로, **BP2D_GameInstance**에 데이터 초기화를 담당하는 함수를 하나 추가한다. 초기 값은 점수를 0, 플레이어 체력을 1000으로 설정한다.



게임 데이터 초기화는 게임플레이 레벨이 시작되는 순간에 적용하는 것이 가장 적절하다. 게임플레이에 사용하는 **LEVEL2D_01**의 **BeginPlay**에서 **HUD(UiPlayGame)**를 생성하기 전에 **InitGameData** 함수를 먼저 호출하여 값을 초기화한다. 이렇게 초기화 과정을 HUD 생성보다 앞에 두면, 화면에 표시되는 점수와 체력이 항상 올바른 초기 값으로 반영된다.



15.2 게임 흐름 구조

일반적인 게임은 다음과 같은 3단계 흐름(Flow)으로 구성된다.

- 1) 게임 시작(Start)
- 2) 게임 플레이(Game Play)
- 3) 플레이 결과(Result)

네트워크 기능이나 광고 시스템이 포함될 경우 전체 흐름이 조금 달라질 수 있지만, 단일 플레이 2D 게임에서는 이 구조가 가장 기본적이며 보편적으로 사용된다.

이번 프로젝트에서는 게임 시작 → 게임 플레이 → 게임 종료(Result)로 이어지는 간단한 구조를 구성할 것이다. 이 중 **LEVEL2D_01** 레벨을 게임 플레이(Game Play) 단계에 사용할 예정이다.

다만 현재 구성만으로는 게임을 시작할 수 있는 전용 시작 화면(Level)과 게임이 끝났을 때 최종 결과를 보여주는 결과 화면(Level)이 없으며, 게임 플레이 중에 **GameOver**와 같은 상황을 표시해 줄 수 있는 팝업 UI도 준비되어 있지 않다.

정리하면, 이제 추가로 구현해야 할 요소는 다음과 같다.

- UI
 - 게임 시작 UI
 - 게임플레이 중 **GameOver** 등 게임을 진행 결과를 알리는 결과 팝업
 - 게임 플레이 종료 후 결과 UI
- Level

- 게임 시작용 Level
- 게임 결과용 Level

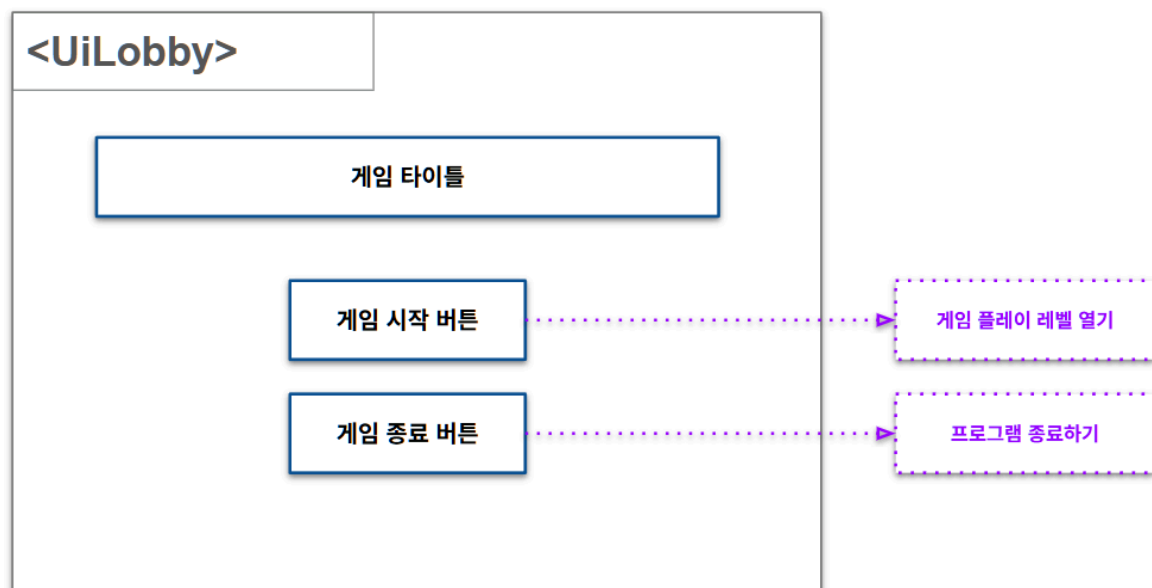
이 작업들을 통해 게임의 전체 흐름을 완성할 수 있으며, 플레이어는 자연스럽게 게임 시작 → 플레이 → 결과 화면으로 이어지는 일관된 사용자 경험을 얻게 된다.

15.3 UiLobby

UiLobby는 게임을 시작하기 전에 표시되는 로비 화면으로, 게임 제목과 두 개의 주요 버튼으로 구성된다.

첫 번째 버튼은 게임 플레이 레벨을 여는 기능을 담당하며, 이를 클릭하면 게임플레이 레벨(LEVEL2D_01)로 전환되어 실제 게임이 시작된다. 두 번째 버튼은 프로그램을 종료하는 기능을 제공한다.

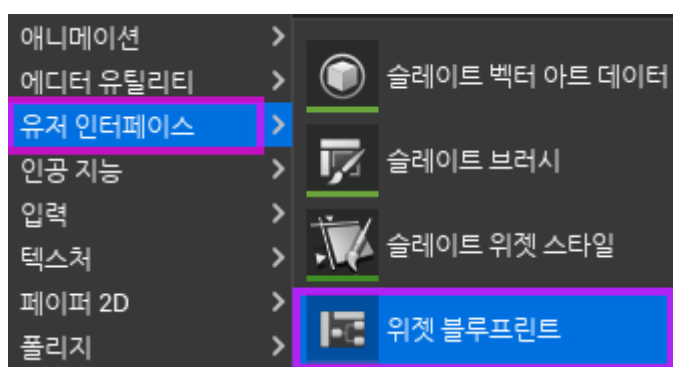
플레이어는 이 로비 화면에서 게임을 시작하거나 프로그램을 종료하는 두 가지 선택을 할 수 있으며, 전체 게임 흐름 중 '게임 시작(Start)' 단계에 해당하는 역할을 한다.



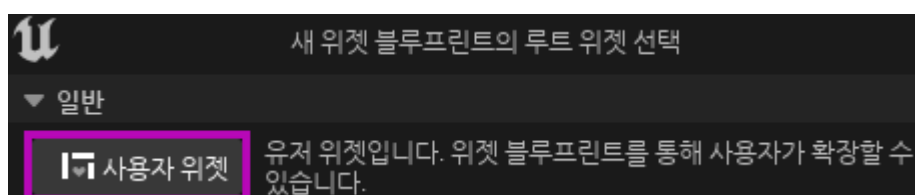
15.3.1 위젯 생성

게임 시작 레벨에서 사용할 UiLobby 위젯을 생성하겠다. 위젯 블루프린트를 만드는 과정은 이전에 진행한 Game UI 제작 과정과 동일하므로 어렵지 않을 것이다. 복습하는 의미로 생성 과정을 반복해 보자.

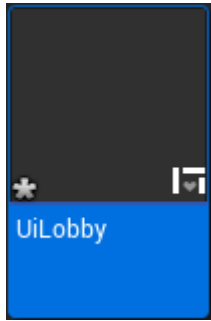
콘텐츠 브라우저에서 “/콘텐츠/Project/UI” 폴더를 선택하고, 폴더의 빈 공간을 우클릭한다. 나타나는 메뉴에서 유저 인터페이스(User Interface) 항목을 선택하면 여러 UI 관련 에셋을 생성할 수 있다.



이 중에서 Widget Blueprint(위젯 블루프린트)를 선택하면, “새 위젯 블루프린트의 루트 위젯 선택”이라는 팝업 창이 나타난다. 여기서 일반 → 사용자 위젯(User Widget)을 선택한다.

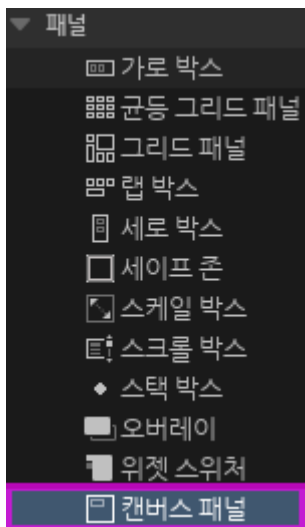


생성된 위젯의 이름은 UiLobby로 정한다.

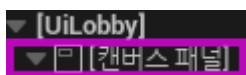


15.3.2 화면 구성: 캔버스 / 이미지 마스크 / 게임 타이틀

UiLobby를 선택한 상태에서 디자이너 창의 팔레트(Palette) 패널에서 캔버스 패널(Canvas Panel)을 선택한다.



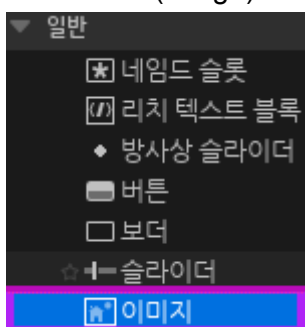
캔버스 패널을 계층 구조 패널의 "[UiLobby]" 아래로 끌어다 배치한다.



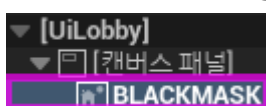
캔버스의 화면 크기는 1280 × 720으로 설정한다.

UMG의 빈 영역은 기본적으로 투명하게 표시되기 때문에, LEVEL_LOBBY에 배치한 3D 배경이 그대로 보이게 된다. 로비 화면에서는 배경을 완전히 가리는 것이 더 명확하므로, UI 배경 역할을 할 이미지를 추가하겠다.

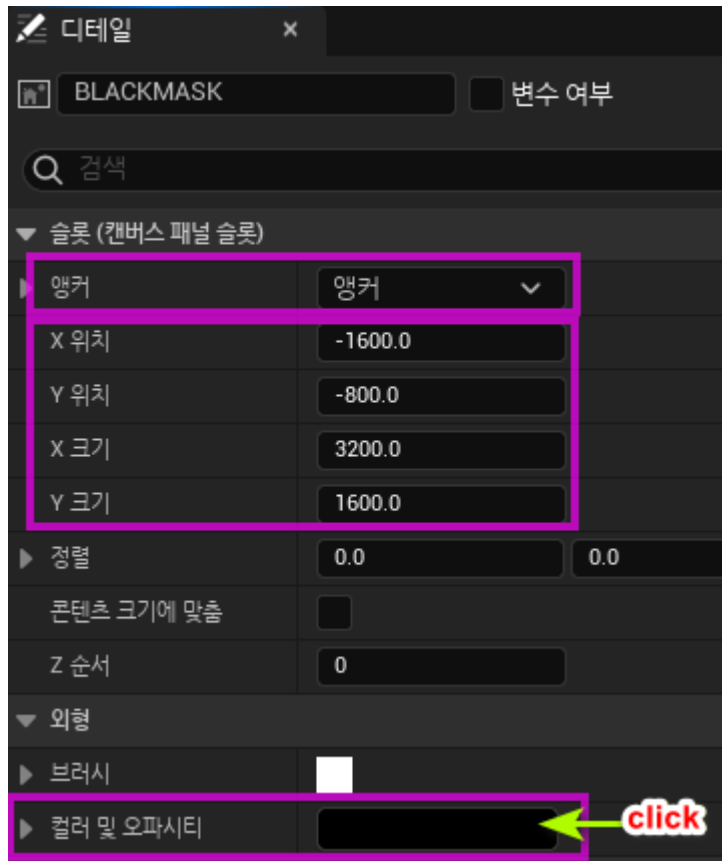
팔레트에서 이미지(Image)를 선택하여 계층 구조 패널의 [캔버스 패널] 아래로 끌어다 배치한다.



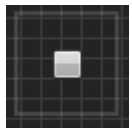
이름을 BLACKMASK로 변경한다.



BLACKMASK를 선택하고 디테일 패널에서 위치와 크기를 조정한다. 화면의 기본 해상도(1280 × 720)보다 2배 이상 큰 3200 × 1600 크기로 설정하는데, 이렇게 크게 설정하는 이유는 다양한 디바이스의 해상도나 비율에서도 화면 전체를 안정적으로 가릴 수 있도록 하기 위함이다.



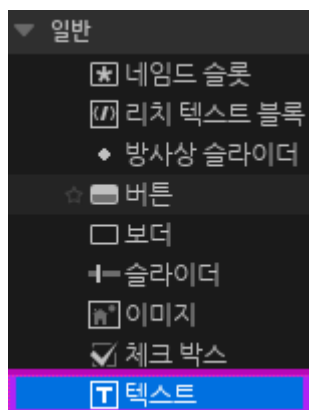
앵커 옵션은 "중앙/중앙" 으로 지정하여 화면 크기의 변화에도 중심을 유지하게 한다.



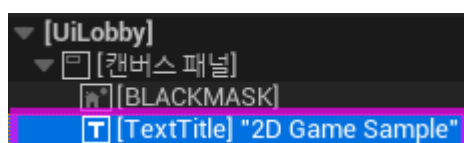
디테일 패널의 컬러 및 오퍼시티(Color & Opacity) 항목의 색상 버튼을 클릭하면 색 선택 창이 열린다. 이곳에서 완전한 검정색을 선택하여 배경 전체가 자연스럽게 어둡게 보이도록 한다.



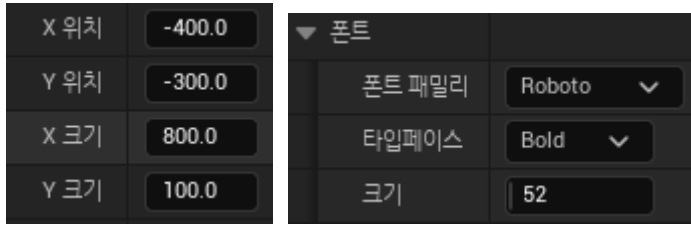
이제 게임 타이틀을 표시할 텍스트를 추가하자. 팔레트에서 텍스트(Text) 항목을 선택하여 [캔버스 패널] 아래로 끌어다 배치한다.



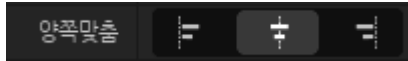
이름을 TextTitle로 변경한다. 디테일 패널의 텍스트 항목에 "2D Game Sample" 을 입력한다.



TextTitle의 크기는 (800, 100) 으로 설정하고, 위치는 (-400, -300) 으로 조정하여 화면 상단에 적당한 크기로 표시되도록 한다. 폰트 크기는 52로 설정한다.

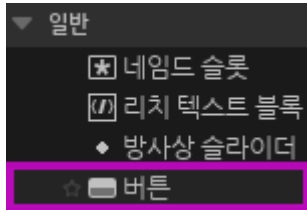


디테일 패널의 외형(Appearance) → 양쪽 맞춤(Justification) 에서 텍스트 중앙 정렬을 선택하여 문자열이 중앙 기준으로 정렬되도록 한다.

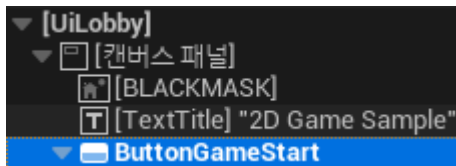


15.3.3 화면 구성: 게임 시작 버튼

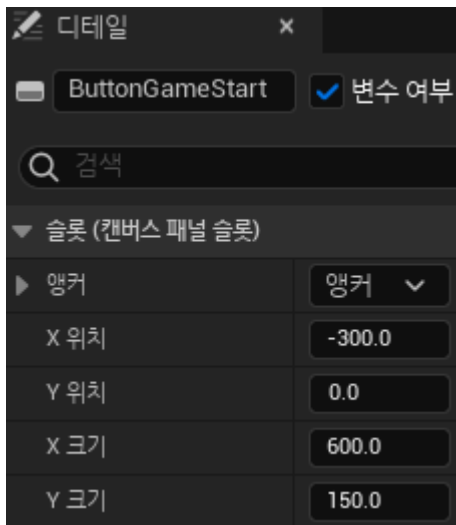
게임 시작 버튼을 추가할 차례다. 팔레트에서 버튼(Button) 항목을 선택하여 [캔버스 패널] 아래로 끌어다 배치한다.



버튼 이름을 ButtonGameStart로 변경한다



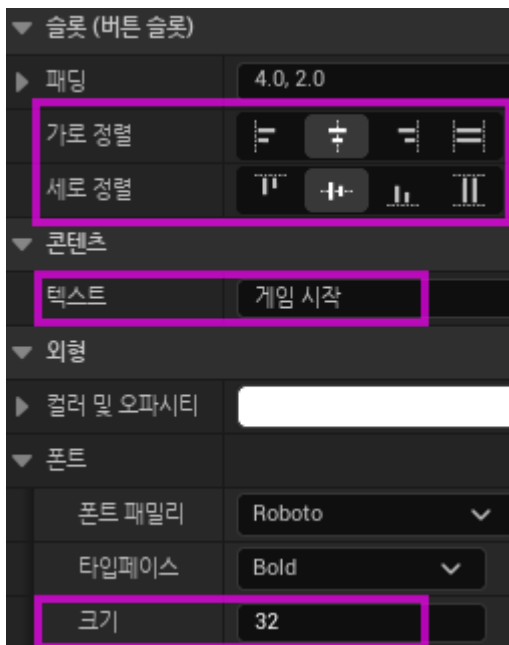
ButtonGameStart를 선택하고 디테일 패널에서 버튼의 크기를 (600, 150) 으로 설정하고, 화면 중앙에 위치하도록 (-300, 0) 좌표로 조정한다.



버튼에 표시될 텍스트는 별도의 텍스트 위젯을 사용한다. 팔레트에서 텍스트(Text) 를 선택하여 ButtonGameStart 아래로 끌어다 배치하고, 디테일 패널의 텍스트 항목에 “게임 시작” 을 입력한다.

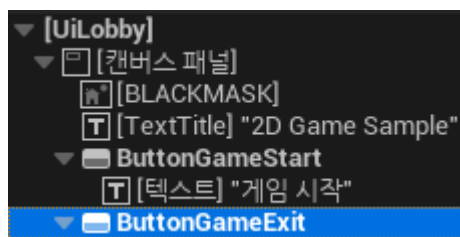


버튼 내부 텍스트는 항상 중앙 배치를 유지해야 한다. 디테일 패널의 패딩(Padding) 항목에서 가운데 가로 정렬과 가운데 세로 정렬을 선택하여 텍스트가 버튼 중앙에 정확히 오도록 한다. 텍스트의 폰트 크기는 32로 설정한다.

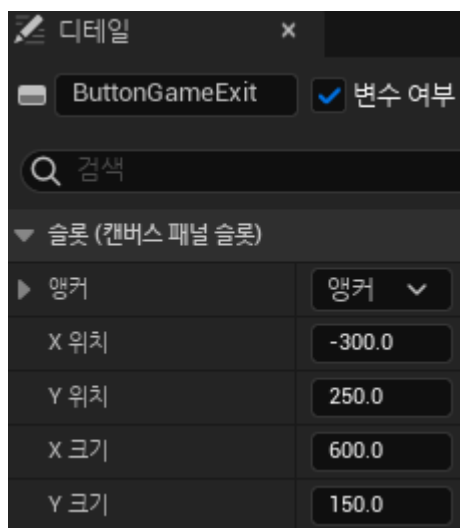


화면 구성: 게임 종료 버튼

게임 종료 버튼은 게임 시작 버튼과 동일한 방식으로 만든다. 팔레트에서 버튼을 선택하여 캔버스 패널 아래로 끌어다 배치하고, 이름을 `ButtonGameExit`으로 정한다.



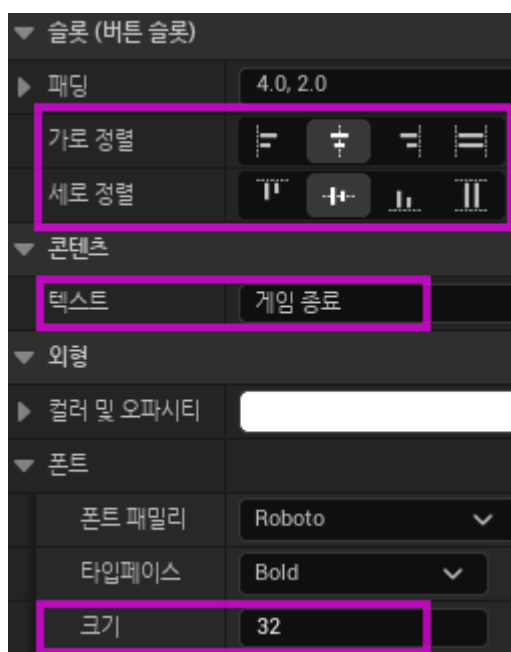
크기는 게임 시작 버튼과 동일하게 (600, 150) 으로 설정하고, 화면에서의 위치는 게임 시작 버튼 아래쪽으로 적당한 거리인 (-300, 250) 위치에 둔다.



`ButtonGameExit` 아래에 텍스트 위젯을 추가하고, 텍스트 항목에 “게임 종료” 를 입력한다. 버튼의 텍스트를 추가하고 디테일 텍스트 항목에 "게임 종료"을 입력 한다.



텍스트의 가로·세로 중앙 정렬을 유지하고 폰트 크기는 동일하게 32로 설정한다.

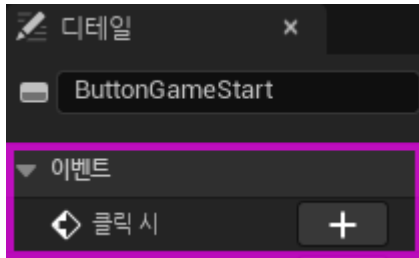


지금까지의 작업을 모두 적용하면 `UiLobby`는 다음과 같은 화면으로 구성된다. 게임 제목이 상단에 표시되고, 그 아래에 게임 시작 버튼과 게임 종료 버튼이 배치된 기본 로비 화면이 완성된다.

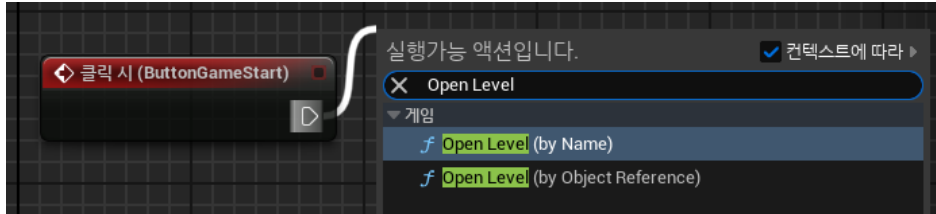


15.3.4 게임 시작 버튼(ButtonGameStart) 이벤트

ButtonGameStart가 클릭되었을 때 게임 플레이 레벨로 이동하도록 이벤트를 설정하자. ButtonGameStart를 선택하고 디테일 패널의 이벤트(Event) 섹션으로 이동하면 “클릭 시(On Clicked)” 항목을 찾을 수 있다. 여기서 추가(+) 버튼을 누르면 해당 버튼의 클릭 이벤트가 이벤트 그래프에 자동으로 생성된다.



이벤트 그래프에서 생성된 클릭 시(OnClick: ButtonGameStart) 이벤트의 실행 핀에 Open Level 노드를 연결한다.

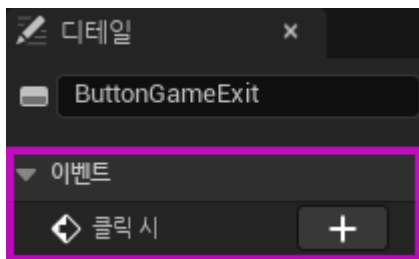


Open Level 노드의 Level Name 항목에는 게임 플레이 레벨 이름인 “LEVEL2D_01”을 입력한다. 이렇게 설정하면 사용자가 게임 시작 버튼을 누를 때 LEVEL2D_01 레벨이 로드되며 게임이 시작된다.



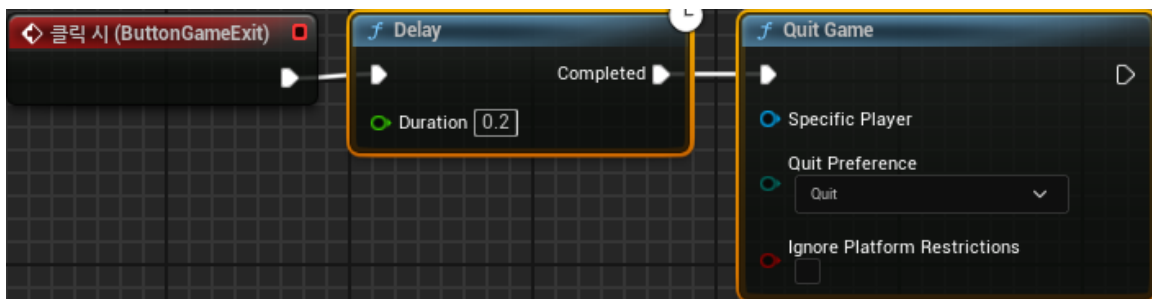
15.3.5 게임 종료 버튼(ButtonGameExit) 이벤트

게임 종료 버튼은 프로그램을 종료하는 기능을 수행한다. ButtonGameStart와 동일하게 ButtonGameExit을 선택하고 디테일 패널의 이벤트(Event) 섹션으로 이동해서 “클릭 시(On Clicked)” 항목의 추가(+) 버튼을 클릭해서 이벤트를 생성한다.



이벤트 그래프에서 생성된 클릭 시(OnClick: ButtonGameExit) 이벤트에 Delay 노드를 연결하고, 지연 시간(Duration)을 0.2초 정도로 설정한다. 이 지연 시간은 버튼 클릭 시 UI가 바로 사라지지 않도록 잠깐의 여유를 주기 위한 것으로, 사용자 경험을 조금 더 자연스럽게 만들기 위해 사용한다.

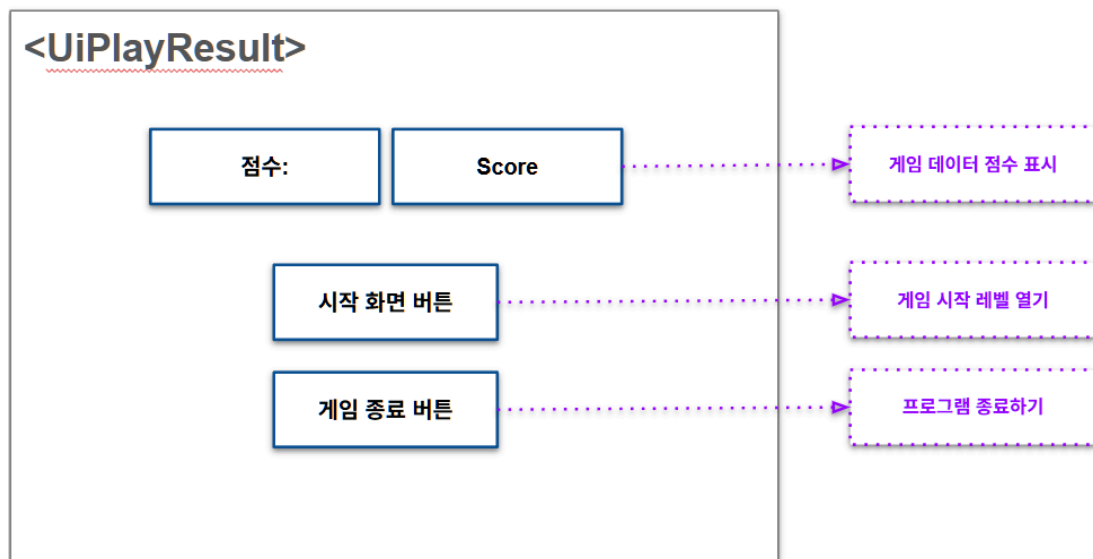
Delay의 Completed 핀에는 Quit Game 노드를 연결한다. Quit Game 노드는 프로그램을 종료하는 기능을 수행하며, 특별한 설정 없이 기본 상태로 사용해도 무방하다. 이렇게 설정하면 사용자가 “게임 종료” 버튼을 누를 때 0.2초 후 프로그램이 종료된다.



15.4 UiPlayResult

UiPlayResult 화면은 전체 구조가 UiLobby와 동일하며, 여기에 점수 표시 영역만 추가된 형태이다.

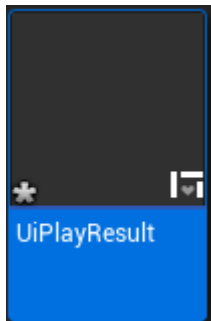
기본 배경 구성(캔버스 패널, BLACKMASK)과 버튼 배치는 UiLobby와 동일하게 만들고, 점수 라벨과 실제 점수를 표시하는 텍스트 위젯만 추가하면 된다.



이 장에서는 기본 UI 를 구성은 반복 설명하지 않고, 점수 표시용 텍스트와 바인딩 처리에 집중하여 **UiPlayResult**를 완성해 보겠다.

15.4.1 화면 디자인

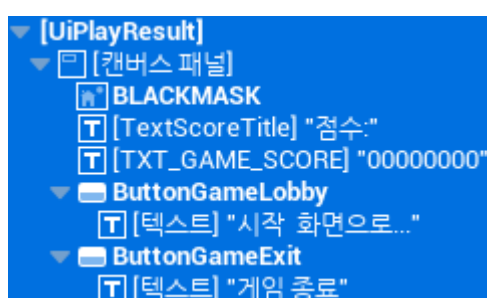
UiLobby와 같은 폴더에 **UiPlayResult**를 생성한다. 생성 과정은 **UiLobby**에서 설명했기 때문에 상세 설명은 생략한다.



UiLobby에서 구성했던 내용을 바탕으로 다음 항목을 배치한다.

- BLACKMASK 이미지
- TextScoreTitle (점수 라벨)
- TXT_GAME_SCORE (점수 표시)
- ButtonGameLobby (“시작 화면으로...” 버튼)
- ButtonGameExit (“게임 종료” 버튼)
- 각 버튼 아래의 텍스트 위젯

최종 계층 구조는 다음과 같다.



15.4.1.1 TextScoreTitle

TextScoreTitle은 “점수:” 라벨을 표시하는 텍스트이다. 디테일 패널에서 다음과 같이 설정한다.

- 텍스트: "점수:"
- 폰트 크기: 52
- 외형(Appearance) → 양쪽 맞춤(Justification): 텍스트 오른쪽 정렬
- 색상: 분홍 계열

이 값들을 적용하면 점수 라벨이 결과 화면에서 좀더 잘 보일 것이다.

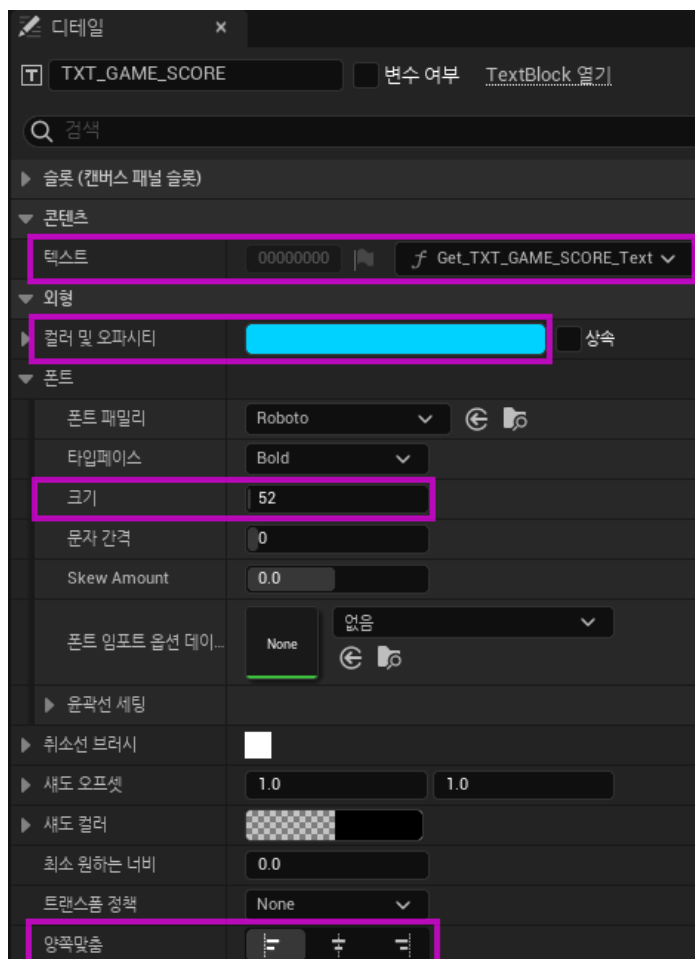


15.4.1.2 TXT_GAME_SCORE

TXT_GAME_SCORE는 실제 점수 숫자를 표시하는 텍스트 위젯이다. 디테일 패널에서 다음과 같이 설정한다.

- 텍스트: “00000000”(임시 값)
- 폰트 크기: 52
- 외형(Appearance) → 양쪽 맞춤(Justification): 텍스트 왼쪽 정렬
- 색상: 시안(Cyan) 계열

점수 라벨(TextScoreTitle)이 텍스트 오른쪽 정렬이므로, TXT_GAME_SCORE는 왼쪽 정렬을 적용하면 두 텍스트가 자연스럽게 나란히 배치된다.



15.4.1.3 완성된 디자인

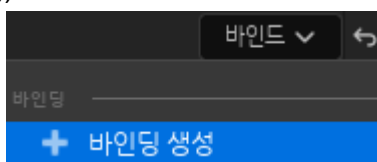
UiPlayResult의 최종 화면은 점수 라벨과 실제 점수를 중심으로, 그 아래에 “시작 화면으로...” 버튼과 “게임 종료” 버튼이 배치된 구조가 된다.



TXT_GAME_SCORE 바인딩

TXT_GAME_SCORE는 실제 점수 숫자를 표시하는 텍스트 위젯이다. 게임의 점수는 BP2D_GameInstance에서 관리하므로, 이 텍스트는 바인딩(Binding) 기능을 사용하여 GameInstance의 스코어 값을 실시간으로 가져오도록 한다.

TXT_GAME_SCORE의 디테일 패널 → 콘텐츠(Content) → 텍스트(Text) 항목 우측의 바인드(Bind) 버튼을 클릭하고 “바인딩 생성(Create Binding)” 을 선택한다.



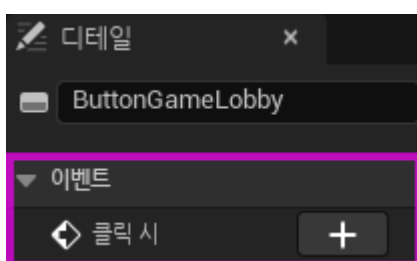
생성되는 바인딩 함수는 HUD용 UiPlayGame에서 이미 작성한 Get_TXT_GAME_SCORE_Text 함수와 완전히 동일한 그래프 구조를 가진다. 그래프 구성 방식도 동일하게 작성하면 되므로 별도의 어려움 없이 구현할 수 있다.



15.4.2 ButtonGameLobby 이벤트

ButtonGameLobby 버튼은 결과 화면에서 시작 화면(LEVEL2D_LOBBY)으로 되돌아가기 위한 기능을 담당한다. 구현 방식은 이미 UiLobby에서 진행했던 버튼 이벤트 처리와 동일하다.

ButtonGameLobby를 선택하고 디테일 패널의 이벤트(Event) 섹션에서 “클릭 시(OnClicked)” 항목 옆의 + 버튼을 클릭하면 해당 이벤트가 이벤트 그래프에 생성된다.



이벤트 그래프에서 생성된 클릭 시(OnClicked: ButtonGameLobby) 실행 핀에 Open Level(by Name) 노드를 연결하고, Level Name 항목에는 앞으로 만들 시작 화면 레벨의 이름인 "LEVEL2D_LOBBY" 를 입력한다. 이렇게 설정하면 “시작 화면으로...” 버튼을 눌렀을 때 LEVEL2D_LOBBY 레벨이 열리게 된다.



15.4.3 ButtonGameExit

ButtonGameExit의 동작은 UiLobby에서 구현했던 프로그램 종료(Quit Game) 처리와 동일하다. 구현 방법이 완전히 같으므로, UiLobby에서 작성한 내용을 참고하여 동일한 방식으로 설정해 완성하도록 한다.

15.5 UiPopGameOver

UiPopGameOver는 게임 플레이 도중 Game Over 상태를 알리는 결과 팝업 UI이다. 간단한 정보만 표시하는 팝업 형태이며, “확인” 버튼을 클릭하면 게임 플레이 결과 레벨로 이동하도록 구성한다.



15.5.1 화면 디자인

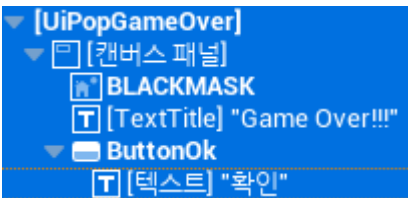
UiLobby와 같은 폴더에 UiPopGameOver 위젯을 생성한다. 생성 과정은 이미 다른 Ui에서 이미 설명했기 때문에 상세 과정은 생략한다.



팔레트에서 다음 항목을 배치하여 화면을 구성한다.

- BLACKMASK 이미지
- TextTitle ("Game Over!!!")
- ButtonOk ("확인" 버튼)
- 버튼 아래의 텍스트 위젯

최종 계층 구조는 다음과 같다.

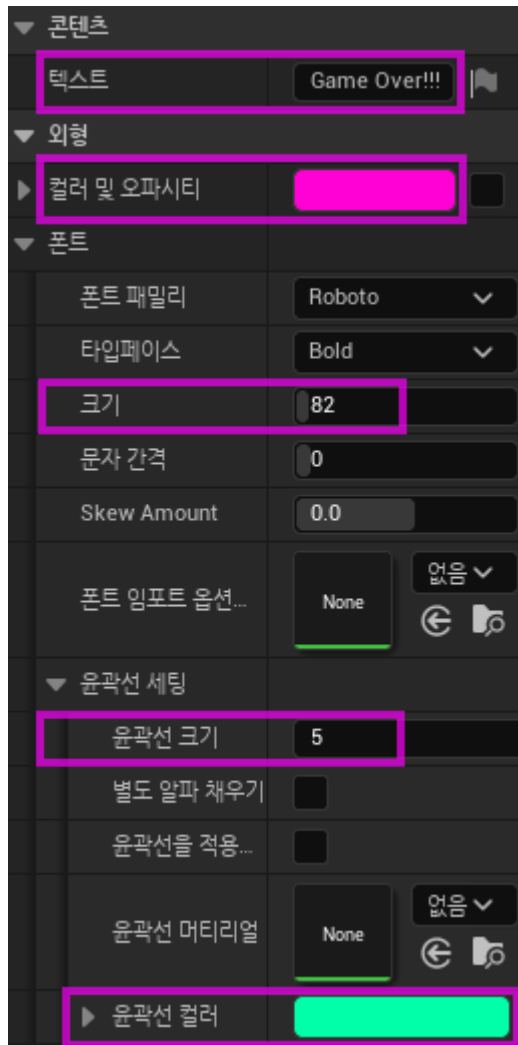


15.5.1.1 TextScoreTitle

TextTitle은 “Game Over!!!” 메시지를 크게 표시하는 텍스트이다. 디테일 패널에서 다음과 같이 설정한다.

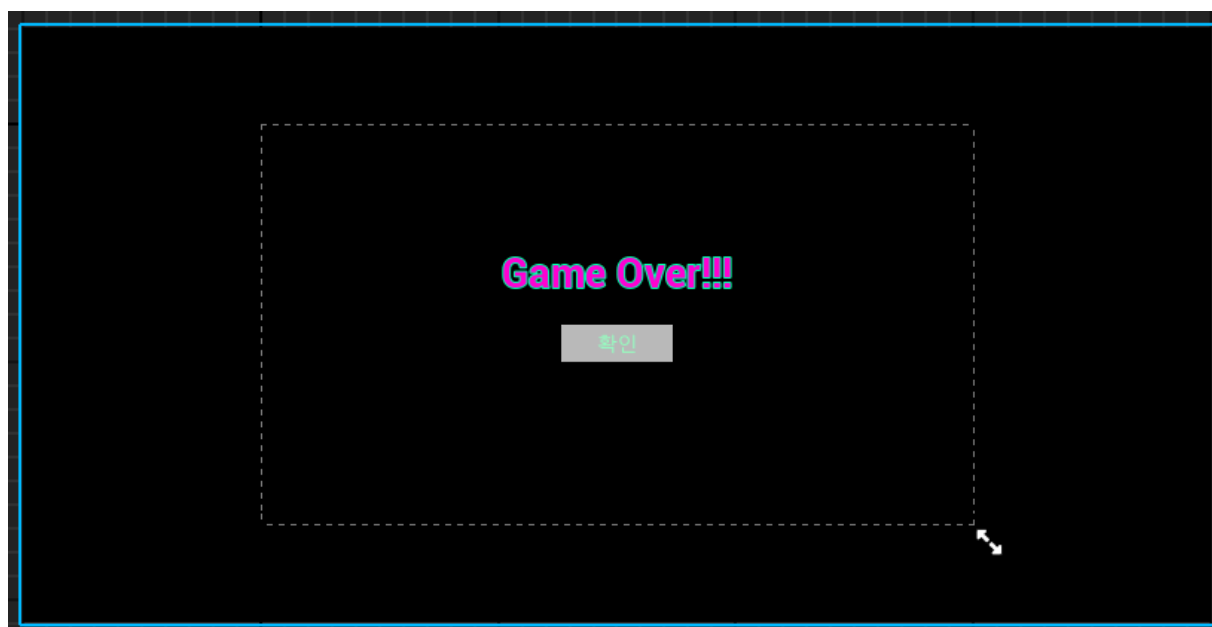
- 텍스트: "Game Over!!!"
- 폰트 크기: 82
- 외형(Appearance) → 양쪽 맞춤(Justification): 텍스트 중앙 정렬
- 색상(Color & Opacity): 분홍 계열
- 윤곽선 크기(Outline Size): 5
- 윤곽선 컬러(Outline Color): 시안(Cyan) 계열

윤곽선을 추가하면 배경과의 대비가 높아져 Game Over 메시지가 더 강조된다.



15.5.1.2 완성된 디자인

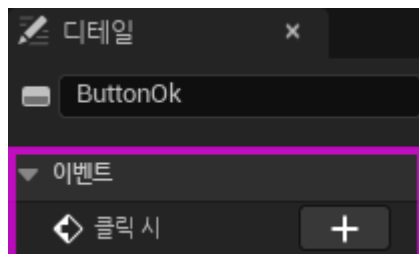
UiPopGameOver는 BLACKMASK 위에 크게 표시된 “Game Over!!!” 텍스트와 그 아래의 “확인” 버튼으로 구성된다. 이 팝업은 게임 오버 상황에서 플레이어에게 결과 화면으로 이동하기 전의 안내 역할을 한다.



15.5.2 ButtonOk 이벤트

ButtonOk 버튼은 GameOver 상태에서 결과 화면(LEVEL2D_PLAY_RESULT)으로 이동하는 기능을 담당한다. 구현 방식은 이미 다른 Ui에서 OpenLevel을 구현했던 방식과 동일하다.

ButtonOk를 선택하고 디테일 패널의 이벤트(Event) 섹션에서 “클릭 시(OnClicked)” 항목 옆의 + 버튼을 클릭해서 이벤트 노드를 생성한다.



이벤트 그래프에서 생성된 클릭 시(OnClicked: ButtonOk) 실행 핀에 Open Level(by Name) 노드를 연결하고, Level Name 항목에는 앞으로 만들 시작 화면 레벨의 이름인 "LEVEL2D_PLAY_RESULT"를 입력한다. 이렇게 설정하면 “확인” 버튼을 눌렀을 때 LEVEL2D_PLAY_RESULT레벨이 열리게 된다.



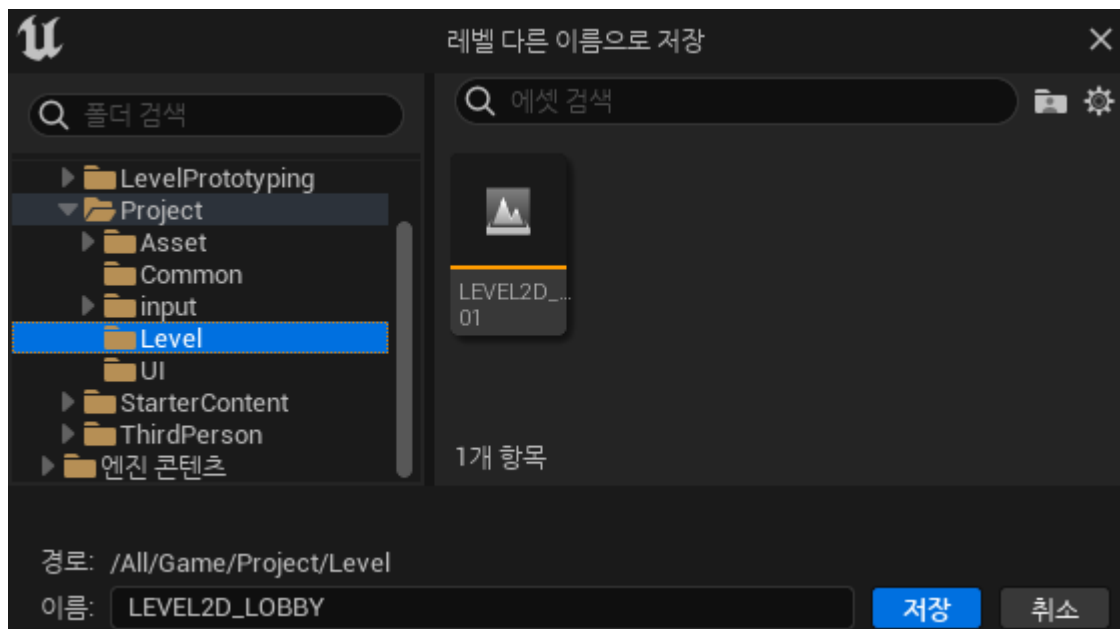
15.6 게임 시작 LEVEL(LEVEL_LOBBY)

이 레벨은 복잡한 로직이 필요하지 않고, 주로 **UI** 중심으로 구성되는 시작 화면 역할을 한다. 따라서 레벨 내부에는 최소한의 빛과 배치만 두고, 실제 **UI**는 위젯으로 표시하는 방식으로 구성한다.

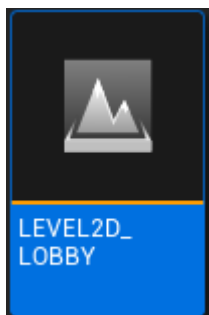
15.6.1 LEVEL_LOBBY 생성

메뉴에서 파일(File) → 새 레벨(New Level) 을 선택하고, 새 레벨 종류 중 빈 레벨(Empty Level) 을 선택한다. **Ctrl + S** 로 저장하면 “레벨 다른 이름으로 저장” 팝업이 나타난다. 저장 위치와 이름을 다음과 같이 지정한다.

- 폴더: /콘텐츠/Project/Level
- 이름: LEVEL_LOBBY



이제 프로젝트에 시작 레벨이 추가되었다.

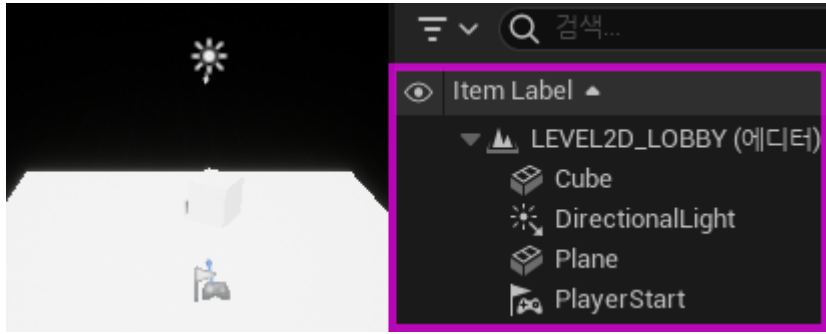


15.6.2 레벨 초기 구성

Ui 중심 레벨이기 때문에 아무것도 없는 화면이어도 동작에는 문제가 없다. 하지만 완전히 어두운 화면만 보이면 개발 과정에서 동작 상황을 확인하기 어렵다. 따라서 레벨의 화면을 확인할 정도의 액터만 배치한다.

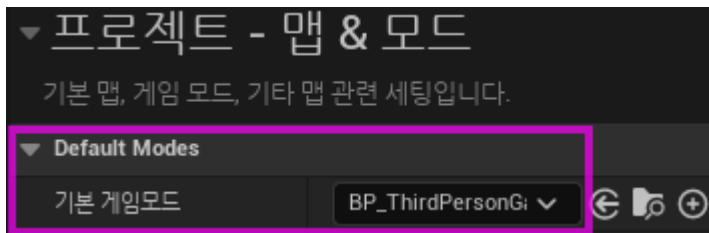
- Plane : 위치: 0,0,0
 - Scale: 10, 10, 1
- Cube
 - 위치: 200, 50, 50
- DirectionalLight
 - 위치: 0,0,0
 - 회전: 0, -45, 0
 - 스케일: 2.5, 2.5, 2.5
- PlayerStart
 - 위치: -250.0, 0, 100

이 정도만 배치해도 실행에서 3D 공간을 명확히 확인할 수 있다.



15.6.3 실행 시 나타나는 문제

LEVEL_LOBBY를 실행하면, 프로젝트 세팅 → 맵 & 모드(Maps & Modes)의 기본 게임모드(Default GameMode) 값이 BP_ThirdPersonGameMode로 되어 있을 경우 기본 캐릭터인 BP_ThirdPersonCharacter가 레벨에 자동으로 생성된다. 또한 ThirdPersonCharacter는 기본 입력을 가지고 있기 때문에 WASD 키로 마음대로 움직일 수 있다. 하지만 LEVEL_LOBBY는 UI 전용 레벨이며, 캐릭터가 등장하거나 움직이는 것은 우리가 원하는 동작이 아니다.

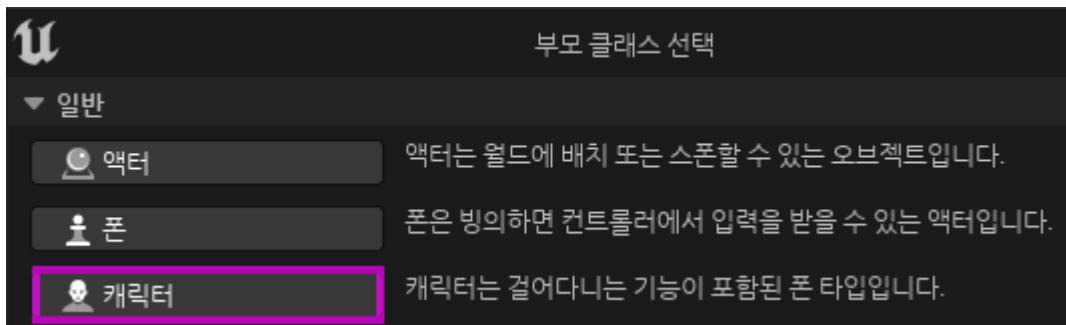


15.6.4 Dummy Character

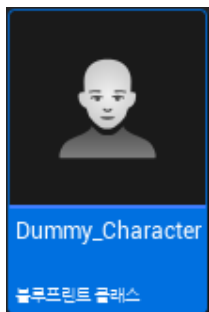
현재 게임의 시작인 로비 레벨과 게임 결과 레벨은 UI 중심 화면이기 때문에 플레이어 캐릭터가 입력(Input)에 반응할 필요가 없다. 따라서 입력에 전혀 반응하지 않는 Dummy Character를 만들어서 사용한다.

콘텐츠 브라우저에서 "/콘텐츠/Project/Common" 폴더의 빈 공간을 우클릭하고 블루프린트 클래스(Blueprint Class)를 선택한다.

부모 클래스 선택 창에서 일반 → 캐릭터(Character)를 클릭한다.



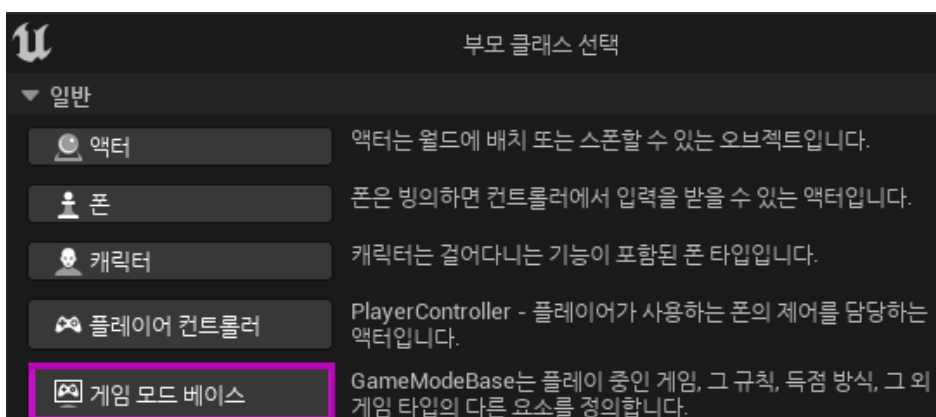
생성된 블루프린트 이름을 Dummy_Character로 정한다.



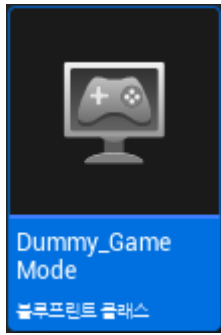
15.6.5 Dummy GameMode

Dummy_Character를 로비 레벨과 게임 결과 레벨에서 사용하기 위해서는 이 캐릭터를 기본 Pawn으로 사용하는 전용 GameMode가 필요하다.

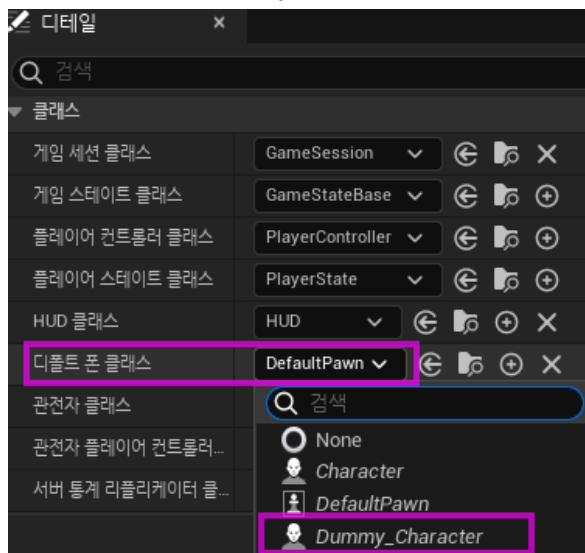
동일한 /콘텐츠/Project/Common 폴더에서 우클릭하여 블루프린트 클래스 → 일반 → GameModeBase를 선택한다.



생성된 블루프린트 이름을 **Dummy_GameMode** 로 정한다.



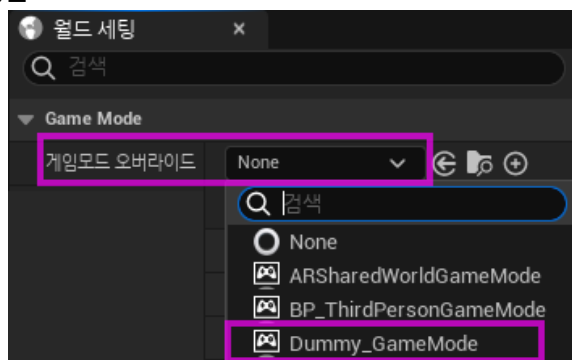
생성된 **Dummy_GameMode**를 열고, 디테일 패널의 클래스(Class) 섹션에서 디폴트 폰 클래스(Default Pawn Class)를 찾는다. 디폴트 폰 클래스를 "**Dummy_Character**" 로 변경한다.



15.6.6 Dummy GameMode 적용

이제 **Dummy GameMode**를 레벨에 적용할 차례다. **LEVEL2D_LOBBY**의 월드 세팅을 보면 "**Game Mode**" 가 보인다. 버튼을 확장하면 "게임모드 오버라이드" 항목이 있는데 선택 버튼을 누르면 변경할 수 있는 **GameMode**가 나온다.

Dummy_GameMode을 선택 한다.



이제 **LEVEL2D_LOBBY**에서 “에디터에서 플레이(Play in Editor)” 를 실행해 보면, 입력에 전혀 반응하지 않는 **Dummy_Character**가 플레이어 Pawn으로 생성되는 것을 확인할 수 있다.

또한 **Dummy GameMode**를 게임 결과 레벨에도 동일하게 적용하면, 그 레벨에서도 인풋에 반응하지 않는 **Dummy_Character**가 플레이어로 설정되어 오직 UI만 표시되는 정적인 화면으로 유지할 수 있다.

15.6.7 마우스 포인터 보이기

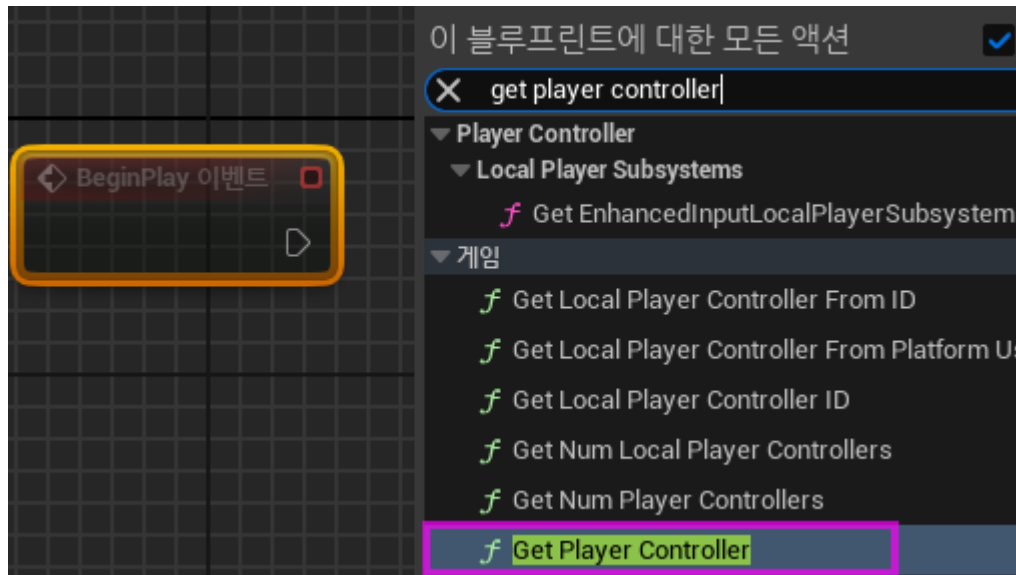
UI 화면에서는 마우스를 사용해야 하므로, 레벨이 시작될 때 플레이어 컨트롤러에서 마우스 커서 표시를 활성화해야 한다.

툴바에서 레벨 블루프린트 열기를 선택하고, 그래프에서 **BeginPlay** 이벤트를 준비한다.



마우스 커서 보이는는 **Player Controller**에서 설정한다.

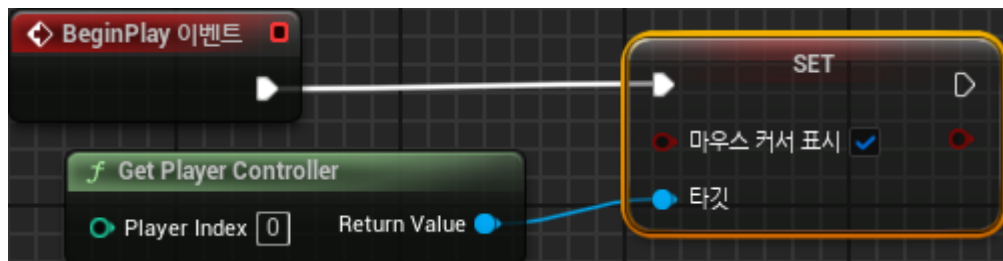
BeginPlay 이벤트 근처의 빈 공간에서 우클릭하고 "get player controller"를 검색하여 "**Get Player Controller**" 노드를 추가한다.



"Get Player Controller" 노드의 "Return Value" 핀에서 끌어와 "set show mouse cursor" 검색 하고 "Set Show Mouse Cursor" 노드를 추가한다.



"Set Show Mouse Cursor" 노드의 "마우스 커서 표시"를 true로 체크한다.



이렇게 설정하면 LEVEL_LOBBY처럼 UI 중심의 레벨에서 마우스 커서가 화면에 표시되어 버튼을 정상적으로 클릭할 수 있다.

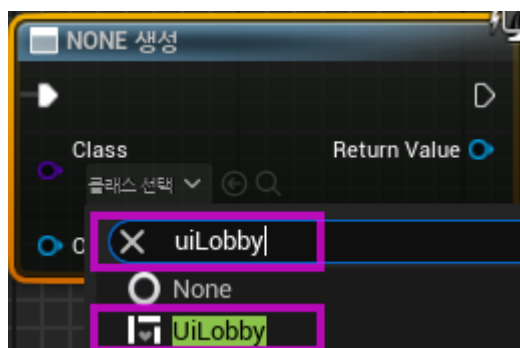
15.6.8 LEVEL2D_LOBBY에 UiLobby 연결하기

LEVEL2D_LOBBY에서 UiLobby 연결은 BeginPlay에서 시작한다.

앞서 구현한 "Set Show Mouse Cursor" 노드의 실행 핀에서 끌어와 "create widget" 검색 하고 위젯 생성 노드를 추가한다.



NONE 위젯의 입력 핀 Class의 "클래스 선택" 버튼을 클릭, "uiLobby" 를 검색하고 "UiLobby" 선택한다.



변경된 "Ui Lobby 위젯 생성" 노드에 "Add to Viewport" 노드 추가해서 UiLobby가 LEVEL2D_LOBBY 레벨에 연결을 완료한다.



15.7 게임 플레이 결과 LEVEL(LEVEL2D_PLAY_RESULT)

LEVEL_LOBBY는 “게임 시작” 또는 “프로그램 종료” 중 하나를 선택하는 단순한 흐름으로 구성되어 있다.

게임 플레이 결과 화면인 LEVEL2D_PLAY_RESULT 역시

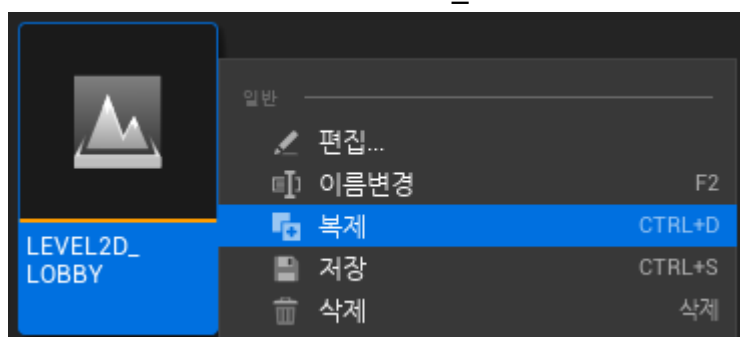
- 시작 화면(LEVEL_LOBBY)으로 이동
- 프로그램 종료

라는 두 가지 기능만 필요하며, 전체 구조가 UI 중심이라는 점도 동일하다.

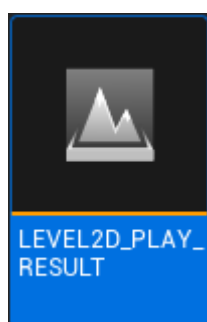
이러한 이유로 지금은 LEVEL2D_PLAY_RESULT는 새로운 레벨을 처음부터 만들지 않고, 기존 LEVEL_LOBBY를 복제하여 일부 내용만 수정할 것이다.

15.7.1 LEVEL_LOBBY 복제하기

복제 방법은 콘텐츠 브라우저에서 LEVEL_LOBBY를 선택하고 LEVEL_LOBBY 위에 마우스 우클릭 하면 복제 메뉴가 나온다.



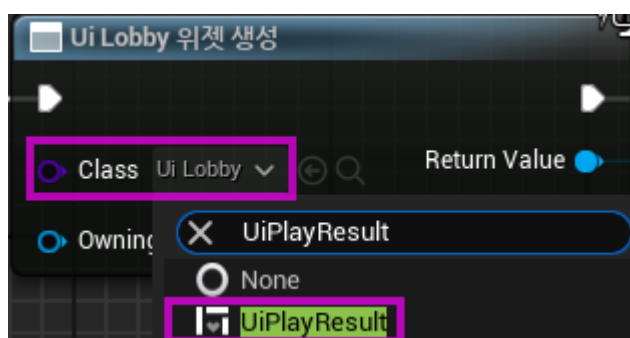
복제된 레벨의 이름을 LEVEL2D_PLAY_RESULT 로 변경한다.



15.7.2 레벨 블루프린트 수정

LEVEL2D_PLAY_RESULT를 열고, 레벨 블루프린트(Level Blueprint) 를 열고, BeginPlay 이벤트에 연결되어 있는 "Ui Lobby 위젯 생성(Create Widget: UiLobby)" 노드를 선택한다. (복제된 상태에서는 UiLobby가 그대로 연결되어 있을 것이다.)

Class 입력 핀의 드롭다운 버튼을 클릭하고 “UiPlayResult”를 검색하여 선택한다. 이제 LEVEL2D_PLAY_RESULT가 시작될 때 UiPlayResult 위젯이 표시될 것이다.



변경 사항을 적용하기 위해 컴파일을 수행하고 레벨을 저장한다.

이제 LEVEL2D_PLAY_RESULT는 게임 플레이가 종료된 뒤 점수 결과 화면 UiPlayResult을 표시하는 전용 레벨로 사용할 수 있다.



15.8 게임 플레이 LEVEL(LEVEL2D_01)

게임 시작 화면과 게임 결과 화면(UI 및 레벨 구성)을 모두 완성했으므로, 이제 실제 게임이 진행되는 **LEVEL2D_01**에서 게임 종료(Game Over)를 판단하고 팝업을 띄우는 로직을 구현해야 한다.

Game Over가 발생하는 대표적인 조건은 다음과 같다.

- 플레이어 체력이 0이 되었을 때
- 목표 점수 달성(예: 100,000점)
- Enemy 처치 수 등 특정 미션 목표 달성

본 교재에서는 플레이어 체력이 0이 되거나 점수가 100,000점이 되었을 때 **Game Over** 팝업을 띄우는 방식으로 구현한다.

15.8.1 IsGameOver 함수 생성

먼저 **Game Over** 조건을 판단하는 **IsGameOver** 함수를 LEVEL2D_01의 레벨 블루프린트에 추가한다. 이 함수는 단순히 조건을 체크하는 역할이므로 **Pure** 함수로 만들면 된다.

간단히 **C++** 코드로 다음과 같이 표현할 수 있다.

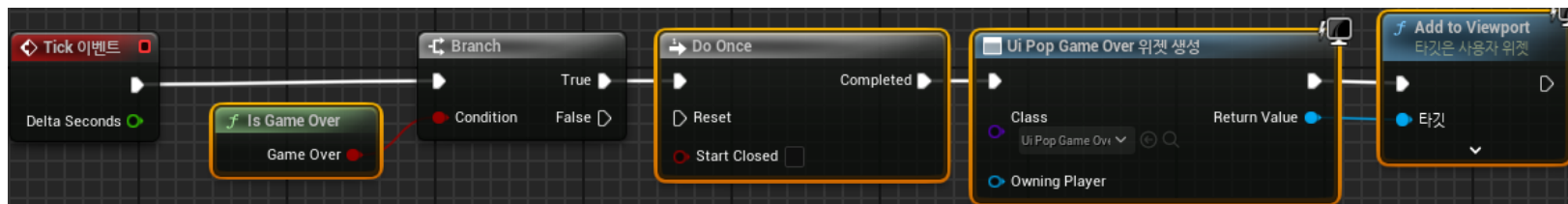
```
bool IsGameOver() {  
    return (false == BP2D_GameInstance->IsPlayerAlive()  
           || BP2D_GameInstance->prj_gameScore >= 100000)  
}
```

IsGameOver 함수는 정확한 판단만 수행하므로 다른 노드를 연결하지 않는 **Pure** 함수가 적합하다.



LEVEL2D_01의 Tick에서 **IsGameOver**이면 게임오버 팝업 띄우는데 **Tick**은 매 프레임마다 실행되는 함수다. **DoOnce**를 사용하면 한 번만 띄울 수 있다.

DoOnce노드 다음에 변경된 위젯을 생성하고 "Ui Pop Game Over" 위젯으로 변경한다. 이 노드를 **DoOnce**에 연결하고, "Add to Viewport" 노드를 추가해서 작업을 완료한다.



컴파일 하고 실행하면 점수가 100,000이 되면 **GameOver** 팝업창이 뜨는 것을 확인 할 수 있다.

16. 패키징

게임 플레이 레벨과 UI를 모두 완성했다면, 프로젝트를 실제 실행 파일 형태로 만들기 위해 패키징 작업을 해야 한다. 언리얼 엔진에서 패키징은 비교적 단순하지만, 시작 레벨 설정과 빌드에 포함될 레벨 지정을 정확히 해두어야 문제가 없다.

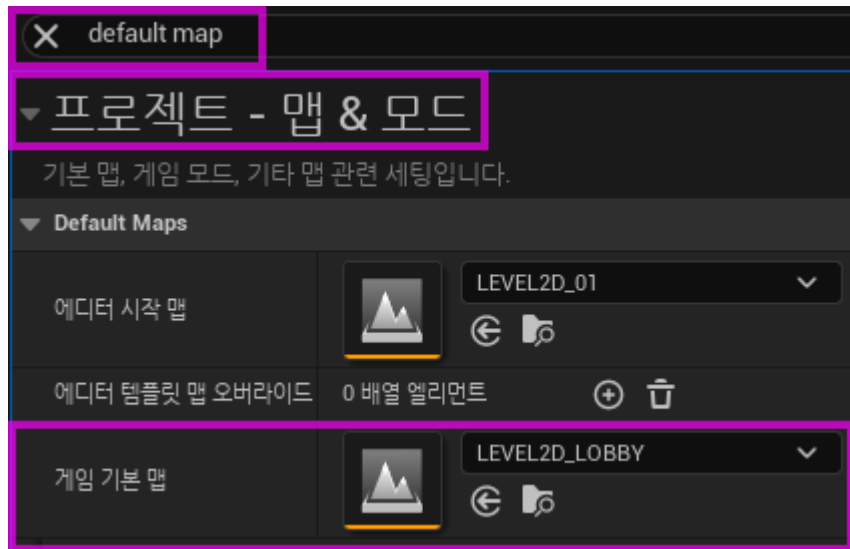
16.1 기본 맵 설정(Default Maps)

패키징된 게임이 실행될 때 가장 먼저 열리는 레벨을 지정해야 한다.

프로젝트 설정(Project Settings)에서 "default map" 을 검색하여 다음 항목을 설정한다.

- 게임 기본 맵(Game Default Map) → LEVEL2D_LOBBY
- 플레이어가 처음 실행 시 반드시 로비 화면부터 보이도록 하기 위함이다.

이 설정이 되어 있지 않으면, 패키징 후 실행 시 엉뚱한 레벨이 뜨거나 빈 화면이 나올 수 있다.



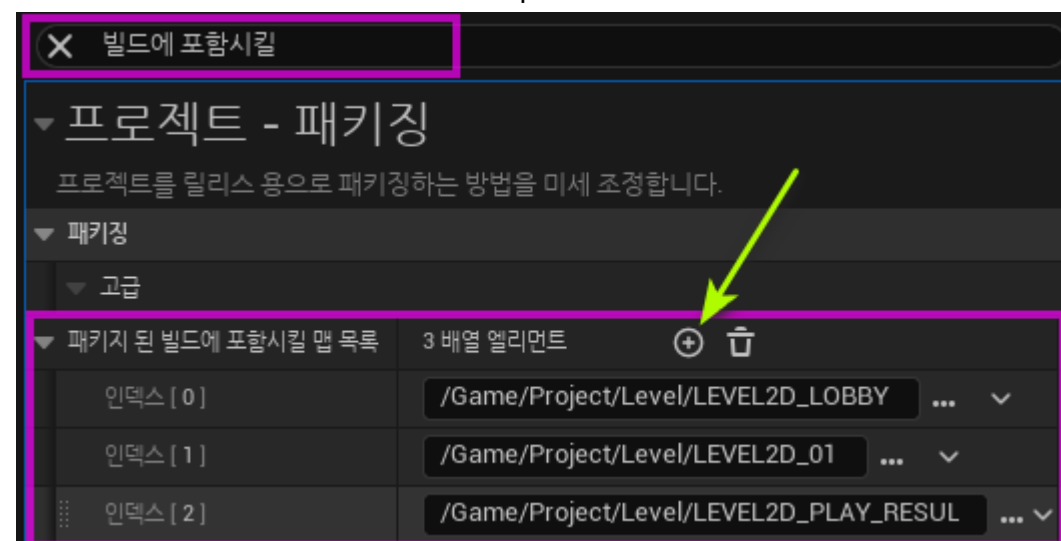
16.2 빌드에 포함될 레벨 지정

패키징된 실행 파일에는 레벨이 자동으로 모두 포함되지 않는다. 특히 콘텐츠가 많은 프로젝트에서는 필요한 레벨만 수동으로 지정해야 한다.

프로젝트 설정에서 “빌드에 포함시킬”을 검색한다. 프로젝트 → 패키징(Packaging) → "패키지된 빌드에 포함시킬 맵 목록" 추가 버튼(+) 을 누르고 레벨을 등록한다.

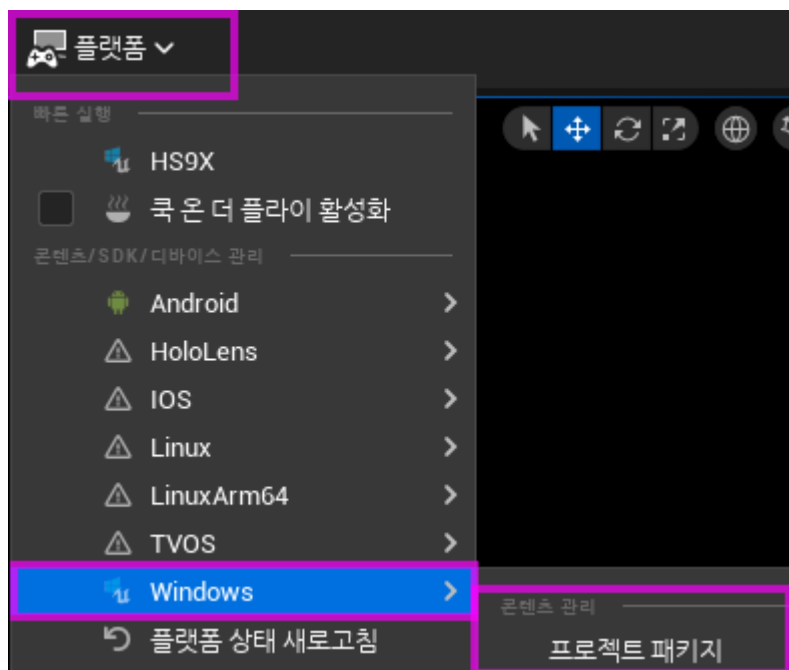
- /Game/Project/Level/LEVEL2D_LOBBY
- /Game/Project/Level/LEVEL2D_01
- /Game/Project/Level/LEVEL2D_PLAY_RESULT

여기서 누락된 레벨이 있으면 게임 내부에서 OpenLevel을 호출해도 레벨이 열리지 않는다



16.3 Windows 실행 파일 패키징

모든 설정을 마쳤다면 실행 파일을 만든다. 툴바 상단의 플랫폼(Platforms) 메뉴 클릭 → Windows 선택 → 프로젝트 패키지(Project Package) 클릭



저장 폴더 선택 하면 Windows용 실행 파일을 빌드한다. 패키징이 완료되면 폴더 안에 다음과 같은 구조로 파일이 생성된다.
<저장폴더>/Windows/xxxx.exe

이제 이 폴더 전체를 복사해서 다른 PC에서도 실행할 수 있다.

맺음말

지금까지 **Paper2D**를 사용하여 하나의 **2D** 언리얼 게임을 완성해 보았다. 이 정도 수준의 구조와 흐름을 이해했다면 간단한 슈팅 게임 정도는 충분히 구현할 수 있을 것이다. 하지만 **2D** 게임이라도 복잡한 액션과 연출을 요구하는 장르가 많으며, **Paper2D**만으로는 표현할 수 있는 범위에 한계가 존재한다. 이러한 부분은 **PaperZD**를 도입하면 훨씬 자연스럽게 확장할 수 있다.

Paper2D로 2~3개의 작은 게임을 만들어 보면 기본 체계가 손에 익고, 이후 **PaperZD**를 배우는 과정도 어렵지 않다. **PaperZD** 역시 몇 가지 프로젝트를 만들어 보면 금세 익숙해지며, 이 두 기술만 제대로 다루게 되면 언리얼에서 **2D** 게임을 제작하는 데 필요한 대부분의 능력을 갖추게 된다.

2D 게임을 만들었다고 해서 **3D** 게임 개발이 곧바로 쉬워지는 것은 아니지만, **2D** 개발을 통해 쌓은 경험은 **3D** 게임 개발의 문턱을 낮추어 준다. 언리얼의 전체 구조, 입력 시스템, **UI**, 게임플레이 흐름, 레벨 구성 등 핵심 원리는 동일하기 때문이다. 결국 시간과 노력을 들여 꾸준히 경험을 쌓는다면 어떤 장르든 충분히 극복할 수 있다는 자신감을 얻게 될 것이다.

긴 여정을 함께한 독자에게 감사드리며, 앞으로 만들어 나갈 여러분만의 게임에 큰 건투를 빈다.

이제 여러분의 새로운 도전이 시작될 차례다.

—끝—

~Next